Everybody

In This Country
Should Learn
How to Program a Computer,
Because it Teaches You

How to *Th!nk.*

— Steve Jobs

J a v a

the complete reference

for

p a t t e r n

programming

By

M O H M A D | Y A K U B

Technical Review : Sadique S. Naikwadi
Proof Reader : Mrs.Shabana Mursal

Editing
Mohmad Yakub

Copyright @ 2019 Mohmad Yakub

All rights reserved. This book or any portion thereof may not be reproduced or used in any manner whatsoever without the express written permission of the author except for the use of brief quotations in a book review.

The author has made every effort in the preparation of this book to ensure the accuracy of the information. However, information in this book is sold without warranty either expressed or implied. The author will not be held responsible for any damages caused or alleged to be caused either directly or indirectly by this book.

Contact Information: **I will appreciate if any queries raised or corrections needs to be applied. I do appreciate honest feedback. Readers of this book can reach me at the following mentioned email address.**

Email : programming.pattern@gmail.com

I dedicate this book as a medium of hope for those;

- **Who are unaware of any approach to craft any programming logic?**
- **Who had a hard time learning programming?**
- **Who had some experience in programming and have no confidence.**
- **Who carries false notion that coding is only for super smart people.**
- **Who are looking for 1st solid move to become self-taught programmer.**
- **Who are victim of discouragement comments similar like the following;**

  - Actually, you aren't interested.
  - You lack patience and determination.
  - Your IQ is well below average.

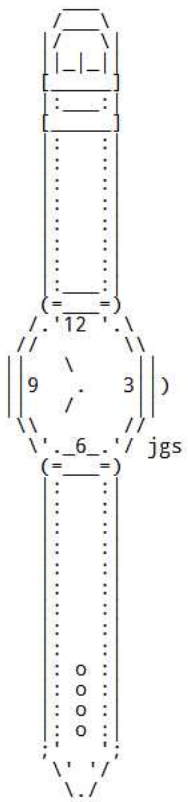
Preface

EVEN THE GREATEST WAS ONCE A BEGINNER.
DON' T BE AFRAID TO TAKE FIRST STEP.

- Muhammad Ali

Developing a fertile ground for **visualization of programming logic** should be the prime focus for an absolute beginner. Unfortunately this perspective is almost alien not only to most of the beginners but also among the teaching group as well. Solve pattern based problems because it will improve the visualization of logic. After some level of practice based on the formulated approach mentioned in this guide book, a **learner's mind will work like a mini-debugger.** A learner not only could be able to visualize the flow of data, but also able to easily relate how different variables are taking values at different point of execution of code.

If a problem asked in the interview or anywhere else, then we should be able to get the logic correctly and in a single chance, instead of playing with guessing. For example consider the following case;

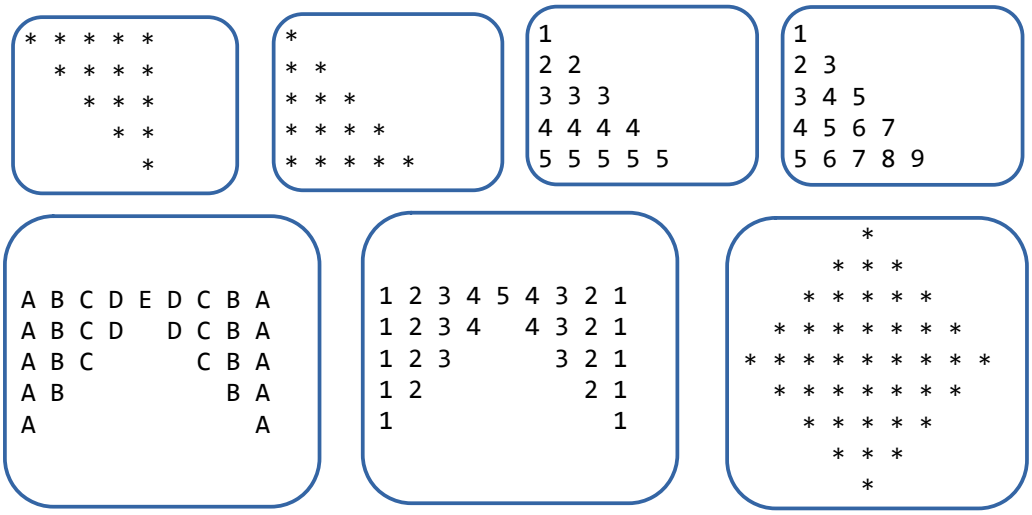


The programmer should be able to write a generalized solution, so that based on input he/she able to generate a wrist-watch of customized sizes.

In the learning of any subject when the visualization and thought process becomes active then only it will turn out to be actually learning. Not only the person will be able to understand the concept but also able to apply the concept naturally. Programming is not about downloading algorithms and

memorizing programming logic in our mind. It is about applying the approach to solve the problems. Practicing approach by solving exercises mentioned in this book, in general improves coding skills.

Patterns are 2-dimensional data output and whether a desktop/mobile or web based applications, there also mostly it deals with 2-dimensional data output. Practice with different patterns helps in developing visualization of logic. This way it builds good foundation and will surely help when a person actually work on real-time application. Following are just few of them and better to practice with many possibilities.



Learning programming with such simple picture based problems surely helps in improving coding skills. The non-matching output results from wrong logical condition applied. This is also a learning, since we may need such type of logical condition in an unfamiliar situation of solving a problem. So, failures will become our lessons. Learning in this way leads learning to interest and force us to put efforts without losing focus. So, in this way it helps in logic building.

On the other hand, learning to use mathematical based problems like prime numbers and other similar kind of problems doesn't create much of interest. Furthermore, on practical ground such mathematical based exercises are just boredom on steroids for most of the non-programmers or beginners.

In my opinion pattern-based problem gives lot many reasons to think logically in different directions and to work on it. The improvement in building logical skills coupled with interest is much higher than any other approach. In general, it suits all learners whoever like to invest some consistent time. I had experimented this with lot many students and i found that this is, the most interesting way to learn programming quickly.

This approach is faster, better and easier way to improve coding skills if followed a proper approach and **not by applying dozens of guesswork to solve a particular problem**. Otherwise, a person will somewhat get improved in guessing and not in finding the approach to solve a particular problem, which is basically against the very definition of an algorithm.

*It reveals logical weaknesses of a programmer if pattern based problems not handled properly. One of the main reason whether at school level, beginners at college level or **junior level interviews**, the pattern based problems appear commonly. This book explains the approach to solve any pattern based problems and in general, it basically teaches **how to get the logic right.***

Without sufficient logic building skills, the efforts on learning advance topics of programming will be like actual talking to Gorilla, it will not be productive. This stuff is unique which I discovered accidentally during my career time while in teaching. I was supposed to complete the teaching of programming subject to first year graduate students and in a very less time of 2-3 months. I did some experiments and devised a plan and that plan was a great success. The results and response was amazingly nice. This book is the explanation of that teachings which can definitely help any student, to learn programming language not only fast but also with a lot of enthusiasm. It is proven formula. It layout an approach based on common sense so that practically anyone can learn to program and interestingly from the very minimal.

If you don't want to waste time on YouTube and other resources, here is the chance to learn quickly. Will you…?

After completing this book and further if you refer any number of resources, you will be in better position to make good use of those resources.

Programming is a kind of mental aerobics which teaches us to keep focus, nourishes good amount of patience, discipline and forecasting. If we observe these are some main qualities of an achiever.

Programming is for <u>everybody</u> and not just an academic exercise.

Happy learning !!!

Table of Contents

Preface ***5***

PART 1: BASICS OF PROGRAMMING LOGIC ***13***

Introduction ***14***

The Right Attitude 15
The Clarity Principles 16
Any pre-requisite? 17
About the book 17
How to make the best use of this book.? 18

Basic Format of Java Program ***19***

SampleFile.java 19
Welcome.java 19
Console Output 20
PrintInSameLine.java 21
PrintInEveryLine.java 21
PrintInEveryLine1.java 22
PrintInEveryLine2.java 22
PrintInEveryLine3.java 23
PrintInEveryLine4Ways.java 23

Storing different data types **24**

Variable declaration 24
Variable assignment 25
Operators 27
Arithmetic Operators 27
ArithmeticOperationsInteger.java 28
IncrementDecrementOperators.java 30
Comparison Operators 31
ComparisonOperators.java 32
Logical Operators 34

Decision Making **35**

a) if-statement 35
StatementIf.java 36
StatementIf1.java 37
b) if-else statement 38
StatementIfElse.java 39
StatementIfElse.java 40
c) if-else if-else statement 41
StatementIfElseIf.java 42
StatementIfElseIf.java 43
StatementIfElseIf.java 44
d) switch-case 46
ConvertLadderIfElseToSwitchCase.java 47

About for-loop ***48***

Basics **49**

ForLoopIncrement.java 50
ForLoopIncrement1.java 50
ForLoopIncrement2.java 51
ForLoopDecrement.java 52

For-loop with {break} and {continue} keywords....53
ForLoopContinueNext.java....53
Nested for-loop/Inner for-loop....54
NestedForLoop.java....55
Nested Loop: Beginner Programmer's Best Friend....55
Why nested loops are such an important deal?....56
ForLoopEveryRowAllColumns.java....57
ForLoopEveryRowAllColumnsHorizontally.java....58
ForLoopCoordinateRepresentation.java....59
RowColumnStars.java....60
JustLoopIt: Write a separate program for each of the given output....61
Art of Applying Logical Conditions....62
Convert Logical Conditions to Simple Picture....64
Convert Simple Picture to Logical Conditions....67
VerifyAlphabetA.java....69
VerifyAlphabetB.java....72
VerifyAlphabetC.java....74
VerifyAlphabetD.java....76
VerifyAlphabetE.java....78
VerifyAlphabetF.java....80
VerifyAlphabetG.java....82
VerifyAlphabetK.java....87
VerifyAlphabetM.java....91
You can do it....94
Learning by experiments and guesswork....97
Logic trace table: Brain behind programming logic....100
PART 2:* PROGRAMMING IN ACTION....*101
Cursor Movement....103
Pattern 1A....104
Analysis of output....104
Pattern1A1If.java....106
PatternA1NestedLoop.java....109
Pattern1A_1.java....117
Pattern1A_1.java....118
Pattern1B....119
Analysis of output....119
PatternB1If.java....121
PatternB1NestedLoop.java....123
Approach to solution....125
PatternB1.java....129
Alternative solution2....130
PatternB2.java....131
Alternative solution3....132
PatternB3.java....133
Pattern 1C....135
Analysis of output....135
Approach to solution....138
PatternC1.java....140
Alternative solution2....141

PatternC2.java....143
Alternative solution3....144
PatternC3.java....146

Pattern1D....147
Analysis of output....147
PatternD1.java....151

Pattern1E....153
PatternE1.java....159
Alternative solution2....160
PatternE2.java....163

Pattern 1F....164
PatternF1.java....169
Alternative solution2....170

Pattern 1G....171
Approach to solution....175
PatternG1.java....180
PatternG2.java....181
Alternative solution2....182
PatternG3.java....183

Pattern1H....184
Approach to solution1....187
PatternH1.java....191
PatternH1_2.java....192
Alternative solution2....193
PatternH2_1.java....196
PatternH2_2.java....199
Alternative solution3....200
PatternH3.java....203

PatternAA....205

PatternCC....205

PatternII....206

NumberPattern 1A....209
Analysis of output....210
NumberPattern1A.java....213

NumberPattern 1B....214
Analysis of output....215
Observations....215
NumberPattern1B.java....219
Alternative solution....220
NumberPattern1B_2.java....221

NumberPattern 1C....222
Analysis of output....223
NumberPattern1C_1.java....225
NumberPattern1C_2.java....226
NumberPattern1C_3.java....227
NumberPattern2C.java....229

NumberPattern 1D.....230
Analysis of output.....231
Observations.....231
NumberPattern1D.java.....234
Alternative solution.....235
NumberPattern2D.java.....235
Alternative solution.....236
NumberPattern3D.java.....238

NumberPattern 1E.....239
Analysis of output.....240
NumberPattern1E.java.....243

NumberPattern 1F.....244
Analysis of output.....245
NumberPattern1F.java.....248

NumberPattern AA.....249
Analysis of output.....250
Observations.....250
NumberPatternAA.java.....252

NumberPattern BB.....253
Analysis of output.....254
Observations.....254
NumberPatternBB.java.....257
Alternative solution.....257
NumberPatternBB2.java.....258

NumberPattern CC.....259
Analysis of output.....260
NumberPatternCC1.java.....262
NumberPatternCC2.java.....263
NumberPatternCC3.java.....265

NumberPattern DD.....266
Analysis of output.....267
NumberPatternDD1.java.....269
NumberPatternDD2.java.....270
NumberPatternDD3.java.....272

NumberPattern EE.....273
Analysis of output.....274
NumberPatternEE.java.....277

NumberPattern FF.....279
Analysis of output.....280
Alternate solution.....282
NumberPatternFF.java.....286
If-else Shortcut.....287
When to apply if-else shortcut.....287
NumberPatternFF2.java.....289

NumberPattern GG.....290

Writing our own functions.....293

NumberPattern EE.....296

Analysis of output....297

NumberPattern AAA....301

Analysis of output....302
Observations....302
NumberPatternAAA.java....304

NumberPattern BBB....305

Observations....306
NumberPatternBBB1.java....309
Alternative solution....310
NumberPatternBBB2.java....310

NumberPattern CCC....311

Analysis of output....312
NumberPatternCCC1.java....314
Alternative solution....315

NumberPattern DDD....319

Analysis of output....320
Observations....320

NumberPatternEEE....325

Analysis of output....326

String concatenation....333

Patterns by Single Looping....334

Accessing each string's character....339

Substrings....341

Appendix-A: Installation of eclipse IDE and project set-up....347

References:....358

Acknowledgments....359

PART 1

BASICS OF PROGRAMMING LOGIC

Introduction

Without a doubt every subject has some key fundamental basis. If key concepts of any subject are truly internalize then the rest of the learning will be productive.

Many novice programmers and many frustrated programmers do ask similar kind of questions which are as follows;

“How to learn programming?”

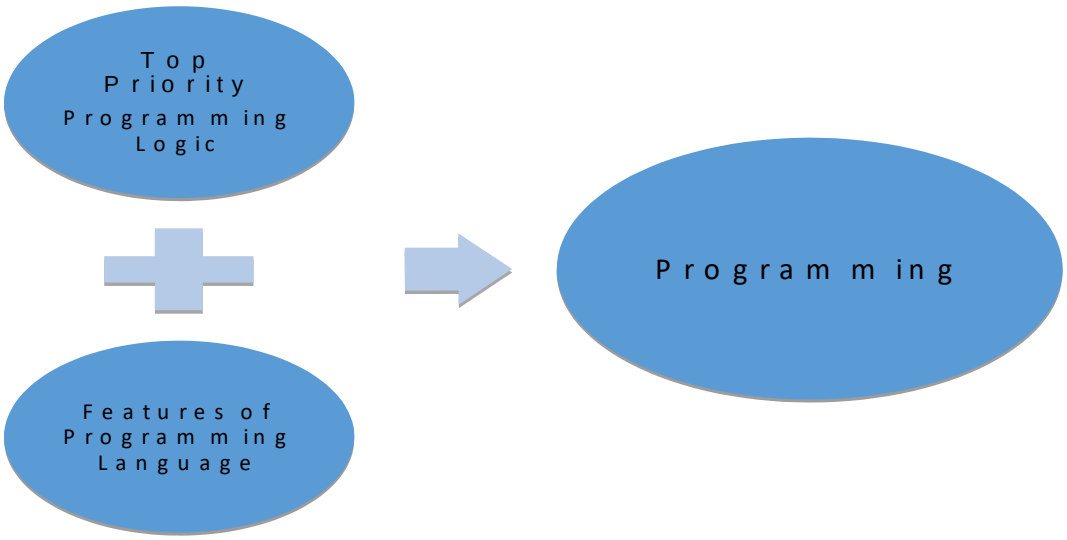
“How to develop logic building skill?”

“How to learn coding?”

“How to improve programming logic?”

Programming is an applied field. In my opinion if a programmer who is weak in programming logic is not a programmer even. We know that complaining is the easiest job and if someone is expert in that haven’t actually achieved anything big. Whenever there is a problem, we just need to think “**what could be the easiest possible way out**?”

In order to become confident in any programming language, the first step should be to gain some level of proficiency to self-hack programming logic. This technical manual is totally dedicated to a non-programmer/beginner or intermediate students who are just tired of hitting hard on many different places in order to become confident in programming. Additionally, if you are among those who got limited time to learn programming, this is the guide that can serve you well too. **So the rule of the thumb is, in order to learn programming language fast and properly, first learn the tactics of programming logic sufficiently**.



The most common mistake that many non-programmers/absolute beginners or trainers does is that they focus more on grammar/syntax and feature(s) of the programming language. Such shallow outlook about programming language doesn’t help beginners much. Computer science is an applied field to solve practical problems. It is foremost and essential need that a beginner should get trained sufficiently in area of programming logic skills at the first priority. How does an appropriate physical exercises and healthy food tones the body muscles, similar it goes with developing programming logical skills. It does need right kind of logic based programming exercises to groom visualization of programming logic.

It also happens that a beginner may know almost all the concepts of programming language but still feels something is missing. That missing element is due to insufficient logic building capabilities. Without a comfortable level of logic building skills, the beginner struck a lot and, on top of that, they face lot of confusions while developing programs.

The Right Attitude

> ***Failure taught me things about myself that I could have learned no other way.***
>
> **- J.K. Rowling**

By default, children are not afraid to be wrong, and so they are creative. It's well said **hiding mistakes is a mistake of higher order**. So it is always good to notice our mistakes as early as possible so that it shouldn't give a hard blow in the future during the much-needed time.
Experimenting and learning from mistakes work like a some sort of body detoxification program. Initially, any detoxification program doesn't taste nice but still person go for it because that person will love to see the way the body will going to respond after 2-3 weeks. Anything new that we do take some practice. Everything that is valuable has a cost associated with it. Failures are one such type of cost that are to be paid to learn and to remember lessons.
Getting accurate output after executing a program is just a by-product. So the total learning will be failures plus the success part. So we need to think in this way, more initial failures will going to help us to reduce the chances of failures in the future instead of thinking that we are not a good learner. When a learner knowingly or unknowingly applies mismatching logical conditions, may encounter unexpected output. This in turn creates a curiosity that which specific logical condition is impacting the output differently. It's easy for a brain to remember surprises. Moreover, picture helps brain to remember more than the flowing texts, let it be any fact or concept. So it's much useful if a learner experiment with a weird or with some typical logical conditions and try to find the reasons behind it. This will work like a self-motivating exercise and it simply creates urge to learn more.

We all have hidden potential and it is up to us to explore ourselves. You are the only person who can explore your hidden potential by taking challenges by experimenting it with your capabilities. Unless and until you take this journey of challenges, you will not know **actual YOU** or you will remain rather somebody else a common class. ***You, I and everybody*** *had to do this exercise of treasure hunt within ourselves.*

It is all about game of attitude and applying our actions in trying for excellence. We just need to bounce back repetitively with another honest attempt. One of the main aim of this guide is to ignite thought process and to develop a natural tendency to solve problems just by touching bare minimal technical details. We know that attitude measures altitude. A right intention gives a lot of power to manage even tough task with confidence. It is all about the perspective, the way we think and approach the situation accordingly.

The Clarity Principles

Let help ourself by putting these principles as our reminders. We must give reminders to ourself, the key important values that helps to cleanse our subconscious mind. Definitely there are benefits in reminders, so give reminders to others or our-self.

> ***You can't manage what you can't measure.***
>
> **- Peter Drucker**

> ***Science is simply common sense at its best, that is, rigidly accurate in observation, and merciless to fallacy in logic.***
>
> **- Thomas Huxley**

> ***Most of man's problems upon this planet, in the long history of the race, have been met and solved either partially or as a whole by experiment based on common sense and carried out with courage.***
>
> **- Frances Perkins**

> ***I read, I study, I examine, I listen, I think, and out of all that I try to form an idea into which I put as much common sense as I can.***
>
> **- Marquis de Lafayette**

> ***Common sense is calculation applied to life.***
>
> **- Henri-Frédéric Amiel**

The most important aspect of calculation or measurement is that it gives the clarity. We will try to tap this "**clarity**" factor to get the hidden programming logic right using **logic trace table.**
All learning covered by this guide is majorly revolves around the concept of **logic trace table.** In the section **Programming Patterns,** its practice **is dealt in greater details.**
We can't expect quality software without quality programs. Good programming logic is one of the essential quality to write error-free program and quality software.

Any pre-requisite?

Intention to give an honest try to learn programming. No prior experience in programming language is required. If you have any, it is good and this is optional as we are not dealing much of the syntactical technical matter. We only need very minimal language's features to practice programming exercises in order to cultivate interest and confidence in programming. We are majorly dealing with two main technical concepts.

a) Conditional statements (If-statement, if-else statement)

b) Looping construct— just *for-loop*.

Programming syntax of conditional statements and ***for-loop*** of Java resembles with many other programming languages like that of JavaScript, C#, C and C++. By learning from this book they can easily experiment in other programming language of their interest.

One of the common quality of highly successful people are as soon as they acquire information, their speed of implementation is very fast. This quality helps them to be stayed ahead in the competition. The best thing to **compete with ourselves and try to** break our own record.

About the book

All the material presented here is only meant for action, a practical to do sessions. We are not covering Java as a language but programming logic as a subject using Java programming language's syntax. This book is not a product of guesswork. This book is outcome of experiments conducted among college students in order to improve their logic building skills or to learn programming from the scratch but in a quick way.

The learner has to practice the approach mentioned in the book for logic building skills related to 2-dimensional simple picture based exercises. Practice exercises arranged in an increasing challenging way and motivates to refine programming skills continuously in a step by step manner. The solved exercises gives the complete idea and unsolved problems urges applying logical concept. So, it is going to be **YOU versus YOU** in an interesting way.

Two major requirements expected from every learner which are as follows:

a) Intention that you want to learn programming

b) Need to spend some time consistently and it depends on how much quickly you want to become a confident programmer. **Don't break the learning pattern at least for 3 months, spend at least minimum 20 – 30 minutes daily.** Just follow the mentioned course of action step by step, you will surely get substantial grip in the programming.

It's up to you afterwards to explore new horizons in programming or software development. I can only promise an excellent start in a short period of time regarding your journey of learning a programming logic. ***So let's start the journey of learning.***

How to make the best use of this book.?

- Start a topic and learn till you reach a point where you are ready to analyze a solved example. Concentrate on that example till you get a feel that you have understood everything about the program.

  Then try to implement that solved example without referring it. Make a strong will that once you start implementing already studied program, you are not going to refer it back that particular solved program. You can refer theoretical explanation but not program. There are a lot of chances that you may get many errors syntactically or logically as well. In that situation, you can refer theoretical explanation about the concept but do not allow yourself to see the solution. Try your best to resolve issues by yourself referring theoretical explanation.

- Please maintain an Excel or spreadsheet file to note down the errors you encountered in a program. When you are able to resolve the errors, please note down the reasons why you have adopted that approach. Also note down, what was your first observations and afterwards when you had resolved a particular logical error. Study of such gap analysis help us to know, how to think in right direction.

- Every new day when you want to start your learning, please have a quick look to your error file that you were maintaining every time whenever encountered any error(s). Referring back to those errors while learning every new day will help to do better next time and it will also help you to quickly revise the concept. We should maintain this practice to avoid common logical errors so those errors should not become part of our habit. So it is good to correct those loopholes in the beginning itself, since breaking a habit consumes sufficient efforts. What we practice, that will become our habit whether good or bad. Here, we are trying to **convert failures into lessons**. It is more rewarding when we get improved from mistakes as early as possible so that we can invest more time learning other important subjects in our life.

- If two times you are able to solve all the solved examples from the book back to back without referring to the solutions, then you will be in a better position to write program effectively.

- Hints are there fro unsolved problems in the explanations of solved exercises. If you are unable to solve unsolved problem, just try 2-3 attempts and move forward to complete the book. Come back again and try to attempt the remaining ones.

- Two times back to back, if you are able to solve all the solved exercise without referring to the solutions and also unsolved problems, then logic wise you will be in a very comfortable position. You will get the courage to build programming logic for more difficult level and can be able to deal other heavier subjects of programming like data-structures and design patterns with confidence.

Basic Format of Java Program

The general format/template for our program will be something like as follows.

SampleFile.java

```
public class SampleFile {

	public static void main(String[] args) {

		// Your program logic goes here

	}

}
```

How we can write our programming logic using Java file?

In order to check the correctness of our programming logic we are required to do two things:

1) From **SampleFile.java file** create your program file, give program's name.
2) Place all your programming code in between code block of main method.

```
public static void main(String[] args) {

	// Your program logic goes here

}
```

Code Block starts with **open curly bracket {** .
It ends with **closing curly bracket }**.

Suppose we want to write a program and our name of program is Welcome, please do the following;
a) Save ***File*** with name as **Welcome.java**.
b) ***Java Class Name*** in the Welcome.java program file will be **Welcome**.

For now just ignore **public class**, **public static void** and other theoretical stuff.
Just reuse this **SampleFile.java** file whenever you want to write a new program.

The following **Welcome.java** Java program outputs the text **Welcome to Java Programming.**

Welcome.java

```
/*
 * This is my first program.
 * Program to print welcome message.
 */
public class Welcome {

	public static void main( String[] args ) {
		// program logic goes here
		System.out.println("Welcome to Java Programming.");
	}

}
```

Output

```
Welcome to Java Programming.
```

The multi-line comments are given by /* multi-line...*/
The single line comment is given by // single-line...

Comments are for our understanding/documentation purpose and not a part of our programming code. The Java compiler ignores it.

How to execute a Java program in order to verify programming logic?

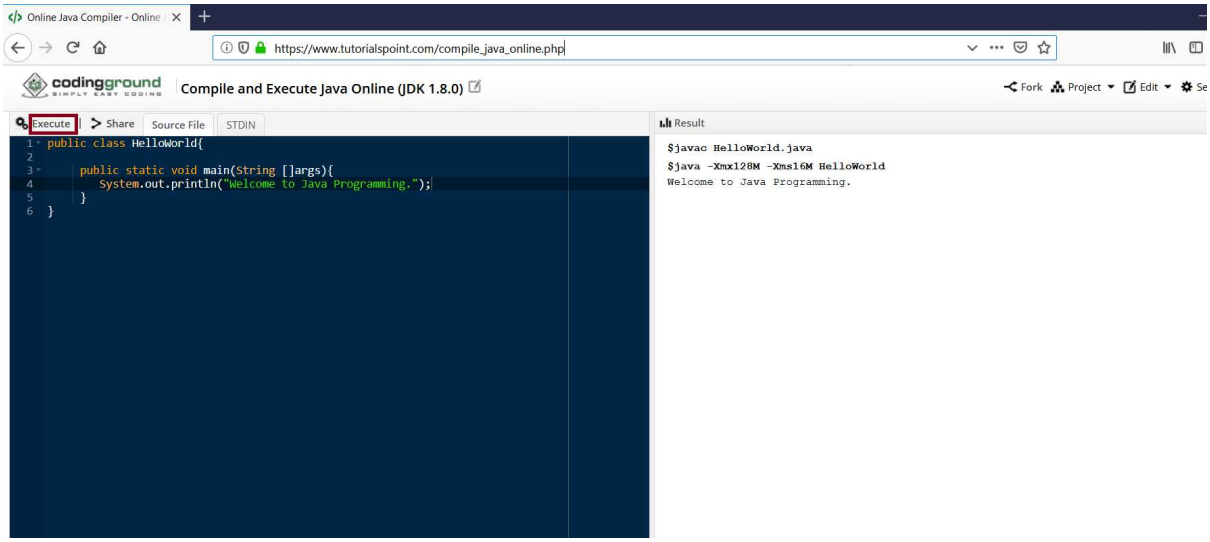
Please refer the following title at end of this book.

Appendix-A: Installation of eclipse IDE and project set-up

It shows step-by-step procedure to create a **Java project** using eclipse IDE and further shows how to create and **execute a Java program** with screen-shots. Eclipse IDE is for free and widely used in software industry also.

Alternatively, you can use one of the online editor shown as follows.

https://www.tutorialspoint.com/compile_java_online.php



Click the **Execute** button to run the program. By default, **Source File** tab will be opened to write the program. A separate Result window is shown, to display the output of the program.

Console Output

Let's try to use Java's inbuilt methods **System.out.print**() and **System.out.**println() to output result in different ways. We can use these methods to generate 2-dimensional output to verify logic.

System.out.println("text");	**Print text to the console and moves the cursor to the next line.**
System.out.print**("**text**");**	**Print text to the console.**
System.out.println**();**	Moves the cursor to the next line.

Following programming statement(s) means the same. **It outputs the given text and moves to the next line in the console outputting window.**		
System.out.println**("**text**");**		
System.out.print**("**text**");** **System.out.**println**();**	**System.out.**print("text**\n**");	**System.out.**print**("**text**"+** "\n**");**

We will now start with learning programming logic, simply by trying to repeatedly print a small text **horizontally** in the same line or **vertically** in every line.

Exercise 1. Printing text "**Java:**" in the same line.

PrintInSameLine.java

```
public class PrintInSameLine {
	public static void main(String[] args) {
		System.out.print("Java: ");
		System.out.print("Java: ");
		System.out.print("Java: ");
	}
}
```

Output

```
Java: Java: Java:
```

Exercise 2. Let's try to output "**Java:**" three times in every line.

PrintInEveryLine.java

```
public class PrintInEveryLine {
	public static void main(String[] args) {
		System.out.println("Java: ");
		System.out.println("Java: ");
		System.out.println("Java: ");
	}
}
```

Output

```
Java:
Java:
Java:
```

The following exercises are showing **different ways of single line break** in the output.

Exercise 2.1. Printing "**Java:**" in every line and use of `System.out.println()` (single line break).

PrintInEveryLine1.java

```
public class PrintInEveryLine1 {
	public static void main(String[] args) {
		System.out.print("Java: ");
		System.out.println();
		System.out.print("Java: ");
		System.out.println();
		System.out.print("Java: ");
	}
}
```

Output

```
Java:
Java:
Java:
```

Exercise 2.2. Printing "**Java:**" in every line and use of "**\n**" (single line break character).

PrintInEveryLine2.java

```
public class PrintInEveryLine2 {
	public static void main(String[] args) {
		System.out.print("Java: " + "\n");
		System.out.print("Java: " + "\n");
		System.out.print("Java: ");
	}
}
```

Output

```
Java:
Java:
Java:
```

Exercise 2.3. Printing "**Java:**" in every line and use of "**\n**" (single line break character).

PrintInEveryLine3.java

```
public class PrintInEveryLine3 {

	public static void main(String[] args) {

		System.out.print("Java:\n");

		System.out.print("Java:\n");

		System.out.print("Java: ");

	}

}
```

Output

```
Java:
Java:
Java:
```

Exercise 2.4. Printing "**Java:**" in every line and in different ways.

PrintInEveryLine4Ways.java

```
public class PrintInEveryLine4Ways {

	public static void main(String[] args) {
		//1st way to print to console
		System.out.println("Java: ");

		//2nd way to print to console
		System.out.print("Java: " + "\n");

		//3rd way to print to console
		System.out.print("Java:\n");

		//4th way to print to console
		System.out.print("Java:");
		System.out.println();
	}
}
```

Output

```
Java:
Java:
Java:
Java:
```

We will learn required only minimum technical knowledge to sharpen our intelligence in programming logic. We are targeting to dive deep in art of building programming logic rather than going deep in basic grammar/syntax of programming language. Lets checkout some basic building blocks of programming logic.

Storing different data types

The intelligence of software is due to programming language(s). The processing of different types of data in some form, is something very core to almost every software. Facility to manipulate **different types of data correctly** should be a necessary feature of any programming language. Example of commonly used different data types.

General data type	Value examples
Integer	**23, -4, 8908**
Real Numbers	**23.45, 3.1456, -45.27**
Truth Values	**true, false**
text	**"Just do it", "impossible"**
Single Character	A, a, @, $, %,*, 9

Don't we use a different variety of data in our daily life? We actually do, for example;

Name : …….

Age : ……

Weight(In Kgs) : ……..

Personal loan (true/false): ……

Gender(M/F) : ……

We are sometimes required to fill correctly personal details in some way or the other way. Few examples are many, like school or college admission process, flight booking or any online purchasing. We can easily distinguish different types of data whether it's numeric, textual or any other kind.

How can be a programming language made intelligent to distinguish different variety of data?

Yes, it is made possible through the facility of declaring variables.

Variable declaration

The format for variable declaration is as follows:

data-type variable_name;

Consider the following sample personal details needs to be filled in, for example;

The variable declaration for the considered common personal details can be expressed as;

Equivalent variable declaration	Required to fill personal details in some real world situation
String name; **int** age; **float** weightInKgs ; **boolean** personalLoan ; **char** gender;	**Name :** …….. **Age :** …… **Weight**(In Kgs) : …….. **Personal loan** (true/false): …… **Gender**(M/F) : ……

For example, the data value 25 could be an age, price and there are many more possibilities. The name of variable should be appropriately chosen by giving full idea about the data it is going to refer. The chosen name of variable helps us to know meaning of stored value contained in it.

How to store data/value in a variable?

Yes, it is made possible through the facility of variable assignment using assignment operator (=).

Variable assignment

Any real time situation of manually filling data can be simulated using feature of a programming language known as variable assignment. Variable assignment is a simple act of storing an appropriate value in an appropriate variable. We can assign the value to the variable using assignment operator (=).

The value of a variable can be over-written programmatically and that is why it's called variable.

In a classroom, the space of board are re-used by wiping out the details for every new class. In the similar way these variables can be over-written programmatically during the course of running/executing the program. **We can store or assign only one data at a time.**

The format for variable assignment is as follows:

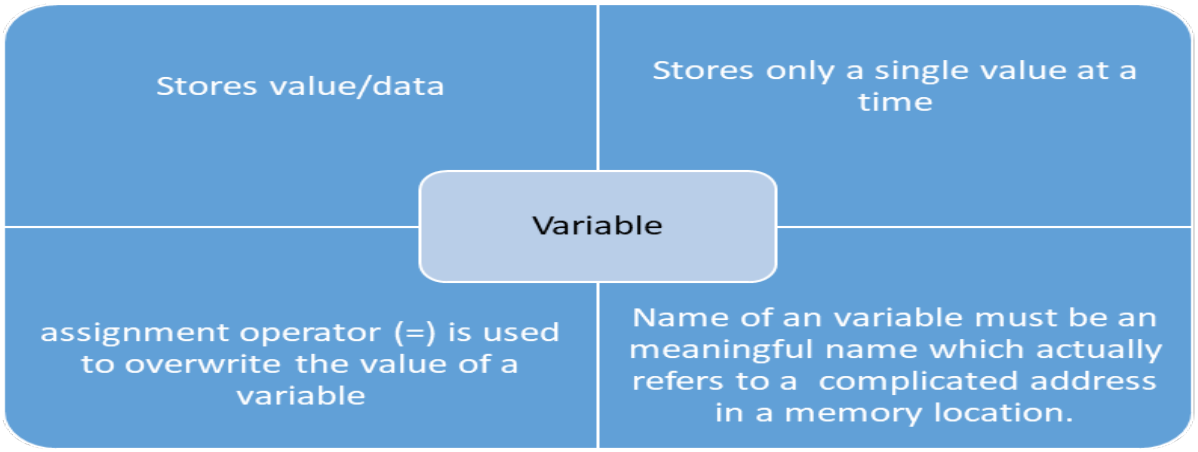
data_type variable_name = value; // assign only one data at a time

Every single programming statement separated by character semi-colon(**;**) similarly like how a character full-stop (.) separates each sentence in the act of writing or typing text.

Equivalent variable assignment	Filled in personal details
String name = "James Arthur Gosling"; **int** age = 63 ; **float** weightInKgs = 53.6; **boolean** personalLoan = false; **char** gender = 'M';	**Name** : James Arthur Gosling **Age**: 63 **Weight**(In Kgs) : 53.6 **Personal loan** (true/false): false **Gender** : M

The names of variables represent memory locations. It is very difficult to do programming using name of memory locations. Memory address location resembles not less than any secret key. The name of a variable that we choose should be a user-friendly name which represents a specific memory location allocated for the purpose to store data.

Please keep these points in your mind and revise it before you start writing programs.



The space in memory location can be expressed in terms of byte units. So, the size of primitive data types too can be expressed in terms of bytes to store the value. (**Note: - 1 byte = 8 bits.**)

Data type	Size	Minimum Value	Maximum Value	Default Value
byte	1byte	-128	127	**0**
short	2bytes	-32768	32767	**0**
int	4bytes	-2,147,483,648	2,147,483,648	**0**
long	8bytes	-9,223,372,036,854,775,808	9,223,372,036,854,775,808	**0**
float	4bytes		upto 6 to 7 decimal digits	**0.0f**
double	8bytes		upto 15 decimal digits	**0.0d**
char	2bytes		65536 characters	**'\u0000' or '' (single space)**
boolean	1-bit	true or false (Only two possible values)		true or false

What is there in the variable's name?

We know that in geographical maps, the locations can be represented as combination of longitude & latitude. Think for a moment if our city, town, street places are replaced by longitude and latitude values, how much it will become difficult to use it in our normal communication. So the choice of variable should be a meaningful name and must indicate some purpose associate with it. It is not uncommon to find such programmers who have this special **SLVNS** sickness.

What is SLVNS sickness?

SLVNS stands for **Single Letter Variable Name Syndrome**. Many programmers has this special habit of creating suspense through name of a variable. They allocate just a single letter for variable's name in writing programs. When such program become lengthy and someone else has to fix a bug/error for that programming code, it will surely turn out to be irritating experience. Even such **SLVNS** programmers may find it difficult to get his/her own programming logic quickly on revisiting after a gap of time. If you find those programmers, please request them to leave this habit, it is an offense. Please prevent yourself from this sickness and **be patient to write meaningful variable name.**

Mind it, Better You Mind It.

Operators

Few among special keyboard characters used as operators in programming to denote frequent mathematical operations like addition, subtraction, comparing two numeric values and many such operations. We can categorize operators as per their purpose. In short, it's a simulation of simple school math calculation activities but using programming. These simple mathematical operations are very much common in our daily life and plays a major role in dealing daily business or transactions. Such calculation based activities with accuracy and many such related activities are very much common in our daily routine.

Arithmetic Operators

Operator	Arithmetic Operation Description
+	**Addition**
-	**Subtraction**
*	**Multiplication**
/	**Division**
%	**Modulus (Remainder on division)**
++	**Increment**
--	**Decrement**

For example, simple **addition operation** using "**+**" **operator** can be given as follows:

```
int intNum1 = 6;
int intNum2 = 4;
int intResult = intNum1 + intNum2;
```

We know that using assignment operator (=), at a time we can store single value in a variable. So in the considered example, **intNum1** will store the value 6 & **intNum2** will store the value 4.

Finally, **intResult** will store the result value of addition of **intNum1** and **intNum2** as addition operator (+) is applied between these two variables.

Variable Declaration	Memory Address	Memory Content
`int` intNum1;	1221	**Nil**
`int` intNum2;	1222	**Nil**
	1223	
Variable Assignment	**Memory Address**	**Memory Content**
`int` intNum1 = 6;	1221	**6**
`int` intNum2 = 4;	1222	**4**
	1223	
`int` intResult = intNum1 + intNum2;	1234	10

Exercise 3. Demonstrate basic arithmetic operations using arithmetic operators.

ArithmeticOperationsInteger.java

```
public class ArithmeticOperationsInteger {

	public static void main(String[] args) {
		int intNum1 = 6, intNum2 = 4, intResult;
		String operatorStr;
		String expressionStr;
		String resultStr;

		System.out.print("Integer numbers: ");
		System.out.print("intNum1 = " + intNum1 + ", " );
		System.out.print("intNum2 = " + intNum2);
		System.out.println();
		System.out.println();
		String header = "Operations | Operator | Expression | Result";
		String line = "-----------------------------------------";
		System.out.println(header);
		System.out.println(line);

		operatorStr = "Add        | +        | ";
		expressionStr = intNum1 + " + " + intNum2 + "      | ";
		intResult = intNum1 + intNum2;
		resultStr = operatorStr + expressionStr + intResult;
		System.out.println(resultStr);

		operatorStr = "Subtract   | -        | ";
		expressionStr = intNum1 + " - " + intNum2 + "      | ";
		intResult = intNum1 - intNum2;
		resultStr = operatorStr + expressionStr + intResult;
		System.out.println(resultStr);
```

```
            operatorStr = "Multiply   | *        | ";
            intResult = intNum1 * intNum2;
            expressionStr = intNum1 + " * " + intNum2 + "      | ";
            resultStr = operatorStr + expressionStr + intResult;
            System.out.println(resultStr);

            operatorStr = "Divide     | /        | ";
            intResult = intNum1 / intNum2;
            expressionStr = intNum1 + " / " + intNum2 + "      | ";
            resultStr = operatorStr + expressionStr + intResult;
            System.out.println(resultStr);

            operatorStr = "Remainder  | %        | ";
            intResult = intNum1 % intNum2;
            expressionStr = intNum1 + " % " + intNum2 + "      | ";
            resultStr = operatorStr + expressionStr + intResult;
            System.out.println(resultStr);

      }

}
```

Output

```
Integer numbers: intNum1 = 6, intNum2 = 4

Operations | Operator | Expression | Result
-----------------------------------------
Add        | +        | 6 + 4      | 10
Subtract   | -        | 6 - 4      | 2
Multiply   | *        | 6 * 4      | 24
Divide     | /        | 6 / 4      | 1
Remainder  | %        | 6 % 4      | 2
```

Exercise4. Write a program showing value changes using incrementing & decrementing operator.

IncrementDecrementOperators.java

```
public class IncrementDecrementOperators {
	public static void main(String[] args) {
		int  value = 100;
		int intResult ;
		System.out.println("Given values.");
		System.out.println("value = " + value + "\n");
		System.out.println("Incrementing 3 times");
		value++;
		value++;
		value++;
		System.out.println("Result = " + value + "\n");
		System.out.println("Decrementing 2 times");
		value--;
		value--;
		System.out.println("Result = " + value );
	}
}
```

Output

```
Given values.
value = 100

Incrementing 3 times
Result = 103

Decrementing 2 times
Result = 101
```

Problem. Write a program named **ArithmeticOperatorsOnFloat.java** showing arithmetic operations Add, Subtract, Multiply, Divide & Remainder using the following variable names & values;

float floatVal1 = 2.55 ;

float floatVal2 = 1.77.;

Problem. Write a program to decrement the value of variable total 5 times and then increment it 5 times. The decrementing /incrementing counter value is 1.0

float total = 5.55;

Comparison Operators

The comparison result of two values either will be **true** or **false** and these values also known as **boolean values or truth values**. This feature is of very much importance since it will help to make decision in terms of true or false. The computer memory mainly based on 1(true) or 0(false) value called as **bit value**. Every data in computer is a combination of one's and zero's called bit pattern.

Unique value or data in a computer's memory means a group of unique bit patterns.

Operator	Comparison Operation Description
==	**Equal**
!=	**Not Equal**
<	**Less than**
<=	**Less than or equal**
>	**Greater than**
>=	**Greater than or equal**

Using comparison operators we can compare two numeric values. The following example shows the use of comparison operators using numeric values.

Operator	Operand1(Operator) Operand2	Result
==	4 == 4	**true**
	4 == 10	**false**
!=	6 != 4	**true**
	4 != 4	**false**
<	6 < 10	**true**
	6 < 6	**false**
<=	6 <= 6	**true**
	10 <= 6	**false**
>	6 > 6	**false**
	10 > 6	**true**
>=	4 >= 6	**false**
	10 >= 6	**true**

We can also test the above results by storing the input values in variables and then applying comparison operators one by one. **Philosophically, true and false are the only two perfect answers in this world**. In a program it requires us to do decision-making based on perfect answers. Developing logic based on operators is the need of implementation of decision-making. It allows flexibility in handling different logical situations in a program. Sense and sensibility of applying logical conditions is one of the core aspect of any good programmer.

So please, invest quality time studying this topic with special attention.

Let's verify the results using comparison operators on operands 4,6 and 10.

ComparisonOperators.java

```
public class ComparisonOperators {

	public static void main(String[] args) {
		int four = 4;
		int six = 6;
		int ten = 10;
		boolean isResult;
		String operatorStr;
		String expressionStr;
		String resultStr;

		String header = "Description    | Operator | Expression  | Result ";
		 String line ="-----------------------------------------------------";
		System.out.println("*** Comparison Operators ***");
		System.out.println();

		System.out.println(header);
		System.out.println(line);

		operatorStr = "equal                  | ==       | ";
		expressionStr = four + " == " + four + "      | ";
		isResult = ( four == four ) ;
		System.out.println( operatorStr + expressionStr + isResult );

		expressionStr = four + " == " + ten + "     | ";
		isResult = ( four == ten ) ;
		System.out.println( operatorStr + expressionStr + isResult );
		System.out.println(line);

		operatorStr = "not equal              | !=       | ";
		expressionStr = six + " != " + four + "      | ";
		isResult = ( six != four ) ;
		System.out.println( operatorStr + expressionStr + isResult );

		expressionStr = four + " != " + four +"      | ";
		isResult = ( four != four ) ;
		System.out.println( operatorStr + expressionStr + isResult );
		System.out.println(line);

		operatorStr = "less than              | <        | " ;
		expressionStr = six + " < " + six + "       | ";
		isResult = ( six < ten ) ;
		System.out.println( operatorStr + expressionStr + isResult );

		expressionStr = six + " < " + ten + "      | ";
		isResult = ( six < six ) ;
		System.out.println( operatorStr + expressionStr + isResult );
		System.out.println(line);

		operatorStr = "less than or equal     | <=       | " ;
		expressionStr = six + " <= " + six + "      | ";
		isResult = ( six <= six ) ;
		System.out.println( operatorStr + expressionStr + isResult );

		expressionStr = ten + " <= " + six + "     | ";
		isResult = ( ten <= six ) ;
		System.out.println( operatorStr + expressionStr + isResult )
		System.out.println(line);
```

```
            operatorStr = "greater than          | >        | " ;
            expressionStr = six + " > " + six + "       | ";
            isResult = ( six > six ) ;
            System.out.println( operatorStr + expressionStr + isResult );

            expressionStr = ten + " > " + six + "      | ";
            isResult = ( ten > six ) ;
            System.out.println( operatorStr + expressionStr + isResult );
            System.out.println(line);

            operatorStr = "greater than or equal | >=       | "  ;
            expressionStr = four + " >= " + six + "      | ";
            isResult = ( four >= six ) ;
            System.out.println( operatorStr + expressionStr + isResult );

            expressionStr = ten + " >= " + six + "     | ";
            isResult = ( ten >= six ) ;
            System.out.println( operatorStr + expressionStr + isResult );
            System.out.println(line);
        }
}
```

Output

```
Given values.
four = 4,six = 6,ten = 6
equal

4 == 4|true
4 == 10|false
--------------------------------------
not equal

6 != 4|true
4 != 4|false
--------------------------------------
less than

6 < 10 | true
6 < 6 | false
--------------------------------------
less than or equal

6 <= 6 | true
10 <= 6 | false
--------------------------------------
greater than

6 > 6 | false
10 > 6 | true
--------------------------------------
greater than or equal

4 >= 6|false
10 >= 6 | true
--------------------------------------
```

Logical Operators

It also returns an ultimate value in either **true or false** similar like expression evaluation in case of comparison operator. It is primarily used to chain many logical expressions based on comparison operators. Following are types of logical operator's name and their respective symbol.

1) Logical AND (**&&**) **2)** Logical OR (**||**) **3)** Logical NOT (**!**)

1) Logical AND: Returns **true** if and only if when the result of **both logical expressions A and B are true** else false. The more we use this operator in a logical expression, the lesser the size of the data set we get out of total data. It usually applied in a situation to get specific data or data of lesser range or size. Let, there are two integer variables, month = 10 and days = 5. Please observe the column "**Result**".

A	B	A&&B	A	B	A&&B = Result
true	true	true	month > 7 (true)	days < 7 (true)	**(true) && (true) = true**
true	false	false	month > 7 (true)	days > 7 (**false**)	**(true) && (false) = false**
false	true	false	month < 10 (**false**)	days < 7 (true)	**(false) && (true) = false**
false	false	false	month < 10 (**false**)	days > 7 (**false**)	(**false**) && (**false**) = **false**

2) Logical OR: Returns **false** if and only if when the result of **both logical expressions A and B are false** else true. The more we add OR condition using this operator in a logical expression, the more it increases the size of data as an output result. So it is used to get a broader range of data. Let, there are two integer variables, month = 10 and days = 5. Please observe the column "**Result**

A	B	A ||B	A	B	A ||B = Result
true	true	true	month > 7 (true)	days < 7 (true)	**(true) || (true) = true**
true	false	true	month > 7 (true)	days > 7 (**false**)	**(true) || (false) = true**
false	true	true	month < 10 (**false**)	days < 7 (true)	**(false) || (true) = true**
false	false	false	month < 10 (**false**)	days > 7 (**false**)	(**false**) || (**false**) = **false**

3) Logical NOT: Returns just opposite, **true** when the outcome of the logical expression is false or returns false when the outcome of a logical expression is true.

A	!A
true	**false**
false	true

Please note that true/false values also known as boolean values or truth values.

Usage of logical operators are quite common and plays a very important role in a designing logic of programs. Every programmer must and should have an obvious idea about usage of these operators. It is worth time spent if a beginner put extra efforts experimenting with these operators.

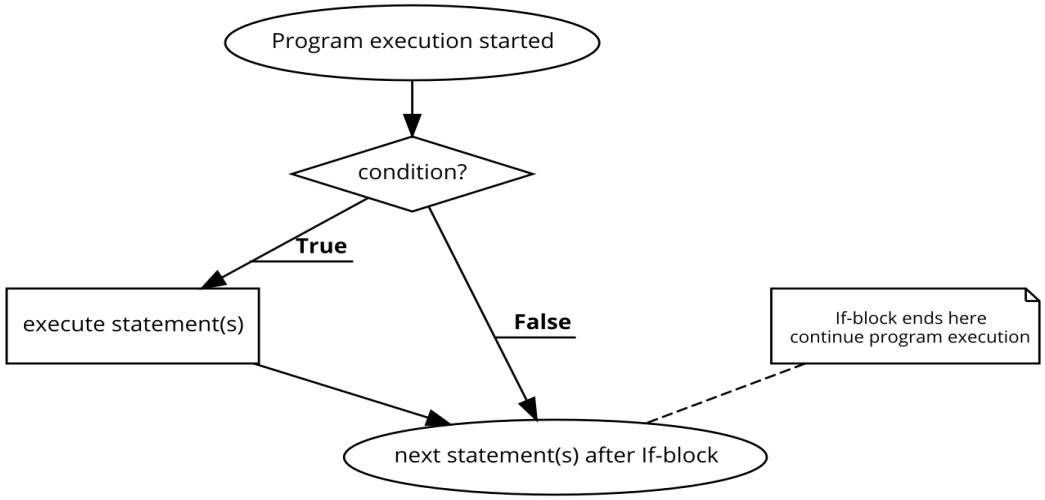
Decision Making

During the design of the program's logic compute the decision-making first. There are only two perfect answers in this world either **YES** or **NO**. In programming, these perfect answers represent the boolean data type. The boolean data type variable takes a single value at a time which can be **true** or **false**. So boolean value **true** represents **YES** and the boolean value **false** represents **NO**. While running the program, the control of the program may find logical expression. It then tests this logical expression to find the result. It expresses the result in terms of either **true** or **false**. Based on the boolean result **true or false** of the logical expression, the control of the program decides whether to execute or skip a bunch of programming statements. We can compute decision making using the following different versions of decision-making **if-statements**.

a) **If-statement**
b) **If-else statement**
c) **If-else if …statement (**Also known as **ladder if-else statement)**

Flow-chart helps to visualize the flow of control when condition becomes either **true** or **false**.

a) if-statement



The general syntax for **if-statement** can be given as:

```
If (condition) {
  execute statement(s);
}
```

`If-block`

```
next statement(s); // continue program execution
```

- **`condition:`** It will be an expression consisting of variables and operators in such a way so that whose outcome should be a boolean value either **true** or **false**. Since logical and comparison operators gives result in either **true** or **false** and commonly used in conditional expression.
- When the result of **`condition`** is **true**, it will execute statement(s) enclosed in the **If-block** {….}.

Note: - We place **if-block or else-block** in a program in order to group a set of statements by enclosing it in an **open curly brace** and **closing curly brace**.

An example:

```
if(logical_condition){
      statement 1;
      statement 2;
      ……...
      statement N;
 }
```

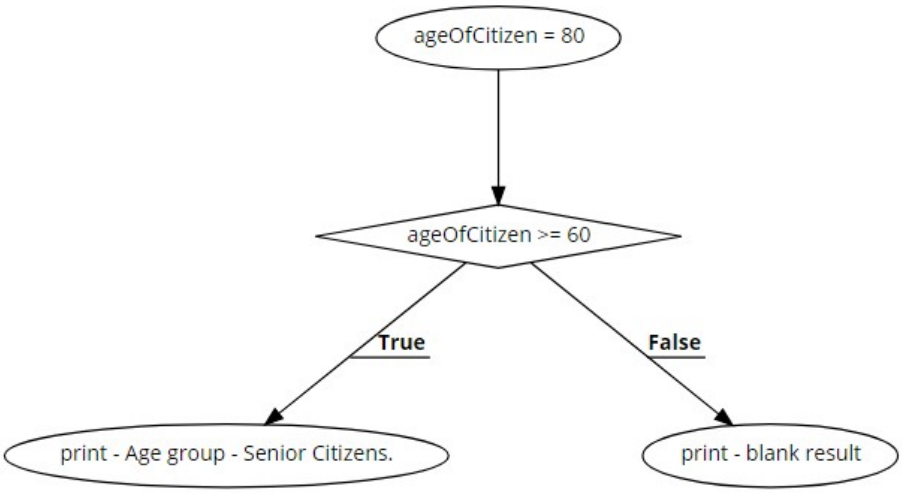
```
else {
      statement 1;
      statement 2;
      ……...
      statement N;
 }
```

Case 1.

`int ageOfCitizen = 80;`

condition: `( ageOfCitizen >= 60 )` will evaluate to **true**

So, `if (80 >= 60)` or if (**condition**) will be **true**, then it executes the if-block.



StatementIf.java

```
public class StatementIf {
      public static void main(String[] args) {
            int ageOfCitizen = 80;
            if( ageOfCitizen >= 60 ) {
                  System.out.println("Age group - Senior Citizens.");
            }
      }
}
```

Output

```
Age group - Senior Citizens.
```

Case 2. Change the value of ageOfCitizen in the program.

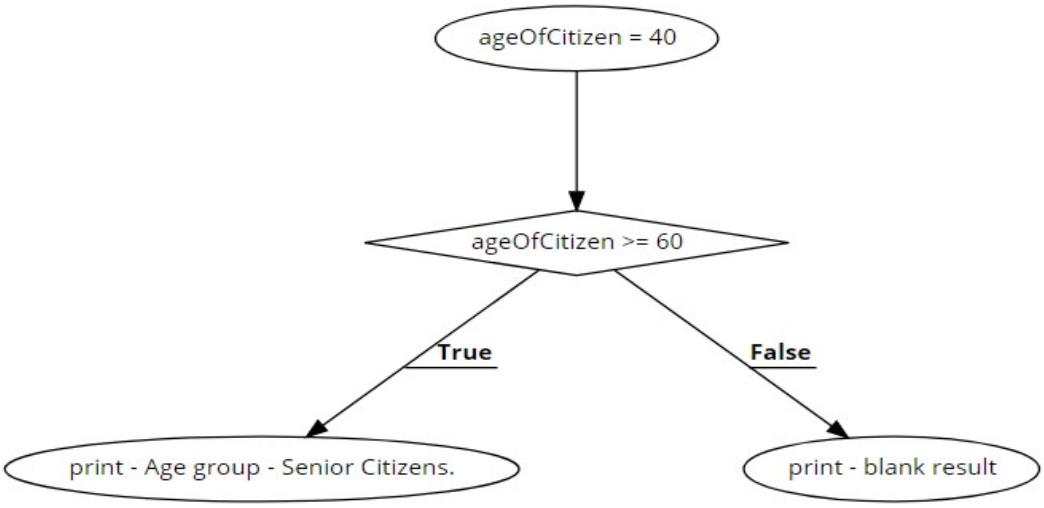
`int ageOfCitizen = 40;`

condition: `( ageOfCitizen >= 60 )`

`if(40 >= 60)`, will evaluate to **false.**

`if( ageOfCitizen >= 60 )` will be false and it will not execute the if-block.

The output will be blank.



StatementIf1.java

```
public class StatementIf1 {

	public static void main(String[] args) {

		int ageOfCitizen = 40;

		if( ageOfCitizen >= 60 ) {

			System.out.println("Age group - Senior Citizens.");

		}

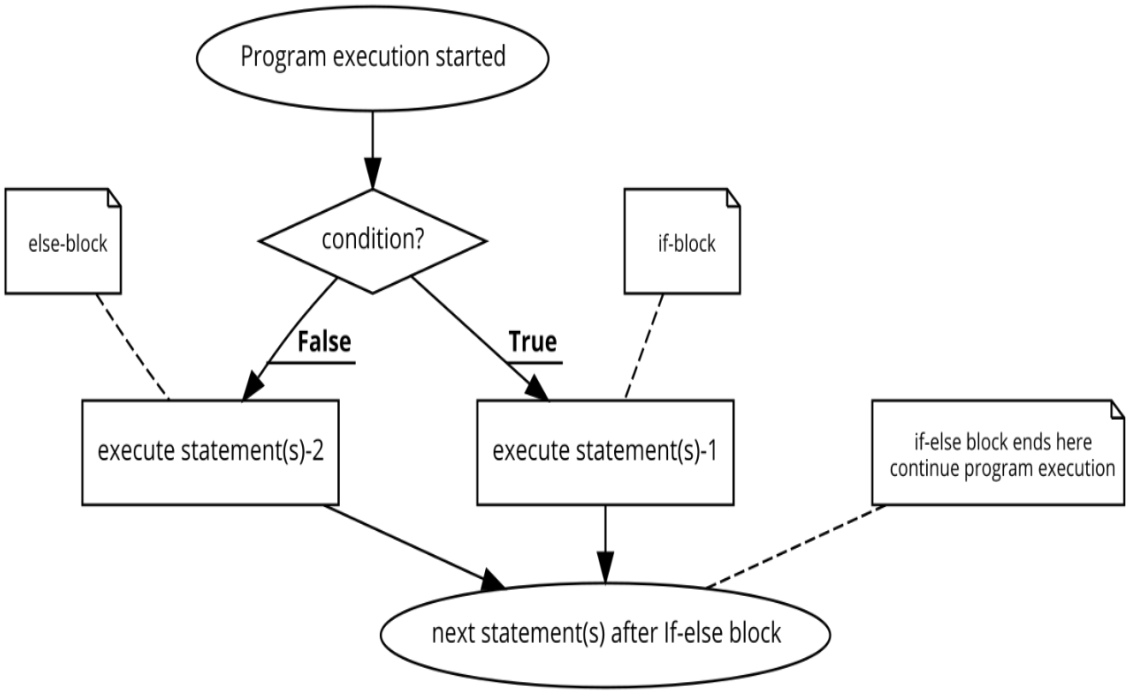
	}

}
```

Output

It didn't print anything, since condition becomes failed and code block within **if-statement** doesn't get executed.

b) if-else statement



The general syntax for **if-else statement** can be given as:

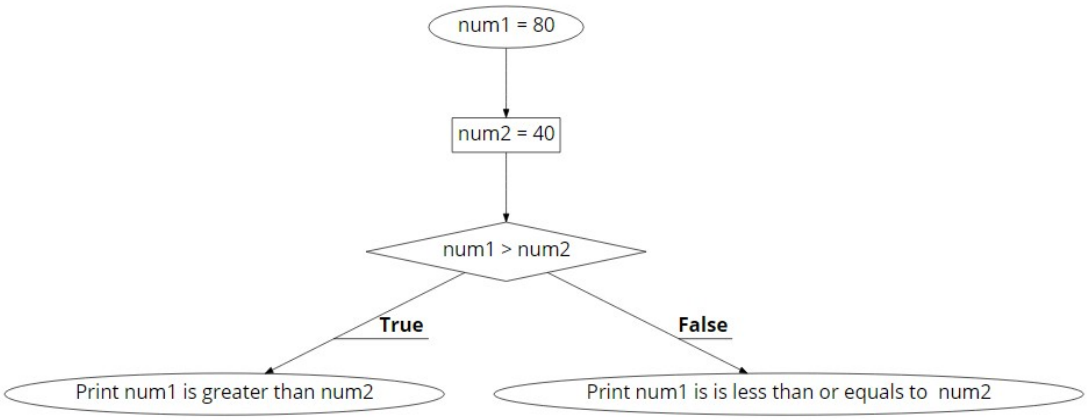
```
if (condition) {
 execute statement(s)-1;
} else {
 execute statement(s)-2;
}
next statement(s); // continue program execution
```

If-block

else-block

Case 1.

`int num1 = 80;`
`int num2 = 40;`
condition: `( num1 > num2)`
`if(80 > 40)`, will evaluate to **true.**
`if( num1 > num2)` conditional statement will be **true** and executes the if-block.



StatementIfElse.java

```
public class StatementIfElse {

  public static void main(String[] args) {

      int num1 = 80;

      int num2 = 40;

      if( num1 > num2) {

        System.out.println(num1 + " is greater than " + num2);

      }else {

        System.out.println(num1 + " is less than or equals to " + num2);

      }

  }

}
```

Output

```
80 is greater than 40
```

Case 2. Change the value of num1 and num2 in the program.

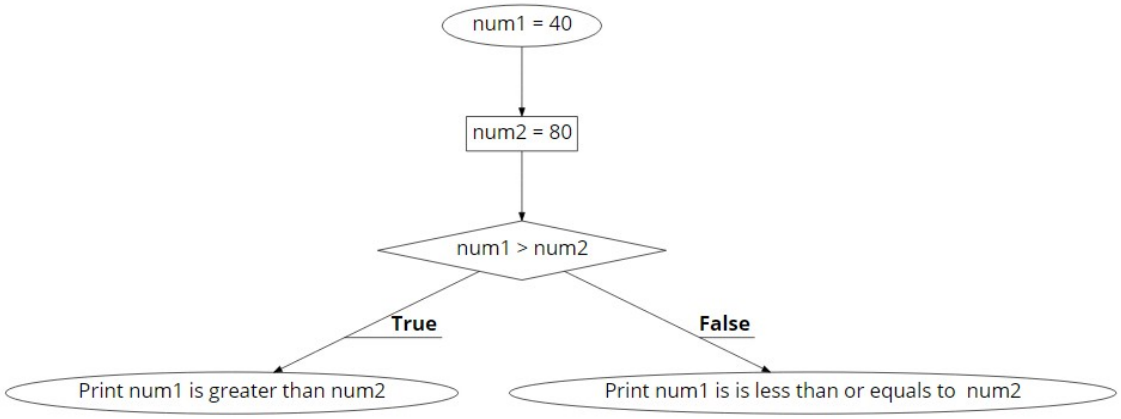
```
int num1 = 40;
int num2 = 80;
```

condition: (num1 > num2)

`if(40 > 80)`, will evaluate to **false**.

`if( num1 > num2)` conditional statement will be **false.**

Program should execute the **else code block**.



StatementIfElse.java

```
public class StatementIfElse {

  public static void main(String[] args) {

      int num1 = 40;

      int num2 = 80;

      if( num1 > num2) {

        System.out.println(num1 + " is greater than " + num2);

      }else {

        System.out.println(num1 + " is less than " + num2);

      }

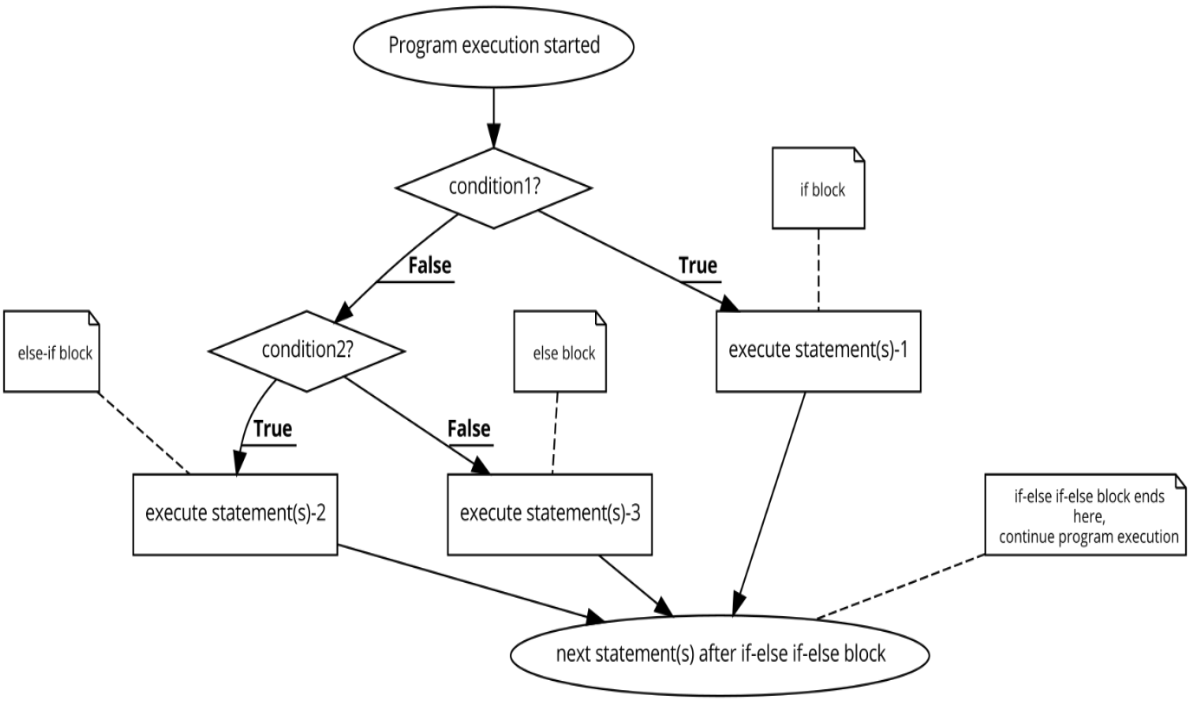
  }

}
```

Output

```
40 is less than 80
```

c) if-else if-else statement



The general syntax for **if-else if-else statement** can be given as:

```
if (condition1) {
  execute statement(s)-1;          → if-block
}else if(condition2) {
    execute statement(s)-2;        → else-if-block
}else{
    execute statement(s)-3;        → else-block
}
```

Case 1.

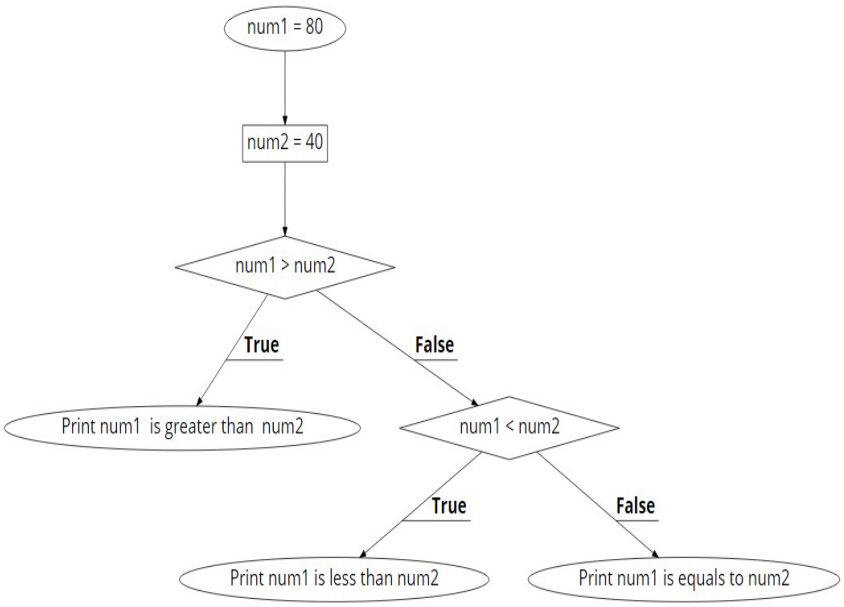
```
int num1 = 80;
int num2 = 40;
```

condition : `(num1 > num2)`

`if` (`80 > 40`), will evaluate to **true**

Hence,`if` (`num1 > num2`) conditional statement will be **true** .

Program should execute the **if code block**.



StatementIfElseIf.java

```
public class StatementIfElseIf {

	public static void main(String[] args) {

		int num1 = 80;

		int num2 = 40;

		if( num1 > num2 ) { // if(true)

			System.out.println(num1 + " is greater than " + num2);

		}else if( num1 < num2 ) {

			System.out.println(num1 + " is less than " + num2);

		}else {

			System.out.println(num1 + " equals to " + num2);

		}

	}

}
```

Output

```
80 is greater than 40
```

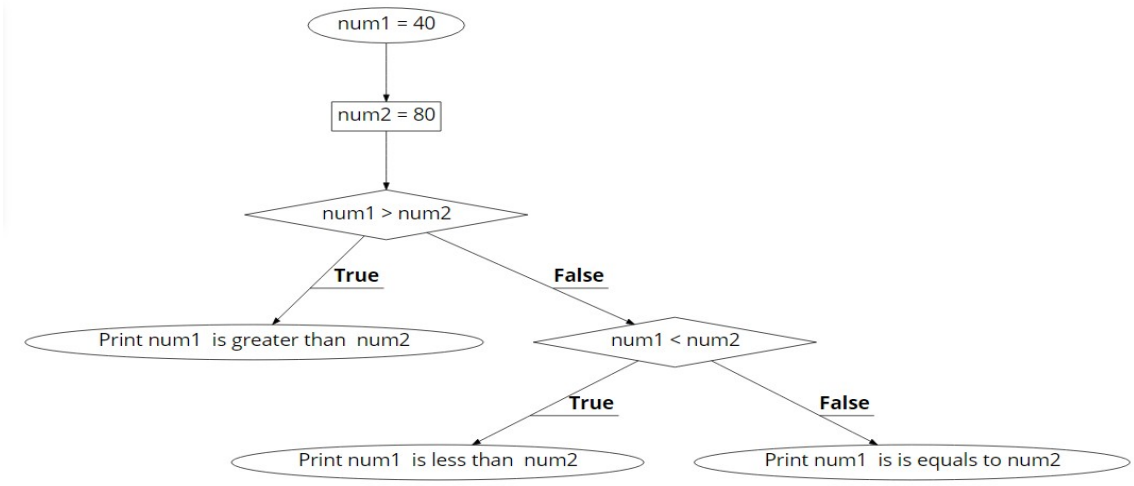
Case 2. Change the value of num1 and num2 in the program.

```
int num1 = 40;
int num2 = 80;
```

`if( num1 > num2 )` conditional statement will evaluate to **false** and program's control will check next if conditional statement, which is `else if( num1 < num2 )`.

`if( num1 < num2 )` conditional statement will be **true,** since (`40 < 80`) evaluates to **true**.

The program should execute the `else if` **code block**.



StatementIfElseIf.java

```
public class StatementIfElseIf {

	public static void main(String[] args) {

		int num1 = 40;

		int num2 = 80;

		if( num1 > num2 ) {// if(false)

			System.out.println(num1 + " is greater than " + num2);

		}else if( num1 < num2 ) {// if(true)

			System.out.println(num1 + " is less than " + num2);

		}else {

			System.out.println(num1 + " equals to " + num2);

		}

	}

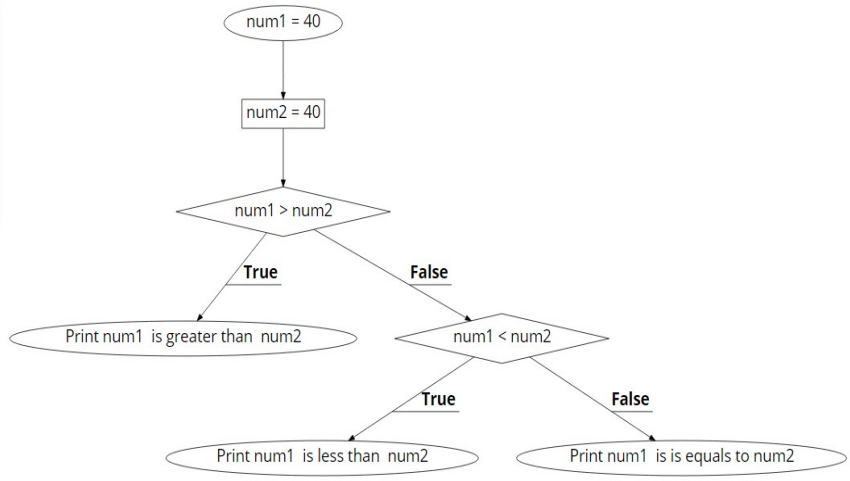
}
```

Output

```
40 is less than 80
```

Case 3. Change the value of num2 in the program.

```
int num1 = 40;
int num2 = 40;
```



The (num1 > num2) logical expression will test to **false, and** then the program's control will test the next logical expression (num1 < num2). The logical expression (num1 < num2) will test as **false** because (40 < 40) will test as false**.** So, else if (num1 < num2) will also **not get executed** and finally execute the last else code block. The else-block has no condition attached to it. The **last else code block** will get executed when all the conditions will become failed. It's a default block and at-least this block will get executed if none other block.

StatementIfElseIf.java

```
public class StatementIfElseIf {

	public static void main(String[] args) {

		int num1 = 40;

		int num2 = 40;

		if( num1 > num2 ) {// if(false)

			System.out.println(num1 + " is greater than " + num2);

		}else if( num1 < num2 ) {// if(false)

			System.out.println(num1 + " is less than " + num2);

		}else {// default

			System.out.println(num1 + " equals to " + num2);

		}

	}

}
```

Output

```
40 equals to 40
```

ProblemLadderIfElse: A college came up with an idea to encourage CS students by refunding some percentage of their bi-semester CS Course fees. College management laid the following conditions to be eligible for the same. Moreover, if a student scored in Grade-0, will be declared failed.

If a student scored in Grade-5, will be eligible for 35% refund of the college fees.

If a student scored in Grade-4, will be eligible for 25% refund of the college fees.

If a student scored in Grade-3, will be eligible for 20% refund of the college fees.

If a student score scored in Grade-2, will be eligible for 10% refund of the college fees.

If a student scored in Grade-1, will be passed with margin.

LadderIfElse.java

```
public class LadderIfElse {

	public static void main(String[] args) {

		int grade = 2;

		if(grade == 1) {
			System.out.println("35% fee refund.");
		}else if(grade == 2) {
			System.out.println("25% fee refund.");
		}else if(grade == 3) {
			System.out.println("20% fee refund.");
		}else if(grade == 4) {
			System.out.println("10% fee refund.");
		}else if(grade == 5) {
			System.out.println("Passed with margin.");
		}else if(grade == 0) {
			System.out.println("Failed,please retry.");
		}else  {
			System.out.println("Invalid grade ? " + grade);
		}
	}

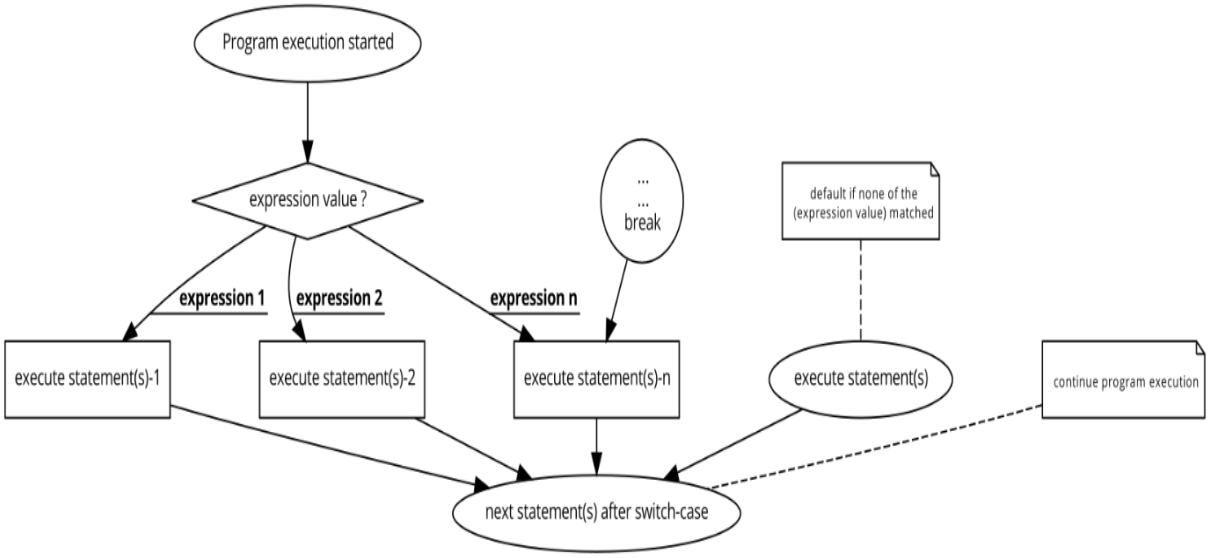
}
```

It's going to test **if-conditions** sequence-wise, unless and until any one of **if-condition** becomes **true**. This type of many else-if is also called a **ladder else-if**, which is basically not a good practice. Two or three **else-if conditions** considered to be okay and furthermore **else-if conditions** strongly discouraged as it reduces the code readability.

Then what is the way-out?

The answer is **switch-case** construct.

d) switch-case



The general syntax for **switch-case statement** can be given as:

```
switch(expression value ?){
  case expression 1:
            execute statement(s)-1;
    break;
  case expression 2:
            execute statement(s)-2;
      break;
   …
   …
   …
   …
   …
   …
  case expression n:
            execute statement(s)-n;
    break;
  default:
            // default if none of the case expression value matched
            execute statement(s);
  }// switch-case ends
next statement(s); // continue program execution
```

Let's convert ladder if-else statement to switch-case. The previous code `LadderIfElse.java` can be re-written as:

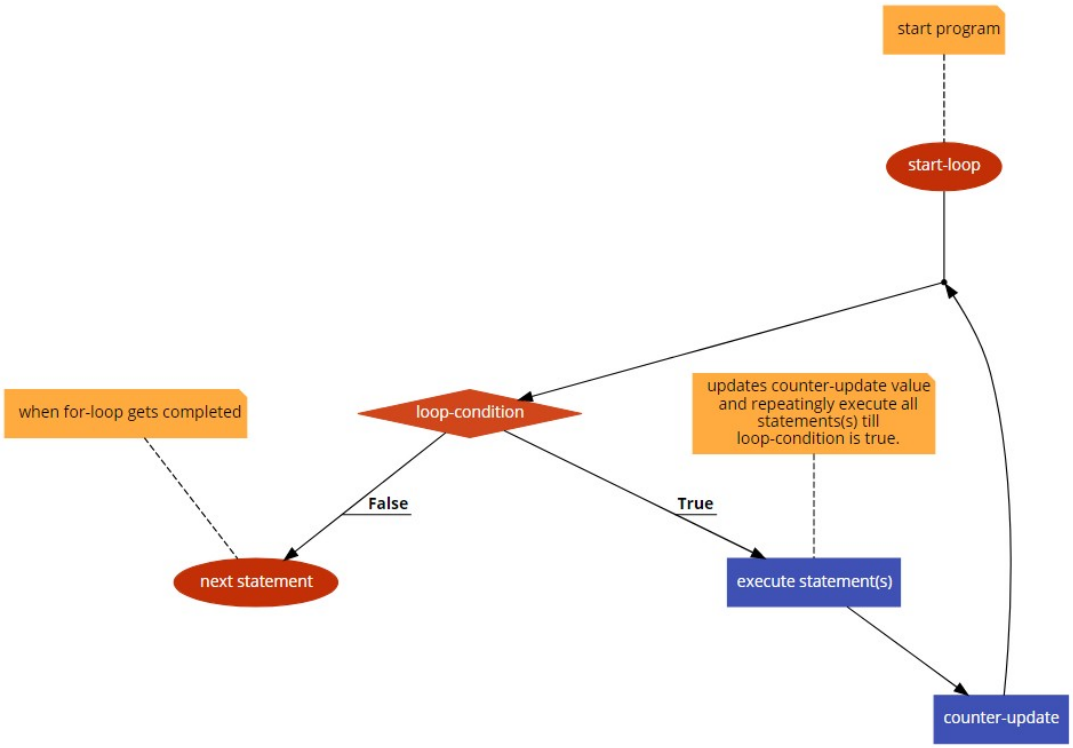
`ConvertLadderIfElseToSwitchCase.java`

```
public class ConvertLadderIfElseToSwitchCase {
	public static void main(String[] args) {
		int grade = 21;
		switch (grade) {
		case 1:
			System.out.println("35% fee refund.");
			break;
		case 2:
			System.out.println("25% fee refund.");
			break;
		case 3:
			System.out.println("20% fee refund.");
			break;
		case 4:
			System.out.println("10% fee refund.");
			break;
		case 5:
			System.out.println("Passed with margin.");
			break;
		case 0:
			System.out.println("Failed,please retry.");
			break;
		default:
			System.out.println("Invalid grade ? " + grade);
			break;
		}
	}
}
```

switch-case is the elegant way of dealing with **ladder if-conditions.**

About *for-loop*

In case of high calculation intensive data or large data processing activities, humans are not reliable in terms of accuracy and productivity as compared to computers. Nothing is automatic unless someone does it. The feature of repetitiveness helps to automate tedious data processing work amazingly efficient as compared to doing it manually. One of the types of repetitive control in programming is known as **for statement** or ***for-loop.*** The ***for-loop*** can be expressed as follows:



for (***start-loop***; ***loop-condition*** ; ***counter-update***) {

    execute statement(s);

}

next statement;

- ***start-loop*** is a simple way of initialization of a variable and anyway from some point the process of repetitive cycle has to be started.
- ***loop-condition*** is a logical expression which evaluates a boolean value either ***true*** or ***false***. Based on this (***true/false***), **for-loop** control structure decides whether to repeat the set of statement(s) or not.
- ***counter-update*** is a change in variable's value in a fixed smaller step (increment/decrement) only after all statement(s) gets executed in **for-loop** block. Usually it holds strong bonding with ***start-loop*** and ***loop-condition***.

The ***for-loop*** helps to repeat set of programming statement(s) over a **range of values**. When we talk about a range, then it must have a minimum value and a maximum value.

We will use the following notations:

min_val → minimum value from where the for-loop starts the repetition.

max_val → maximum value to the extent where for-loop will do the repetition.

It means ***for-loop*** will operate between *min_val* and *max_val* values. We can understand this concept better by doing examples based on incrementing or decrementing ***counter-update*** expression of the ***for-loop.***

Basics

Here are some basic examples to get clarity about how the ***for-loop*** behaves.

Let's express range [*min_val*, *max_val*] where *min_val* < *max_val* with counter-update incremental value 1 .The for-loop can be expressed as;

for (**int** *variable_name* = *min_val* ; *variable_name* <= *max_val* ; *variable_name* ++){

    statement(s);

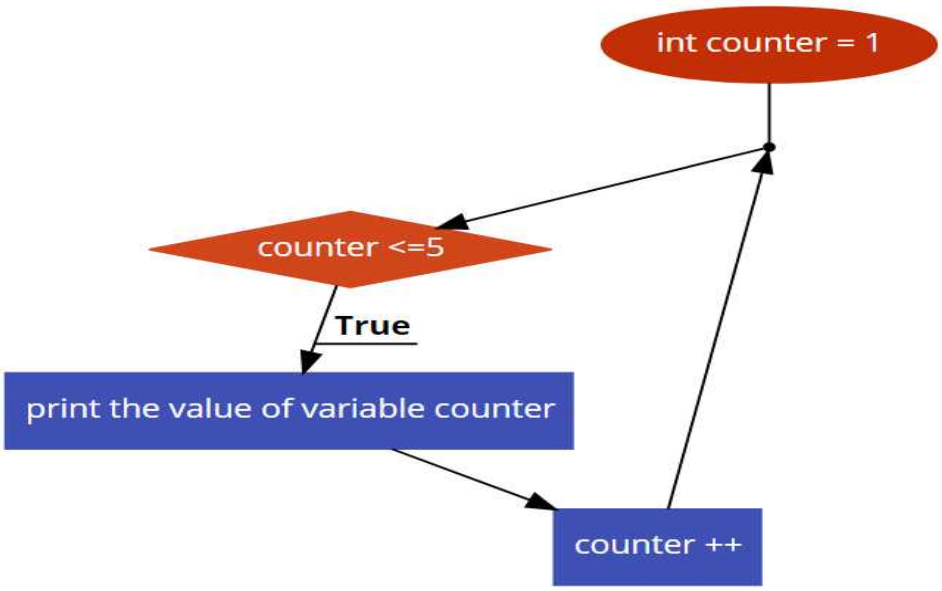
}

Case a:

Let the range be [1,5] with counter-update incremental value 1 ,then the for-loop can be expressed to print counter variable's value 1 to 5 in ascending order which is as follows;

```
for ( int counter = 1 ; counter <=5 ; counter ++ ){
      System.out.println(counter);//print output to console
}
```

Hence the above ***for-loop*** can be expressed as a flow chart which is as follows:



ForLoopIncrement.java

```
public class ForLoopIncrement {
	public static void main(String[] args) {
		for ( int counter = 1 ; counter <=5 ; counter ++ ){
			System.out.println(counter);//print to console
		}
	}
}
```

Output

```
1
2
3
4
5
```

Case b: Changing the ***loop-condition*** counter <= 5 to counter < 5, and let's see the result.

ForLoopIncrement1.java

```
public class ForLoopIncrement1 {
	public static void main(String[] args) {
		for ( int counter = 1 ; counter <5 ; counter ++ ){
			System.out.println(counter);//print to console
		}
	}
}
```

Output

```
1
2
3
4
```

It printed from 1 to 4.

Case c: Let's modify `ForLoopIncrement1.java` keeping **loop-condition(** `counter <5 )` same and yet still print 1 to 5.

`ForLoopIncrement2.java`

```
public class ForLoopIncrement2 {
	public static void main(String[] args) {
		for ( int counter = 0 ; counter <5 ; counter ++ ){
			int  result = counter + 1; //add 1 to the counter
			System.out.println( result);//print to console
		}
	}
}
```

Output

```
1
2
3
4
5
```

The variable counter started with the value 0 instead of 1.

Case d: Let's express range[min_val, max_val] where min_val < max_val with counter-update decremented value 1 .

The for-loop can be expressed as;

for (**int** *variable_name* = *max_val*; *variable_name* >= *min_val* ; *variable_name* --){

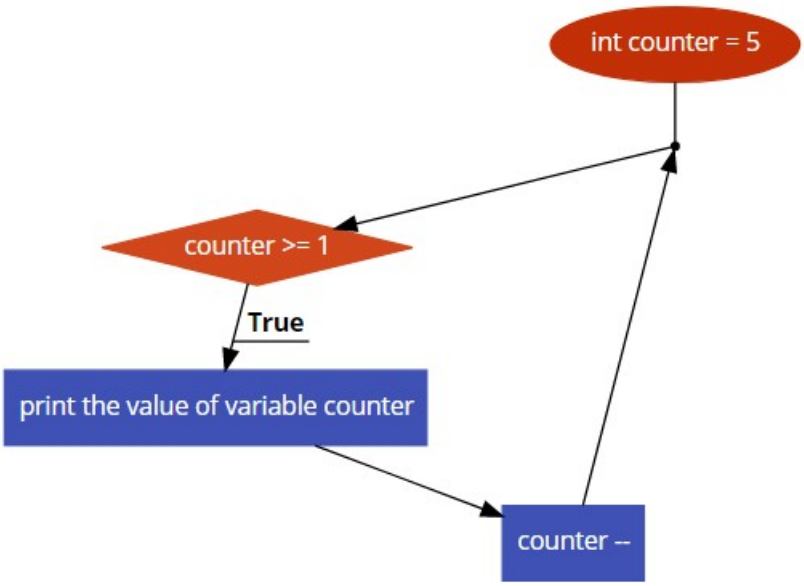
	statement(s);

}

To print numbers 1 to 5 in descending order with ***counter-update*** decremented value 1, then the ***for-loop*** can be expressed as;

```
	for ( int counter = 5 ; counter >= 1; counter -- ){
		System.out.println( counter ); //print to console
	}
```

Flow chart of ***for-loop*** with decrement counter:



ForLoopDecrement.java

```
public class ForLoopDecrement {

	public static void main(String[] args) {

		for ( int counter = 5 ; counter >= 1; counter -- ){

			System.out.println( counter ); //print to console

		}

	}

}
```

Output

```
5
4
3
2
1
```

For-loop with {break} and {continue} keywords

- Any logical condition that executes the keyword **break** within the for-loop, it stops further repetition of the loop. It's similar to a **moving car that stops** exactly whenever the driver applies the **break**.
- Any logical condition that executes the keyword **continue** within the for-loop, it continues with the next iteration without executing programming statement(s) that come after it within the for-loop's code block.

ForLoopContinueNext.java

```
public class ForLoopContinueNext {

	public static void main(String[] args) {

		for( int cnt = 1 ; cnt <= 10 ; cnt ++ ) {

			if( cnt <= 3) {

				System.out.print("keyword{continue}");

				System.out.println(" is throwing back.");

				continue;

			}

			System.out.println("Please Join Us...!!!");

			if( cnt == 5) {

				System.out.println("keyword{break} at cnt = " + cnt);

				break;

			}

			System.out.println("Please look back..." );

		}

	}

}
```

Output

```
keyword{continue} is throwing back.
keyword{continue} is throwing back.

keyword{continue} is throwing back.

Please Join Us...!!!

Please look back...

Please Join Us...!!!

keyword{break} at cnt = 5
```

Nested for-loop/Inner for-loop

We have previously seen examples of ***for-loop*** that have if-statements or any other programming statements. Instead of programming statement(s) within for-loop, if we place another ***for-loop*** within the ***for-loop***, then how things will be. For example;

```
for ( int counter = 5 ; counter >= 1; counter -- ){
      // print the variable counter's value
      System.out.println( counter );  //statement1
      System.out.println( 2*counter );//statement2
      System.out.println( 3*counter );//statement3
}
```

We know it will execute these considered statements in order. Firstly **statement1** will get executed, then **statement2** and lastly **statement3** will get executed. These statements will repeat five times when we execute the considered piece of the program. If we place another for-loop instead of any simple statement, the major style of work doesn't change. We need to think in a way that **the inner for-loop** itself is like some other single programming statement. Suppose the control of a program executes programming statements in a sequence and finds an **inner for-loop.** It will then fully execute the **inner for-loop** and then execute the next programming statement(s) if there are any available.

For example;

```
for ( int counter = 5 ; counter >= 1; counter -- ){
      // print the variable counter's value
      System.out.println( counter );  //statement1
      //The for-loop is a statement2
     for(int inner = 1 ; inner <= 5; inner ++){
           System.out.println( inner );

     }
      System.out.println( 3* counter );//statement3
}
```

Please treat **nested for-loop** as a **single** programming statement

As usual, **statement1** gets completed first. The inner for-loop will perform with 5 times repetitions, and it will finally execute **statements3**. These three statements will repeat five times, as suggested by the structure of the outer for-loop. A for-loop placed within the code block of another for-loop is known as **inner for-loop** or **nested for-loop**.

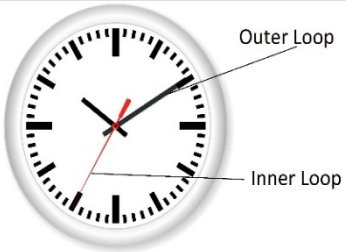
The inner for-loop will **restart from the beginning** for every new value of a outer for-loop.

It is easy to follow this concept if we try to relate it with time piece clock with following assumptions;

Inner Loop → It's like a "**seconds" hand** in a clock

Outer Loop → It's like a "**minute" hand** in a clock

In a clock when "**seconds**" hand completes one cycle then only afterwards "**minute**" hand will get updated by counter value 1 and not before.



The *inner* for-loop will execute completely before every new counter value of *outer* for-loop to continue repetition depends upon it's loop-condition.

NestedForLoop.java

```
public class NestedForLoop {

	public static void main(String[] args) {

		for( int out = 1 ; out <= 2 ; out++ ) {

			System.out.println("OUTER count No." + out);

			// nested for-loop: always re-starts and finishes
			// completely for every new value of outer for-loop.
			for( int in = 1; in <= 3; in++ ) {
				System.out.println(" <INNER> count No." + in );
			}
		}
	}
}
```

Output

```
OUTER count No.1
 <INNER> count No.1
 <INNER> count No.2
 <INNER> count No.3
OUTER count No.2
 <INNER> count No.1
 <INNER> count No.2
 <INNER> count No.3
```

Nested Loop: Beginner Programmer's Best Friend

A best friend is a person whenever required, who not only appreciates our wonderful qualities, but also gives us a reality check by showing our weaknesses or mistakes. The nested loop plays such a best friend's role while learning to program logic, and it also gives a reality check of how much true our programming logic is. This is my opinion that **practicing programming logic just by juggling with numbers is not a good idea.** This mostly held true for a person who had a less or no inclination in high school mathematics. Many of them feel bored practicing programming logic based on mathematical problems. Everybody is different, and everyone has a different buffer of patience and the pace of learning. There should be a simple approach to learning a program. Any beginner who wants to learn/improve programming should be able to do it with his own learning pace on the backdrop of minimal technical stuff. The approach is to use a three-step learning process, guided by pictorial exercises, to ensure beginners become comfortable with the core of programming.

Step 1. The clarity about how logical conditions can be explored.

Step 2. The clarity about how nested-loop works.

Step 3. Research alternatives to simplify programming logic.

After studying these three steps properly, almost anyone will feel confident exploring advanced features of the programming language.

Why nested loops are such an important deal?

In the outer for-loop and inner for-loop, each one is used for outputting data in different directions horizontally and vertically to give a 2-dimensional output. Usually outer for-loop moves the control vertically and inner for-loop outputs the data horizontally. This gives a grid kind of output. This particular feature of looping can be utilized to perfect the imagination of the learner. Yes, logic building is a skill and can be developed using simple math and common sense. It's not a must for a person to be very good at math, in order to be a successful programmer or software developer. If a learner is able to imagine the behavior of outer and nested loops correctly, the person will gain enough confidence that he/she can code. The first and foremost requirement for any subject-related teaching should be to create interest among the student or learner groups. If a learner gets deeply connected with the subject, then the rest of the learning will be without burden. This course is strictly meant to enjoy learning. If any learner takes this course and follows honestly, it will surely see the fruits of effort without a doubt.

Nested loop gives good drilling exercise to a new/novice/beginner/unconfident programmer.

Once a learner becomes interested in programming then all the learning will become thrilling and exciting. That interest will come naturally when learner gets some confidence in developing programming logic on its own. For that purpose only very few things needs to be practice well and that is covered in appropriate depth. I strongly suggest reading and practicing this book first entirely before seeking out any other resources.
This book helps you progress faster and it's kind of essential preworkout for programming.
I personally don't agree that those who give-up learning programming, that they are less capable but actually it doesn't excite them to put efforts. Whatever may be the learning pace of a person it doesn't matter but what matters is involvement in learning. True kind of involvement is not at all possible without interest. This book is all about proper installation of interest and confidence that any person can do programming by putting some efforts in the right direction.
The selection of exercises and problems are done for the main reason that learning should be naturally adaptable to any learner. The learner just need to follow step-by-step instructions mentioned in this book.

The confidence and interest in programming will be assured as you naturally learn and complete the training. This is not just a book, basically a training guide. Initial learning phase definitely requires some effort and patience. Once a learner understood the required fundamentals, the further improvement will be sky-rocketing due to immense love for a subject. It will come naturally during the practice. These sets of exercises are the result of experiments conducted in order to teach programming for the beginner's level and who had no or some programming experience.

After completion of these exercises everyone will be able to develop a good level of confidence within a span of 2-3 months of time. High point was that students became interested in programming. Programming has never remained boring or a burden for them. Learners will be able to naturally develop a tendency of patience in order to push himself/herself to solve the problems and be totally focused. In short, it makes the learning interesting.

ExerciseGB1.For every single value of outer for-loop in an iteration, inner for-loop executes completely.

The complete execution of inner for-loop is treated like an execution of a normal statement.

Unless and until inner for-loop get executed completely, the outer for-loop next iteration won't happen. The following program gives the demonstration of printing the value of columns vertically for every single row value.

ForLoopEveryRowAllColumns.java

```
public class ForLoopEveryRowAllColumns {
	public static void main(String[] args) {
		int TOTAL_ROWS = 3;
		int TOTAL_COLUMNS = 3;
		for( int row = 1 ; row <= TOTAL_ROWS ; row ++ ) {
			for( int col = 1 ; col <= TOTAL_COLUMNS ; col ++ ) {
			  System.out.println( "Row = " + row +"|" +" Col = " + col);
			}
		   System.out.println("--------------------------------");
		}
	}
}
```

Output

```
Row = 1| Col = 1
Row = 1| Col = 2
Row = 1| Col = 3
--------------------------------
Row = 2| Col = 1
Row = 2| Col = 2
Row = 2| Col = 3
--------------------------------
Row = 3| Col = 1
Row = 3| Col = 2
Row = 3| Col = 3
--------------------------------
```

ExerciseGB2. Printing column iteration values horizontally for every iteration value of row. The row value should come in every different row. So row level iteration values print vertically.

ForLoopEveryRowAllColumnsHorizontally.java

```
public class ForLoopEveryRowAllColumnsHorizontally {
	public static void main(String[] args) {
		int TOTAL_ROWS = 3;
		int TOTAL_COLUMNS = 3;

		for( int row = 1 ; row <= TOTAL_ROWS ; row ++ ) {
			System.out.print( "Row = " + row + " |" );
			for( int col = 1 ; col <= TOTAL_COLUMNS ; col ++ ) {
				System.out.print( " Col = " + col);
			}
			// moves the control to the next line and prints
			// next row iteration value
			System.out.println();
		}
	}
}
```

Output

```
Row = 1 | Col = 1 Col = 2 Col = 3
Row = 2 | Col = 1 Col = 2 Col = 3
Row = 3 | Col = 1 Col = 2 Col = 3
```

ExerciseGB3.Printing row iteration and column iteration values as (row, col) as shown in the following output

(1,1) (1,2) (1,3) (1,4) (1,5)

(2,1) (2,2) (2,3) (2,4) (2,5)

(3,1) (3,2) (3,3) (3,4) (3,5)

(4,1) (4,2) (4,3) (4,4) (4,5)

(5,1) (5,2) (5,3) (5,4) (5,5)

ForLoopCoordinateRepresentation.java

```
public class ForLoopCoordinateRepresentation {
	public static void main(String[] args) {
		int TOTAL_ROWS = 5;
		int TOTAL_COLUMNS = 5;

		for( int row = 1 ; row <= TOTAL_ROWS ; row ++ ) {
			for( int col = 1 ; col <= TOTAL_COLUMNS ; col ++ ) {
				System.out.print("(" + row + "," + col + ") ");
			}
			// moves the control to the next line and
			// prints next row iteration value
			System.out.println();
			System.out.println();
		}
	}
}
```

Output

```
(1,1) (1,2) (1,3) (1,4) (1,5)
(2,1) (2,2) (2,3) (2,4) (2,5)
(3,1) (3,2) (3,3) (3,4) (3,5)
(4,1) (4,2) (4,3) (4,4) (4,5)
(5,1) (5,2) (5,3) (5,4) (5,5)
```

ExerciseGB4.Generating pattern using character "*" by printing it for every (row, col) iteration values.

RowColumnStars.java

```
public class RowColumnStars {
	public static void main(String[] args) {
		int TOTAL_ROWS = 10;
		int TOTAL_COLUMNS = 10;
		for( int row = 1 ; row <= TOTAL_ROWS ; row ++ ) {
			for( int col = 1 ; col <= TOTAL_COLUMNS ; col ++ ) {
				System.out.print("* " );
			}
			// moves the control to the next line and
			// prints next row iteration value
			System.out.println();
		}
	}
}
```

Output

```
* * * * * * * * * *
* * * * * * * * * *
* * * * * * * * * *
* * * * * * * * * *
* * * * * * * * * *
* * * * * * * * * *
* * * * * * * * * *
* * * * * * * * * *
* * * * * * * * * *
* * * * * * * * * *
```

ProblemGB3: Write a program for the following output considering rows and columns are 5.

Output

```
*-*-*-*-*
*-*-*-*-*
*-*-*-*-*
*-*-*-*-*
*-*-*-*-*
```

***JustLoopIt*: Write a separate program for each of the given output.**

LoopingProblemA

```
1==1 1==2 1==3 1==4 1==5
2==1 2==2 2==3 2==4 2==5
3==1 3==2 3==3 3==4 3==5
4==1 4==2 4==3 4==4 4==5
5==1 5==2 5==3 5==4 5==5
```

LoopingProblemB

```
1!=1 1!=2 1!=3 1!=4 1!=5 1!=6 1!=7 1!=8
2!=1 2!=2 2!=3 2!=4 2!=5 2!=6 2!=7 2!=8
3!=1 3!=2 3!=3 3!=4 3!=5 3!=6 3!=7 3!=8
4!=1 4!=2 4!=3 4!=4 4!=5 4!=6 4!=7 4!=8
5!=1 5!=2 5!=3 5!=4 5!=5 5!=6 5!=7 5!=8
```

LoopingProblemC

```
1<=1 1<=2 1<=3 1<=4 1<=5
2<=1 2<=2 2<=3 2<=4 2<=5
3<=1 3<=2 3<=3 3<=4 3<=5
4<=1 4<=2 4<=3 4<=4 4<=5
5<=1 5<=2 5<=3 5<=4 5<=5
```

LoopingProblemD

```
3<1 3<2 3<3 3<4 3<5 3<6 3<7 3<8
4<1 4<2 4<3 4<4 4<5 4<6 4<7 4<8
5<1 5<2 5<3 5<4 5<5 5<6 5<7 5<8
6<1 6<2 6<3 6<4 6<5 6<6 6<7 6<8
7<1 7<2 7<3 7<4 7<5 7<6 7<7 7<8
8<1 8<2 8<3 8<4 8<5 8<6 8<7 8<8
```

LoopingProblemE

```
1>=1 1>=2 1>=3 1>=4
2>=1 2>=2 2>=3 2>=4
3>=1 3>=2 3>=3 3>=4
4>=1 4>=2 4>=3 4>=4
1>=1 1>=1 1>=1 1>=1
2>=2 2>=2 2>=2 2>=2
3>=3 3>=3 3>=3 3>=3
4>=4 4>=4 4>=4 4>=4
```

LoopingProblemF

```
1>=1 1>=2 1>=3 1>=4 1>=5 1>=6 1>=7 1>=8
2>=1 2>=2 2>=3 2>=4 2>=5 2>=6 2>=7 2>=8
3>=1 3>=2 3>=3 3>=4 3>=5 3>=6 3>=7 3>=8
4>=1 4>=2 4>=3 4>=4 4>=5 4>=6 4>=7 4>=8
5>=1 5>=2 5>=3 5>=4 5>=5 5>=6 5>=7 5>=8
6>=8 5>=7 5>=6 5>=5 5>=4 5>=3 5>=2 5>=1
7>=8 5>=7 5>=6 5>=5 5>=4 5>=3 5>=2 5>=1
8>=8 5>=7 5>=6 5>=5 5>=4 5>=3 5>=2 5>=1
```

LoopingProblemG

```
1!=1  1!=3  1!=5  1!=7
2!=1  2!=3  2!=5  2!=7
3!=1  3!=3  3!=5  3!=7
4!=1  4!=3  4!=5  4!=7
5!=1  5!=3  5!=5  5!=7
4!=1  4!=3  4!=5  4!=7
3!=1  3!=3  3!=5  3!=7
2!=1  2!=3  2!=5  2!=7
1!=1  1!=3  1!=5  1!=7
```

LoopingProblemH

```
2>=2  2>=4  2>=6  2>=8
3>=2  3>=4  3>=6  3>=8
2>=2  2>=4  2>=6  2>=8
 1>=2  1>=4  1>=6  1>=8
 2>=2  2>=4  2>=6  2>=8
 3>=2  3>=4  3>=6  3>=8
 4>=2  4>=4  4>=6  4>=8
 5>=2  5>=4  5>=6  5>=8
```

Art of Applying Logical Conditions

Logical conditions are the defense mechanism of working features of any software application. Most of the bugs/errors which occurred are because of the improper handling of logical condition(s). It mostly comprises **logical and comparison operators**. One of the most important element of programming logic is to apply logical conditions. It controls the flow of program similarly like how road traffic control system works. Road traffic control system logically distributes the crowd by conditionally allows or stops the portion of the crowd directing towards a particular road. Similarly, execution of logical conditions work like traffic control system that allow or disallow execution of chosen set of programming statements in a program. If roadside traffic control system doesn't work appropriately, then instead of guiding traffic it may create traffic jams and accidents. Similarly, wrong logical conditions give unpredictable results. We are going to learn how can we become fluent in applying logical conditions accurately.

The roadside sign boards are for a specific purpose. Similarly, with programming we have to understand the motive behind usage of each operator. We have to use operators in logical conditions based on their specific purpose which can precisely influences the output of a program. Same way we will try to understand how logical conditions influences the final output of the program. We will use our version of a simple scratch pad to verify our logical conditions manually first. It may remind you of cross-words puzzle box. Let us consider a similar box and to make point across we consider that it comprises 8 rows and 8 columns. For representing rows and columns we can consider two variables named as **row** and **col**, respectively.

Variable **row** → denotes values of row=1, 2,3,4,5,6,7,8.

Variable **col** → denotes values of col=1, 2,3,4,5,6,7,8.

Each one box recognized by (row, col) value pair. So value pair (1, 1) means 1st row and 1st column.

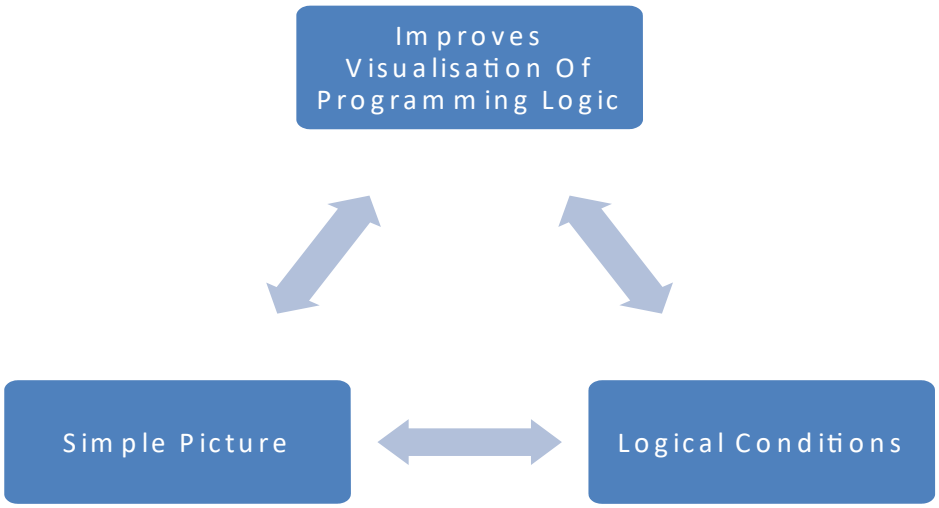
	col=1	col=2	col=3	col=4	col=5	col=6	col=7	col=8
row=1	1,1	1,2	1,3	1,4	1,5	1,6	1,7	1,8
row=2	2,1	2,2	2,3	2,4	2,5	2,6	2,7	2,8
row=3	3,1	3,2	3,3	3,4	3,5	3,6	3,7	3,8
row=4	4,1	4,2	4,3	4,4	4,5	4,6	4,7	4,8
row=5	5,1	5,2	5,3	5,4	5,5	5,6	5,7	5,8
row=6	6,1	6,2	6,3	6,4	6,5	6,6	6,7	6,8
row=7	7,1	7,2	7,3	7,4	7,5	7,6	7,7	7,8
row=8	8,1	8,2	8,3	8,4	8,5	8,6	8,7	8,8

Rule of the game: *Where ever logical condition matches, need to put character * in a small box.*

All (row, col) value pairs represent the total information or data. It has 8 rows and 8 columns which makes total 64 (row, col) value pairs. We will select correct (row, col) value pairs from the total available 64(row, col) value pairs into this box to map logical conditions. Applying different logical conditions means manually selecting (row, col) value pairs matching each logical condition. This is the, very simple observation based exercises that can help us understand how logical condition

works at the very core level. Overall, it will help to improve our logical thinking capabilities definitively not in unsure or guessing way.

To begin, we practice writing logical conditions manually. We try to confirm understanding of applying logical conditions on simple picture based exercises such as alphabets of very simple structure. Yes, of course we can verify all these logical conditions by writing programs also which will be our next level of learning. Important, each logical condition can be verified easily by writing program using **nested loops**. Working with nested loops will be our next major step in learning. We will try to transform our action of writing alphabets manually into a set of logical conditions. The countable experience we will get from such type of picture based exercises are that we can learn step by step how to apply logical conditions. The considered alphabetic pictures suggests selecting only particular set of (row, col) value pairs and leave the others. It will force us to think what could be the possible accurate logical condition. Since it's based on simple math and common sense, so verifying manually these logical conditions won't be difficult. When we do such exercises in order to transform from simple picture to logical conditions or do the reverse, it helps us to improve our visualization of programming logic.



Different alphabets mean different pictures. So in totality it helps us in knowing how certain program works to give a targeted result. Additionally, we can verify all these logical conditions by placing it exactly in a program and executing it to verify the results. The right time has not yet come for novice /absolute beginner programmers to verify it through writing programs.
For those who had already been experienced in any programming language to the extent that they know conditional statements and nested loops, surely they can go ahead and test their logical conditions. The first important lesson we should know how to give proper logical condition. We can clearly learn this important aspect of logic building by practicing different alphabets of simple structure into equivalent logical conditions.

Convert Logical Conditions to Simple Picture

We need to express logical conditions in the form of simple picture-based exercises. We need to mark those boxes with * character which rightly fits a logical condition. A unique combination of (row, col) value pair represents each box. To understand how the selection of boxes done using specific logical conditions is shown in the following exercises.

***ExerciseYB1*: Express Logical condition: (row == col) in the following 5X5 box.**

The condition **(row == col)** is asking to find those (row, column) pairs in the **5X5** box where row value and column value are equal. We notice that the values of row and column are equal on the diagonal squares.

Just to get attention easily, boxes are marked with thick borders where row and col values are equal.

	col=1	col=2	col=3	col=4	col=5
row=1	1,1	1,2	1,3	1,4	1,5
row=2	2,1	2,2	2,3	2,4	2,5
row=3	3,1	3,2	3,3	3,4	3,5
row=4	4,1	4,2	4,3	4,4	4,5
row=5	5,1	5,2	5,3	5,4	5,5

So as per rule of the game we should put * character on those boxes where row and col values are equal.

	col=1	col=2	col=3	col=4	col=5
row=1	1,1 *	1,2	1,3	1,4	1,5
row=2	2,1	2,2 *	2,3	2,4	2,5
row=3	3,1	3,2	3,3 *	3,4	3,5
row=4	4,1	4,2	4,3	4,4 *	4,5
row=5	5,1	5,2	5,3	5,4	5,5*

Removing other (row, col) value pairs where row and col values are not equal to indicate that considered logical condition doesn't select them. It simply means that only those boxes were chosen and marked with * character that matches the given logical condition. This way we can represent particular logical condition by filling small boxes with * character.

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*				
row=2		2,2*			
row=3			3,3*		
row=4				4,4*	
row=5					5,5*

So the meaning of logical condition now took the visual form. The expression of logical condition get transformed into the simple picture of an line drawn diagonally using * characters. This help us to understand how filtering/selection of data takes place when use logical condition.

	col=1	col=2	col=3	col=4	col=5
row=1	*				
row=2		*			
row=3			*		
row=4				*	
row=5					*

*

*

*

*

*

We turned an expression of logic condition into a simple picture of a diagonal line drawn using * characters. Such targeted attempt refines our skill of applying logical condition.

***ExerciseYB2*. Express Logical condition: (row == 5) || (col==1) in the following 5X5 box.**

Let's us take one part of logical condition **(row==5) ,** which means rows equal to 5.The boxes where value of **row** is 5 will be our selection of boxes irrespective of value of **col**. It can be brought into notice by highlighting border of the selected boxes.

Just to get attention easily, boxes are marked with thick borders where row equals to 5.

	col=1	col=2	col=3	col=4	col=5
row=1	1,1	1,2	1,3	1,4	1,5
row=2	2,1	2,2	2,3	2,4	2,5
row=3	3,1	3,2	3,3	3,4	3,5
row=4	4,1	4,2	4,3	4,4	4,5
row=5	5,1	5,2	5,3	5,4	5,5

So as per rule of the game we should put * character on those boxes where row equals to 5.

	col=1	col=2	col=3	col=4	col=5
row=1	1,1	1,2	1,3	1,4	1,5
row=2	2,1	2,2	2,3	2,4	2,5
row=3	3,1	3,2	3,3	3,4	3,5
row=4	4,1	4,2	4,3	4,4	4,5
row=5	*5,1	*5,2	*5,3	*5,4	*5,5

In the similar way, let's take the other part of logical condition which is **(col ==1).** Let's select the boxes where value of col is 1 irrespective of value of row and are given by highlighting border of the selected boxes.

Just to get attention easily, boxes are marked with thick borders where col equals to 1.

	col=1	col=2	col=3	col=4	col=5
row=1	1,1	1,2	1,3	1,4	1,5
row=2	2,1	2,2	2,3	2,4	2,5
row=3	3,1	3,2	3,3	3,4	3,5
row=4	4,1	4,2	4,3	4,4	4,5
row=5	*5,1	*5,2	*5,3	*5,4	*5,5

So as per rule of the game we should put * character on those boxes where col equals to 1.

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2	1,3	1,4	1,5
row=2	2,1*	2,2	2,3	2,4	2,5
row=3	3,1*	3,2	3,3	3,4	3,5
row=4	4,1*	4,2	4,3	4,4	4,5
row=5	*5,1	*5,2	*5,3	*5,4	*5,5

Removing (row, col) value pairs where character * is not present. The selected (row ,col) value pair with character * will be like the following.

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*				
row=2	2,1*				
row=3	3,1*				
row=4	4,1*				
row=5	*5,1	*5,2	*5,3	*5,4	*5,5

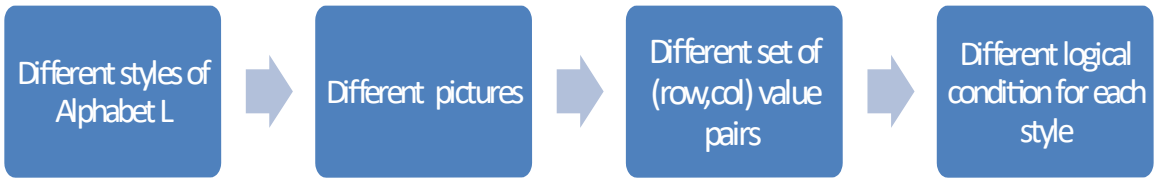
Removing selected (row ,col) value pair details. Removing the row and column details too. This looks like similar to English alphabet L.

	*				
	*				
	*				
	*				
	*	*	*	*	*

We can see few different styles of alphabet L.



What possible thoughts shall cross our mind if we try to fit each different style of alphabet L in an 8 rows and 8 column box using * character. Different styles of alphabet means different pictures and each will have different set of (row, col) value pairs.



It's obvious that we can't get pixel perfect picture by using *character. The most important point we need to understand is that **different pictured output mean different logical conditions**. These set of pictures are somewhat advanced to start with it in order to learn to apply logical conditions.

We can surely start with something so simple that even school going kids can't able to make any excuse that they can't learn programming. We will pick something which will be so easy and familiar to most of us. We will consider coding English alphabets of very simplest structure as shown in the following examples.

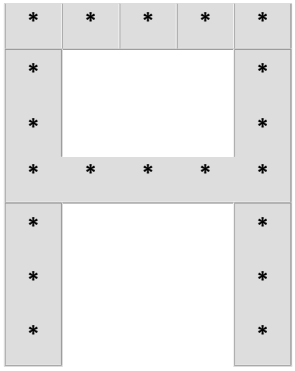
Alphabet A					Alphabet C						Alphabet D					
*	*	*	*	*			*	*	*	*		*	*	*	*	
*				*		*							*			*
*				*		*							*			*
*	*	*	*	*		*							*			*
*				*		*							*			*
*				*		*							*			*
*				*			*	*	*	*		*	*	*	*	

Let's recall our learning experiences with respect to the English alphabets from our school days. In order to learn how to write alphabets, we were taught by our teachers, parents, and others. They taught us in some logical way to write alphabets. The total logic applied to writing one alphabet differs from another. Since each alphabet's total structure is different, we have the opportunity to practice various kinds of logical conditions.

Convert Simple Picture to Logical Conditions

We will do some exercises by trying to get logical conditions from an English alphabet of very simple nature drawn using * character. We may consider 7 rows and 5 columns box. We have to select only those pair of (row, col) values which are helpful to make a required alphabet. We will be putting logical condition only for those (row, col) value pairs that are matching * characters.

***ExerciseYBA*: Alphabet A needs to be fit in 7X5 box.**



	col=1	col=2	col=3	col=4	col=5
row=1	1,1	1,2	1,3	1,4	1,5
row=2	2,1	2,2	2,3	2,4	2,5
row=3	3,1	3,2	3,3	3,4	3,5
row=4	4,1	4,2	4,3	4,4	4,5
row=5	5,1	5,2	5,3	5,4	5,5
row=6	6,1	6,2	6,3	6,4	6,5
row=7	7,1	7,2	7,3	7,4	7,5

In order to draw alphabet A let start it by putting * characters in the first row.

So logical condition will be **row==1** where == is equals comparator operator**.**

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2	2,3	2,4	2,5
row=3	3,1	3,2	3,3	3,4	3,5
row=4	4,1	4,2	4,3	4,4	4,5
row=5	5,1	5,2	5,3	5,4	5,5
row=6	6,1	6,2	6,3	6,4	6,5
row=7	7,1	7,2	7,3	7,4	7,5

We will use logical OR (||) conditional operator to add more data. In our case we need to add some more * characters to complete one more part of the alphabet A**.** Now we need to put * characters in the first column and the last column. So adding **(col==1)** and **(col==5)** logical conditions using **OR (||)** logical operator means more asterisk (*) characters are to be added in the following boxes**.**

The logical conditions updates are shown in the following

(row==1)||(col==1)

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1*	2,2	2,3	2,4	2,5
row=3	3,1*	3,2	3,3	3,4	3,5
row=4	4,1*	4,2	4,3	4,4	4,5
row=5	5,1*	5,2	5,3	5,4	5,5
row=6	6,1*	6,2	6,3	6,4	6,5
row=7	7,1*	7,2	7,3	7,4	7,5*

(row==1)||(col==1)||(col==5)

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1*	2,2	2,3	2,4	2,5*
row=3	3,1*	3,2	3,3	3,4	3,5*
row=4	4,1*	4,2	4,3	4,4	4,5*
row=5	5,1*	5,2	5,3	5,4	5,5*
row=6	6,1*	6,2	6,3	6,4	6,5*
row=7	7,1*	7,2	7,3	7,4	7,5*

Adding one more condition **(row == 4)** to the previous logical conditions to complete the middle part of the alphabet A**.** Now total * characters that displays alphabet A can be shown as follows with logical conditions **(row==1)||(col==1)||(col==5)||(row==4).**

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1*	2,2	2,3	2,4	2,5*
row=3	3,1*	3,2	3,3	3,4	3,5*
row=4	4,1*	4,2*	4,3*	4,4*	4,5*
row=5	5,1*	5,2	5,3	5,4	5,5*
row=6	6,1*	6,2	6,3	6,4	6,5*
row=7	7,1*	7,2	7,3	7,4	7,5*

Now removing the (row, col) value pairs which doesn't have * characters.

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1*				2,5*
row=3	3,1*				3,5*
row=4	4,1*	4,2*	4,3*	4,4*	4,5*
row=5	5,1*				5,5*
row=6	6,1*				6,5*
row=7	7,1*				7,5*

Alphabet A

*	*	*	*	*
*				*
*				*
*	*	*	*	*
*				*
*				*
*				*

Logical conditions

(row == 1) ||

(col == 1) ||

(col == 5) ||

(row == 4)

Let's verify following logical condition of alphabet A in 7 rows and 5 columns given as:
(row==1) || (col==1) ||(col==5)||(row==4)

VerifyAlphabetA.java

```
public class VerifyAlphabetA{

	public static void main(String[] args) {

		int row,col;
		int TOTAL_ROWS = 7;
		int TOTAL_COLUMNS = 5;
		boolean condition; //logical condition variable
		// A space characters are used to show the formatted output.
		String  printChar = "* ";
		String  blankSpaces = "  " ;// 2 space characters

		for( row = 1 ; row <= TOTAL_ROWS ; row ++ ) {
			for( col = 1 ; col <= TOTAL_COLUMNS ; col ++ ) {

				condition = ( row == 1 ) ||
					    ( col == 1 ) ||
					    ( col == 5 ) ||
					    ( row==4) ;

				if( condition ) {
					System.out.print(  printChar);
				}else {
					System.out.print( blankSpaces );
				}
			}
			// printing starts at the next line
			System.out.println();// Move  to the next row
		}

	}
}
```

Output

```
* * * * *
*       *
*       *
* * * * *
*       *
*       *
*       *
```

***ExerciseYBB*: Alphabet B**

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2	2,3	2,4	2,5
row=3	3,1	3,2	3,3	3,4	3,5
row=4	4,1	4,2	4,3	4,4	4,5
row=5	5,1	5,2	5,3	5,4	5,5
row=6	6,1	6,2	6,3	6,4	6,5
row=7	7,1*	7,2*	7,3*	7,4*	7,5*

In order to draw alphabet B let start it by putting * characters in the first row and the last row. The equivalent logical conditions will be using logical condition **row==1** and **row ==7.**

The logical condition becomes

(row==1)|| (row==7)

Now we need to put * characters in the second column and the last column. So adding (col==2) and (col==5) logical conditions using OR (||) logical operator means more * characters are to be added in the following boxes**.** The updated logical conditions can be shown as the following:

(row==1)|| (row==7)|| (col==2)||(col==5)

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2*	2,3	2,4	2,5*
row=3	3,1	3,2*	3,3	3,4	3,5*
row=4	4,1	4,2*	4,3	4,4	4,5*
row=5	5,1	5,2*	5,3	5,4	5,5*
row=6	6,1	6,2*	6,3	6,4	6,5*
row=7	7,1*	7,2*	7,3*	7,4*	7,5*

Adding one more condition **(row ==4).** One issue with this logical condition is that

we don't want * character in the (row == 4) especially in the box with value pair (4, 1).

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2*	2,3	2,4	2,5*
row=3	3,1	3,2*	3,3	3,4	3,5*
row=4	4,1*	4,2*	4,3*	4,4*	4,5*
row=5	5,1	5,2*	5,3	5,4	5,5*
row=6	6,1	6,2*	6,3	6,4	6,5*
row=7	7,1*	7,2*	7,3*	7,4*	7,5*

We apply logical AND (&&) conditional operator to select fewer data. In our case we need to select one * character less in the **(row == 4**) to complete the middle part of the alphabet B.

The updated logical condition for middle part of the alphabet B will be **(row ==4 && col>=2).**

Now the logical condition becomes;

(row==1)|| (row==7)|| (col==2)||(col==5)|| (row ==4 && col>=2)

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2*	2,3	2,4	2,5*
row=3	3,1	3,2*	3,3	3,4	3,5*
row=4	4,1	4,2*	4,3*	4,4*	4,5*
row=5	5,1	5,2*	5,3	5,4	5,5*
row=6	6,1	6,2*	6,3	6,4	6,5*
row=7	7,1*	7,2*	7,3*	7,4*	7,5*

Now removing the (row, col) value pairs which doesn't have * characters.

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2		2,2*			2,5*
row=3		3,2*			3,5*
row=4		4,2*	4,3*	4,4*	4,5*
row=5		5,2*			5,5*
row=6		6,2*			6,5*
row=7	7,1*	7,2*	7,3*	7,4*	7,5*

OR

*	*	*	*	*
	*			*
	*			*
	*	*	*	*
	*			*
	*			*
*	*	*	*	*

The complete logical condition becomes

(row==1)|| (row==7)|| (col==2)||(col==5)|| (row ==4 && col>=2)

Let's verify following logical condition of alphabet B in 7 rows and 5 columns given as:
(row==1)|| (row==7)|| (col==2)||(col==5)|| (row ==4 && col>=2)

VerifyAlphabetB.java

```
public class VerifyAlphabetB{

	public static void main(String[] args) {

		int row,col;
		int TOTAL_ROWS = 7;
		int TOTAL_COLUMNS = 5;
		boolean condition; //logical condition variable
		// A space characters are used to show the formatted output.
		String  printChar = "* ";
		String  blankSpaces = "  " ;// 2 space characters

		for( row = 1 ; row <= TOTAL_ROWS ; row ++ ) {
			for( col = 1 ; col <= TOTAL_COLUMNS ; col ++ ) {

				condition = ( row == 1 )||
					    ( row == 7 )||
					    ( col == 2 )||
					    ( col == 5 )||
					    ( row == 4 && col >= 2 );

				if( condition ) {
					System.out.print(  printChar);
				}else {
					System.out.print( blankSpaces );
				}
			}
			// printing starts at the next line
			System.out.println();// Move  to the next row
		}

	}
}
```

Output

```
* * * * *
  *     *
  *     *
  * * * *
  *     *
  *     *
* * * * *
```

***ExerciseYBC*: Alphabet C**

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2	2,3	2,4	2,5
row=3	3,1	3,2	3,3	3,4	3,5
row=4	4,1	4,2	4,3	4,4	4,5
row=5	5,1	5,2	5,3	5,4	5,5
row=6	6,1	6,2	6,3	6,4	6,5
row=7	7,1	7,2	7,3	7,4	7,5

On observing alphabet C and trying to fit into the box of size 7X5, we can start putting * characters in the first row. Now we need to put * characters in the first row. So logical condition will become **row==1**.

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1*	2,2	2,3	2,4	2,5
row=3	3,1*	3,2	3,3	3,4	3,5
row=4	4,1*	4,2	4,3	4,4	4,5
row=5	5,1*	5,2	5,3	5,4	5,5
row=6	6,1*	6,2	6,3	6,4	6,5
row=7	7,1*	7,2	7,3	7,4	7,5

The equivalent logical condition for the first column will be **(col==1)** condition.
Using **logical OR (||) conditional operator** to combine logical conditions and it becomes
(row==1)||(col==1)

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1*	2,2	2,3	2,4	2,5
row=3	3,1*	3,2	3,3	3,4	3,5
row=4	4,1*	4,2	4,3	4,4	4,5
row=5	5,1*	5,2	5,3	5,4	5,5
row=6	6,1*	6,2	6,3	6,4	6,5
row=7	7,1*	*7,2	*7,3	*7,4	*7,5

Adding one more logical condition **(row == 7)** to the previous. Now the * character set resembling the alphabet C and complete logical conditions can be:
(row==1)||(col==1)||(row==7)

Now removing the (row, col) value pair which doesn't have * character. Hence , the Logical condition: (row==1) ||(col==1)||(row==7)

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1*				
row=3	3,1*				
row=4	4,1*				
row=5	5,1*				
row=6	6,1*				
row=7	7,1*	7,2*	7,3*	7,4*	7,5*

OR

```
* * * * *
*
*
*
*
*
* * * * *
```

Let's verify following logical condition of alphabet C in 7 rows and 5 columns given as:
(row==1) ||(col==1)||(row==7)

VerifyAlphabetC.java

```
public class VerifyAlphabetC{
	public static void main(String[] args) {
		int row,col;
		int TOTAL_ROWS = 7;
		int TOTAL_COLUMNS = 5;
		boolean condition; //logical condition variable
		// A space characters are used to show the formatted output.
		String  printChar = "* ";
		String  blankSpaces = "  " ;// 2 space characters

		for( row = 1 ; row <= TOTAL_ROWS ; row ++ ) {
			for( col = 1 ; col <= TOTAL_COLUMNS ; col ++ ) {

				condition = (row==1) ||(col==1)||(row==7);

				if( condition ) {
					System.out.print(  printChar);
				}else {
					System.out.print( blankSpaces );
				}
			}
			// printing starts at the next line
			System.out.println();// Move  to the next row
		}

	}
}
```

Output

```
* * * * *
*
*
*
*
*
* * * * *
```

***ExerciseYBD*: Alphabet D**

We need to put * characters in the first row and the last row. So matching logical conditions will be (row==1) and (row==7)**.**

*	*	*	*	*
	*			*
	*			*
	*			*
	*			*
	*			*
*	*	*	*	*

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2	2,3	2,4	2,5
row=3	3,1	3,2	3,3	3,4	3,5
row=4	4,1	4,2	4,3	4,4	4,5
row=5	5,1	5,2	5,3	5,4	5,5
row=6	6,1	6,2	6,3	6,4	6,5
row=7	7,1*	7,2*	7,3*	7,4*	7,5*

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2*	2,3	2,4	2,5*
row=3	3,1	3,2*	3,3	3,4	3,5*
row=4	4,1	4,2*	4,3	4,4	4,5*
row=5	5,1	5,2*	5,3	5,4	5,5*
row=6	6,1	6,2*	6,3	6,4	6,5*
row=7	7,1*	7,2*	7,3*	7,4*	7,5*

Now by putting * characters in the second column and the last column will complete the alphabet D.
The logical conditions columns will be **(col==2) and (col==5).**
Total logical conditions becomes;
(row==1) || (row==7)|| (col==2)||(col==5)

Removing all the (row, col) value pair that doesn't have * characters.

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2*	2,3	2,4	2,5*
row=3	3,1	3,2*	3,3	3,4	3,5*
row=4	4,1	4,2*	4,3	4,4	4,5*
row=5	5,1	5,2*	5,3	5,4	5,5*
row=6	6,1	6,2*	6,3	6,4	6,5*
row=7	7,1*	7,2*	7,3*	7,4*	7,5*

OR

*	*	*	*	*
	*			*
	*			*
	*			*
	*			*
	*			*
*	*	*	*	*

Let's verify following logical condition of alphabet D in 7 rows and 5 columns given as:
(row==1) || (row==7)|| (col==2)||(col==5)

VerifyAlphabetD.java

```
public class VerifyAlphabetD{

	public static void main(String[] args) {

		int row,col;
		int TOTAL_ROWS = 7;
		int TOTAL_COLUMNS = 5;
		boolean condition; //logical condition variable
		// A space characters are used to show the formatted output.
		String  printChar = "* ";
		String  blankSpaces = "  " ;// 2 space characters

		for( row = 1 ; row <= TOTAL_ROWS ; row ++ ) {
			for( col = 1 ; col <= TOTAL_COLUMNS ; col ++ ) {

				condition = ( row == 1 ) ||
					    ( row == 7 ) ||
					    ( col == 2 ) ||
					    ( col == 5 ) ;

				if( condition ) {
					System.out.print(  printChar);
				}else {
					System.out.print( blankSpaces );
				}
			}
			// printing starts at the next line
			System.out.println();// Move  to the next row
		}


	}

}
```

Output

```
* * * * *
  *     *
  *     *
  *     *
  *     *
  *     *
* * * * *
```

***ExerciseYBE*: Alphabet E**

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2	2,3	2,4	2,5
row=3	3,1	3,2	3,3	3,4	3,5
row=4	4,1	4,2	4,3	4,4	4,5
row=5	5,1	5,2	5,3	5,4	5,5
row=6	6,1	6,2	6,3	6,4	6,5
row=7	7,1*	7,2*	7,3*	7,4*	7,5*

We need to put * characters in the first row and the last row.
So matching logical conditions will be **(row==1) and (row==7).**

Also by putting * characters in the second column will complete the alphabet E.

On adding logical condition (col==2), the total logical conditions becomes

(row==1)|| (row==7)|| (col==2)

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2*	2,3	2,4	2,5
row=3	3,1	3,2*	3,3	3,4	3,5
row=4	4,1	4,2*	4,3	4,4	4,5
row=5	5,1	5,2*	5,3	5,4	5,5
row=6	6,1	6,2*	6,3	6,4	6,5
row=7	7,1*	7,2*	7,3*	7,4*	7,5*

Adding one more condition

(row==4 && col>=2) to the previous conditions.

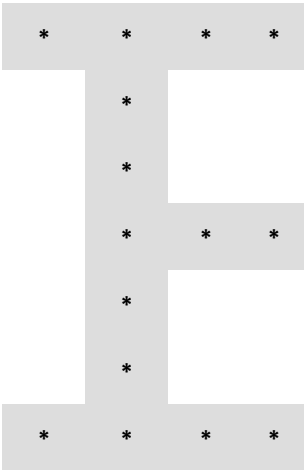
The AND logical condition **(&& col>=2)** is because we don't want to put * character in the box with value pair **(4, 1).**

The AND logical condition is applied to provide restriction in selecting data and our data are **(row, col)** value pairs, the boxes in general.

The total logical conditions becomes **(row==1)|| (row==7)|| (col==2)|| (row ==4 && col>=2)**

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2*	2,3	2,4	2,5
row=3	3,1	3,2*	3,3	3,4	3,5
row=4	4,1	4,2*	4,3*	4,4*	4,5*
row=5	5,1	5,2*	5,3	5,4	5,5
row=6	6,1	6,2*	6,3	6,4	6,5
row=7	7,1*	7,2*	7,3*	7,4*	7,5*

OR



Let's verify following logical condition of alphabet E in 7 rows and 5 columns given as:
(row==1)|| (row==7)|| (col==2)|| (row ==4 && col>=2)

VerifyAlphabetE.java

```
public class VerifyAlphabetE{


	public static void main(String[] args) {

		int row,col;
		int TOTAL_ROWS = 7;
		int TOTAL_COLUMNS = 5;
		boolean condition; //logical condition variable
		// A space characters are used to show the formatted output.
		String  printChar = "* ";
		String  blankSpaces = "  " ;// 2 space characters

		for( row = 1 ; row <= TOTAL_ROWS ; row ++ ) {

			for( col = 1 ; col <= TOTAL_COLUMNS ; col ++ ) {

				condition = ( row == 1 )||
					    ( row == 7 )||
					    ( col == 2 )||
					    ( row == 4 && col >= 2 );

				if( condition ) {
					System.out.print(  printChar);
				}else {
					System.out.print( blankSpaces );
				}
			}
			// printing starts at the next line
			System.out.println();// Move  to the next row
		}
	}
}
```

Output

```
* * * * *
  *
  *
  * * * *
  *
  *
* * * * *
```

***ExerciseYBF*: Alphabet F**

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2*	2,3	2,4	2,5
row=3	3,1	3,2*	3,3	3,4	3,5
row=4	4,1	4,2*	4,3	4,4	4,5
row=5	5,1	5,2*	5,3	5,4	5,5
row=6	6,1	6,2*	6,3	6,4	6,5
row=7	7,1	7,2*	7,3	7,4	7,5

We need to put * characters in the first row and the second column.

So matching logical conditions will be **(row==1) and (col==2).** The logical conditions for now becomes;

(row==1)|| (col==2).

Adding one more condition **(row ==4 && col>=2)** to the previous conditions. The AND logical condition **(&& col>=2)** is because we don't want to put * character in the box with **value pair (4, 1).**

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2*	2,3	2,4	2,5
row=3	3,1	3,2*	3,3	3,4	3,5
row=4	4,1	4,2*	4,3*	4,4*	4,5*
row=5	5,1	5,2*	5,3	5,4	5,5
row=6	6,1	6,2*	6,3	6,4	6,5
row=7	7,1	7,2*	7,3	7,4	7,5

Now the collection of * characters that makes alphabet F can be given by the logical conditions which are as follows:

(row==1)|| (col==2)||
(row ==4 && col>=2).

Removing all the (row, col) value pair that doesn't have * characters.

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2		2,2*			
row=3		3,2*			
row=4		4,2*	4,3*	4,4*	4,5*
row=5		5,2*			
row=6		6,2*			
row=7		7,2*			

OR

*	*	*	*	*
	*			
	*			
	*	*	*	*
	*			
	*			
	*			

Let's verify following logical condition of alphabet F in 7 rows and 5 columns given as:
(row==1)|| (col==2)|| (row ==4 && col>=2)

VerifyAlphabetF.java

```
public class VerifyAlphabetF{

	public static void main(String[] args) {

		int row,col;
		int TOTAL_ROWS = 7;
		int TOTAL_COLUMNS = 5;
		boolean condition; //logical condition variable
		// A space characters are used to show the formatted output.
		String  printChar = "* ";
		String  blankSpaces = "  " ;// 2 space characters

		for( row = 1 ; row <= TOTAL_ROWS ; row ++ ) {
			for( col = 1 ; col <= TOTAL_COLUMNS ; col ++ ) {

				condition = ( row == 1 )||
					    ( col == 2 )||
					    ( row == 4 && col >= 2 );


				if( condition ) {
					System.out.print(  printChar);
				}else {
					System.out.print( blankSpaces );
				}
			}
			// printing starts at the next line
			System.out.println();// Move  to the next row
		}

	}

}
```

Output

```
* * * * *
  *
  *
  * * * *
  *
  *
  *
```

***ExerciseYBG*: Alphabet G**

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2	2,3	2,4	2,5
row=3	3,1	3,2	3,3	3,4	3,5
row=4	4,1	4,2	4,3	4,4	4,5
row=5	5,1	5,2	5,3	5,4	5,5
row=6	6,1	6,2	6,3	6,4	6,5
row=7	7,1*	7,2*	7,3*	7,4*	7,5*

We need to put * characters in the first row and the last row. So matching logical conditions will be **(row==1)** and **(row==7**).

Also we need to put * characters in the first column. So the matching logical condition will be **(col==1).**

The logical conditions become
(row==1)|| (row==7)|| (col==1)

Adding one more condition **(row == 4 && col>=3)** to the previous condition to exclude * character in the box with value pair **(4, 2)**. We don't want * character in the box with value pairs **(2, 5) and (3, 5).** Once again, adding one more condition **(col == 5 && row >=5)** to the previous condition.

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1*	2,2	2,3	2,4	2,5
row=3	3,1*	3,2	3,3	3,4	3,5
row=4	4,1*	4,2	4,3	4,4	4,5
row=5	5,1*	5,2	5,3	5,4	5,5
row=6	6,1*	6,2	6,3	6,4	6,5
row=7	7,1*	7,2*	7,3*	7,4*	7,5*

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1*	2,2	2,3	2,4	2,5
row=3	3,1*	3,2	3,3	3,4	3,5
row=4	4,1*	4,2	4,3*	4,4*	4,5*
row=5	5,1*	5,2	5,3	5,4	5,5*
row=6	6,1*	6,2	6,3	6,4	6,5*
row=7	7,1*	7,2*	7,3*	7,4*	7,5*

Now the complete collection of * characters along with logical conditions can be shown as follows **(row==1) || (row==7)|| (col==1)|| (row ==4 && col>=3) || (col==5 && row>=5).** Removing all the (row, col) value pair that doesn't have * characters.

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1*				
row=3	3,1*				
row=4	4,1*		4,3*	4,4*	4,5*
row=5	5,1*				5,5*
row=6	6,1*				6,5*
row=7	7,1*	7,2*	7,3*	7,4*	7,5*

OR

*	*	*	*	*
*				
*				
*		*	*	*
*				*
*				*
*	*	*	*	*

Let's verify following logical condition of alphabet G in 7 rows and 5 columns given as:
(row==1) || (row==7)|| (col==1)|| (row ==4 && col>=3) || (col==5 && row>=5)

VerifyAlphabetG.java

```
public class VerifyAlphabetG{


	public static void main(String[] args) {

		int row,col;
		int TOTAL_ROWS = 7;
		int TOTAL_COLUMNS = 5;
		boolean condition; //logical condition variable
		// A space characters are used to show the formatted output.
		String  printChar = "* ";
		String  blankSpaces = "  " ;// 2 space characters

		for( row = 1 ; row <= TOTAL_ROWS ; row ++ ) {
			for( col = 1 ; col <= TOTAL_COLUMNS ; col ++ ) {

				condition = ( row == 1 )||
					    ( row == 7 )||
					    ( col == 1 )||
					    ( row == 4 && col >=3 ) ||
					    ( col == 5 && row >=5 ) ;

				if( condition ) {
					System.out.print(  printChar);
				}else {
					System.out.print( blankSpaces );
				}
			}
			// printing starts at the next line
			System.out.println();// Move  to the next row
		}

	}
}
```

Output

```
* * * * *
*
*
*   * * *
*       *
*       *
* * * * *
```

***ExerciseYBJ*: Alphabet J**

Let fill * characters in the first row & in the middle column. So matching logical conditions will be **(row==1) || (col==3)**.We have used logical OR (||) conditional operator to add more * characters.

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2	2,3*	2,4	2,5
row=3	3,1	3,2	3,3*	3,4	3,5
row=4	4,1	4,2	4,3*	4,4	4,5
row=5	5,1	5,2	5,3*	5,4	5,5
row=6	6,1	6,2	6,3*	6,4	6,5
row=7	7,1	7,2	7,3*	7,4	7,5

Since we have to restrict column count to 3 at row=7, so we have to use conditional operator **logical AND (&& col <= 3)**. Adding logical condition **(row==7 && col<=3)** to previous conditions using logical OR (||) conditional operator will give * character set can be shown in the box. The logical condition becomes;

(row==1) || (col==3) ||
(row==7 && col<=3)

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2	2,3*	2,4	2,5
row=3	3,1	3,2	3,3*	3,4	3,5
row=4	4,1	4,2	4,3*	4,4	4,5
row=5	5,1	5,2	5,3*	5,4	5,5
row=6	6,1	6,2	6,3*	6,4	6,5
row=7	7,1*	7,2*	7,3*	7,4	7,5

Please Pay Attention

Using logical AND we have restricted or limited the placing of * character at row=7.The condition used is **(row==7 && col<=3)**.

If condition were row==7 only, then the complete row=7 would have then got filled with * characters. The additional condition **&&col<=3** in the expression **(row==7 && col<=3)** actually limited the * characters in the row=7, started from column 1 till column 3.

Again, adding condition to previous **(col==1 && row>=4).** The selected * character set shown with thick border is representing condition **(col==1 && row>=4)**.

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2	2,1	2,2	2,3*	2,4	2,5
row=3	3,1	3,2	3,3*	3,4	3,5
row=4	4,1*	4,2	4,3*	4,4	4,5
row=5	5,1*	5,2	5,3*	5,4	5,5
row=6	6,1*	6,2	6,3*	6,4	6,5
	7,1*	7,2*	7,3*	7,4	7,5

Please Pay Attention

Using logical AND we have restricted or limited the placing of * character at col=1.The condition is **(col==1 && row>=4)**.

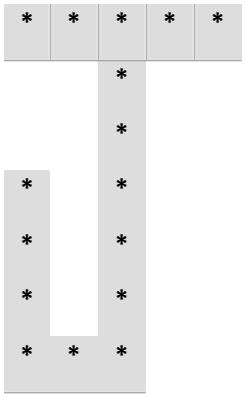
If condition were col==1 only, then the complete col=1 would have then get filled with * characters.

The additional condition **&&row>=4** in the expression **(col==1 && row>=4)** actually limited the * characters at the column 1 started from row 4 till row 7.

Removing the (row, col) value pair which doesn't have * characters.

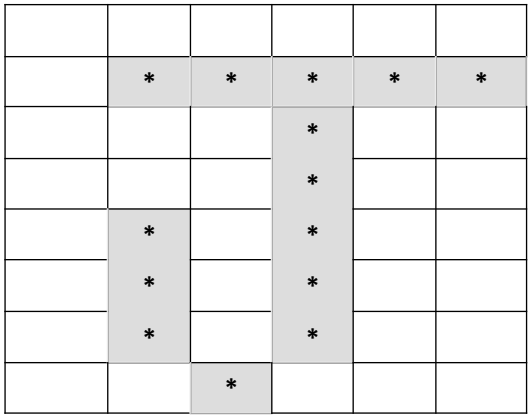
	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2*	1,3*	1,4*	1,5*
row=2			2,3*		
row=3			3,3*		
row=4	4,1*		4,3*		
row=5	5,1*		5,3*		
row=6	6,1*		6,3*		
row=7	7,1*	7,2*	7,3*		

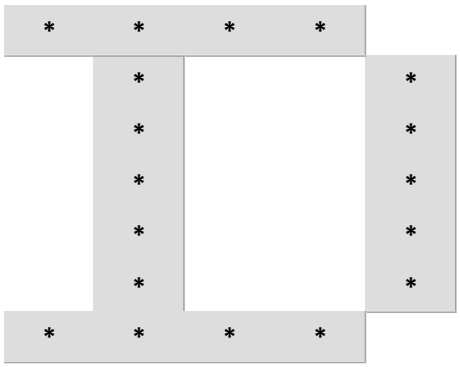
OR



Logical condition: (row==1) || (col==3) || (row==7 && col<=3) || (col==1 && row>=4).

ProblemYBJD: **Write the program for the following style of alphabet J and D.**





***ExerciseYBK*: Alphabet K**

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2	1,3	1,4	1,5
row=2	2,1*	2,2	2,3	2,4	2,5
row=3	3,1*	3,2	3,3	3,4	3,5
row=4	4,1*	4,2	4,3	4,4	4,5
row=5	5,1*	5,2	5,3	5,4	5,5
row=6	6,1*	6,2	6,3	6,4	6,5
row=7	7,1*	7,2	7,3	7,4	7,5

We need to put * characters in the first row. So matching logical conditions will be **(row==1)**. We need to find a relationship among highlighted boxes as it suggests one of the arm of alphabet K.

The highlighted boxes which are represented by **(row, col)** value pairs as **(4,1),(3,2),(2,3),(1,4).**

If we pay little bit of attention we can observe that if we add row value/number to column value/number, we will get value 5 consistently.

(4,1),(3,2),(2,3),(1,4) ➔ (4+1),(3+2),(2+3),(1+4) ➔(5),(5),(5),(5),(5).

We need to verify if this holds true for other **row+col** value combinations or not. Let check by adding each **row+col** value for every box represented by **(row, col)** value pair. All **row+col** value are as follows and highlighted where **row+col=5.**

(1+1=2) (1+2=3) (1+3=4) **(1+4=5)** (1+5=6)
(2+1=3) (2+2=4) **(2+3=5)** (2+4=6) (2+5=7)
(3+1=4) **(3+2=5)** (3+3=6) (3+4=7) (3+5=8)
(4+1=5) (4+2=6) (4+3=7) (4+4=8) (4+5=9)
(5+1=6) (5+2=7) (5+3=8) (5+4=9) (5+5=10)
(6+1=7) (6+2=8) (6+3=9) (6+4=10) (6+5=11)
(7+1=8) (7+2=9) (7+3=10) (7+4=11) (7+5=12)

Nowhere **row + col = 5** holds true in other than our considered boxes. So our guess now becomes valid which uniquely identifies or selects the considered value pairs. This how we do selection of in general data by applying condition. In this case boxes with (**row, col)** combination are selected. Hence our next logical condition becomes **(row+col == 5)**.

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2	1,3	1,4*	1,5
row=2	2,1*	2,2	2,3*	2,4	2,5
row=3	3,1*	3,2*	3,3	3,4	3,5
row=4	4,1*	4,2	4,3	4,4	4,5
row=5	5,1*	5,2	5,3	5,4	5,5
row=6	6,1*	6,2	6,3	6,4	6,5
row=7	7,1*	7,2	7,3	7,4	7,5

So logical condition becomes

(col == 1) || (row+col==5).

Similarly we need to find the relationship between **(4, 1), (5, 2), (6, 3), (7, 4) boxes**, the other arm of the alphabet K shown by highlighted cell boarder for row value more than 4.

Previously, addition of (row +col) worked and this time seems to be subtraction may work. Let's study this case

(4,1),(5,2),(6,3),(7,4) ➔(4-1),(5-2),(6-3),(7-4) ➔(3),(3),(3),(3).

It's giving consistent result and we need to verify if any other **(row, col)** combination too follows this rule of selection or not. If we find any other **(row, col),** then we need to find some other relationship formula. The selection rule or logical condition should uniquely select **(4,1),(5,2),(6,3),(7,4).** Let check by subtracting each **row - col** value. All **(row – col)** value are shown as follows and highlighted ones were follows the rule **(row - col=3).**

(1-1=0)	(1-2=-1)	(1-3=-2)	(1-4=-3)	(1-5=-4)
(2-1=1)	(2-2=0)	(2-3=-1)	(2-4=-2)	(2-5=-3)
(3-1=2)	(3-2=1)	(3-3=0)	(3-4=-1)	(3-5=-2)
(4-1=3)	(4-2=2)	(4-3=1)	(4-4=0)	(4-5=-1)
(5-1=4)	**(5-2=3)**	(5-3=2)	(5-4=1)	(5-5=0)
(6-1=5)	(6-2=4)	**(6-3=3)**	(6-4=2)	(6-5=1)
(7-1=6)	(7-2=5)	(7-3=4)	**(7-4=3)**	(7-5=2)

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2	1,3	1,4*	1,5
row=2	2,1*	2,2	2,3*	2,4	2,5
row=3	3,1*	3,2*	3,3	3,4	3,5
row=4	4,1*	4,2	4,3	4,4	4,5
row=5	5,1*	5,2*	5,3	5,4	5,5
row=6	6,1*	6,2	6,3*	6,4	6,5
row=7	7,1*	7,2	7,3	7,4*	7,5

Nowhere **row – col = 3** holds true in other than our considered value pairs**.** So our guess now becomes valid to select these boxes under consideration**.**

Hence our next logical condition becomes

(row – col == 3). So final logical condition becomes

(col ==1) || (row+col==5) ||(row - col == 3).

Removing the (row, col) value pair where * character is not present, we will get as follows;

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*			1,4*	
row=2	2,1*		2,3*		
row=3	3,1*	3,2*			
row=4	4,1*				
row=5	5,1*	5,2*			
row=6	6,1*		6,3*		
row=7	7,1*			7,4*	

OR

*				*
*			*	
*		*		
*				
*		*		
*			*	
*				*

We have seen in this case addition and subtraction relationship among row and column values worked. Sometimes other arithmetic relationship like division or remainder among row and column value holds true to select the boxes means **(row, col)** value pairs. Logical condition can be implemented in many different ways. There could be possible different alternative ways of giving conditions for the same set of selection of **(row, col)** value pairs.

Let's verify following logical condition of alphabet K in 7 rows and 5 columns given as:
(col ==1) || (row + col == 5) || (row - col == 3)

VerifyAlphabetK.java

```
public class VerifyAlphabetK{

	public static void main(String[] args) {

		int row,col;
		int TOTAL_ROWS = 7;
		int TOTAL_COLUMNS = 5;
		boolean condition; //logical condition variable
		// A space characters are used to show the formatted output.
		String  printChar = "* ";
		String  blankSpaces = "  " ;// 2 space characters

		for( row = 1 ; row <= TOTAL_ROWS ; row ++ ) {
			for( col = 1 ; col <= TOTAL_COLUMNS ; col ++ ) {

				condition = ( col == 1 ) ||
					    ( row + col == 5 ) ||
					    ( row - col == 3 ) ;

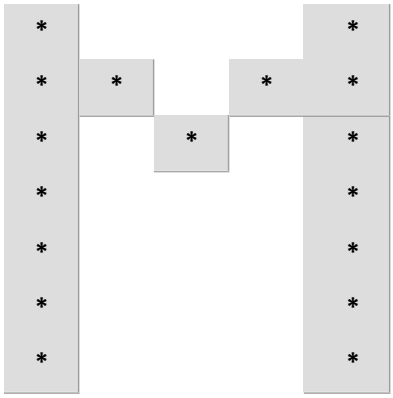
				if( condition ) {
					System.out.print(  printChar);
				}else {
					System.out.print( blankSpaces );
				}
			}
			// printing starts at the next line
			System.out.println();// Move  to the next row
		}

	}
}
```

Output

```
*     *
*   *
* *
*
* *
*   *
*     *
```

***ExerciseYBM*: Alphabet M needs to be fit in 7X5 box.**



	col=1	col=2	col=3	col=4	col=5
row=1	1,1	1,2	1,3	1,4	1,5
row=2	2,1	2,2	2,3	2,4	2,5
row=3	3,1	3,2	3,3	3,4	3,5
row=4	4,1	4,2	4,3	4,4	4,5
row=5	5,1	5,2	5,3	5,4	5,5
row=6	6,1	6,2	6,3	6,4	6,5
row=7	7,1	7,2	7,3	7,4	7,5

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2	1,3	1,4	1,5*
row=2	2,1*	2,2	2,3	2,4	2,5*
row=3	3,1*	3,2	3,3	3,4	3,5*
row=4	4,1*	4,2	4,3	4,4	4,5*
row=5	5,1*	5,2	5,3	5,4	5,5*
row=6	6,1*	6,2	6,3	6,4	6,5*
row=7	7,1*	7,2	7,3	7,4	7,5*

Let start with putting * characters in the first column & in the last column.

So matching logical conditions will be:
(col==1)||(col==5).

We need to find a relationship among highlighted boxes which are represented by **(row, col)** value pairs as **(1, 1), (2, 2), (3, 3).**

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2	1,3	1,4	1,5*
row=2	2,1*	2,2	2,3	2,4	2,5*
row=3	3,1*	3,2	3,3	3,4	3,5*
row=4	4,1*	4,2	4,3	4,4	4,5*
row=5	5,1*	5,2	5,3	5,4	5,5*
row=6	6,1*	6,2	6,3	6,4	6,5*
row=7	7,1*	7,2	7,3	7,4	7,5*

Let's try to understand relationship that uniquely selects these specific set of row and column value pair.

It's easy to observe that row value/number and column value/number are the same. So we can express our selection of boxes by logical condition **(row==col).**

The logical condition (row==col) will select (4, 4) and (5, 5) also.

The box (5, 5) is already been selected by our previous condition.

We don't want box **(4, 4)** to be get selected in our case. We have to do selection till value pair **(3, 3)** starting from the value pair **(1, 1).** We have to use logical AND operator to apply restriction in selection of boxed and logical condition then can be given as follows;

(row==col) && (row <=3 && col<=3)

These selection of boxes till now can be expressed by logical condition

(col==1)|| (col==5) || ((row==col) && (row <=3 && col<=3))

Again, we need to find a relationship among given highlighted boxes in order to uniquely select them out all the given boxes and which are represented by (row, col) value pairs as **(3, 3), (2, 4), (1, 5).**

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2	1,3	1,4	1,5*
row=2	2,1*	2,2*	2,3	2,4	2,5*
row=3	3,1*	3,2	3,3*	3,4	3,5*
row=4	4,1*	4,2	4,3	4,4	4,5*
row=5	5,1*	5,2	5,3	5,4	5,5*
row=6	6,1*	6,2	6,3	6,4	6,5*
row=7	7,1*	7,2	7,3	7,4	7,5*

Let's try to understand relationship that uniquely selects these specific set of row and column value pairs which are as follows;

- ➔ **(3,3),(2,4),(1,5) or**
- ➔ **(3+3),(2+4),(1+5) or**
- ➔ **(6),(6),(6)**

If we pay little bit of attention we can observe that if we add row value/number to column value/number, we will get value 6 consistently. We need to verify if this holds true for other **row+col** value combinations or not. Let check by adding each **row+col** value for every box represented by **(row, col)** value pair**.** All **row+col value** are as follows and highlighted ones were that follows **row+col=6.**

	col=1	col=2	col=3	col=4	col=5
row=1	1+1=2	1+2=3	1+3=4	1+4=5	1+5=6
row=2	2+1=3	2+2=4	2+3=5	2+4=6	2+5=7
row=3	3+1=4	3+2=5	3+3=6	3+4=7	3+5=8
row=4	4+1=5	4+2=6	4+3=7	4+4=8	4+5=9
row=5	5+1=6	5+2=7	5+3=8	5+4=9	5+5=10
row=6	6+1=7	6+2=8	6+3=9	6+4=10	6+5=11
row=7	7+1=8	7+2=9	7+3=10	7+4=11	7+5=12

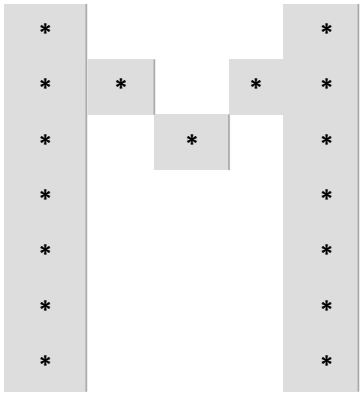
	col=1	col=2	col=3	col=4	col=5
row=1	1,1*	1,2	1,3	1,4	1,5*
row=2	2,1*	2,2*	2,3	2,4*	2,5*
row=3	3,1*	3,2	3,3*	3,4	3,5*
row=4	4,1*	4,2	4,3	4,4	4,5*
row=5	5,1*	5,2	5,3	5,4	5,5*
row=6	6,1*	6,2	6,3	6,4	6,5*
row=7	7,1*	7,2	7,3	7,4	7,5*

Now **row+col=6** also holds true for the value pair **(4, 2)** and we don't want to put character * over there. We have to restrict it to value pair **(3, 3).**The overall logical condition specifically for this can be given as; **(row+col ==6) && (row<=3 && col>=3)**

Removing the other (row, col) value pairs and we get the alphabet M.

	col=1	col=2	col=3	col=4	col=5
row=1	1,1*				1,5*
row=2	2,1*	2,2*		2,4*	2,5*
row=3	3,1*		3,3*		3,5*
row=4	4,1*				4,5*
row=5	5,1*				5,5*
row=6	6,1*				6,5*
row=7	7,1*				7,5*

OR



The complete selection of boxes now can be expressed by logical condition
(col==1)||(col==5)||
((row==col) &&(row <=3 && col<=3))||
((row+col ==6) &&(row<=3 && col>=3))
In the logical condition **((row==col) &&(row <=3 && col<=3)),** due to logical condition **(row==col)** the part of the condition given by **(row <=3 && col<=3) is** either **row<=3 or col<=3** is enough.
This could be further given by
((row==col) && row <=3) or ((row == col) && col<=3).

(row, col) value pair	row value	col value	Description
(1, 5)	1	5	In the logical condition **((row+col ==6) &&(row<=3 && col>=3)),** every **(row, col)** value pair in **(row+col ==6)** are given by **(5, 1), (4, 2) ,(3, 3), (2, 4), (1, 5)** .
(2, 4)	2	4	
(3, 3)	3	3	
(4, 2)	4	2	
(5, 1)	5	1	

Every row value is unique within their group of values and is similar with the case for col values too. Hence, considering either row value or col value is sufficient. It is not necessary to consider both the values of row and col to apply restriction condition as given by **&& (row <=3 && col>=3).**
So part of total logical condition which is given by **(row <=3 && col>=3**), considering either **row<=3 or col>=3** is enough**.** This could be further given by
((row+col ==6) && row <=3) or ((row+col ==6) && col>=3).

We can rewrite the actual logical condition as;
(col==1)||(col==5)||
((row==col) && row <=3)||
((row+col ==6) && row<=3)

Please notice the restriction condition **&&row <=3** is same for both logical condition expressions**.**
We can again rewrite logical condition in more compact form and can be given as follows:

(col == 1)||(col == 5)||

(((row == col)|| (row + col == 6)) && row <= 3)

Let's verify following logical condition of alphabet M in 7 rows and 5 columns given as:

(col == 1) || (col == 5) ||
(((row == col) || (row + col == 6)) && row <= 3)

VerifyAlphabetM.java

```
public class VerifyAlphabetM{


	public static void main(String[] args) {

		int row,col;
		int TOTAL_ROWS = 7;
		int TOTAL_COLUMNS = 5;
		boolean condition; //logical condition variable
		// A space characters are used to show the formatted output.
		String  printChar = "* ";
		String  blankSpaces = "  " ;// 2 space characters

		for( row = 1 ; row <= TOTAL_ROWS ; row ++ ) {
			for( col = 1 ; col <= TOTAL_COLUMNS ; col ++ ) {

				condition = ( col == 1 )||( col == 5 )||
					    ( ( (row == col ) || ( row+col == 6 ) )
					       && row <= 3 );

				if( condition ) {
					System.out.print(  printChar);
				}else {
					System.out.print( blankSpaces );
				}
			}
			// printing starts at the next line
			System.out.println();// Move  to the next row
		}

	}
}
```

Output

```
*       *
* *   * *
*   *   *
*       *
*       *
*       *
*       *
```

Correct the given logical conditions.

Write a program to update the given logical conditions to match the given output.

ProblemYBM1

```
(col==1)||(col==5)||
( (row==col) &&( row <=3 && col<=3) )||
( (row+col ==6) &&(row<=3 && col>=3) )
```

	col=1	col=2	col=3	col=4	col=5
row=1	0				
row=2	0	0			
row=3	0		0		
row=4	0	0			
row=5	0				
row=6	0				
row=7	0				

ProblemYBM2

```
(col==1)||(col==5)||
( (row==col) &&( row <=3 && col<=3) )||
( (row+col ==6) &&(row<=3 && col>=3) )
```

	col=1	col=2	col=3	col=4	col=5
row=1	0				0
row=2	0	0		0	0
row=3	0		0		0
row=4	0	0		0	0
row=5	0				0
row=6	0				0
row=7	0				0

ProblemYBM3

```
(col==1)||(col==5)||
( (row==col) &&( row <=3 && col<=3) )||
( (row+col ==6) &&(row<=3 && col>=3) )
```

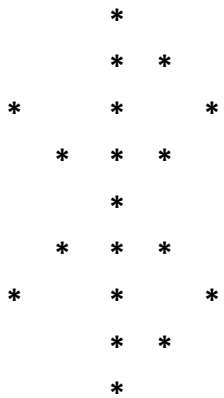
	col=1	col=2	col=3	col=4	col=5
row=1	0				0
row=2		0		0	
row=3			00		
row=4		0	00	0	
row=5	0		00		0
row=6			00		
row=7			00		

ProblemYBM4

```
(col==1)||(col==5)||
( (row==col) &&( row <=3 && col<=3) )||
( (row+col ==6) &&(row<=3 && col>=3) )
```

	col=1	col=2	col=3	col=4	col=5
row=1	$$		0		$$
row=2		$$	0	$$	
row=3	0	0	$$	0	0
row=4		$$	0	$$	
row=5	$$		0		$$
row=6			0		
row=7			0		

ProblemYBM5



	col=1	col=2	col=3	col=4	col=5
row=1			1,3*		
row=2			2,3*	2,4*	
row=3	3,1*		3,3*		3,5*
row=4		4,2*	4,3*	4,4*	
row=5			5,3*		
row=6		6,2*	6,3*	6,4*	
row=7	7,1*		7,3*		7,5*
row=8			8,3*	8,4*	
row=9			9,3*		

ProblemYBN

	col=1	col=2	col=3	col=4	col=5
row=1	*				*
row=2	*	*			*
row=3	*	*	*		*
row=4	*	*	*	*	*
row=5	*		*	*	*
row=6	*			*	*
row=7	*				*

ProblemYBP

	col=1	col=2	col=3	col=4	col=5
row=1		*	*	*	*
row=2		*			*
row=3		*			*
row=4		*	*	*	*
row=5		*			
row=6		*			
row=7		*			

ProblemYBR

	col=1	col=2	col=3	col=4	col=5
row=1	*	*	*	*	
row=2		*		*	
row=3		*		*	
row=4		*	*	*	
row=5		*	*		
row=6		*		*	
row=7		*			*

ProblemYBS

	col=1	col=2	col=3	col=4	col=5
row=1	*	*	*	*	*
row=2	*	*			
row=3	*	*			
row=4	*	*	*	*	*
row=5					*
row=6	*	*	*	*	*
row=7	*	*	*	*	*

You can do it.

Write program for each of the following outputs.

Task M

	col=1	col=2	col=3	col=4	col=5
row=1	M				M
row=2	M				M
row=3	M	M		M	M
row=4	M	M	M	M	M
row=5	M		M		M
row=6	M				M
row=7	M				M

Task T

	col=1	col=2	col=3	col=4	col=5
row=1	T	T	T	T	T
row=2	T		T		T
row=3			T		
row=4			T		
row=5			T		
row=6			T		
row=7			T		

Task U

	col=1	col=2	col=3	col=4	col=5
row=1					
row=2	U				U
row=3	U				U
row=4	U				U
row=5	U				U
row=6	U	U	U	U	U
row=7		U	U	U	

Task V

	col=1	col=2	col=3	col=4	col=5
row=1	V				V
row=2					
row=3		V		V	
row=4					
row=5			V		
row=6					
row=7					

Task W **Columns →**

	1	2	3	4	5	6	7	8	9	0	1	2	3	4	5	6	7	8	9
row=1	W																		W
row=2		W																W	
row=3			W							W							W		
row=4				W						W						W			
row=5					W	W	W	W	W		W	W	W	W	W				
row=6						W	W	W				W	W	W					
row=7							W						W						

Task Y

	col=1	col=2	col=3	col=4	col=5
row=1	XX				XX
row=2	XX	XX		XX	XX
row=3		XX	XXX	XX	
row=4			\V/		
row=5			| |		
row=6			| |		
row=7			| |		

Task 9

	col=1	col=2	col=3	col=4	col=5
row=1			99	99	
row=2		99			99
row=3		99			99
row=4			99	99	99
row=5				99	
row=6			99		
row=7		99			

Task 5

	col=1	col=2	col=3	col=4	col=5
row=1	5	5	5	5	5
row=2	5	5			
row=3	5	5	5		
row=4	5			5	
row=5				5	
row=6				5	
row=7	5	5	5		

Task 8

	col=1	col=2	col=3	col=4	col=5
row=1	8	| |		8	8
row=2	8	| |		| |	8
row=3	8	| |		| |	8
row=4	@	@		@	@
row=5	| |	8		8	| |
row=6	| |	8		8	| |
row=7	| |	8		8	| |

Task 5

	col=1	col=2	col=3	col=4	col=5
row=1	5	5	5	5	5
row=2	5				
row=3	5	5	5	5	
row=4				5	5
row=5				5	5
row=6	5			5	5
row=7		5	5	5	

Task 6

	col=1	col=2	col=3	col=4	col=5
row=1				6	
row=2			6		
row=3		6	6		
row=4	6	6	6	6	
row=5	6			6	
row=6	6			6	
row=7		6	6		

Task 7

	col=1	col=2	col=3	col=4	col=5
row=1	7	7	7	7	7
row=2					7
row=3					7
row=4					7
row=5					7
row=6					7
row=7					7

Task LT

	col=1	col=2	col=3	col=4	col=5
row=1				#	
row=2			#	#	
row=3		#	#		
row=4	#	#			
row=5		#	#		
row=6			#	#	
row=7				#	

Task 2

	col=1	col=2	col=3	col=4	col=5
row=1			#	#	
row=2		#	#	#	#
row=3	#				#
row=4					#
row=5				#	
row=6			#		
row=7	#	#	#	#	#

Task 4

	col=1	col=2	col=3	col=4	col=5
row=1				#	
row=2			#	#	
row=3		#	#	#	
row=4	#	#		#	
row=5	#	#	#	#	#
row=6				#	
row=7				#	

Task Z

	col=1	col=2	col=3	col=4	col=5
row=1	Z	Z	Z	Z	Z
row=2	#	#	#	#	
row=3			#		
row=4		#			
row=5	#	#	#	#	#
row=6	Z	Z	Z	Z	Z
row=7					

Task 1

	col=1	col=2	col=3	col=4	col=5
row=1			#		
row=2		#	#		
row=3	#		#		
row=4			#		
row=5			#		
row=6			#		
row=7	#	#	#	#	#

Learning by experiments and guesswork

The usual and common way to learn programming is just by randomly experimenting it. This takes much more time to learn programming as it required hell lot of practice. No doubt it has advantages especially if we want to explore the api/in-built libraries or explore the features of a programming language. This particular style of learning by guessing for beginners may act as a killer of interest in the programming subject itself. This way it takes more time to learn programming and yes for some percentage of people it is an interesting way. **Everyone can't make a good guess in every subject area and there is no formula for guess work that anybody can utilize it**. It's difficult for a person to experiment with programming who couldn't able to make good guess or even awkward guessing. The imagination regarding programming doesn't flow for many of them and person's mind just goes blank. So It should not be a surprise if that person is good in many other activities and lacks patience in learning programming. This is not in question specific to learning programming as a subject, it could happen with learning any other subject also.

The ground reality is that if imagination and thought process doesn't flow for a person who trying to learn something, it just doesn't go into the head. Learning programming by guessing is a switch off to many of them. It doesn't reflect any standard way of learning. Instead, a systematic approach to learn programming based on simple math and common sense makes learner a better thinker. That approach dealt in depth at different levels of difficulties to properly learn programming logic and covered in the next section **programming patterns**.

In order to do programming in patterns the person has to know for-loop and applying logical condition well. Looping are the hardest. Nested loop challenges us to think logically in two different directions at the same time to output a pattern and any pattern is a 2-dimensional simple picture. These type of activities stimulates logical thinking. During learning, to get multiple and different thoughts are common. Let's try to think confusions or doubts from a different perspective. It creates a kind of suspense and makes the learning exciting. Confusions and doubts are good for learning programming and it is an indication that learner's thought process and imagination becomes active. It does happen many times that clearing one doubt opens the door to explore knowledge in areas. So doubt basically works like a surprise knowledge capsule. A learner just need to keep working in order to try to resolve that specific doubt or confusion. So whenever we get any doubt, we just need to think that we got one more chance to improve our intelligence.

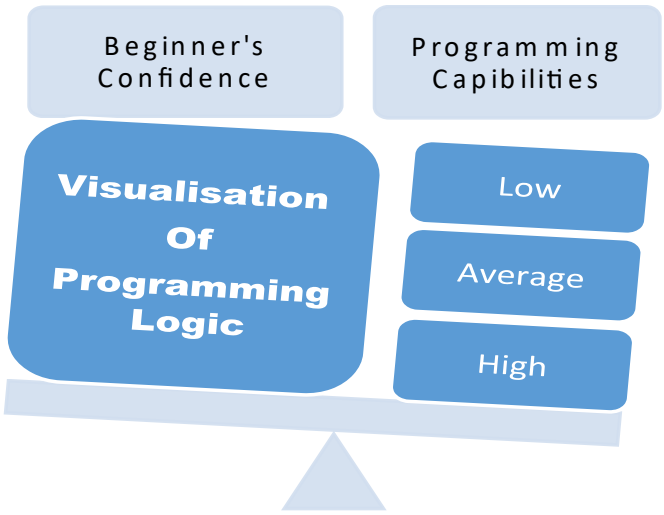
Obviously, learning will not last longer if it is not supported by a systematic approach. A person who is somewhat good at developing programming logic usually struck lesser whenever it is required to deal with harder subjects of programming. So good programming logic is the key and this book is all about developing programming logic skills and it doesn't matter if someone initially a big zero in it. The content in this book surely helps a person to become interested in programming. The exercises designed in a way to naturally suit any new learner in increasing order of difficulty. Some exercises specially included only for the purpose of giving surprise, forcing learner to think out of box.

Programming Patterns

Any pattern is simply a 2-dimensional picture. Most of the time we deal with 2-dimensional type of output, whether it's on the desktop/web/mobile application. So is the reason to select simple picture based problems to address common programming logic issues. Using programming language if someone is good at outputting result in two-dimensional then we can expect that person will be able to take care of three-dimensional aspect of programming too, for example, in case of game programming.

Main idea behind selection of pictorial pattern based exercises is that it clearly validates programmer's logic. The mismatching output forces learner to think for appropriate reason. This way of repetitively working on programming logic to get the matching output greatly helps in perfecting the visualization of programming logic. It is one type of reverse engineering, and we will see how it really works to improve programming skills. These exercise makes learner alert, involved, self-motivated and sure teaches good amount of patience with ease. These set of exercise does create curiosity a lot. The programmer's imagination should work in a direction to trap the programming logic. If this happens, then programming will be an interesting job. In a software development workplaces, one programmer has to understand other programmers programs. This is also true, a programmer can learn good programming by seeing other programmer's good programs. In all cases, some mastery over visualization of programming logic will definitely help to understand other programmers programs. So practicing visualization of programming logic should be the prime focus for any beginner programmer. Looping is the only hardest thing in a program. Mastering the flow of control in nested loops are enough to boost the confidence of beginner programmers.

In a nutshell, beginner's confidence in programming capabilities has a direct relation with low/average/high level of visualization of programming logic.



What we will be going to learn is a simple math based approach to find the actual program logic. This technique requires some level of practice and more we practice more it will refine our imagination to trap the programming logic.

If you can imagine, you can do it.

- Enzo Ferrari

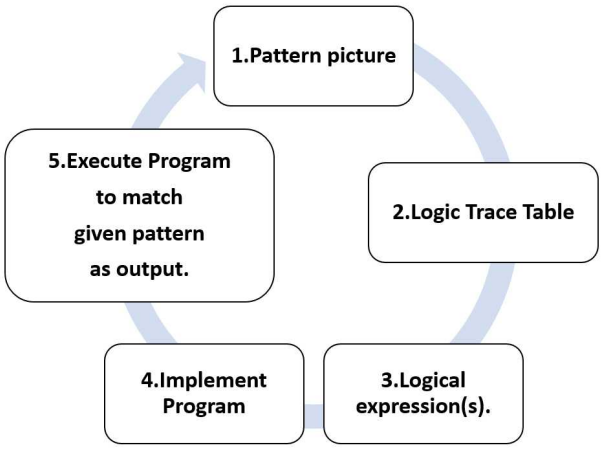
Just by practicing nested loops along with correct sense of applying logical conditions can immensely help to build confidence in a novice programmer.

In general, programmer is always expected to be good at programming logic. If a programmer has got that confidence, then further advance technical learning will be with much confidence. Every programmer might wish to have that level of imaginative sense so that programming logic should get click naturally, matching the demand of the problem to be solved. It will be just a wishful thinking to expect clarity about visualization of programming logic without proper practice. Solid groundwork is required to acquire talent and to have fair level of clarity about visualization of programming logic.

The most important protocol which makes visualization of programming logic to work correctly is to understand the intelligence involved in the flow of data.
The trick to get this intelligence is to express flow of data in one or more logical expressions using our version of observation table known as logic trace table. The logical expression is made up of one or more variables and operators. Finally, programming logic is made up of one or more logical expressions. We can easily learn this art of making logical expressions through appropriate practice mentioned in this section.
Our approach will be to extract **logical expression(s)** by observing the output of the program. This thought process is captured in the following approach which is nothing but a programming logic building exercise of one of its kind. The main steps of the approach are depicted in the following picture. The programmer should be able to visualize the logic clearly even before writing the program. This quality is possible only when programmer is practiced to such a level to be able to visualize clearly how any repeating control structure works or looping in general. Simple picture based exercises helps greatly in that regard which are basically based on nested loop. Given is the complete framework to be followed for improving logic building skills.

The steps need to follow are in a specific order which are given as follows;

Step	Activity
1.	Select the **pattern** picture.
2.	Create the **Logic Trace Table** by seeing the pattern picture.
3.	Extract the **logical expression(s)** from Logic Trace Table.
4.	Implementing the **program** using logical expression(s).
5.	Execute the program and check **program's output to match selected pattern picture**.
Note:	**If step 5 goes wrong, need to repeat step2 to step5.**



If person has one quality and that is having patience to stick to work ethics, then no one can stop that person becoming talented in that field.

Logic trace table: Brain behind programming logic

Do not deny the classical approach, simply as a reaction, or you will have created another pattern and trapped yourself there.

- Bruce Lee

Conceptually logic trace table just one type of observation table. All major observations are put into this table so that we should be able to relate the different pieces of information. You can create your own one or many depends upon your needs. In our case, the rows and columns of logic trace table selected with the motive to trace programming logic. We can use observation table based approach to solve other types of problems also not just related to patterns.

The main concern is to get involve in perfecting novice programmer's visualization of logic through the usage of logic trace table and ultimately to install confidence in programming. Furthermore, clarity and work ethics helps to achieve our small goals. Small achievements raise our confidence level to achieve next level of difficult challenges. So clarity and work ethics are the foundation for every event of achievement to happen.

What purpose does logic trace table solve?

Simplicity is the key to brilliance.

- Bruce Lee

It's a simple trick to train novice programmer's mind on logical thinking. It's main purpose is to apply thinking in more structured manner to get programming logic. It is a simple table structure meant to derive the **logical expression**. The table structure basically gives clarity about relationship among the variable parameters.

Clarity is power.

-The more clear you are about EXACTLY what it is that you want, the more your brain knows how to get there.

- Unknown

In my opinion, without taking help of table structure most of the novice programmer will find it difficult to think clearly in terms of building programming logic. The clear thinking to find logical expression is a process which can be improved with little bit of practice. Unless and until a person is able to put his/her understanding in a simplified way, it means that person had not really understood the concept.

What is the importance of logical expression?

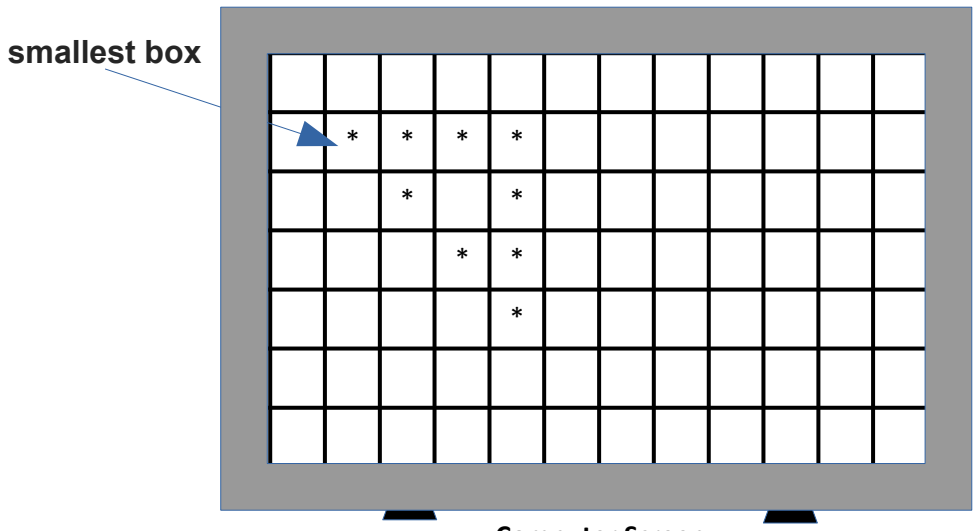
Programming logic is a brain behind program. It is made up of logical expression(s).So, at the very core level it is a logical expressions when executed helps in decision making. Most of the time it is applied in condition of looping, in decision structures like **if-condition**, **if-elseIf condition**.

PART 2

PROGRAMMING IN ACTION

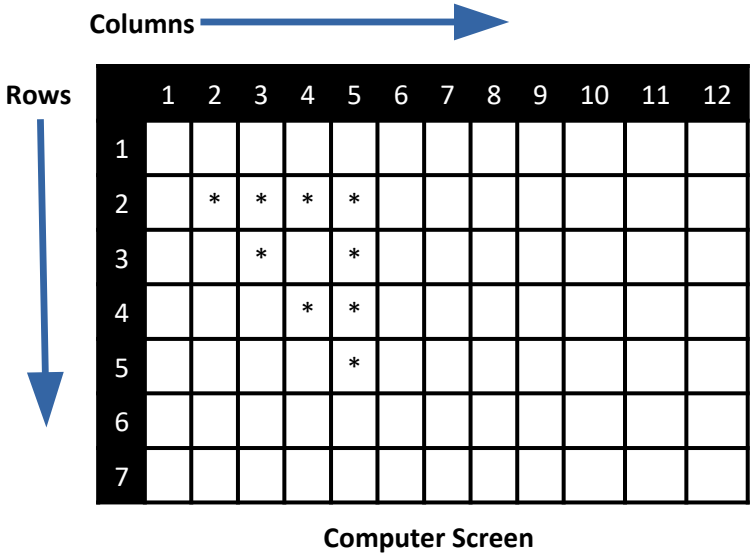
Outputting Simple Picture on Computer Screen

The key point here is to understand that the computer screen is nothing more than a logical collection of small boxes where individual small box can output individual keyboard character. These boxes are arranged in such a way to give a clear idea about rows and columns. Following is the conceptual diagram of computer screen showing some rows and columns of a simple picture outputted into the computer screen. Each keyboard character is in one of the single smallest boxes.



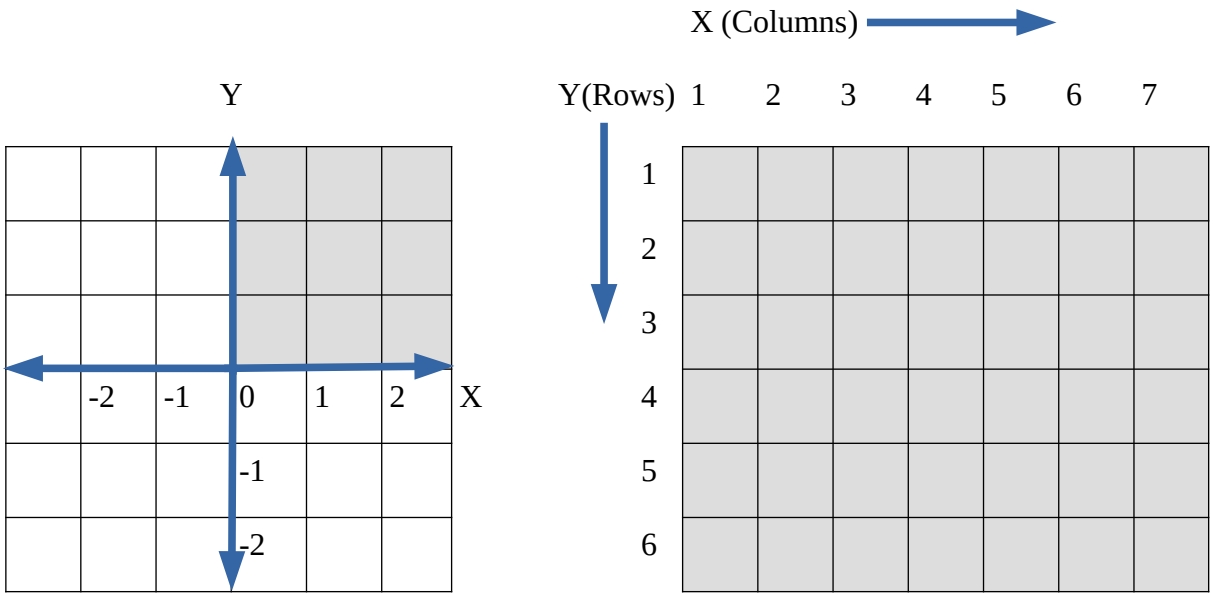
Before we begin programming with simple picture-based exercises, we must first learn to conceptually allocate row number and column number. This will give justice to uniquely identify each smallest box with a singular keyboard character. The (**row number, column number**) numeric combination will imply a single keyboard character. Mentioning (row number, column number) numeric combination is simply a reason to locate correctly, any keyboard character on the computer screen. It is quite analogous to locating a city on the geographical map using **(longitude, latitude)** combination. Hence, our version of single smallest box maps to a single keyboard character on the computer screen and equivalently maps to a unique (row number, column number) numeric pair. In this way, the collection of (row number, column number) numeric pairs can be used to represent a simple picture completely.

Single Smallest Box → Single keyboard character → (row number, column number) numeric pair.



Alternately, we can think of the computer screen as a sophisticated version of graph paper or grid paper. Each smallest box of the computer screen is a coordinate, a unique combination of two numbers, an "X (Columns)" (**horizontal**) and a "Y(Rows)" (**vertical**) - that determines the location of a keyboard character in computer screen space.

It is our programming job, how can a simple picture-based pattern should appear at these smallest box coordinates represented by (**row number, column number**) numeric pairs.



School Grade Graph Paper Computer Screen

As discussed previously, we can represent any 2-dimensional simple picture in terms of rows and columns on the computer screen. So, such simple picture which are made up of keyboard characters can be completely caged into collection of small boxes. Each keyboard character of a considered simple picture can have a unique (**row number, column number**) numeric pair. Hence, the collection of such numeric pairs, then can totally represent a complete simple picture.

The collection of related unique (**row number, column number**) numeric pairs of this simple picture can be given as;

(2,2), (2,3), (2,4), (2,5)

(3,3), (3,5), (4,4), (4,5),

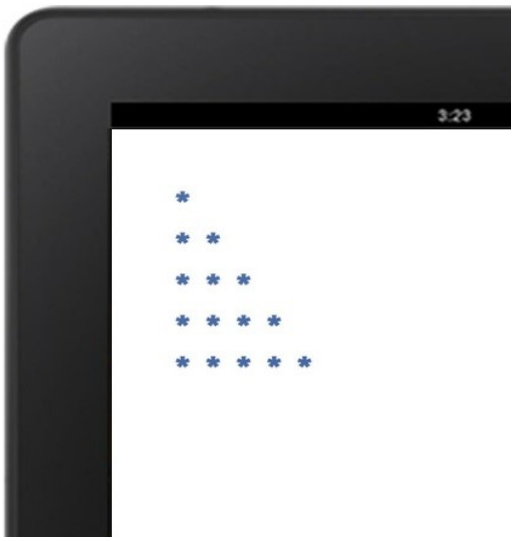
(5,5)

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1					
Row=2		*(2,2)	*(2,3)	*(2,4)	*(2,5)
Row=3			*(3,3)		*(3,5)
Row=4				*(4,4)	*(4,5)
Row=5					*(5,5)

Cursor Movement

A moving position-indicator shown on a computer monitor where the next move or operation will occur. We will use this concept to explain better how each character of a pattern got generated on the computer screen or console screen of the programming IDE. In our case, keyboard characters are to be outputted using JAVA'S inbuilt libraries onto the computer screen at specific locations. So the cursor movement to the next position on the screen and then printing of character will be taken care by JAVA'S inbuilt methods.

Pattern 1A



Our programming job is to print this particular logic-based pattern on a part of the computer screen. **Our approach will be assumed that the computer screen is a combination of rows and columns virtually dividing it into smaller boxes.** The smallest box is treated as a resident of just a single character only. The address in each smallest box on the display screen will logically become a unique **(row, column)** numeric combination. With this assumption computer screen will simply be a unique combination of (row, column) numeric value paired boxes which are uniformly spread all across the screen.

Data Representation

Although logically, we can exactly locate the position of each printable character on the display screen by knowing it's uniquely identifiable **(row, column)** numeric paired-values.

Row, Col	Col = 1	Col = 2	Col = 3	Col = 4	Col = 5
Row = 1	* (1, 1)				
Row = 2	* (2, 1)	*(2, 2)			
Row = 3	* (3, 1)	*(3, 2)	*(3, 3)		
Row = 4	* (4, 1)	*(4, 2)	*(4, 3)	*(4, 4)	
Row = 5	* (5, 1)	*(5, 2)	*(5, 3)	*(5, 4)	*(5, 5)

The pair (4, 3) represents a single character * at the assumed position of 4th row and 3rd column on the display screen. The set of all value pairs (row = value, col = value) now represent this particular pattern.

Analysis of output

Computer dominantly works on very precise calculations. One of the interesting way to practice programming is to try converting manual steps of writing a given pattern into programming instructions. We can use paper or spreadsheet. To analyze a pattern correctly, we can mark row numbers and column numbers to logically identify position-indicator.

		Col = 1	Col = 2	Col = 3	Col = 4	Col = 5	WRITING ACTION	
ROW	Move to Row = 1	1 → *					Write *	1 time
MOVEMENT	Move to Row = 2	1 → *	2 → *				Write *	2 times
ACTION	Move to Row = 3	1 → *	2 → *	3 → *			Write *	3 times
	Move to Row = 4	1 → *	2 → *	3 → *	4 → *		Write *	4 times
	Move to Row = 5	1 → *	2 → *	3 → *	4 → *	5 → *	Write *	5 times

Decomposition

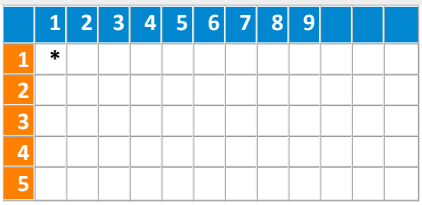
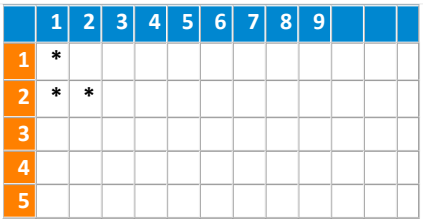
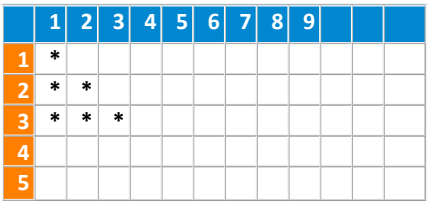
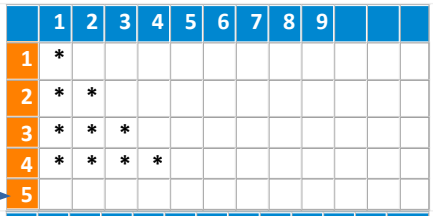
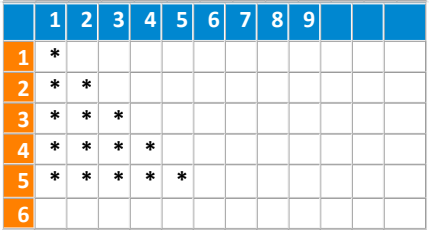
We can decompose our repetitive actions of printing pattern into the following categories;

Repetitive Actions

a) Row Level: Repetitively moving down, marking the next row number in sequence-wise.
b) Column Level: Repetitively writing * characters at the column level.

The given pattern is a 2-dimensional simple picture such that it can be expressed in terms of rows and columns**.** Our next step will be to transform these manual, repetitive actions into a working program. We will use * characters in our program to print it on the computer screen. The working program should behave in the following way.

Steps for generation of pattern on the screen.

		COLUMN LEVEL	Computer Screen
ROW LEVEL	row = 1	1) Print "*" <u>once</u> in the same 1st row with col = 1. 2) Move to row number 2.	
	row = 2	1) Print "*" <u>twice</u> in the same 2nd row with col = 1, 2. 2) Move to row number 3.	
	row = 3	1) Print "*" <u>thrice</u> in the same 3rd row with col = 1, 2, 3. 2) Move to row number 4.	
	row = 4	1) Print "*" <u>four</u> times in the same 4th row with col = 1, 2, 3, 4. 2) Move to row number 5.	
	row = 5	1) Print "*" <u>five</u> times in the 5th row with col = 1, 2, 3, 4, 5 2) Move to row number 6.	

Let's try to mimic the steps by writing a program using given in-built methods.

Purpose	Method Name
In order to shift program's control to the next row.	**System.*out*.println();**
To move program's control in the **same row** for the next column position (**same row**, new column) and then print the required character. Each column movement on the screen will be a single character sized jump from left to right direction in the same row.	System.*out*.print("*");

We can easily observe that the row value increases by 1, multiple times. This is the exact feature of ***for-loop*** control structure. Hence we can apply ***for-loop*** on row values.

Pattern1A1If.java

```
public class PatternA1If{

	public static void main( String[] args )

		for ( int row = 1; row <= 5; row = row + 1 ) {

			if ( row == 1 ) {
				System.out.print("*" );// col = 1
			}
			if ( row == 2 ) {
				System.out.print("*" );// col = 1
				System.out.print("*" );// col = 2
			}
			if ( row == 3 ) {
				System.out.print("*" );// col = 1
				System.out.print("*" );// col = 2
				System.out.print("*" );// col = 3
			}
			if ( row == 4 ) {
				System.out.print("*" );// col = 1
				System.out.print("*" );// col = 2
				System.out.print("*" );// col = 3
				System.out.print("*" );// col = 4
			}
			if ( row == 5 ) {
				System.out.print("*" );// col = 1
				System.out.print("*" );// col = 2
				System.out.print("*" );// col = 3
				System.out.print("*" );// col = 4
				System.out.print("*" );// col = 5
			}
			System.out.println();// Move to the next row
		}
	}
}
```

Fixed Output Implementation

```
*
**
***
****
*****
```

*Whenever we execute this program, we will always get fixed output in 5 rows. We are unable to output bigger or smaller pattern for different number of rows unless we manually change the given **for-loop** and add/subtract **if-conditions**. Implementing generalized programming logic essentially means we should be able to get customized output without modifying the actual logic of a program.*

How to get generalized programming logic

One of the suitable way to get generalized programming logic is to use our own version of logic trace table. Basically logic trace table is a concept and it can be applied in general to get logic of a program. Obviously, its structure will get differ accordingly depends upon the type of problem to be solved. We are learning how to apply this concept of logic trace table. One of the interesting way to do so is to try out in implementing console based programs of the type pattern-recognition based problems of various shapes. To get program's logic is easy if we clearly understand how the changes in logic trace table will be going to impact the implementation of program under consideration. Let's try to understand the principles on which it works.

Principles of Logic trace Table/Observation Table

It's a set of ways for interacting with the program's logic that haven't actually explored yet. It helps in doing that. The final generalized programming logic is by nature made up of one more generalized logical expressions. So we need to trace out generalized logical expressions from logic trace table. The way to get generalized logical expression is to follow few of the given facts;

- ✔ The order of execution of variables in a given program. In our case, in general the working program consists of outer-loop and inner-loop. Since we know that outer-loop executes before nested-loop, the **variable: row** will execute before the **variable: col.**

  GENERAL APPROACH TO PRINTING PATTERN

  Row level Repetitive Action - Outer Loop

  Column level Repetitive Action - Nested Loop

  Move To Next Row

- ✔ The logic trace table is a snap-shot of how different variables store different value during the execution of program. If two variables follows the same numeric pattern during the execution of program, then the variable which execute first will act as replacement for the numeric pattern followed by the other variable. In this particular way we need to try to replace set of hard-coded numerical values with the available variable(s) of a given program. Hence, it is utmost important to check if any numeric pattern matching exists between the columns of logic trace table.
- ✔ After doing numeric pattern matching and it's replacement using available variable(s), we need to check consistency of an expression in a particular column. If it holds true, then it is a valid generalized logical expression. A generalized logical expression may consist of one or many variables.
- ✔ We need to check, if there exist same numeric value in a particular column throughout. If this hold true, then it is a required to be generalized numeric expression, a singular numerical value in the whole column.

So the bottom-line is we need to update columns of a logic trace table till we get consistent numeric or logical expression(s) for all the rows. We will see how to apply these points practically, to build logic of a program. This is an attempt to design program's logic by applying **principles of logic trace table** on the columns of given table to get the suitable numeric/logical expressions. We can start with creating our first version of the logic trace table using **PatternA1If.java** program.

COLUMN LEVEL REPETITIVE ACTION

Abstracting out details of **PatternA1If.java** program to create logic trace table. The given table decomposes the given pattern into relationship of **one-to-many** between row and column values.

INPUT	ROW LEVEL	COLUMN LEVEL
TOTAL ROWS = 5	REPETITIVE ACTION	REPETITIVE ACTION
*	row = 1	col = 1
* *	row = 2	col = 1, 2
* * *	row = 3	col = 1, 2, 3
* * * *	row = 4	col = 1, 2, 3, 4
* * * * *	row = 5	col = 1, 2, 3, 4, 5

For the purpose of uniquely identifying each printing character of a given pattern let's consider variables **row** and **col**. Choosing variables **row** and **col** to precisely mark printing character's row and column number position respectively in a given pattern. Every value of **variable: row** can be mapped to a set of **variable: col** values at the column level. The **variable: col** values maintains common difference of value 1 if we move from *starting value* to the *ending value*. Wait for a second, doesn't it very much sound like another case of ***for-loop*** construction.

ROW LEVEL	COLUMN LEVEL		
ROW VALUE	COLUMN VALUES		COMMON DIFFERENCE
row = 1	col = 1	Start value = 1 End value = 1	{1 = 1}
row = 2	col = 1, 2	Start value = 1 End value = 2	{2-1=1}
row = 3	col = 1, 2, 3	Start value = 1 End value = 3	{2-1=1,3-2=1}
row = 4	col = 1, 2, 3, 4	Start value = 1 End value = 4	{2-1=1,3-2=1,4-3=1}
row = 5	col = 1, 2, 3, 4, 5	Start value = 1 End value = 5	{2-1=1,3-2=1,4-3=1,5-4=1}

By considering **variable: row** and **variable: col** we can further refine logic trace table with the intention of building ***for-loop*** against each value of row number. The ***for-loop*** will be basically a nested-loop for each row's value.

INPUT	ROW LEVEL FOR LOOP		COLUMN LEVEL FOR LOOP	
TOTAL ROWS = 5	Variable: row		Variable: col	
PRINT PATTERN	start = 1	end = 5	start	end
*	row = 1		col = 1	col = 1
* *	row = 2		col = 1	col = 2
* * *	row = 3		col = 1	col = 3
* * * *	row = 4		col = 1	col = 4
* * * * *	row = 5		col = 1	col = 5

Based on the recent updated logic trace table we can update program's logic given as follows.

PatternA1NestedLoop.java

```
public class PatternA1NestedLoop{

	public static void main( String[] args ) {

		for ( int row = 1; row <= 5; row = row + 1 ) {

			if ( row == 1 ) {
				// start = 1 | end = 1
				for(int col = 1 ; col <= 1 ; col++) {
					System.out.print("* ");
				}
			}

			if ( row == 2 ) {
				// start = 1 | end = 2
				for(int col = 1 ; col <= 2 ; col++) {
					System.out.print("* ");
				}
			}

			if ( row == 3 ) {
				// start = 1 | end = 3
				for(int col = 1 ; col <= 3 ; col++) {
					System.out.print("* ");
				}
			}

			if ( row == 4 ) {
				// start = 1 | end = 4
				for(int col = 1 ; col <= 4 ; col++) {
					System.out.print("* ");
				}
			}

			if ( row == 5 ) {
				// start = 1 | end = 5
				for(int col = 1 ; col <= 5  ; col++) {
					System.out.print("* ");
				}
			}

			System.out.println();// Move to the next row
		}
	}
```

Output

```
*
* *
* * *
* * * *
* * * * *
```

Our program become better than before by using nested loops. For each row's value we had successfully formulated a single nested-loop. Hence, the count of nested-loops will go up or down based on the number of rows used. Next step in generalization will be to convert many nested-loops into a one nested-loop. Such conversion to single nested-loop suggests that the number of rows inputted to the program should not impact our version of solution. We will see all these in action while applying the principles of logic trace table. Primarily, our goal will be to get consistent numeric or logical expression(s) in the columns of logic trace table. We will use these obtained consistent numeric or logical expression(s) results to design our program. Derived expressions only when considered as correct when it gives correct output, otherwise we need to re-work on logic trace table to find out our own mistake(s). This is the "**computational thinking**" way to learn the design of program's logic as compared to trial-error method.

We can find a column where a numeric value is consistent.



INPUT VARIABLE	ROW LEVEL FOR LOOP		COLUMN LEVEL FOR LOOP	
totalRows = 5	Variable: row		Variable: col	
PRINT PATTERN	start = 1	end = 5	start	end
*	row = 1		**col = 1**	col = 1
* *	row = 2		**col = 1**	col = 2
* * *	row = 3		**col = 1**	col = 3
* * * *	row = 4		**col = 1**	col = 4
* * * * *	row = 5		**col = 1**	col = 5

The column **start** contains the data **col = 1** which is same for every row. Hence, column **start** is already being in generalized state and hence we got the consistent logical expression in order to **start** the column level for-loop. The **column level for-loop** must require a singular *starting value* and a singular *end value*.

INPUT VARIABLE	ROW LEVEL FOR LOOP		COLUMN LEVEL FOR LOOP	
totalRows = 5	Variable: row		Variable: col	
PRINT PATTERN	start = 1	end = 5	start	end
*	**row = 1**		**col = 1**	**col = 1**
* *	**row = 2**			**col = 2**
* * *	**row = 3**		**consistent numerical expression**	**col = 3**
* * * *	**row = 4**			**col = 4**
* * * * *	**row = 5**			**col = 5**

Clearly, the column **end** with singular value is not our case. The other remained possibility is to derive a logical expression which should be consistent for this column. We can observe numeric

pattern variations of the **variable: col** and the **variable: row.** The column **end** matches exactly with numeric pattern variations of the **variable: row**, shown in the above logic trace table.

We can find a column where a logical expression is consistent.



INPUT VARIABLE	ROW LEVEL FOR LOOP		COLUMN LEVEL FOR LOOP	
totalRows = 5	Variable: row		Variable: col	
PRINT PATTERN	start = 1	end = 5	start	end
*	row = 1		col = 1	col = row
* *	row = 2			col = row
* * *	row = 3		consistent numerical expression	col = row
* * * *	row = 4			col = row
* * * * *	row = 5			col = row

It means column level for-loop with **variable: col** starts at value 1 and ends at the most recent value taken by the **variable: row**. Hence, we got the consistent logical expression **(col = row)** for the column **end.**

INPUT VARIABLE	ROW LEVEL FOR LOOP		COLUMN LEVEL FOR LOOP	
totalRows = 5	Variable: row		Variable: col	
PRINT PATTERN	start = 1	end = 5	start	end
*	row = 1		col = 1	col = row
* *	row = 2			
* * *	row = 3		consistent numerical expression	consistent logical expression
* * * *	row = 4			
* * * * *	row = 5			

So, during the execution of the program row level for-loop with **variable: row** takes the value 1 to 5. Then column level for-loop with **variable: col** will take values between 1 and the current value of **variable: row**.
Mathematically, it can be given as;

1 <= col <= row

Where **row = 1 to 5**

The row level for-loop will get started at the value 1 and it will be get stopped and controlled by a changing value of **variable: row** during the execution of the program. At each row's value how column level ***for-loop*** should stop looping is controlled by the logical expression **(col <= row).** The consistent logical expression **(col <= row)** will be the reason to produce dynamic output at the

column level for different values of **variable: row**. Extracting the consistent logical expression is a way to generalize and to get a customized solution.When we are able to establish a relationship between variables, then we can have a chance to automate the process of computing.
We can now clearly define the action of column level ***for-loop***.

ELEMENTS OF COLUMN LEVEL FOR LOOP			
Variable: col		INCREMENTAL ORDER	ACTION
start	end	step-size	Print character
1	row	1	*

COLUMN LEVEL FOR LOOP OF INCREMENTAL ORDER

Column level Repetitive Action

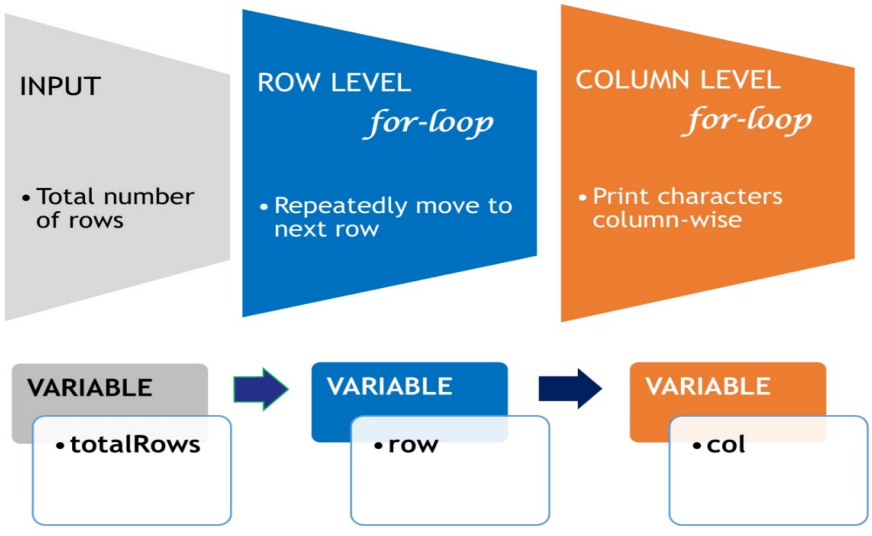
```
for( int col = 1; col <= end; col = col + 1) {

        Print character *

}
```

ROW LEVEL REPETITIVE ACTION

We have use logic trace table as a tool to get the generalized solution. The logic trace table is the snap-shot of how different variables **totalRows**, **row** and **col** store different value and in what matching pattern during the execution of program.



ORDER OF EXECUTION OF VARIABLES IN A PROGRAM.

The following logic trace table is basically **for-loop** in action at row level. The only action expected from **row level for-loop** in our case is to shift program's control to the next row. It should be done dynamically based on the **total number of rows** inputted by the **variable: totalRows**. Our target is to get singular numeric/logical expression for **start** and **end** of a **row level for-loop**.

We can find a column where a numeric value is consistent.



ROW LEVEL FOR LOOP			
INPUT VARIABLE	Variable: row		INCREMENTAL ORDER
TOTAL ROWS	start	end	step-size = 1
totalRows = 3	1	3	Move To Next Row 3 times
totalRows = 5	1	5	Move To Next Row 5 times
totalRows = 7	1	7	Move To Next Row 7 times
totalRows = 10	1	10	Move To Next Row 10 times

This **for-loop's** looping **starts with value 1** and is same for every different input given by total number of rows. Hence, it is already being in generalized state and let marked it down and updating the given logic trace table with single numeric expression value 1. Moreover, the numeric pattern which are shown in **shaded columns** of the given table are matching exactly. During the execution of the program the **variable: totalRows** is the first one to get executed and other columns which are marked shaded followed the same data trend. Hence, **variable: totalRows** is a good replacement for the numeric pattern that exist in different column and followed the numeric pattern exactly. This will establish the relationship between input **variable: totalRows** and **row-level for-loop's variable: row.** It means that during the execution of the program, when **variable: totalRows** contains the value **3,** then the **row level for-loop** should **end** after **3** repetitions.

For **5** repetitions of **row level for-loop**, the control of program moves to the next rows **3** times or **variable: totalRows** should contain value 5.

For **7** repetitions of **row level for-loop**, the control of program moves to the next rows **7** times or **variable: totalRows** should contain value 7.

For **10** repetitions of **row level for-loop**, the control of program moves to the next rows **10** times or **variable: totalRows** should contain value 10.

ROW LEVEL FOR LOOP					
INPUT	Variable: row		INCREMENTAL ORDER		
TOTAL ROWS	start	end	STEP-SIZE = 1		
totalRows = 3	1	3	Move To Next Row	3	times
totalRows = 5		5	Move To Next Row	5	times
totalRows = 7	CONSISTENT	7	Move To Next Row	7	times
totalRows = 10	NUMERIC EXPRESSION	10	Move To Next Row	10	times

After doing numeric pattern matching and applying replacement using **variable: totalRows**, the logic trace table looks like the following.

We can find a column where a logical expression is consistent. 

ROW LEVEL FOR LOOP						
INPUT	Variable: row		INCREMENTAL ORDER			
TOTAL ROWS	start	end	STEP-SIZE = 1			
totalRows = 3	1	totalRows	Move To Next Row	totalRows	times	
totalRows = 5		totalRows	Move To Next Row	totalRows	times	
totalRows = 7		totalRows	Move To Next Row	totalRows	times	
totalRows = 10		totalRows	Move To Next Row	totalRows	times	

The set of data in the border marked columns shows a consistent logical expression after updating the previously logic trace table with single consistent logical expression. The following logic tracing table suggests that maximum possible repetition of **row level for-loop** and moving program's control to the next row can be controlled by the value stored in the **variable: totalRows**. Now, the stored value of **totalRows** will act as maximum boundary value for the **row level for-loop** repetitions. This is the way it formulates a flexible solution to resize the length of the pattern on the basis of given input value of the **variable: totalRows**. Whenever a row-level repetitive action become generalized, it can print pattern of matching length using the value of total number of rows inputted to the program.

ROW LEVEL FOR LOOP					
INPUT	Variable: row		INCREMENTAL ORDER		
TOTAL ROWS	start	end	STEP-SIZE = 1		
totalRows = 3	1	totalRows	Move To Next Row	totalRows	times
totalRows = 5					
totalRows = 7		CONSISTENT LOGICAL EXPRESSION		CONSISTENT LOGICAL EXPRESSION	
totalRows = 10					

We can now clearly define the action of column level ***for-loop***.

ELEMENTS OF ROW LEVEL FOR LOOP			
Variable: row		INCREMENTAL ORDER	ACTION
start	end	step-size	Statements
1	totalRows	1	1# COLUMN LEVEL For Loop 2# Move To Next Row

ROW LEVEL FOR LOOP OF INCREMENTAL ORDER

Row level Repetitive Action

```
for( int row = 1; row <= totalRows; row = row + step-size) {

        Column level Repetitive Action
        Move To Next Row
}
```

POINTS TO PONDER:

WAY TO CUSTOMIZED SOLUTION IS TO FIND CONSISTENT NUMERICAL/LOGICAL EXPRESSION

- The way to generalized/customized solution is to pass through the stage of abstraction. The key to abstraction in our case is to obtain the consistent numeric or logical expression by matching data collectively among the columns of the logic trace table. We refer to this activity as pattern matching. Let's apply pattern matching to find the program's logic.
- Numeric pattern in a logic trace table's column is a foot-print of variable obtained during the execution of program. Observe the overlapping of numeric patterns. Replacing the matching numeric pattern by available variable(s) targeting a consistent logical expression.
- The order of execution of variable is very much important to apply numeric pattern matching between the columns of logic trace table. This is the basic step to get generalized logical expressions and finally leads to complete generalization of a program.

Whenever program does not take care of moving program's control from one row to another, then for example, someone may end up printing straight line instead of printing a triangle shaped-pattern.

CASE A. WHEN PROGRAM'S CONTROL MOVE TO THE NEXT LINE CORRECTLY.

```
*  Move to next line

*  *  Move to next line

*  *  *   Move to next line

*  *  *   *   Move to next line

*  *  *   *   *
```

Case B. When program's control does not move to the next line.

```
*   Move to next line – Skip this step.

*   *   Move to next line - Skip this step.

*   *   *   Move to next line - Skip this step.

*   *   *   *   Move to next line - Skip this step.

*   *   *   *   *
```

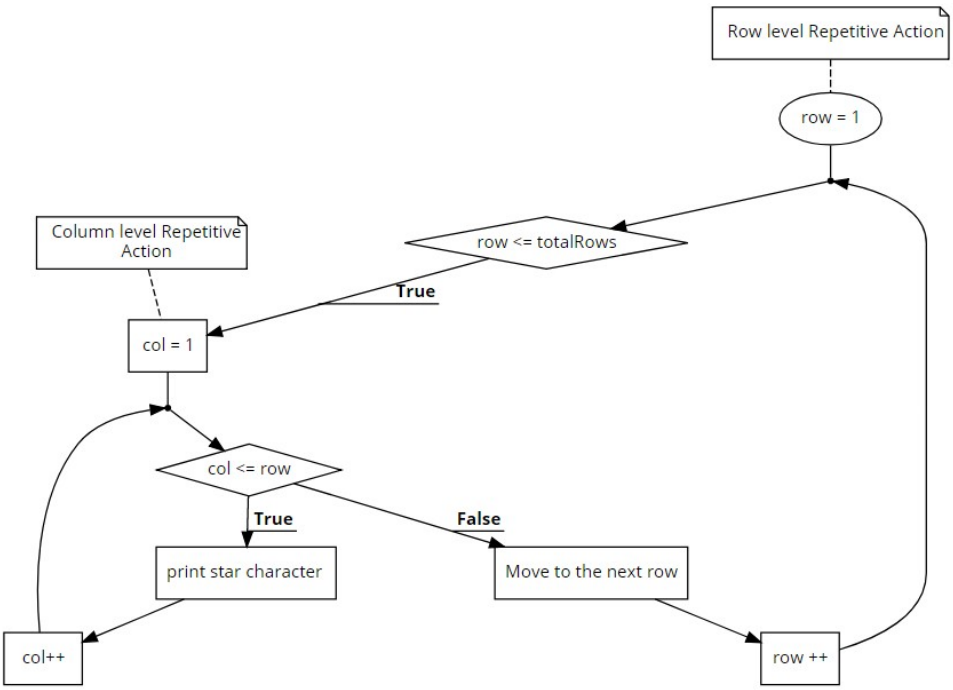
Program's Output—The pattern goes flat.

```
*   *   *   *   *   *   *   *   *   *   *   *   *   *   *
```

Most of the beginner programmers experience such fun-filled surprises while learning to code. The Beauty of learning to code in such a way is that code failures do not put learners on a depressing note. It should stimulate a higher level of interest and motivates to do it better next time. The real challenges a learner will face while handling **column level for-loop**. It requires learners to find a suitable consistent logical expression.

Flow Chart

We got everything related to ***for-loop***.



Let's execute the following program.

Pattern1A_1.java

```
public class PatternA1_2{
	public static void main(String[] args) {
		int totalRows = 5;  //number of rows to display

		// Row level Repetitive Action:
		// Action1.Executes Column level Repetitive Action
		// Action2.Move cursor to next row.
		for( int row = 1 ; row <= totalRows ; row ++ ) {
			// Column level Repetitive Action :
			for( int col = 1 ; col <= row  ; col++ ) {
				// Action1.Move cursor in the same row
				// Action2.print "*" character
				System.out.print("* ");
			}
			System.out.println();// Move cursor to the next row
		}
	}
}
```

Output

```
*
* *
* * *
* * * *
* * * * *
```

Now, test for a different number of lines, let it be 10. It should work correctly, so to confirm that it generalizes our solution.

Pattern1A_1.java

```
public class PatternA1_2{
	public static void main(String[] args) {
		int  totalRows = 10;  //number of rows to display

		// Row level Repetitive Action:
		// Action1.Executes Column level Repetitive Action
		// Action2.Move cursor to next row.
		for( int row = 1 ; row <= totalRows ; row ++ ) {
			// Column level Repetitive Action :
			for(int col = 1 ; col <= row  ; col++) {
				// Action1.Move cursor in the same row
				// Action2.print “*” character
				System.out.print("* ");
			}
			System.out.println();// Move cursor to the next row

		}
	}
}
```

Output

```
*
* *
* * *
* * * *
* * * * *
* * * * * *
* * * * * * *
* * * * * * * *
* * * * * * * * *
* * * * * * * * * *
```

Pattern1B



Our programming job is to print this particular logic-based pattern on a part of the computer screen. **Our approach will be assumed that the computer screen is a combination of rows and columns virtually dividing it into smaller boxes.** The smallest box is treated as a resident of just a single character only. The address in each smallest box on the display screen will logically become a unique **(row, column)** numeric combination. With this assumption computer screen will simply be a unique combination of (row, column) numeric value paired boxes which are uniformly spread all across the screen.

Data Representation

Although logically, we can exactly locate the position of each printable character on the display screen by knowing it's uniquely identifiable (row, column) numeric paired-values.

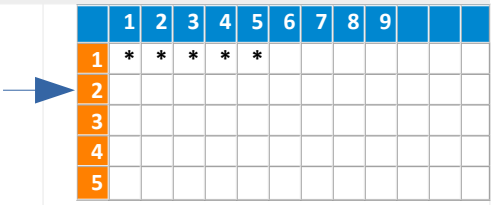
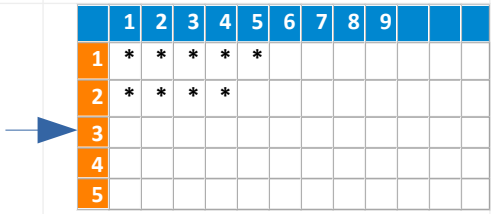
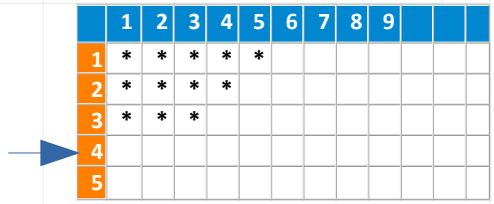
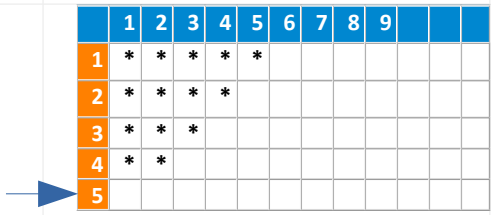
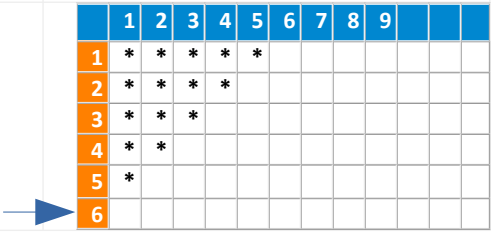
Row, Col	Col = 1	Col = 2	Col = 3	Col = 4	Col = 5
Row = 1	* (1, 1)	* (1, 2)	*(1, 3)	* (1, 4)	* (1, 5)
Row = 2	* (2, 1)	* (2, 2)	* (2, 3)	* (2, 4)	
Row = 3	* (3, 1)	* (3, 2)	* (3, 3)		
Row = 4	* (4, 1)	* (4, 2)			
Row = 5	* (5, 1)				

The pair (2, 4) represents a single character * at the assumed position of 2nd row and 4th column on the display screen. The set of all value pairs (row = value, col = value) now represent this particular pattern.

Analysis of output

Computer dominantly works on very precise calculations. One of the interesting way to practice programming is to try converting manual steps of writing a given pattern into programming instructions. We can use paper or spreadsheet. To analyze a pattern correctly, we can mark row numbers and column numbers to logically identify position-indicator.

		Col = 1	Col = 2	Col = 3	Col = 4	Col = 5	WRITING ACTION	
ROW	Move to Row = 1	1 → *	2 → *	3 → *	4 → *	5 → *	Write *	5 times
MOVEMENT	Move to Row = 2	1 → *	2 → *	3 → *	4 → *		Write *	4 times
ACTION	Move to Row = 3	1 → *	2 → *	3 → *			Write *	3 times
	Move to Row = 4	1 → *	2 → *				Write *	2 times
	Move to Row = 5	1 → *					Write *	1 time

Steps for pattern generation (Total Rows = 5)		Computer Screen
For row = 1	Print "*" five times in the same 1st row then move to row number 2	
For row = 2	Print "*" four times in the same 2nd row then move to row number 3	
For row = 3	Print "*" thrice in the same 3rd row then move to row number 4	
For row = 4	Print "*" twice in the same 4th row then move to row number 5	
For row = 5	Print "*" once in the 5th row then move to row number 6	

Let's convert steps into Java program.

Use of Java in-built method	
In order to move cursor to the next row	**System.out.println( );**
Move cursor to the next (row, column) position and then print star "*" character. Each movement on the computer screen will be a single keyboard character sized jump from left to right direction in the same row.	System.out.print("*");

These are two kinds of statements that will get repeated many times in the program.

```
System.out.println( ); & System.out.print("*");
```

Hence we can use ***for-loop*** for the same.

PatternB1If.java

```
public class PatternB1If{

	public static void main( String[] args ) {

		for ( int row = 1; row <= 5; row = row + 1 ) {

			if ( row == 1 ) {
				System.out.print("*" );// col = 1
				System.out.print("*" );// col = 2
				System.out.print("*" );// col = 3
				System.out.print("*" );// col = 4
				System.out.print("*" );// col = 5
			}

			if ( row == 2 ) {
				System.out.print("*" );// col = 1
				System.out.print("*" );// col = 2
				System.out.print("*" );// col = 3
				System.out.print("*" );// col = 4
			}

			if ( row == 3 ) {
				System.out.print("*" );// col = 1
				System.out.print("*" );// col = 2
				System.out.print("*" );// col = 3
			}

			if ( row == 4 ) {
				System.out.print("*" );// col = 1
				System.out.print("*" );// col = 2

			}
			if ( row == 5 ) {
				System.out.print("*" );// col = 1
			}
			System.out.println();// Move to the next row
		}
	}
}
```

Fixed Output Implementation

```
*****
****
***
**
*
```

Whenever we execute this program, we will always get fixed output in 5 rows. We are unable to output bigger or smaller pattern for different number of rows unless we manually change the given **for-loop** and add/subtract **if-conditions**. Implementing generalized programming logic essentially means we should be able to get customized output without modifying the actual logic of a program.

COLUMN LEVEL REPETITIVE ACTION

Abstracting out details of **PatternB1If.java** program to create logic trace table. The given table decomposes the given pattern into relationship of **one-to-many** between row and column values.

INPUT	ROW LEVEL	COLUMN LEVEL
TOTAL ROWS = 5	REPETITIVE ACTION	REPETITIVE ACTION
* * * * *	row = 1	col = 1, 2, 3, 4, 5
* * * *	row = 2	col = 1, 2, 3, 4
* * *	row = 3	col = 1, 2, 3
* *	row = 4	col = 1, 2
*	row = 5	col = 1

For the purpose of uniquely identifying each printing character of a given pattern let's consider variables **row** and **col**. Choosing variables **row** and **col** to precisely mark printing character's row and column number position respectively in a given pattern. Every value of **variable: row** can be mapped to a set of **variable: col** values at the column level. The **variable: col** values maintains common difference of value 1 if we move from starting value to the ending value. Wait for a second, doesn't it very much sound like another case of ***for-loop*** construction.

ROW LEVEL	COLUMN LEVEL		
ROW VALUE	COLUMN VALUES		COMMON DIFFERENCE
row = 1	col = 1, 2, 3, 4, 5	Start value = 1 End value = 5	{2-1=1,3-2=1,4-3=1,5-4=1}
row = 2	col = 1, 2, 3, 4	Start value = 1 End value = 4	{2-1=1,3-2=1,4-3=1}
row = 3	col = 1, 2, 3	Start value = 1 End value = 3	{2-1=1,3-2=1}
row = 4	col = 1, 2	Start value = 1 End value = 2	{2-1=1}
row = 5	col = 1	Start value = 1 End value = 1	{1 = 1}

By considering **variable: row** and **variable: col** we can further refine logic trace table with the intention of building ***for-loop*** against each value of row number. The ***for-loop*** will be basically a nested-loop for each row's value.

INPUT	ROW LEVEL FOR LOOP		COLUMN LEVEL FOR LOOP	
TOTAL ROWS = 5	Variable: row		Variable: col	
PRINT PATTERN	start = 1	end = 5	start	end
* * * * *	row = 1		col = 1	col = 5
* * * *	row = 2		col = 1	col = 4
* * *	row = 3		col = 1	col = 3
* *	row = 4		col = 1	col = 2
*	row = 5		col = 1	col = 1

Based on the recent updated logic trace table we can update program's logic given as follows.

PatternB1NestedLoop.java

```
public class PatternB1NestedLoop{

	public static void main( String[] args ) {

		for ( int row = 1; row <= 5; row = row + 1 ) {

			if ( row == 1 ) {
				// start = 1 | end = 5
				for(int col = 1 ; col <= 5  ; col++) {
					System.out.print("* ");
				}
			}

			if ( row == 2 ) {
				// start = 1 | end = 4
				for(int col = 1 ; col <= 4  ; col++) {
					System.out.print("* ");
				}
			}

			if ( row == 3 ) {
				// start = 1 | end = 3
				for(int col = 1 ; col <= 3  ; col++) {
					System.out.print("* ");
				}
			}

			if ( row == 4 ) {
				// start = 1 | end = 2
				for(int col = 1 ; col <= 2  ; col++) {
					System.out.print("* ");
				}
			}

			if ( row == 5 ) {
				// start = 1 | end = 1
				for(int col = 1 ; col <= 1  ; col++) {
					System.out.print("* ");
				}
			}

			System.out.println();// Move to the next row
		}
	}
}
```

Output

```
* * * * *
* * * *
* * *
* *
*
```

Our program become better than before by using nested loops. For each row's value we had successfully formulated a single nested-loop. Hence, the count of nested-loops will go up or down based on the number of rows used. Next step in generalization will be to convert many nested-loops into a one nested-loop. It can not be done unless and until we get consistent numeric or logical expression(s) in the columns of logic trace table against all row values.

We can find a column where a numeric value is consistent.



INPUT	ROW LEVEL FOR LOOP		COLUMN LEVEL FOR LOOP	
TOTAL ROWS = 5	Variable: row		Variable: col	
PRINT PATTERN	start = 1	end = 5	start	end
* * * * *	row = 1		col = 1	col = 5
* * * *	row = 2		col = 1	col = 4
* * *	row = 3		col = 1	col = 3
* *	row = 4		col = 1	col = 2
*	row = 5		col = 1	col = 1

Any **for-loop** must require a singular *starting value* and a singular *end value*. The column **end** contains the data **col = 1** which is same for every row. Hence, column **end** is already being in generalized state. We got the consistent numerical expression to **end** the column level for-loop.

INPUT	ROW LEVEL FOR LOOP		COLUMN LEVEL FOR LOOP	
TOTAL ROWS = 5	Variable: row		Variable: col	
PRINT PATTERN	start = 1	end = 5	start	end
* * * * *	row = 1		col = 1	col = 5
* * * *	row = 2			col = 4
* * *	row = 3		consistent	col = 3
* *	row = 4		numerical	col = 2
*	row = 5		expression	col = 1

Target to achieve

We need to find a unique expression for the column start so that if we increase the number of rows, that much row sized pattern it should generate.

Approach to solution

We need to generalize the non-repeating numeric pattern of the column **variable: start** using the available variables. We can use **variable: totalRows.** As an input we had been assigned the value 5, the **total number of rows**. Let's try to derive the consistent logical expression for the column **variable: start.**

ROW LEVEL FOR LOOP		COLUMN LEVEL FOR LOOP			
Variable: row		Variable: col			
start = 1	end = 5	end			start
row = 1		col = 5	= totalRows	= (**totalRows+1**)-1	col = 1
row = 2		col = 4	= totalRows-1	= (**totalRows+1**)-2	
row = 3		col = 3	= totalRows-2	= (**totalRows+1**)-3	
row = 4		col = 2	= totalRows-3	= (**totalRows+1**)-4	
row = 5		col = 1	= totalRows-4	= (**totalRows+1**)-5	

The Logical expression in the column **variable: start** is partially consistent. We can observe, it has partially non-repeating numeric pattern.

ROW LEVEL FOR LOOP		COLUMN LEVEL FOR LOOP			
Variable: row		Variable: col			
start = 1	end = 5	end			start
row = 1		col = 5	= (**totalRows + 1**) -	1	col = 1
row = 2		col = 4	= (**totalRows + 1**) -	2	
row = 3		col = 3	= (**totalRows + 1**) -	3	
row = 4		col = 2	= (**totalRows + 1**) -	4	
row = 5		col = 1	= (**totalRows + 1**) -	5	

The partially non-repeating numeric pattern the same numeric value pattern as **variable: row.**

ROW LEVEL FOR LOOP		COLUMN LEVEL FOR LOOP			
Variable: row		Variable: col			
start = 1	end = 5	end			start
row = 1		col = 5	= (**totalRows + 1**) -	1	col = 1
row = 2		col = 4	= (**totalRows + 1**) -	2	
row = 3		col = 3	= (**totalRows + 1**) -	3	
row = 4		col = 2	= (**totalRows + 1**) -	4	
row = 5		col = 1	= (**totalRows + 1**) -	5	

Further we can express it in terms of **variable: totalRows** & **variable: row**.

ROW LEVEL FOR LOOP		COLUMN LEVEL FOR LOOP			
Variable: row		Variable: col			
start = 1	end = 5	end			start
row = 1		col = 5	= (totalRows + 1) -	row	col = 1
row = 2		col = 4	= (totalRows + 1) -	row	
row = 3		col = 3	= (totalRows + 1) -	row	
row = 4		col = 2	= (totalRows + 1) -	row	
row = 5		col = 1	= (totalRows + 1) -	row	

Now, variable: start has the same logical expression and it is consistent for every value of **row.**

ROW LEVEL FOR LOOP		COLUMN LEVEL FOR LOOP		
Variable: row		Variable: col		
start = 1	end = 5	end		start
row = 1		col = 5	(totalRows + 1) - row	col=1
row = 2		col = 4		
row = 3		col = 3		
row = 4		col = 2		
row = 5		col = 1		

Mathematically, it can be given as;

1 <= col <= (totalRows + 1) – row

Where row = **1 to 5**

The consistent logical expression **(totalRows + 1)** is the reason to produce dynamic output at the column level for different values of variable: row. Extracting the consistent logical expression is a way to generalize and to get a customized solution. When we are able to establish a relationship between variables to get consistent logical expression, then we can have a true chance to automate the process of computing. We can clearly define looping action of column level ***for-loop***.

COLUMN LEVEL FOR LOOP OF INCREMENTAL ORDER

Column level Repetitive Action

```
for( int col = 1; col <= (totalRows + 1) – row; col = col - 1) {
        Print character *
}
```

ELEMENTS OF COLUMN LEVEL FOR LOOP			
Variable: col		DECREMENTING ORDER	ACTION
start	end	step-size	Print character
1	(totalRows + 1) – row	1	*

ROW LEVEL REPETITIVE ACTION

The only action expected from **row level for-loop** in our case is to shift program's control to the next row. It should be done based on the **total number of rows** inputted by the **variable: totalRows**.
It means that during the execution of the program, when **variable: totalRows** contains the value **3,** then the **row level for-loop** should **end** after **3** repetitions.
For **5** repetitions of **row level for-loop**, the control of program moves to the next rows **3** times or **variable: totalRows** should contain value 5.
For **7** repetitions of **row level for-loop**, the control of program moves to the next rows **7** times or **variable: totalRows** should contain value 7.
For **10** repetitions of **row level for-loop**, the control of program moves to the next rows **10** times or **variable: totalRows** should contain value 10.

ROW LEVEL FOR LOOP					
INPUT	Variable: row		INCREMENTAL ORDER		
TOTAL ROWS	start	end	STEP-SIZE = 1		
totalRows = 3	1	3	Move To Next Row	3	times
totalRows = 5		5	Move To Next Row	5	times
totalRows = 7	CONSISTENT	7	Move To Next Row	7	times
totalRows = 10	NUMERIC EXPRESSION	10	Move To Next Row	10	times

We can find a column where a logical expression is consistent.



ROW LEVEL FOR LOOP					
INPUT	Variable: row		INCREMENTAL ORDER		
TOTAL ROWS	start	end	STEP-SIZE = 1		
totalRows = 3	1	totalRows	Move To Next Row	totalRows	times
totalRows = 5		totalRows	Move To Next Row	totalRows	times
totalRows = 7		totalRows	Move To Next Row	totalRows	times
totalRows = 10		totalRows	Move To Next Row	totalRows	times

Clearly, the stored value of `totalRows` will act as upper bound for the **row level for-loop** repetitions. It formulates a flexible solution to resize the length of the pattern in the run-time on the basis of given input value of the `variable: totalRows`.

We can now abstractly define the complete action of row level ***for-loop.***

ELEMENTS OF ROW LEVEL FOR LOOP			
Variable: row		INCREMENTAL ORDER	ACTION
start	end	step-size	Statements
1	totalRows	1	1# COLUMN LEVEL For Loop 2# Move To Next Row

ROW LEVEL FOR LOOP OF INCREMENTAL ORDER

Row level Repetitive Action

```
for( int row = 1; row <= totalRows; row = row + step-size) {

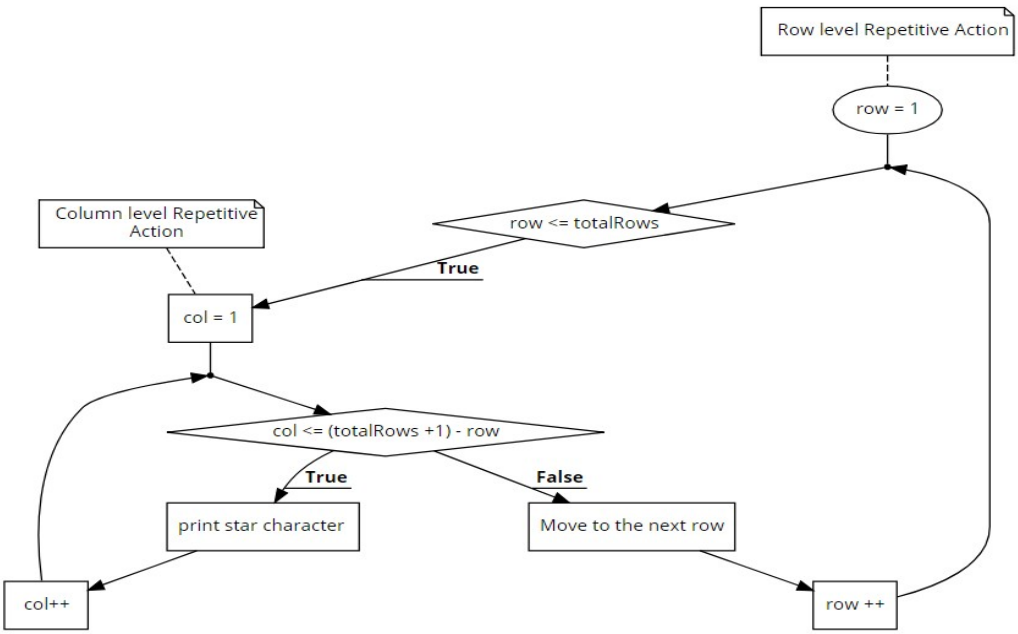
        1# Column level Repetitive Action
        for( int col = 1; col <= (totalRows + 1) – row; col = col - 1) {

           Print caracter *

        }
        2# Move To Next Row
}
```

FLOW CHART

We got everything related to ***for-loop***.



Let's execute the following program.

PatternB1.java

```
public class PatternB1{
	public static void main(String[] args) {
		int  totalRows = 5;  //number of rows to display

		// Row level Repetitive Action :
		// Action1.Executes Column level Repetitive Action
		// Action2.Move cursor to next row.
		for( int row = 1 ; row <= totalRows ; row ++ ) {
			// Column level Repetitive Action :
			for(int col = 1 ; col <= (totalRows +1) - row  ; col++) {
				// Action1.Move cursor in the same row
				// Action2.print "*" character
				System.out.print("* ");
			}
			System.out.println();// Move cursor to the next row
		}
	}
}
```

Output: totalRows = 5

```
* * * * *
* * * *
* * *
* *
*
```

Output: totalRows = 10

```
* * * * * * * * * *
* * * * * * * * *
* * * * * * * *
* * * * * * *
* * * * * *
* * * * *
* * * *
* * *
* *
*
```

Alternative solution2

From previous solution we got the following **Column level Repetitive Action** details as follows:
Let's try to simplify the expression (totalRows +1) – row.

ROW LEVEL FOR LOOP		COLUMN LEVEL FOR LOOP			
Variable: row		Variable: col			
start = 1	end = 5	end			start
row = 1		col = 5	= (totalRows + 1) -	row	col = 1
row = 2		col = 4	= (totalRows + 1) -	row	
row = 3		col = 3	= (totalRows + 1) -	row	
row = 4		col = 2	= (totalRows + 1) -	row	
row = 5		col = 1	= (totalRows + 1) -	row	

Subtracting 1 from row and column **end** , since its logical expression is impacted by the **variable:row**.

ROW LEVEL FOR LOOP		COLUMN LEVEL FOR LOOP			
Variable: row		Variable: col			
start = 1	end = 5	end			start
row = 1 - 1		col = 5	= (totalRows + 1) -	row - 1	col = 1
row = 2 - 1		col = 4	= (totalRows + 1) -	row - 1	
row = 3 - 1		col = 3	= (totalRows + 1) -	row - 1	
row = 4 - 1		col = 2	= (totalRows + 1) -	row - 1	
row = 5 - 1		col = 1	= (totalRows + 1) -	row - 1	

ROW LEVEL FOR LOOP		COLUMN LEVEL FOR LOOP			
Variable: row		Variable: col			
start = 1	end = (totalRows - 1)	end			start
row = 0		col = 5	= totalRows -	row	col = 1
row = 1		col = 4	= totalRows -	row	
row = 2		col = 3	= totalRows -	row	
row = 3		col = 2	= totalRows -	row	
row = 4 = (totalRows - 1)		col = 1	= totalRows -	row	

Now, **column end** has the same expression and it is consistent for every value of **variable:row.**
We can define column level ***for-loop*** while considering every row value.

ELEMENTS OF COLUMN LEVEL FOR LOOP			
Variable: col		DECREMENTING ORDER	ACTION
start	end	step-size	Print character
1	(totalRows – row)	1	*

Outer ***for-loop*** starts at row = 0 & will repeat till row <= (**totalRows** – 1) or row < **totalRows**.

ELEMENTS OF ROW LEVEL FOR LOOP			
Variable: row		INCREMENTAL ORDER	ACTION
start	end	step-size	Statements
row = 0	row < totalRows	1	1# COLUMN LEVEL For Loop 2# Move To Next Row

The new solution changes are in the following program.

PatternB2.java

```
public class PatternB2{

	public static void main(String[] args) {

		int  totalRows = 5;  //number of rows to display

		// Row level Repetitive Action :
		// Action1.Executes Column level Repetitive Action
		// Action2.Move cursor to next row.
		for( int row = 0 ; row < totalRows ; row ++ ) {

			// Column level Repetitive Action :
			for(int col = 1 ; col <= ( totalRows  - row ) ; col++) {

				// Action1.Move cursor in the same row
				// Action2.print “*” character
				System.out.print("* ");
			}

			System.out.println();// Move to the next row

		}
	}
}
```

Output: totalRows = 5

```
* * * * *
* * * *
* * *
* *
*
```

Output: totalRows = 10

```
* * * * * * * * * *
* * * * * * * * *
* * * * * * * *
* * * * * * *
* * * * * *
* * * * *
* * * *
* * *
* *
*
```

Please verify output for different number of rows, let it be 10.

Alternative solution3

The next observation table was one of several logic trace tables we summarized in the very beginning trying to find a solution to this particular pattern.

INPUT	ROW LEVEL FOR LOOP		COLUMN LEVEL FOR LOOP	
TOTAL ROWS = 5	Variable: row		Variable: col	
PRINT PATTERN	start = 1	end = 5	start	end
* * * * *	row = 1		col = 1	col = 5
* * * *	row = 2			col = 4
* * *	row = 3		consistent	col = 3
* *	row = 4		numerical	col = 2
*	row = 5		expression	col = 1

If we observe the numeric pattern for row (1, 2, 3, 4, and 5) is **reverse** of the numeric pattern of the column **end** (5, 4, 3, 2, and 1). If we just reverse the counter of row variable it will match exactly the values of the column **max_val** (5, 4, 3, 2, and 1).Instead of using the incrementing ***for-loop*** for the row variable we need to replace it with the decrementing ***for-loop*** for the row variable.
Reversing the flow of values for the **variable:row**. It can be given as follows:

INPUT	ROW LEVEL FOR LOOP		COLUMN LEVEL FOR LOOP	
TOTAL ROWS = 5	Variable: row		Variable: col	
PRINT PATTERN	start = 5	end = 1	start	end
* * * * *	row = 5		col = 1	col = 5
* * * *	row = 4			col = 4
* * *	row = 3		consistent	col = 3
* *	row = 2		numerical	col = 2
*	row = 1		expression	col = 1

Now numeric pattern of variable row and **column end** matches exactly. So, **column end** can be expressed in variable:row. So, **column end** is consistent for every value of **row.** Now, we are in good position since ***for-loop*** requires one minimum value or a logical expression and one maximum value or a logical expression.

INPUT	ROW LEVEL FOR LOOP		COLUMN LEVEL FOR LOOP		
TOTAL ROWS = 5	Variable: row		Variable: col		
PRINT PATTERN	start = 5	end = 1	start	end	
* * * * *	row = 5			row	
* * * *	row = 4			row	
* * *	row = 3		col = 1	row	row
* *	row = 2			row	
*	row = 1			row	

We can now define column level incrementing ***for-loop***.

Elements Of COLUMN LEVEL For Loop			
Variable: col		Incrementing Order	Action
start	end	step-size	Print character
col = 1	col <= row	1	*

We can now define row level **decrementing *for-loop***.
It starts at current value of **variable: `totalRows`** and repeatedly decreases till it becomes 1. Please verify results with the mentioned updated program.

Elements Of ROW LEVEL For Loop			
Variable: row		Decremental Order	Action
start	end	step-size	Statements
row = totalRows	row >= 1	1	1# COLUMN LEVEL For Loop 2# Move To Next Row

PatternB3.java

```
public class PatternB3{
	public static void main(String[] args) {
		int  totalRows = 10;  //10 rows to display

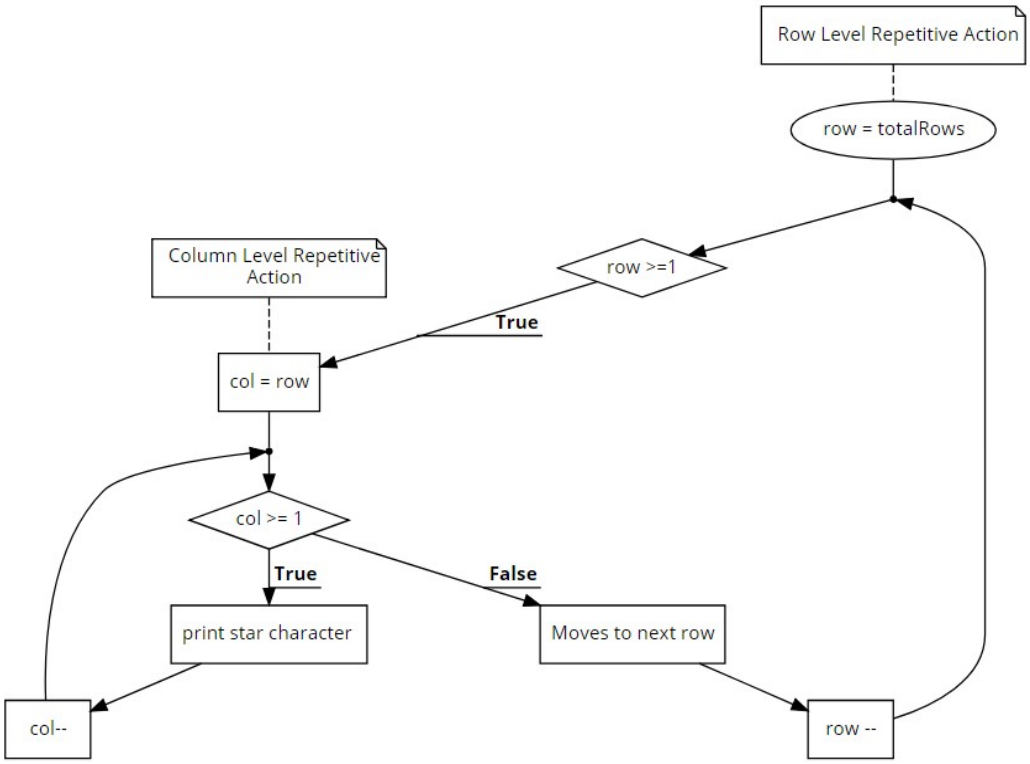
		// Row level Repetitive Action :
		// Action1.Executes Column level Repetitive Action
		// Action2.Move cursor to next row.
		for( int row = totalRows ; row >=1  ; row -- ) {

			// Column level Repetitive Action :
			for(int col = 1; col <= row ; col++) {
				// Action1.Move cursor in the same row
				// Action2.print “*” character
				System.out.print("* ");
			}
			System.out.println();// Move cursor to the next row
		}
	}
}
```

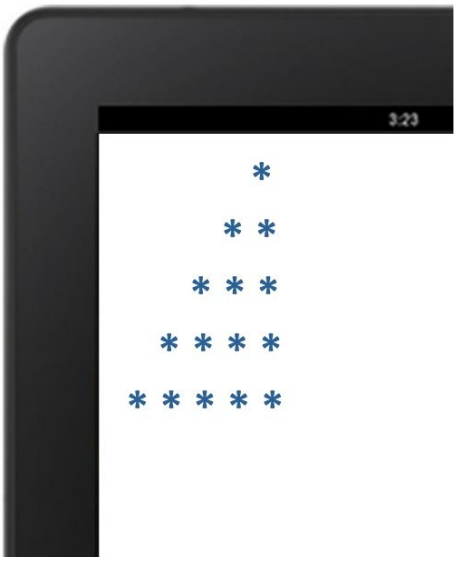
Output

```
* * * * * * * * * *
* * * * * * * * *
* * * * * * * *
* * * * * * *
* * * * * *
* * * * *
* * * *
* * *
* *
*
```

FLOW CHART PROBLEM : The given following flow-chart has every information related to the outer and nested ***for-loop***. Please write a program which suits to the given flow-chart.



Pattern 1C



Our programming job is to print this particular logic-based pattern on a part of the computer screen. **Our approach will be assumed that the computer screen is a combination of rows and columns virtually dividing it into smaller boxes.** The smallest box is treated as a resident of just a single character only. The address in each smallest box on the display screen will logically become a unique **(row, column)** numeric combination. With this assumption computer screen will simply be a unique combination of (row, column) numeric value paired boxes which are uniformly spread all across the screen.

Data Representation

Although logically, we can exactly locate the position of each printable character on the display screen by knowing it's uniquely identifiable **(row, column)** numeric paired-values.

Row, Col	Col = 1	Col = 2	Col = 3	Col = 4	Col = 5
Row = 1					* (1, 5)
Row = 2				* (2, 4)	* (2, 5)
Row = 3			* (3, 3)	* (3, 4)	* (3, 5)
Row = 4		* (4, 2)	* (4, 3)	* (4, 4)	* (4, 5)
Row = 5	* (5, 1)	* (5, 2)	* (5, 3)	* (5, 4)	* (5, 5)

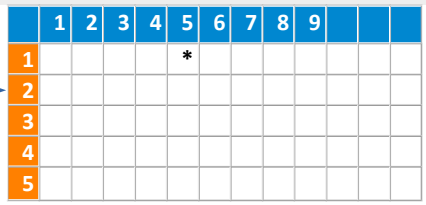
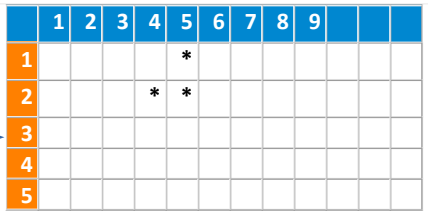
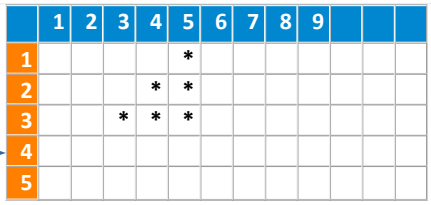
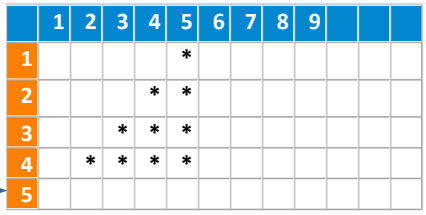
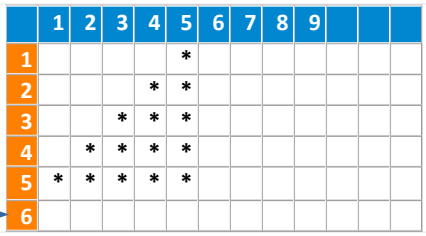
The pair (4, 5) represents a single character * at the assumed position of 4th row and 5th column on the display screen. The set of all value pairs (row = value, col = value) now represent this particular pattern.

Analysis of output

Computer dominantly works on very precise calculations. One of the interesting way to practice programming is to try converting manual steps of writing a given pattern into programming instructions. We can use paper or spreadsheet. To analyze a pattern correctly, we can mark row numbers and column numbers to logically identify position-indicator.

		Col = 1	Col = 2	Col = 3	Col = 4	Col = 5	WRITING ACTION	
ROW	Move to Row = 1					1 → *	4 spaces	Write * 1 time
MOVEMENT	Move to Row = 2				1 → *	2 → *	3 spaces	Write * 2 times
ACTION	Move to Row = 3			1 → *	2 → *	3 → *	2 spaces	Write * 3 times
	Move to Row = 4		1 → *	2 → *	3 → *	4 → *	1 space	Write * 4 times
	Move to Row = 5	1 → *	2 → *	3 → *	4 → *	5 → *	0 space	Write * 5 times

Space is also a character like any other keyboard character. If a word or single non-spacing character is displayed on the screen after a gap, it means one or many space characters has to be print before printing any other non-spacing character(s).

	Steps for pattern generation	Computer Screen
For row = 1	Print space " " 4 times in the same 1st row. Print star "*" once in the same 1st row then move to the next row number 2.	
For row = 2	Print space " " 3 times in the same 2nd row. Print star "*" 2 times in the same 2nd row then move to the next row number 3.	
For row = 3	Print space " " 2 times in the same 3rd row. Print star "*" 3 times in the same 3rd row then move to the next row number 4.	
For row = 4	Print space" " 1 time in the same 4th row. Print star "*" 4 times in the same 4th row then move to the next row number 5.	
For row = 5	Print space" " 0 times in the same 5th row. Print star "*" 5 times in the same 5rd row then move to the next row number 5.	

Let's convert these steps into Java program using following inbuilt methods.	
To move cursor to the next row.	`System.out.println();`
Print space " " character in the same row.	`System.out.print(" ");`
Print star "*" character in the same row.	`System.out.print("*");`

In order to print each space " " or "*" character, each movement on the computer screen will be a single keyboard character sized jump from left to right direction in the same row. These three types of programming statements are required to get repeat many times using looping. Writing a program to calculate the row and column values to print character correctly on the computer screen will be the main reason for improving the logical thinking.

COLUMN LEVEL REPETITIVE ACTION

Let's consider;

row → variable type of integer to store integer value one at a time and represents row number between 1 and 5.

col → variable type of integer to store integer value one at a time and represents column number between 1 and 5.

spaces → variable type of integer to store integer value one at a time and represents space character at column number between 1 and 5.

totalRows → variable type of integer to store integer value for total number of rows.

Logic Trace Tables

(ROW, COL) VALUE PAIR DEPICTS THE POSITION OF "*" CHARACTER ON THE DISPLAY SCREEN				
				row=1, col=5
			row=2, col=4	**row=2**, col=5
		row=3, col=3	**row=3**, col=4	**row=3**, col=5
	row=4, col=2	**row=4**, col=3	**row=4**, col=4	**row=4**, col=5
row=5, col=1	**row=5**, col=2	**row=5**, col=3	**row=5**, col=4	**row=5**, col=5

ROW LEVEL REPETITIVE ACTION				PATTERN
Variable: row	Column Level Repetitive Action			
	Variable : spaces	Variable: col	Character count	
1	1,2,3,4	5	4 Spaces + 1 star	*
2	1,2,3	4, 5	3 Spaces + 2 stars	* *
3	1,2	3, 4, 5	2 Spaces + 3 stars	* * *
4	1	2, 3, 4, 5	1 Spaces + 4 stars	* * * *
5	0(no space)	1, 2, 3, 4, 5	0 Spaces + 5 stars	* * * * *

The space (" ") and star("*") characters are repeated many times. So, based on the count of the space (" ") and star("*") characters we can use ***for-loop*** for each. We can put our initial observation in somewhat more clear form as given in the following table.

ROW LEVEL REPETITIVE ACTION					PATTERN
Variable: row	Column Level Repetitive Action				
	Variable : spaces		Variable: col		
	min_val	max_val	min_val	max_val	
1	1	4	1	1	*
2	1	3	1	2	* *
3	1	2	1	3	* * *
4	1	1	1	4	* * * *
5	0	0	1	5	* * * * *

Approach to solution

The variables are **row** & **totalRows**=5.
We need to generalize the non-repeating numeric pattern of the column **max_val**. The numeric pattern of the **variable col's column max_val** matches exactly the every value of **row**. So all the numeric pattern of **variable col's column max_val** can be expressed in row variable.
Let's try to use variable **totalRows** to generalize the variable **spaces**'s column **max_val.**

Row level Repetitive Action					
Variable : row	Column level Repetitive Action				
	Variable : spaces			Variable : col	
	min_val	min_val		min_val	max_val
		default	1		
1	1	4	totalRows -1	1	row
2	1	3	totalRows - 2	1	row
3	1	2	totalRows - 3	1	row
4	1	1	totalRows - 4	1	row
5	0	0	totalRows - 4	1	row

Part of variable **spaces** column **max_val's** expression follows the same numeric value pattern as **row.** Further we can express it in terms of variables **totalRows & row**.

Row level Repetitive Action					
Variable : row	Column level Repetitive Action				
	Variable : spaces			Variable : col	
	min_val	min_val		min_val	max_val
		default	1		
1	1	4	totalRows - row	1	row
2	1	3	totalRows - row	1	row
3	1	2	totalRows - row	1	row
4	1	1	totalRows - row	1	row
5	0	0	totalRows - row	1	row

We generalized the non-repeating numeric pattern of the column **max_val** into an expression.
We can now define column level ***for-loop*** which is printing spaces for every row value.

Elements Of COLUMN LEVEL For Loop			
Variable: col		Incrementing Order	Action
min_val	max_val	step-size	Print character
spaces = 1	spaces <= totalRows — row	1	space

We can now define column level ***for-loop*** which is printing (*) characters for every row value.

Elements Of COLUMN LEVEL For Loop			
Variable: col		Incrementing Order	Action
min_val	max_val	step-size	Print character
col = 1	col <= row	1	*

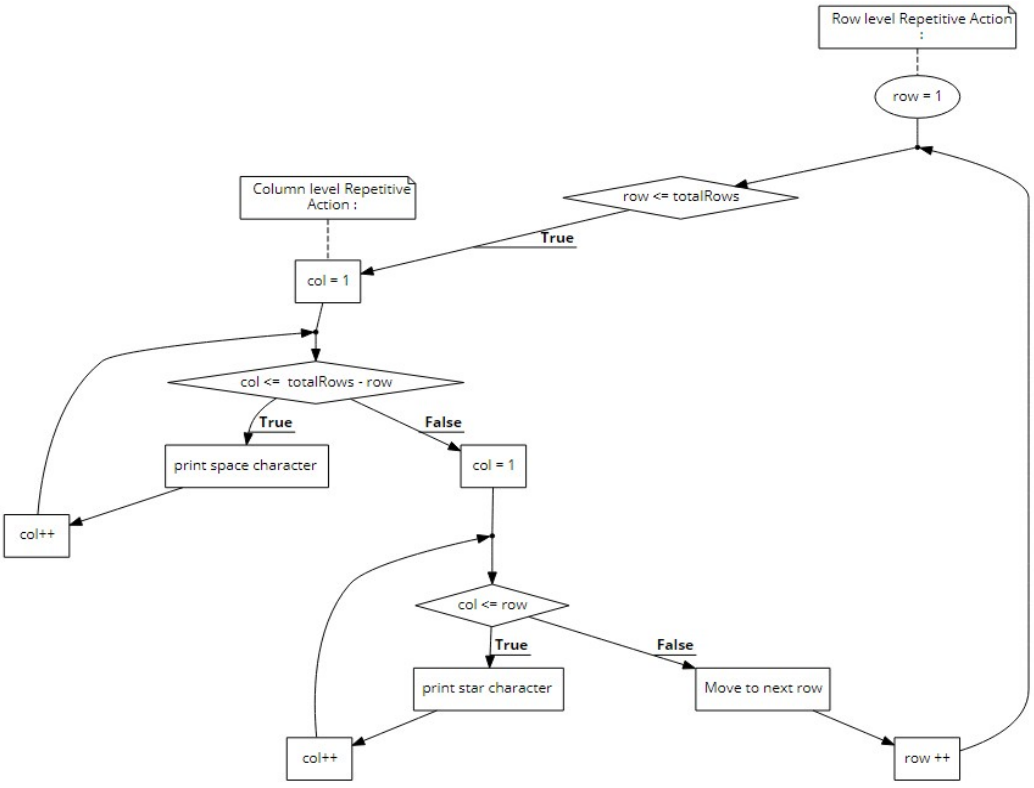
ROW LEVEL REPETITIVE ACTION

The same following statements had been repeated 5 times(total number of rows).
Column level *for-loop* for printing space characters// ***inner for-loop***
Column level *for-loop* for printing star characters// ***inner for-loop***
`System.out.println( );// Move cursor to the next row`
We have assigned unique value to each row starting from 1 till 5. So, it's an easy take now.
The minimum and maximum value can be given for the ***for-loop*** as;
min_val = 1 & max_val = 5 or totalRows

Let's consider;
row → variable type of integer to store integer value one at a time and represents row number between 1 and 5(**totalRows**). The **totalRows** has initially assigned the value 5.

Outer ***for-loop*** starts at row=1 & will repeat till row <= **totalRows**.
We got everything related to ***for-loop*** for row level repetition.

ELEMENTS OF ROW LEVEL FOR LOOP			
Variable: row		INCREMENTAL ORDER	ACTION
start	end	step-size	Statements
row = 1	row = totalRows	1	1# COLUMN LEVEL For Loop for printing spaces 2# COLUMN LEVEL For Loop for printing * 3# Move To Next Row



Let's write complete programming logic and execute the program.

PatternC1.java

```
public class PatternC1{
	public static void main(String[] args) {
		int  totalRows = 5;  //number of rows to display

		// Row level Repetitive Action :
		// Action1.Executes Column level Repetitive Action
		// Action2.Move cursor to next row.
		for( int row = 1 ; row <= totalRows ; row ++ ) {

			// Column level Repetitive Action :
			for(int sp = 1 ; sp <= ( totalRows - row ) ; sp++) {
				// Action1.Move cursor in the same row
				// Action2.print space "  " character
				System.out.print("  ");
			}
			for(int col = 1 ; col <= row  ; col++) {
				// Action1.Move cursor in the same row
				// Action2.print star "* " character
				System.out.print("* ");
			}
			System.out.println();// Move cursor to the next row
		}
	}
}
```

Output

```
        *
      * *
    * * *
  * * * *
* * * * *
```

Alternative solution2

The available variables are **row & totalRows=5**. The variable:col = 1, 2, 3, 4, 5, takes the value 1 to 5. The overall observation is that at each row, either it's printing space characters or star (*) characters in a continues fashion. Based on type of printing the characters the columns grouped into two parts either it's printing space " " character(s) or star("*") characters.

Row level Repetitive Action				Expected Output
Variable: row	Column level Repetitive Action			
	Variable: spaces	Variable: col	Character count	
1	1,2,3,4	5	4 Spaces + 1 star = 5 characters	*
2	1,2,3	4, 5	3 Spaces + 2 stars = 5 characters	* *
3	1,2	3, 4, 5	2 Spaces + 3 stars = 5 characters	* * *
4	1	2, 3, 4, 5	1 Spaces + 4 stars = 5 characters	* * * *
5	0(no space)	1, 2, 3, 4, 5	0 Spaces + 5 stars = 5 characters	* * * * *

The printing of space(s) **comes first** before print star("*") character(s).When care of the condition till it print space character(s),then the remaining are the star("*") character(s). If we closely observe It's cutting the values of variable col into two halves. One part goes to printing of space character(s) and the other goes to printing of star character(s).

Row level Repetitive Action					Expected Output
Variable: row	Column level Repetitive Action				
	Variable: spaces		Variable: col		
	min_val	max_val	min_val	max_val	
1	1	4	5	5	*
2	1	3	4	5	* *
3	1	2	3	5	* * *
4	1	1	2	5	* * * *
5	0	0	1	5	* * * * *

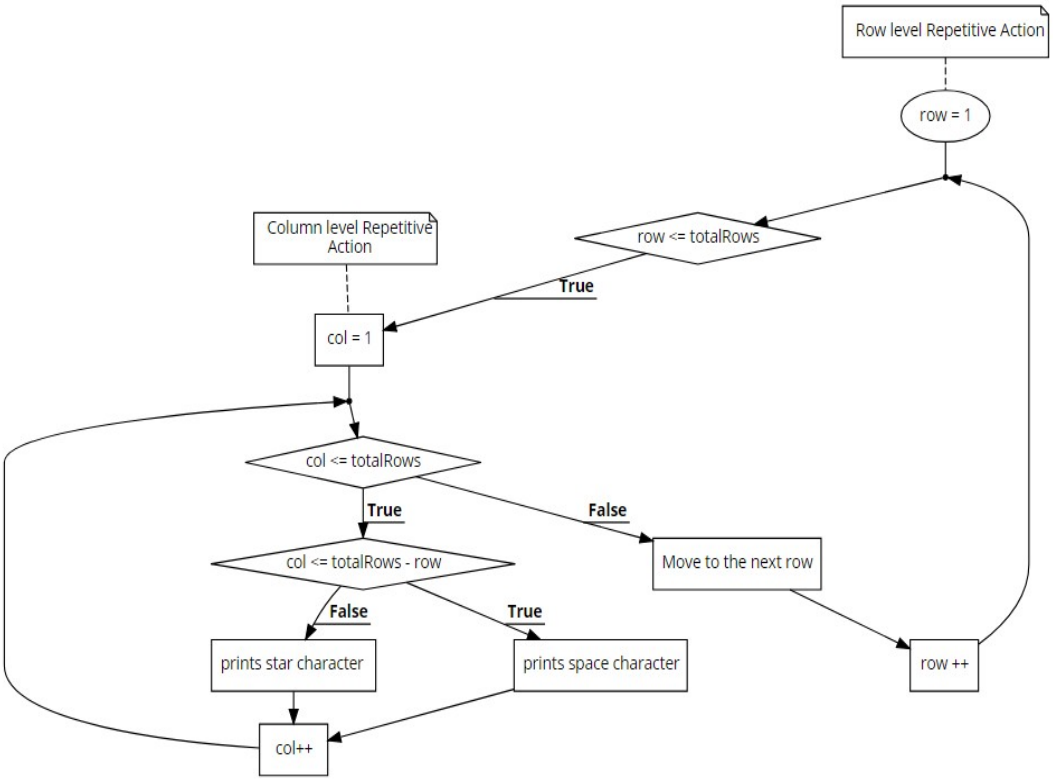
Row level Repetitive Action					Expected Output
Variable: row	Column level Repetitive Action				
	Variable: spaces		Variable: col		
	min_val	max_val	min_val	max_val	
1	1	4 = 5 -1 = totalRows - 1	5	5	*
2	1	3 = 5 – 2 = totalRows - 2	4	5	* *
3	1	2 = 5 – 3 = totalRows - 3	3	5	* * *
4	1	1 = 5 – 4 = totalRows - 4	2	5	* * * *
5	0	0 = 5 – 5 = totalRows - 5	1	5	* * * * *

We partially got generalized logical expression for printing spaces from the previous logic trace table. The column **max_val's** numeric pattern of **variable:spaces** matches with the numeric occurrences of **variable:row** and hence got the logical condition **(totalRows - row)** as shown below.

Row level Repetitive Action				
Variable: row	Column level Repetitive Action			
	Variable: spaces		Variable: col	
	min_val	max_val	min_val	max_val
1	1	totalRows – 1 = **totalRows - row**	5	5
2	1	totalRows – 2 = **totalRows - row**	4	5
3	1	totalRows – 3 = **totalRows - row**	3	5
4	1	totalRows – 4 = **totalRows - row**	2	5
5	0	totalRows – 5 = **totalRows - row**	1	5

The other **for-loop** runs between 1 and (**totalRows =** 5) for **variable: col.** The logical expression (**totalRows - row**) gives the maximum count starting from count 1 to print that many space characters. So, we can use **if-else** for conditional filtering and printing character (space or star).

Print character space when condition is True for every value of variable:col				
col=1	col=2	col=3	col=4	col=5
	If (condition is **True)** Print space character **Else** Print star character	**If** (**col<= totalRows - row**) is **True** Print space character **Else** Print star character		



The updated program with if-else statement is as follows:

PatternC2.java

```
public class PatternC2{

	public static void main(String[] args) {
		int  totalRows = 5;  //number of rows to display

		// Row level Repetitive Action :
		// Action1.Executes Column level Repetitive Action
		// Action2.Move cursor to next row.
		for( int row = 1 ; row <= totalRows ; row ++ ) {
			// Column level Repetitive Action :

			// Action1.Move cursor in the same row
			for(int col = 1 ; col <= totalRows  ; col++) {

				if( col <= ( totalRows - row ) ) {
					// Action2.prints space " " character
					System.out.print("  ");
				}else {
					// Action3.prints star "* " character
					System.out.print("* ");
				}
			}
			System.out.println();// Move cursor to the next row

		}
	}
}
```

Output: totalRows = 5

```
        *
      * *
    * * *
  * * * *
* * * * *
```

Output: totalRows = 10

```
                    *
                  * *
                * * *
              * * * *
            * * * * *
          * * * * * *
        * * * * * * *
      * * * * * * * *
    * * * * * * * * *
  * * * * * * * * * *
```

Now, test for different number of lines, let it be 10

Alternative solution3

From the previous solution, we have checked the **if-condition** to be true to print spaces and in the else part of the **if-statement** we have printed the star("*") characters. In this solution we will reverse situation. When **if-condition** becomes true we will print a star("*") character and in the else part of the **if-else statement** we will print spaces.

Print character star when condition is True for every value of variable:col				
col=1	col=2	col=3	col=4	col=5

If (condition is **True)**
Print star character
Else
Print space character

If (**?**) is **True**
Print star character
Else
Print space character

Let's try to re-learn from given observation table that we had used in the previous solution.

Row level Repetitive Action					Expected Output
Variable: row	Column level Repetitive Action				
	Variable: spaces		Variable: col		
	min_val	max_val	min_val	max_val	
1	1	4	5	5	*
2	1	3	4	5	* *
3	1	2	3	5	* * *
4	1	1	2	5	* * * *
5	0	0	1	5	* * * * *

We know that totalRows = 5 .
The column **min_val** for printing star character(s) can be given as;

Row level Repetitive Action					Expected Output
Variable: row	Column level Repetitive Action				
	Variable: spaces		Variable: col		
	min_val	max_val	min_val	max_val	
1	1	4	5 - 0	5 = totalRows	*
2	1	3	5 - 1	5 = totalRows	* *
3	1	2	5 - 2	5 = totalRows	* * *
4	1	1	5 - 3	5 = totalRows	* * * *
5	0	0	5 - 4	5 = totalRows	* * * * *

If we can try variable totalRows to generalize the numeric pattern of the column **min_val .**
Then it will be totalRows – **0** , totalRows -**1**,….so on. The part of this expression nearly following the numeric pattern of the variable row = 1,2,…,5.The former is having the numeric pattern starting from zero(0) and **variable:row's** numeric pattern starting from one(1). We can start the numeric pattern of **variable:row** from zero(0). In case of row level movements, whether the range is 1 to 5 or 0 to 4, it only means five iterations.

Let's make use of the variables totalRows & row to derive the generalized logical expression for printing character star for the **columns min_val & max_val** given as follows;

Row level Repetitive Action					Expected Output
Variable: row	Column level Repetitive Action				
	Variable: spaces		Variable: col		
	min_val	max_val	min_val	max_val	
0	1	4	totalRows - row	totalRows	*
1	1	3	totalRows - row	totalRows	* *
2	1	2	totalRows - row	totalRows	* * *
3	1	1	totalRows - row	totalRows	* * * *
4	0	0	totalRows - row	totalRows	* * * * *

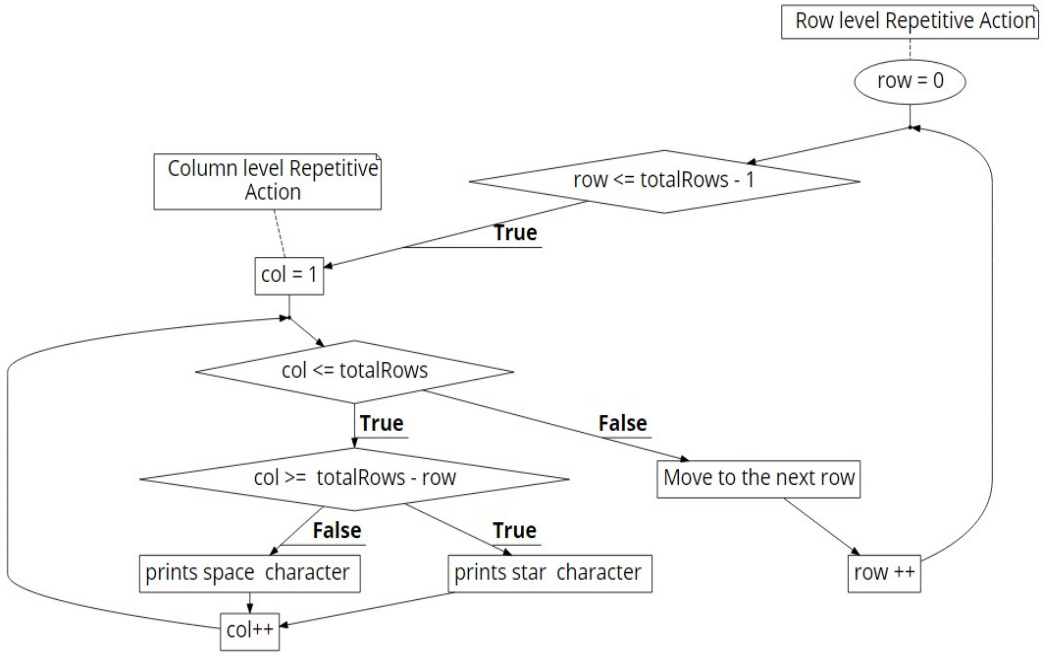
The condition (col >= (**totalRows – row)**) gives the minimum limit, a starting point from which it will start printing star("*") character(s) till the maximum value of variable col which is **totalRows**.
The **if-condition** can be given for in between range of values (**totalRows - row**) and **totalRows** using the comparison and logical operators as:
If (col >= (**totalRows - row**) && col <= **totalRows)** and moreover we already know that;

variable:col = 1,2,3,4,5 or 1,2,3,4, **totalRows**
variable:row = 0,1,2,3,4 or 0,1,2,3,4, **(totalRows - 1) .**

We can eliminate the logical AND(&&) condition**(col <= totalRows) ,**since this condition is already been taken care in the column level inner for-loop. So the if-condition will be reduced to;

If (col >= (**totalRows — row**))

We can summarize it all in the following flow-chart and update it in the program.



Test for total number of lines equals to 7.

PatternC3.java

```
public class PatternC3{

	public static void main(String[] args) {
		int totalRows = 7; //7 rows to display

		// Row level Repetitive Action :
		// Action1.Executes Column level Repetitive Action
		// Action2.Move cursor to next row.
		for( int row = 0 ; row <= totalRows - 1 ; row ++ ) {

			// Column level Repetitive Action :
			// Action1.Move cursor in the same row
			for(int col = 1 ; col <= totalRows  ; col++) {

				if( col >= ( totalRows - row ) ) {
					// Action2.prints star “* ” character
					System.out.print("* ");
				}else {
					// Action3.prints space “ ” character
					System.out.print("  ");
				}
			}
			System.out.println();// Move cursor to the next row
		}
	}
}
```

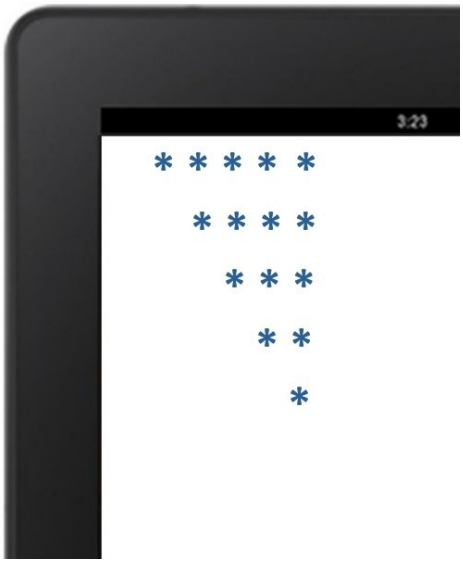
Output

```
            *
          * *
        * * *
      * * * *
    * * * * *
  * * * * * *
* * * * * * *
```

Problem: Write a program to print pattern1C using **decrementing *for-loop*** in the following mentioned two cases.

Case A	Case B
If (**condition = ?** is **True)** Print space character **Else** Print star character	**If** (**condition = ?**) is **True** Print star character **Else** Print space character

Pattern1D



Our programming job is to print this particular logic-based pattern on a part of the computer screen. **Our approach will be assumed that the computer screen is a combination of rows and columns virtually dividing it into smaller boxes.** The smallest box is treated as a resident of just a single character only. The address in each smallest box on the display screen will logically become a unique **(row, column)** numeric combination. With this assumption computer screen will simply be a unique combination of (row, column) numeric value paired boxes which are uniformly spread all over the screen.

Data Representation

Although logically, we can exactly locate the position of each printable character on the display screen by knowing it's uniquely identifiable **(row, column)** numeric paired-values.

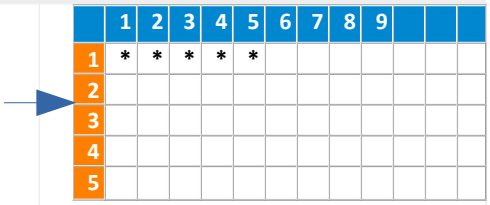
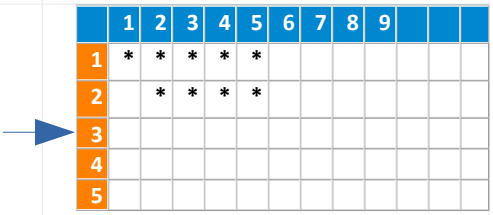
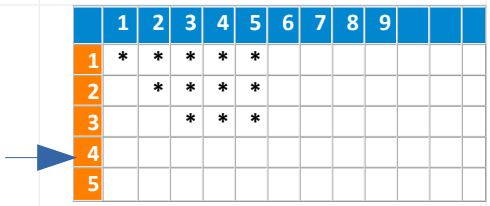
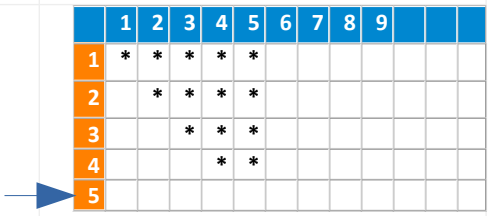
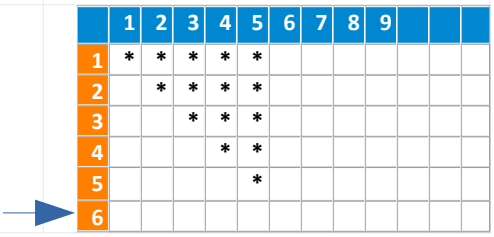
Row, Col	Col = 1	Col = 2	Col = 3	Col = 4	Col = 5
Row = 1	* (1, 1)	* (1, 2)	* (1, 3)	* (1, 4)	* (1, 5)
Row = 2		* (2, 2)	* (2, 3)	* (2, 4)	* (2, 5)
Row = 3			* (3, 3)	* (3, 4)	* (3, 5)
Row = 4				* (4, 4)	* (4, 5)
Row = 5					* (5, 5)

The pair (4, 5) represents a single character * at the assumed position of 4th row and 5th column on the display screen. The set of all value pairs (row = value, col = value) now represent this particular pattern.

Analysis of output

Computer dominantly works on very precise calculations. One of the interesting way to practice programming is to try converting manual steps of writing a given pattern into programming instructions. We can use paper or spreadsheet. To analyze a pattern correctly, we can mark row numbers and column numbers to logically identify position-indicator. ***Space is also a character like any other keyboard character. If a spacing is displayed on the screen, it means one or many space characters has to be print first before printing any other non-blank characters.***

		Col = 1	Col = 2	Col = 3	Col = 4	Col = 5	WRITING ACTION	
ROW	Move to Row = 1	1 → *	2 → *	3 → *	4 → *	5 → *	0 space	Write * 5 times
MOVEMENT	Move to Row = 2		1 → *	2 → *	3 → *	4 → *	1 space	Write * 4 times
ACTION	Move to Row = 3			1 → *	2 → *	3 → *	2 spaces	Write * 3 times
	Move to Row = 4				1 → *	2 → *	3 spaces	Write * 2 times
	Move to Row = 5					1 → *	4 spaces	Write * 1 time

Steps for pattern generation (Total Rows = 5)		Computer Screen
For row = 1	Print "*" five times in the same 1st row then move to row number 2	
For row = 2	Print space " " 1 time in the same 2nd row. Print "*" four times in the same 2nd row then move to row number 3.	
For row = 3	Print space " " 2 times in the same 3rd row. Print "*" thrice in the same 3rd row then move to row number 4.	
For row = 4	Print space " " 3 times in the same 4th row. Print "*" twice in the same 4th row then move to row number 5.	
For row = 5	Print space " " 4 times in the same 5th row. Print "*" once in the same 5th row then move to row number 6.	

Let's convert steps into a program using following inbuilt methods.

To move cursor to the next row	`System.out.println()`
To move cursor and to print space " " character in the same row	`System.out.print(" ")`
To move cursor and to print star "*" character in the same row	`System.out.print("*")`

In order to print each space " " or "*" character, each movement on the computer screen will be a single keyboard character sized jump from left to right direction in the same row. These three types of programming statements are required to get repeat many times in the program. Since looping is meant for repetitive action within a program, hence we need use these printing statements along with the for-loop. Writing a program to calculate the row and column values to print character correctly, into the computer screen will be the main reason for improving the logical thinking.

COLUMN LEVEL REPETITIVE ACTION

Let's consider;

row → variable type of integer to store integer value one at a time and represents row number between 1 and 5.

col → variable type of integer to store integer value one at a time and represents column number between 1 and 5.

spaces → variable type of integer to store integer value one at a time and represents space character at column number between 1 and 5.

totalRows → variable type of integer to store integer value for total number of rows.

Logic Trace Tables

(row, col) value pair depicts the position of "*" character on the display screen				
row=1, col=1	**row=1**, col=2	**row=1**, col=3	**row=1**, col=4	**row=1**, col=4
	row=2, col=2	**row=2**, col=3	**row=2**, col=4	**row=2**, col=4
		row=3, col=3	**row=3**, col=4	**row=3**, col=4
			row=4, col=4	**row=4**, col=4
				row=5, col=5

ROW LEVEL REPETITIVE ACTION				PATTERN
Variable: row	Column Level Repetitive Action			
	Variable : spaces	Variable: col	Character count	
1	0(no space)	1, 2, 3, 4, 5	4 Spaces + 1 star	* * * * *
2	1	2, 3, 4, 5	3 Spaces + 2 stars	* * * *
3	1,2	3, 4, 5	2 Spaces + 3 stars	* * *
4	1,2,3	4, 5	1 Spaces + 4 stars	* *
5	1,2,3,4	5	0 Spaces + 5 stars	*

ROW LEVEL REPETITIVE ACTION					PATTERN
Variable: row	Column Level Repetitive Action				
	Variable : spaces		Variable: col		
	min_val	max_val	min_val	max_val	
1	0	0	1	5	* * * * *
2	1	1	2	5	* * * *
3	1	2	3	5	* * *
4	1	3	4	5	* *
5	1	4	5	5	*

We know that **totalRows** = 5.
If we observe the numeric pattern for variable row (1, 2, 3, 4, and 5) is **matching** the numeric pattern of the column **min_val** of printing star character(s).
Our target is to express our program logic in terms of our chosen variables (**row, col, totalRows**).
Let's use these variables to find generalized expression for printing star character(s).

ROW LEVEL REPETITIVE ACTION					PATTERN
Variable: row	COLUMN LEVEL REPETITIVE ACTION				
	Variable : spaces		Variable: col		
	min_val	max_val	min_val	max_val	
1	0	0	row	totalRows	* * * * *
2	1	1	row	totalRows	* * * *
3	1	2	row	totalRows	* * *
4	1	3	row	totalRows	* *
5	1	4	row	totalRows	*

The condition (col >= **row**) gives the minimum limit ,a starting point from which it will start printing star("*") character(s) till the maximum value of variable col which is **totalRows**.

The if-condition can be given for in between range of values (**row**) and **totalRows** using

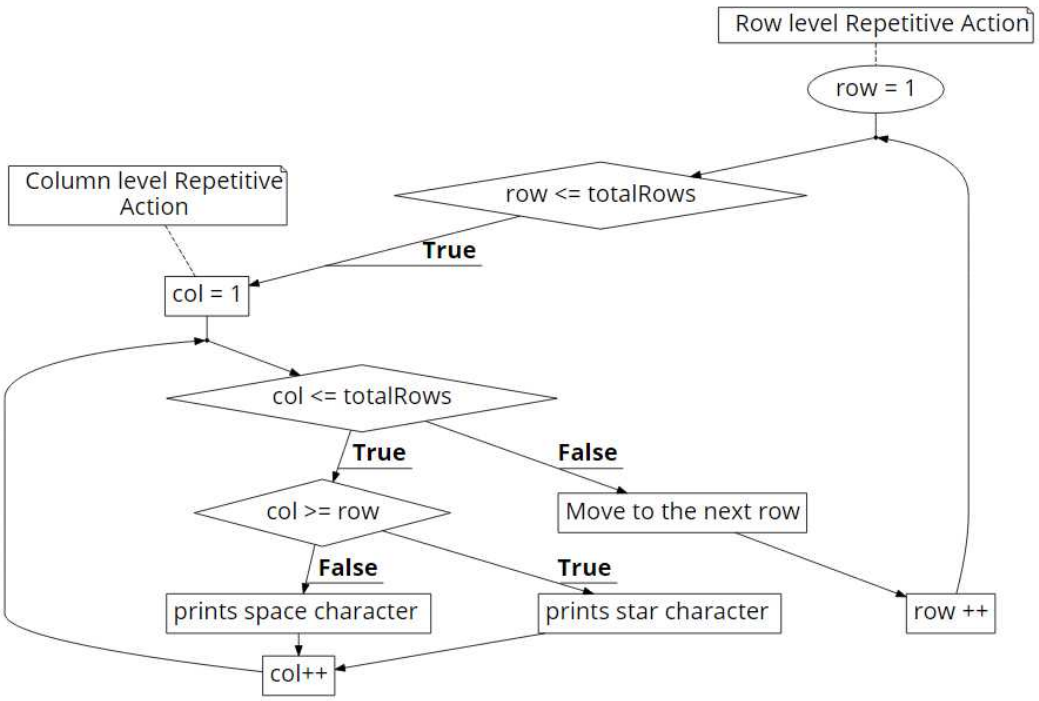
the comparison and logical operators given as follows:

If (col >= **row** **&& col <= totalRows**).

We can eliminate the logical AND(&&) condition(**col <= totalRows**) , since this condition already been taken care in the column level inner for-loop. So the **if-condition** can be given as;

If (col >= **row**)

We summarized our conclusions in a flow-chart which is given as follows:



The program can be written as follows:

PatternD1.java

```
public class PatternD1{

	public static void main(String[] args) {

		int  totalRows = 5;  // total number of rows to display

		// Row level Repetitive Action :
		// Action1.Executes Column level Repetitive Action
		// Action2.Move cursor to next row.
		for( int row = 1 ; row <= totalRows  ; row ++ ) {

			// Column level Repetitive Action :

			// Action1.Move cursor in the same row
			for(int col = 1 ; col <= totalRows  ; col++) {

				if( col >= row  ) {
					// Action2.prints star “* ” character
					System.out.print("* ");
				}else {
					// Action3.prints space “ ” character
					System.out.print("  ");
				}
			}
			System.out.println();// Move cursor to the next row

		}
	}
}
```

Output: totalRows = 5

```
* * * * *
  * * * *
    * * *
      * *
        *
```

Output: totalRows = 10

```
* * * * * * * * * *
  * * * * * * * * *
    * * * * * * * *
      * * * * * * *
        * * * * * *
          * * * * *
            * * * *
              * * *
                * *
                  *
```

Now, test for different number of lines, let it be 10.

Problem D1*:* Write a program to generate the pattern **Pattern1D** by following the condition expression for printing of space characters. When the result of condition expression is **true**, the program should print space character and character star("*") when it's **false**.

Prints a character space when condition is True for every value of variable:col				
col=1	**col=2**	**col=3**	**col=4**	**col=5**
If (condition=(?) is **True)** Print space character **Else** Print star character				

Problem D2. Write a program to generate the pattern **Pattern1D** by using the information given in the mentioned columns. Do not use **variable:col.**

columns →Variable: row, Character count and Total Characters.

The variable totalRows = 5,denotes the number of rows. The program should work for 10 rows also and which should give output by changing the variable **totalRows = 10**.

ROW LEVEL REPETITIVE ACTION					PATTERN
Variable: row	Column Level Repetitive Action				
	Variable : spaces	Variable: col	Character count	Total Characters	
1	0(no space)	1, 2, 3, 4, 5	4 Spaces + 1 star	5	* * * * *
2	1	2, 3, 4, 5	3 Spaces + 2 stars	5	* * * *
3	1,2	3, 4, 5	2 Spaces + 3 stars	5	* * *
4	1,2,3	4, 5	1 Spaces + 4 stars	5	* *
5	1,2,3,4	5	0 Spaces + 5 stars	5	*

Problem D3*:* Write a program to generate the pattern **Pattern1D** using decrementing for-loop. When the result of condition expression is **true**, the program should print space character and character star("*") when it's **false**.

Prints a character space when condition is True for every value of variable:col				
col=1	**col=2**	**col=3**	**col=4**	**col=5**
If (condition=(?) is **True)** Print space character **Else** Print star character				

Pattern1E

Our programming job is to print this particular logic-based pattern on a part of the computer screen. Our approach will be assumed that the computer screen is a combination of rows and columns virtually dividing it into smaller boxes. The smallest box is treated as a resident of just a single character only. The address in each smallest box on the display screen will logically become a unique **(row, column)** numeric combination. Computer screen will simply become a unique combination of **(row, column)** numeric value paired boxes which are spread uniformly all over the screen.



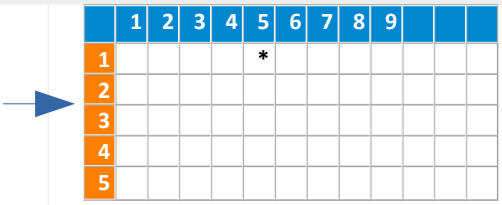
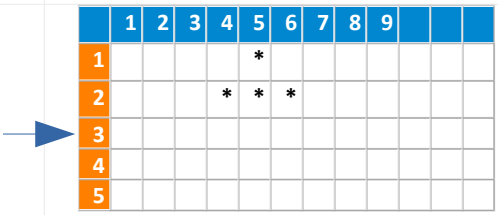
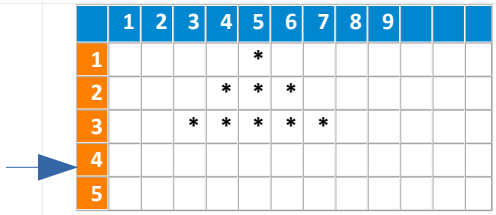
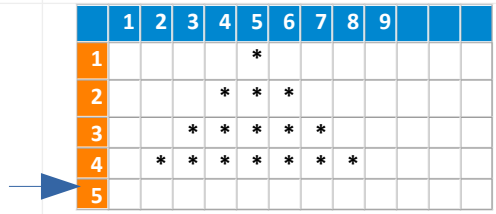
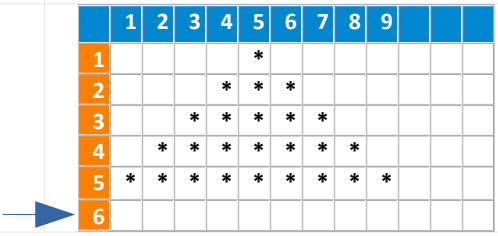
Data Representation

Although logically, we can exactly locate the position of each printable character on the display screen by knowing it's uniquely identifiable **(row, column)** numeric paired-values. For example; **(2, 4)** means character "*" at **2nd row** and **4th column** in the pattern.

The given set of row and col value pairs tries to mimic the location of each printable character of a given pattern on the device's display screen. The set of all value pairs (row = value, col = value) represent this particular pattern.

Row, Col	Col = 1	Col = 2	Col = 3	Col = 4	Col = 5	Col = 6	Col = 7	Col = 8	Col = 9
Row = 1					* (1, 5)				
Row = 2				* (2, 4)	*(2, 5)	* (2, 6)			
Row = 3			*(3, 3)	* (3, 4)	*(3, 5)	*(3, 6)	*(3, 7)		
Row = 4		*(4, 2)	*(4, 3)	*(4, 4)	* (4, 5)	*(4, 6)	*(4, 7)	*(4, 8)	
Row = 5	* (5, 1)	*(5, 2)	*(5, 3)	*(5, 4)	*(5, 5)	* (5, 6)	*(5, 7)	*(5, 8)	*(5, 9)

Space is also a character like any other character. If a non-space character or a text is displayed on the screen after a blank area or gap, it means single or that many space characters has to be print before printing any other non-space character(s).

	Steps for pattern generation	Computer Screen
For row = 1	Print space " " 4 times in the same 1st row. Print star "*" once in the same 1st row then move to the next row number 2.	
For row = 2	Print space " " 3 times in the same 2nd row. Print star "*" 3 times in the same 2nd row then move to the next row number 3.	
For row = 3	Print space " " 2 times in the same 3rd row. Print star "*" 5 times in the same 3rd row then move to the next row number 4.	
For row = 4	Print space" " 1 time in the same 4th row. Print star "*" 7 times in the same 4th row then move to the next row number 5.	
For row = 5	Print space" " 0 times in the same 5th row. Print star "*" 9 times in the same 5rd row then move to the next row number 6.	

Let's convert steps into a program using in-built methods	
To move cursor to the next row	`System.out.println()`
Move cursor to the next (row, column) position and then print space " " character in the same row.	`System.out.print(" ")`
Move cursor to the next (row, column) position and then print star "*" character in the same row.	`System.out.print("*")`

Above three types of programming statements that are expected to get repeat many times in the program. Hence we can use ***for-loop*** for the same.

COLUMN LEVEL REPETITIVE ACTION

Let's consider;

row → variable type of integer to store integer value one at a time and represents row number between 0 and 4.

col → variable type of integer to store integer value one at a time and represents column number between 1 and 5.

spaces → variable type of integer to store integer value one at a time and represents space character at column number between 1 and 5.

totalRows → variable type of integer to store integer value for total number of rows.

Logic Trace Tables

(row, col) value pair depicts the position of "*" character on the display screen								
				row=1, col=5				
			row=2, col=4	**row=2**, col=5	**row=2**, col=6			
		row=3, col=3	**row=3**, col=4	**row=3**, col=5	**row=3**, col=6	**row=3**, col=7		
	row=4, col=2	**row=4**, col=3	**row=4**, col=4	**row=4**, col=5	**row=4**, col=6	**row=4**, col=7	**row=4**, col=8	
row=5, col=1	**row=5**, col=2	**row=5**, col=3	**row=5**, col=4	**row=5**, col=5	**row=5**, col=6	**row=5**, col=7	**row=5**, col=8	**row=5**, col=9

Row level Repetitive Action			Expected Output
Variable: row	Column level Repetitive Action		
	Variable : spaces	Variable: col	
1	1,2,3,4	5	*
2	1,2,3	4, 5,6	* * *
3	1,2	3, 4, 5,6,7	* * * * *
4	1	2, 3, 4, 5,6,7,8	* * * * * * *
5	0(no space)	1, 2, 3, 4, 5,6,7,8,9	* * * * * * * * *

The space (" ") and star ("*") characters are repeated many times.
So, we can use ***for-loop*** for each. We can fill table as follows:

Row level Repetitive Action					Expected Output
Variable : row	Column level Repetitive Action				
	Variable : spaces		Variable : col		
	min_val	max_val	min_val	max_val	
1	1	4	5	5	*
2	1	3	4	6	* * *
3	1	2	3	7	* * * * *
4	1	1	2	8	* * * * * * *
5	0	0	1	9	* * * * * * * * *

We know that **totalRows = 5** (total number of rows).Let's try to use variable **totalRows** to generalize the variable **col**'s columns **min_val** & **max_val**. Also, we need to start the row value from 0 to 4 instead of 1 to 5. This is in order to match the part of the expression of **variable:col's** columns **min_val & max_val**.

Row level Repetitive Action							Expected Output
Variable : row	Column level Repetitive Action						
	Variable : spaces		Variable : col				
	min_val	max_val	min_val		max_val		
0	1	4	totalRows -	0	totalRows +	0	*
1	1	3	totalRows -	1	totalRows +	1	* * *
2	1	2	totalRows -	2	totalRows +	2	* * * * *
3	1	1	totalRows -	3	totalRows +	3	* * * * * * *
4	0	0	totalRows -	4	totalRows +	4	* * * * * * * * *

Using variable **row** to generalize the variable **col**'s columns **min_val** & **max_val.**

Row level Repetitive Action					Expected Output
Variable : row	Column level Repetitive Action				
	Variable : spaces		Variable : col		
	min_val	max_val	min_val	max_val	
0	1	4	totalRows -row	totalRows + row	*
1	1	3	totalRows -row	totalRows + row	* * *
2	1	2	totalRows -row	totalRows + row	* * * * *
3	1	1	totalRows -row	totalRows + row	* * * * * * *
4	0	0	totalRows -row	totalRows + row	* * * * * * * * *

Row level Repetitive Action					Expected Output
Variable : row	Column level Repetitive Action				
	Variable : spaces		Variable : col		
	min_val	max_val	min_val	max_val	
0	1	4	**totalRows - row**	**totalRows + row**	*
1	1	3			* * *
2	1	2			* * * * *
3	1	1			* * * * * * *
4	0	0			* * * * * * * * *

The condition (col >= (**totalRows – row**)) gives the minimum limit, a starting point from which it will start printing star("*") character(s) till the maximum value of variable:col which is (**totalRows + row)**. Then **if-condition** can be given for in between the range of values givens as:

(**totalRows - row**) and (**totalRows + row).**

On applying comparison and logical operators **if-condition** can be given as:

If (col >= (**totalRows - row**) && col <= **(totalRows + row))**

We already know that;

variable:row = 0,1,2,3,4 **or** 0,1,2,3,4, **totalRows - 1**

variable:col = 1,2,3,4,5 **or** 1,2,3,4**, (totalRows + row)**

Why the maximum value of variable:col is kept as dynamic?

We know that the printing star("*") character(s) will continue till the maximum value of variable:**col** which is (**totalRows + row).**

We are not worried about the spaces when printing of all the star("*") character(s) gets completed in a row. It makes sense to restrict the column level looping to the point where the maximum number of printing of star("*") characters reached.

Why the minimum value of variable:col is kept as 1?

It makes sense to start the col=1,2…from the value 1 ,since we have to print space characters before actual task of printing star("*") characters. It is required to count space characters accurately in the beginning of the column level looping.

We can eliminate the logical AND(&&) condition(col <= **(totalRows + row))** in the **if-condition** to print the star("*") characters. It's already planned **to consider** this logical AND(&&) condition as the **loop-condition** already been taken into the column level inner for-loop. So the if-condition will now reduce to the following:

If (col >= (**totalRows - row**))

We can summarize it all as;

Prints a character star when condition is True for every value of variable:col					
col=1	**col=2**	**col=3**	**col=4**	**……**	**col= (totalRows + row)**
If (condition = (col >= (**totalRows - row**)) is **True)** Print star character **Else** Print space character					

We generalized the non-repeating numeric pattern of the column **max_val** into an expression.

We can now define column level ***for-loop*** which is same for every row value.

ELEMENTS OF COLUMN LEVEL FOR LOOP			
Variable: col		INCREMENTAL ORDER	ACTION
start	end	step-size	Print character
col = 1	col <= totalRows + row	1	* or space

ROW LEVEL REPETITIVE ACTION

The following statements had been repeated 5 times(total number of rows).

Column level *for-loop* for printing star characters// ***inner for-loop***

```
System.out.println( );// Move cursor to the next row
```

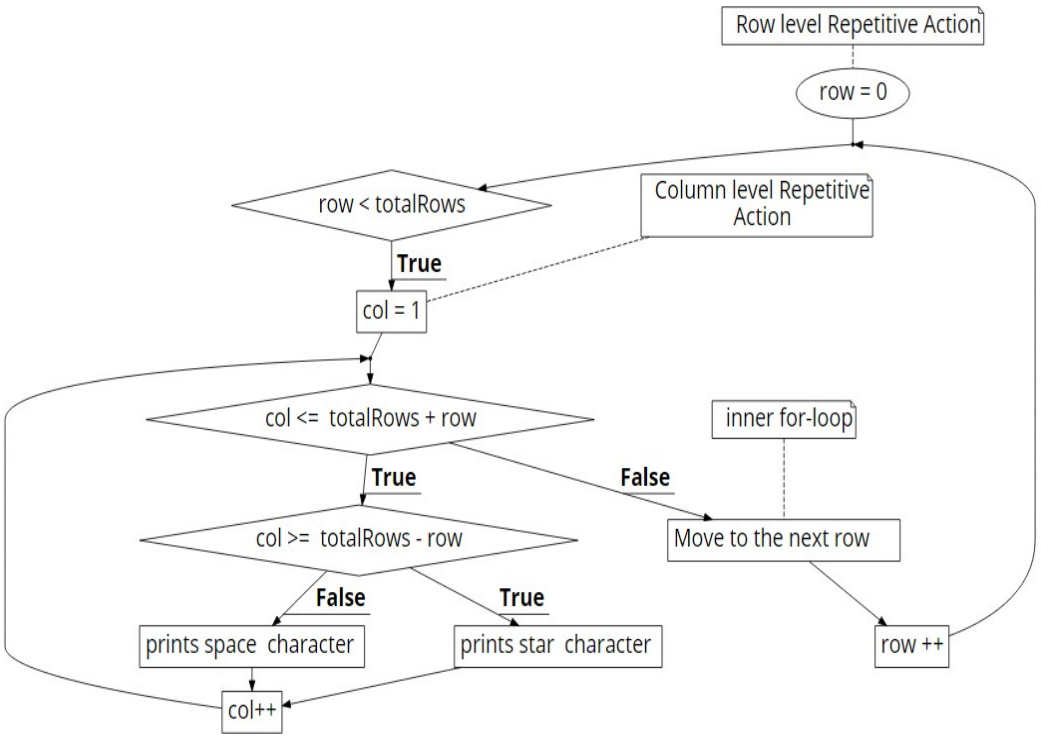
We have assigned unique value to each row starting from 0 till 4.

Let's consider;

row → variable type of integer to store integer value one at a time and represents row number between 0 and 4(totalRows - 1). The **totalRows** has initially assigned the value 5.

ELEMENTS OF ROW LEVEL FOR LOOP				
Variable: row		INCREMENTAL ORDER	ACTION	
start	end	step-size	Statements	
row = 0	row = totalRows - 1	1	1#	COLUMN LEVEL For Loop for printing space or star(*) characters.
			2#	Move To Next Row

We got everything related to ***for-loop*** for row level repetition and related flow-chart can be shown.



Let's execute the following program.

PatternE1.java

```
public class PatternE1 {

	public static void main(String[] args) {

		int  totalRows = 5;  // total number of rows to display

		// Row level Repetitive Action :
		// Action1.Executes Column level Repetitive Action
		// Action2.Move cursor to next row.
		for( int row = 0 ; row < totalRows  ; row ++ ) {

			// Column level Repetitive Action :

			// Action1.Move cursor in the same row
			for(int col = 1 ; col <= ( totalRows + row ) ; col++) {

				if( col >= ( totalRows - row )  ) {
					// Action2.prints star "* " character
					System.out.print("* ");
				}else {
					// Action3.prints space " " character
					System.out.print("  ");
				}

			}// inner for-loop

			System.out.println();// Move cursor to the next row

		}// outer for-loop
	}
}
```

Output

```
        *
      * * *
    * * * * *
  * * * * * * *
* * * * * * * * *
```

Alternative solution2

From the previous solution, when the **if-condition** is true we will print a star("*") character and in the else part of the **if-statement** we will print spaces. Let's start with the observation table that we got from the previous solution.

Row level Repetitive Action					Expected Output
Variable : row	Column level Repetitive Action				
	Variable : spaces		Variable : col		
	min_val	max_val	min_val	max_val	
1	1	4	5	5	*
2	1	3	4	6	* * *
3	1	2	3	7	* * * * *
4	1	1	2	8	* * * * * * *
5	0	0	1	9	* * * * * * * * *

Reversing the counter values of variable row, i.e. row = 5, 4, 3, 2, 1 so that it will match the column **min_val** of the variable **col.** Moreover, we can use the **variable:totalRows** to generalize the column **max_val** for printing star character(s).

Row level Repetitive Action					
Variable : row	Column level Repetitive Action				
	Variable : spaces		Variable : col (For printing *)		
	min_val	max_val	min_val	max_val	
5	1	4	row	5 = 5 + 0	totalRows + 0
4	1	3	row	6 = 5 + 1	totalRows + 1
3	1	2	row	7 = 5 + 2	totalRows + 2
2	1	1	row	8 = 5 + 3	totalRows + 3
1	0	0	row	9 = 5 + 4	totalRows + 4

Part of **variable:col's column max_val's** expression is still having distinct numeric value at each row.

Row level Repetitive Action				
Variable : row	Column level Repetitive Action			
	Variable : col (For printing *)			
	min_val	max_val		
5	row	5 = 5 + 0	totalRows + 0	totalRows + 5 - 5
4	row	6 = 5 + 1	totalRows + 1	totalRows + 5 - 4
3	row	7 = 5 + 2	totalRows + 2	totalRows + 5 - 3
2	row	8 = 5 + 3	totalRows + 3	totalRows + 5 - 2
1	row	9 = 5 + 4	totalRows + 4	totalRows + 5 - 1

Part of **variable:col's column max_val's** expression is still having distinct numeric value at each row. Again, we can use the **variable:totalRows** to generalize the column **max_val.**

Row level Repetitive Action				
Variable : row	Column level Repetitive Action			
	Variable : col (For printing *)			
	min_val	max_val		
5	row	totalRows + 0	totalRows + 5 - 5	totalRows + totalRows - 5
4	row	totalRows + 1	totalRows + 5 - 4	totalRows +totalRows - 4
3	row	totalRows + 2	totalRows + 5 - 3	totalRows + totalRows - 3
2	row	totalRows + 3	totalRows + 5 - 2	totalRows + totalRows - 2
1	row	totalRows + 4	totalRows + 5 - 1	totalRows + totalRows - 1

Part of variable **col's column max_val's** expression matches with the row values.

Row level Repetitive Action				
Variable : row	Column level Repetitive Action			
	Variable : col (For printing *)			
	min_val	max_val		
5	row	totalRows + totalRows - 5	2 * totalRows - row	2 * totalRows - row
4	row	totalRows +totalRows - 4	2 * totalRows - row	
3	row	totalRows + totalRows - 3	2 * totalRows - row	
2	row	totalRows + totalRows - 2	2 * totalRows - row	
1	row	totalRows + totalRows - 1	2 * totalRows - row	

Row level Repetitive Action		
Variable : row	Column level Repetitive Action	
	Variable : col (For printing *)	
	min_val	max_val
5	**row**	**2 * totalRows - row**
4		
3		
2		
1		

The condition (col >= **row**) gives starting point from which ***for-loop*** will start printing star("*") character(s) till the maximum value of variable col which is **(**2 * **totalRows - row**) for every value of row. The if-condition can be given for between the range of values (**row**) and **(**2 * **totalRows - row**) using the comparison and logical operators as:

If (col >= (**row**) && col <= **(**2 * **totalRows - row))**

We already know that;

variable col = 1,2,3,4,5 or 1,2,3,4, **…,(**2 * **totalRows - row)**

Same expression

variable row = 5,4,3,2,1 or **totalRows**,4,3,2,1

We can eliminate the logical AND(&&) condition(col <= **(**2 * **totalRows - row))** in the **if-condition.** This logical AND(&&) condition already been taken care as the **loop-condition** in the column level inner for-loop.

So the if-condition will now reduce to If (col >= **row**).We can summarise it all as;

Prints a star when condition is True for every value of variable col					
col=1	**col=2**	**col=3**	**col=4**	**……**	**col= (2 * totalRows - row)**
If (condition=(col >= **row**) is **True)** Print star character **Else** Print space character					

We generalized the non-repeating numeric pattern of the column **max_val** into an expressions.

We can now define column level ***for-loop***.

Elements of **Column level** ***for-loop***	
min_val	**1**
max_val	**(2 * totalRows - row)**
variable_name	**col**
statement(s)	System.out.print("*");

ROW LEVEL REPETITIVE ACTION

The following statements had been repeated 5 times(total number of rows).

Column level *for-loop* for printing star characters// ***inner for-loop***

```
System.out.println( );// Move cursor to the next row
```

We have assigned unique value to each row starting from 5 till 1.

Let's consider;

row → variable type of integer to store integer value one at a time and represents row number between 5 and 1. The **totalRows** has initially assigned the value 5.

We got everything related to ***for-loop*** for row level repetition.

Elements Of **Row Level decrementing** ***for-loop***	
min_val	**1**
max_val	**totalRows**
variable_name	**row**
statement(s)	**Column level *for-loop*** for printing star characters System.out.println( );//Moves cursor to the next row

Let’s execute the following program.

PatternE2.java

```
public class PatternE2 {

	public static void main(String[] args) {

		int  totalRows = 5;  // total number of rows to display

		// Row level Repetitive Action :
		// Action1.Executes Column level Repetitive Action
		// Action2.Move cursor to next row.
		for( int row = totalRows ; row >= 1 ; row -- ) {

			// Column level Repetitive Action :

			// Action1.Move cursor in the same row
			for(int col = 1 ; col <= ( 2 * totalRows  - row ) ; col++) {

				if( col >= row  ) {
					// Action2.prints star “* ” character
					System.out.print("* ");
				}else {
					// Action3.prints space “ ” character
					System.out.print("  ");
				}

			}// inner for-loop

			System.out.println();// Move cursor to the next row

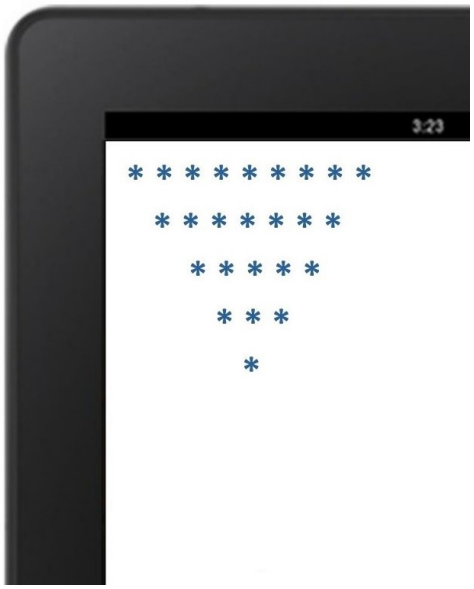
		}// outer for-loop
	}
}
```

Output

```
        *
      * * *
    * * * * *
  * * * * * * *
* * * * * * * * *
```

Pattern 1F

Our programming job is to output this pattern on a part of the computer screen as shown in the following. We can consider the computer screen as a combination of (**row, column**) numeric value paired boxes which are spread all over the screen. Each box is treated as a resident of just a single character only.



Data Representation

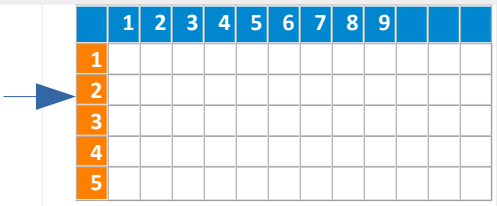
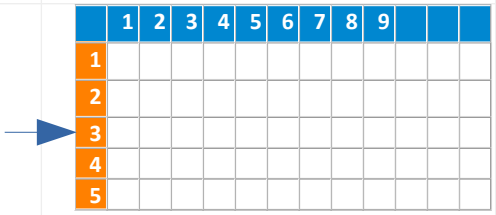
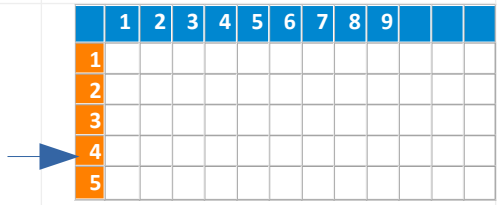
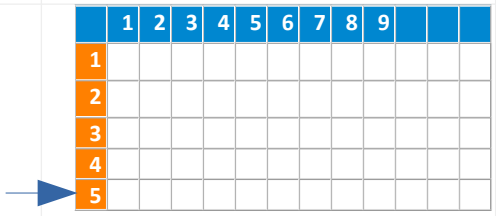
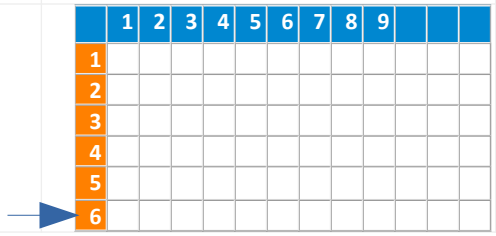
Although logically, we can exactly locate the position of each printable character on the display screen by knowing it's uniquely identifiable **(row, column)** numeric paired-values. For example; **(2, 4)** means character "*" at **2nd row** and **4th column** in the pattern.

The given set of row and col value pairs tries to mimic the location of each printable character of a given pattern on the device's display screen. The set of all value pairs (row = value, col = value) represent this particular pattern.

Row, Col	Col = 1	Col = 2	Col = 3	Col = 4	Col = 5	Col = 6	Col = 7	Col = 8	Col = 9
Row = 1	* (1, 1)	*(1, 2)	*(1, 3)	*(1, 4)	*(1, 5)	* (1, 6)	*(1, 7)	*(1, 8)	*(1, 9)
Row = 2		*(2, 2)	*(2, 3)	*(2, 4)	* (2, 5)	*(2, 6)	*(2, 7)	*(2, 8)	
Row = 3			*(3, 3)	* (3, 4)	*(3, 5)	*(3, 6)	*(3, 7)		
Row = 4				* (4, 4)	*(4, 5)	*(4, 6)			
Row = 5					*(5, 5)				

The actual challenge is to find correct (row, col) positions on the computer screen by writing a program. This will be carried out by applying for-loop along with appropriate logical conditions. This stimulates growth for algorithmic thinking, a fundamental requirement of a confident programmer.

Exercise. Complete the steps for the generation of this logic pattern for every row 1 to 5 by showing output on the computer screen diagram.

Steps for pattern generation(row = 1 to 5)	**Computer Screen**
For row = 1	
For row = 2	
For row = 3	
For row = 4	
For row = 5	

Let's convert steps into a program using inbuilt methods.

Move to the next row.	**System.out.println( )**
Move to the next (row, col) position and print space " " character in the same row	**System.out.print (" ")**
Move to the next (row, col) position and print star "*" character in the same row	**System.out.print ("*")**

The task of printing characters to generate this particular pattern is repetitive in nature. To output these characters on the computer screen repeatedly, mentioned method calls bound to get repeat many a time within ***for-loop*** in a program.

COLUMN LEVEL REPETITIVE ACTION

Let's consider;

row → variable type of integer to store integer value one at a time and represents row number between 0 and 4.

col → variable type of integer to store integer value one at a time and represents column number between 1 and 5.

spaces → variable type of integer to store integer value one at a time and represents space character at column number between 1 and 5.

totalRows → variable type of integer to store integer value for total number of rows.

Logic Trace Tables

(row, col) value pair depicts the position of "*" character on the display screen								
row=1, col1	**row=1**, col2	**row=1**, col3	**row=1**, col4	**row=1**, col5	**row=1**, col6	**row=1**, col7	**row=1**, col8	**row=1, col9**
space	**row=2**, col2	**row=2**, col3	**row=2**, col4	**row=2**, col5	**row=2**, col6	**row=2**, col7	**row=2**, col8	
space	space	**row=3**, col3	**row=3**, col4	**row=3**, col5	**row=3**, col6	**row=3**, col7		
space	space	space	**row=4**, col4	**row=4**, col5	**row=4**, col6			
space	space	space	space	**row=5**, col5				

The space (" ") and star("*") characters are repeated many times.
So, we can use ***for-loop*** for space (" ") and star("*") characters.

Row level Repetitive Action			Expected Output
Variable: row	Column level Repetitive Action		
	Variable : spaces	Variable: col	
1	0(no space)	1, 2, 3, 4, 5,6,7,8,9	* * * * * * * * *
2	1	2, 3, 4, 5,6,7,8	* * * * * * *
3	1,2	3, 4, 5,6,7	* * * * *
4	1,2,3	4, 5,6	* * *
5	1,2,3,4	5	*

We can fill previous table as follows also in order to explore various components of ***for-loop.***

Row level Repetitive Action					Expected Output
Variable : row	Column level Repetitive Action				
	Variable : spaces		Variable : col		
	min_val	max_val	min_val	max_val	
1	0	0	1	9	* * * * * * * * *
2	1	1	2	8	* * * * * * *
3	1	2	3	7	* * * * *
4	1	3	4	6	* * *
5	1	4	5	5	*

We know that totalRows=5(total number of rows).

Let's try to use variable **totalRows** to generalize the variable **col**'s columns are **min_val** & **max_val.**

Row level Repetitive Action					Expected Output
Variable : row	Column level Repetitive Action				
	Variable : spaces		Variable : col		
	min_val	max_val	min_val	max_val	
1	0	0	totalRows -4	totalRows + 4	* * * * * * * * *
2	1	1	totalRows -3	totalRows + 3	* * * * * * *
3	1	2	totalRows -2	totalRows + 2	* * * * *
4	1	3	totalRows -1	totalRows + 1	* * *
5	1	4	totalRows -0	totalRows + 0	*

Let use decrementing ***for-loop*** with range 4 to at the row level repetition.
This is in order to match the part of the expression of variable **col**'s columns **min_val** & **max_val.**
Using variable **row** to generalize the variable **col**'s columns **min_val** & **max_val.**

Row level Repetitive Action						
Variable : row	Column level Repetitive Action					
	Variable : spaces		Variable : col			
	min_val	max_val	min_val		max_val	
4	0	0	totalRows - row		totalRows + row	
3	1	1	totalRows - row		totalRows + row	
2	1	2	totalRows - row	**totalRows - row**	totalRows + row	**totalRows + row**
1	1	3	totalRows - row		totalRows + row	
0	1	4	totalRows - row		totalRows + row	

The condition (col >= (**totalRows – row**)) gives the minimum limit, a starting point from which it will start printing star("*") character(s) till the maximum value of variable col which is (**totalRows + row)**.

The **if-condition** can be given for between the range of values

(**totalRows - row**) and (**totalRows + row)** using the comparison and logical operators as:

If (col >= (**totalRows - row**) && col <= **(totalRows + row))**

We already know that;

Variable col = 1,2,3,4,…,9 or 1,2,3,4,**…., (totalRows + row)**

Same expression

Variable row = 4,3,2,1,0 or (totalRows – 1),3,2,1,0

We can eliminate the logical AND(&&) condition(col <= **(totalRows + row))** in the

if-condition to print the star("*") characters.This logical AND(&&) condition already been **considered** as the upper_limit in the **loop-condition** for column level inner for-loop. So the **if-condition** will now reduce to the following.

If (col >= (**totalRows - row**))It will allow to print star("*") character when result of **if-condition's expression** is **true** or else will prints the space character. We generalized the non-repeating numeric pattern of the column **max_val** into an expressions. We can now define column level ***for-loop***.

Elements of Column level *for-loop*	
min_val	**1**
max_val	**totalRows + row**
variable_name	**col**
statement(s)	System.out.print("*");

Row Level Repetitive Action

The following statements had been repeated 5 times(total number of rows).

Column level *for-loop* for printing star characters**//** ***inner for-loop***

```
System.out.println( );// Move cursor to the next row
```

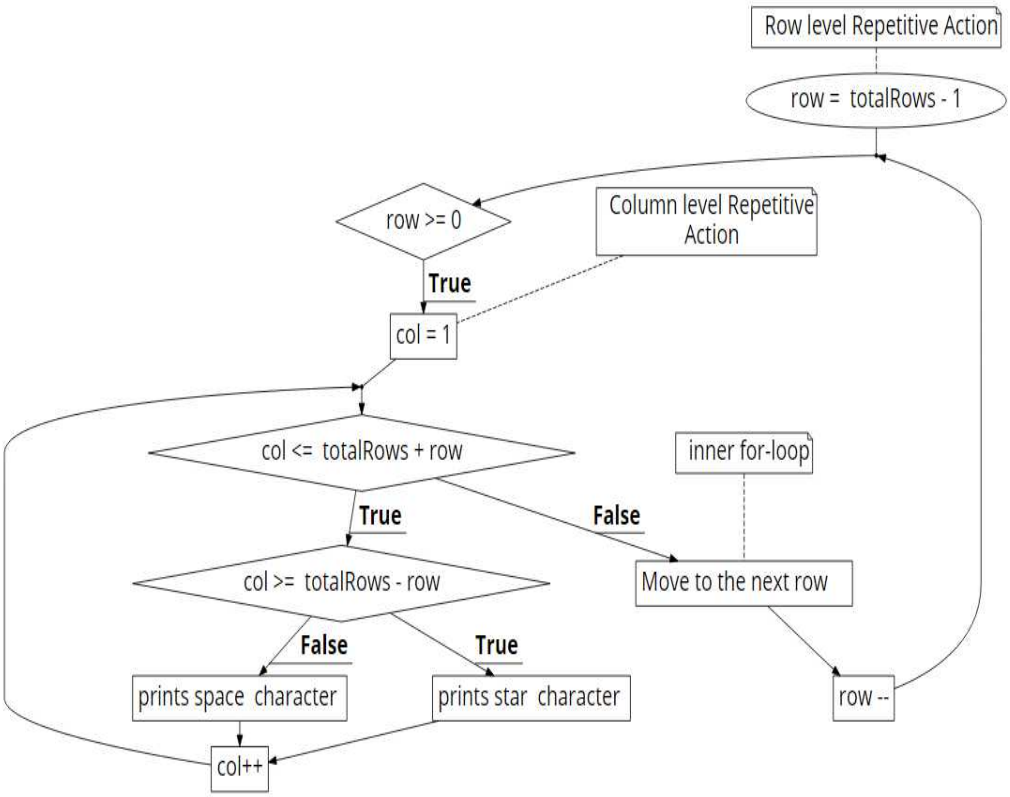
row → variable type of integer to store integer value one at a time and represents

row number between 4 or (**totalRows - 1**) and 0. The **totalRows** has initially assigned the value 5.

We got everything related to ***for-loop*** for row level repetition.

Elements Of Row Level decrementing *for-loop*	
min_val	**0**
max_val	**totalRows - 1**
variable_name	**row**
statement(s)	**Column level *for-loop*** for printing star characters System.out.println( ); // Moveto the next row

The flow-chart can be given as follows:



Let’s execute the following program.

PatternF1.java

```
public class PatternF1 {

	public static void main(String[] args) {

		int  totalRows = 5;  // total number of rows to display

		// Row level Repetitive Action :

		// Action1.Executes Column level Repetitive Action
		// Action2.Move cursor to next row.
		for( int row = ( totalRows - 1) ; row >= 0   ; row -- ) {

			// Column level Repetitive Action :

			// Action1.Move cursor in the same row
			for(int col = 1 ; col <= ( totalRows + row ) ; col++) {

				if( col >= ( totalRows - row )  ) {
					// Action2.prints star “* ” character
					System.out.print("* ");
				}else {
					// Action3.prints space “ ” character
					System.out.print("  ");
				}

			}// inner for-loop

			System.out.println();// Move cursor to the next row

		}// outer for-loop
	}
}
```

Output

```
* * * * * * * * *
  * * * * * * *
    * * * * *
      * * *
        *
```

Alternative solution2

By using incrementing ***for-loop*** at the row level repetition we can give alternative solutions.

***Problem* F1**.Find the different logic and write the program to print the following pattern for 5 rows. Test the logic for 12 rows. Some of the details are mentioned in the form of table to start with.

Row level Repetitive Action					Expected Output
Variable : row	Column level Repetitive Action				
	Variable : spaces		Variable : col		
	min_val	max_val	min_val	max_val	
1	0	0	1	9	* * * * * * * * *
2	1	1	2	8	* * * * * * *
3	1	2	3	7	* * * * *
4	1	3	4	6	* * *
5	1	4	5	5	*

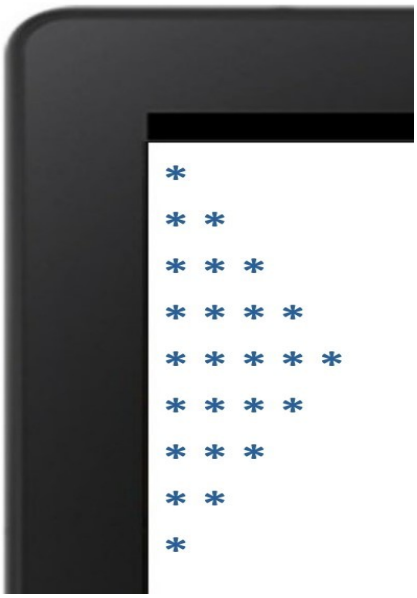
When the **if-condition** is true program should print a star("*") character and in the **else** part of the **if-statement** program should print space character.

Problem* F2**.Continuation to previous problem (Problem* F1**) write the program for the following situation.

When the **if-condition** is true, then program should print a space(" ") character and in the **else** part of the **if-statement** program should print star("* ")character.

Pattern 1G

Our programming job is to output this pattern on a part of the computer screen as shown in the following. We can treat computer screen as a combination of (**row, column**) numeric value paired boxes which are spread all over the screen. Each box can hold a single character only.



Data Representation

The complete pattern can be easily expressed as a collection of selected (**row, column**) numeric value pairs. The first value represents the row number and second value represents a column number. For example;

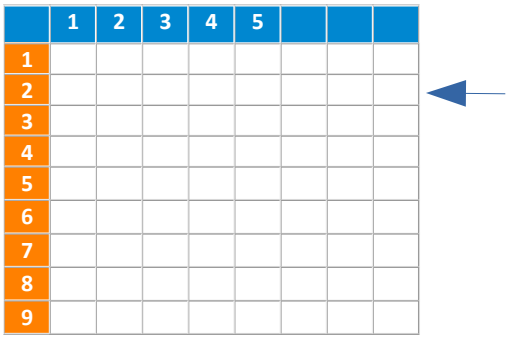
(2, 1) means character "*" at **second row** and **first column** in the table.

The real challenge is to find the correct (row, col) positions and to print required character on the computer screen by writing a program. This will be carried out by applying ***for-loop*** in combination with appropriate logical conditions. Finding logical conditions in a step-by-step approach stimulates the growth for algorithmic thinking, a fundamental requirement to become a confident programmer.

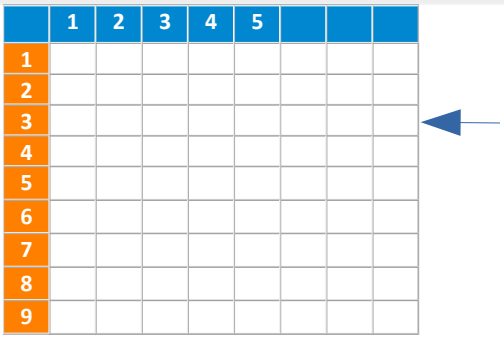
	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	* (1,1)				
Row=2	*** (2 ,1)**	*(2,2)			
Row=3	* (3,1)	*(3,2)	*(3,3)		
Row=4	***(4,1)**	*(4,2)	***(4,3)**	*(4,4)	
Row=5	*(5,1)	*(5,2)	*(5,3)	*(5,4)	*(5,5)
Row=6	***(6,1)**	*(6,2)	***(6,3)**	*(6,4)	
Row=7	*(7,1)	*(7,2)	*(7,3)		
Row=8	***(8,1)**	*(8,2)			
Row=9	*(9,1)				

Exercise. Print logic pattern step-by-step from row 1 to 9 by showing output similar like in the computer screen.

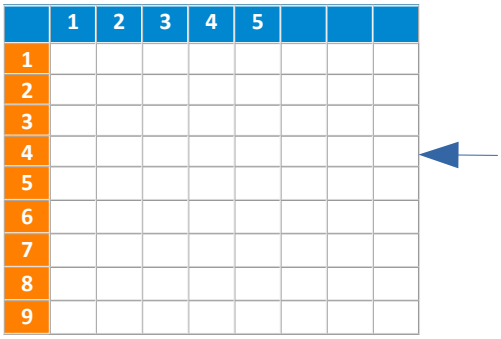
Steps for pattern generation(row = 1 to 9) with computer screen diagram.



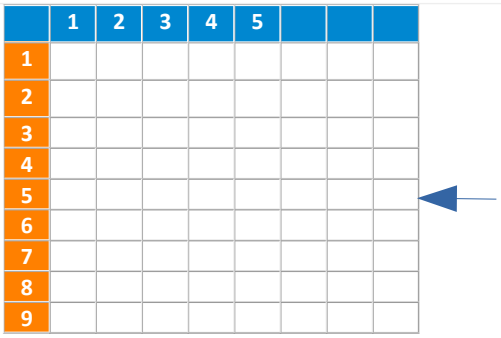
For row = 1



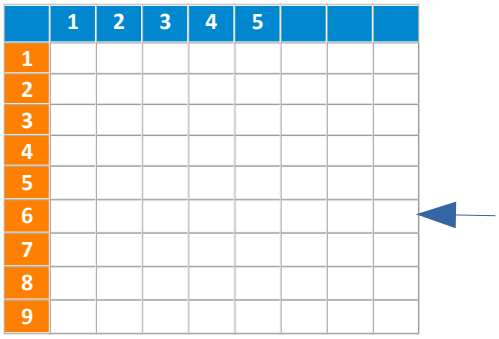
For row = 2



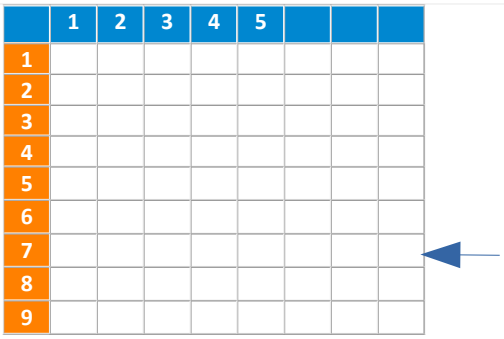
For row = 3



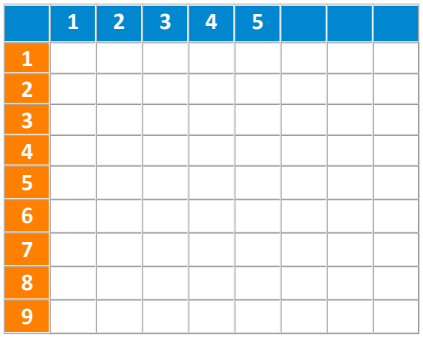
For row = 4



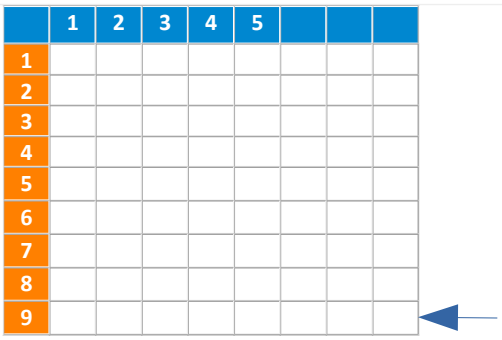
For row = 5



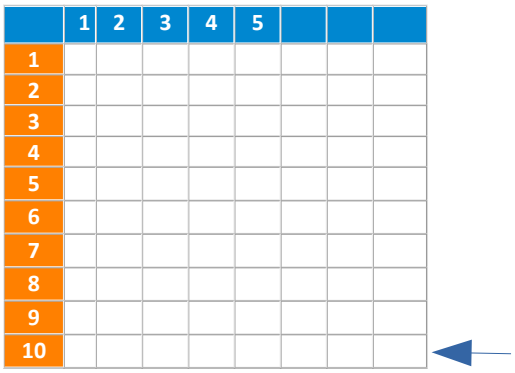
For row = 6



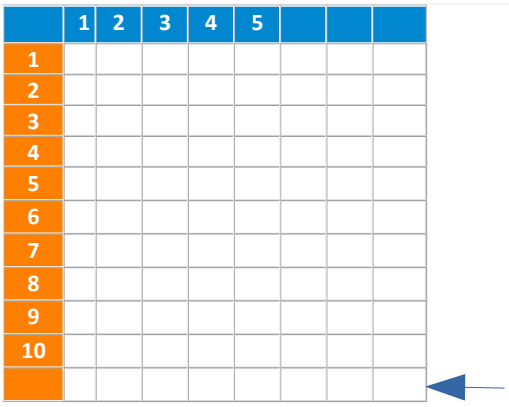
For row = 7



For row = 8



For row = 9



For row = 10

Let's convert steps into a program using in-built methods.	
Move cursor to the next row	**System.*out*.println();**
Move cursor to the next (row, column) position and then print star "*" character in the same row.	**System.*out*.print("*");**

In order to print star "*" character, each movement on the computer screen will be a single keyboard character sized jump from left to right direction in the same row. Since looping is meant for repetitive action within a program, hence we need to use these printing statements along with the ***for-loop***. Writing a program to calculate the row and column values to print character correctly, into the computer screen will be the actual drill for improving logical thinking capabilities.

Let's consider following variables which are type of integer to store integer value one at a time.

row → represents row number between 1 and 9.

col → represents column number between 1 and 5.

totalRows → represents input value in the program for total number of rows.

COLUMN LEVEL REPETITIVE ACTION

We need to explore various components of required **outer and inner *for-loops.***

(row, col) value pair depicts the position of "*" character on the display screen				
row=1, col=1				
row=2, col=1	**row=2**, col=2			
row=3, col=1	**row=3**, col=2	**row=3**, col=3		
row=4, col=1	**row=4**, col=2	**row=4**, col=3	**row=4**, col=4	
row=5, col=1	**row=5**, col=2	**row=5**, col=3	**row=5**, col=4	**row=5, col=5**
row=6, col=1	**row=6**, col=2	**row=6**, col=3	**row=6**, col=4	
row=7, col=1	**row=7**, col=2	**row=7**, col=3		
row=8, col=1	**row=8**, col=2			
row=9, col=1				

We required a set of **min_val** & **max_val** boundary values so that looping shall be performed.

Row level Repetitive Action		Expected Output
Variable: row	Column level Repetitive Action Variable: col	
1	1	*
2	1,2	* *
3	1,2,3	* * *
4	1,2,3,4	* * * *
5	1,2,3,4,5	* * * * *
6	1,2,3,4	* * * *
7	1,2,3	* * *
8	1,2	* *
9	1	*

Row level Repetitive Action			Expected Output
Variable: row	Column level Repetitive Action Variable: col		
	min_val	max_val	
1	1	1	*
2	1	2	* *
3	1	3	* * *
4	1	4	* * * *
5	1	5	* * * * *
6	1	4	* * * *
7	1	3	* * *
8	1	2	* *
9	1	1	*

Approach to solution

The variables are row & totalRows=5.We need to generalize the non-repeating numeric pattern of the column **max_val** using these remaining variables. The numeric pattern (1 to 5) of the column **max_val** matches exactly the every value of row. Hence, it can be expressed in row variable.

Row level Repetitive Action					Expected Output
Variable: row		Column level Repetitive Action			
		Variable: col			
		min_val	max_val		
1	1	1	1	row	*
2	2	1	2	row	* *
3	3	1	3	row	* * *
4	4	1	4	row	* * * *
5	totalRows	1	5	row	* * * * *
6	6	1	4	totalRows - 1	* * * *
7	7	1	3	totalRows - 2	* * *
8	8	1	2	totalRows - 3	* *
9	totalRows + 4	1	1	totalRows - 4	*

The column **min_val** contains only the data **col = 1,** means it is same for every row. Hence, column **min_val** is already being in generalized state. Hence, we got the consistent numerical expression in order to **start** the column level for-loop. The column **max_val** is partially generalized (row = 1 to 5).

Row level Repetitive Action					Expected Output
Variable: row		Column level Repetitive Action			
		Variable: col			
		min_val	max_val		
1	1	1	1	row	*
2	2		2		* *
3	3		3		* * *
4	4		4		* * * *
5	totalRows		5		* * * * *
6	6		4	totalRows - 1	* * * *
7	7		3	totalRows - 2	* * *
8	8		2	totalRows - 3	* *
9	totalRows + 4		1	totalRows - 4	*

We need to find the generalized expression for the maximum value of **variable:row** also.

There are still non-repeating numeric values in the column **max_val** of the col variable.

Row level Repetitive Action					
Variable: row		Column level Repetitive Action			
		Variable: col			
		min_val	max_val		
1	1	1	row	row	row
2	2				
3	3				
4	4				
5	totalRows				
6	6		totalRows – 1	totalRows – (6- 5)	totalRows – (row -5)
7	7		totalRows – 2	totalRows – (7- 5)	totalRows – (row -5)
8	8		totalRows – 3	totalRows – (8 -5)	totalRows – (row -5)
9	totalRows + 4		totalRows – 4	totalRows – (9 -5)	totalRows – (row -5)

Row level Repetitive Action				
Variable: row		Column level Repetitive Action		
		Variable: col		
		min_val	max_val	
1	1	1	row	row
2	2			
3	3			
4	4			
5	totalRows			
6	6		totalRows – (row -5)	totalRows – row +5
7	7		totalRows – (row -5)	totalRows – row +5
8	8		totalRows – (row -5)	totalRows – row +5
9	totalRows + totalRows – 1		totalRows – (row -5)	totalRows – row +5

There are still non-repeating numeric values in the column **max_val** of the col variable.

Row level Repetitive Action				
Variable: row		Column level Repetitive Action		
		Variable: col		
		min_val	max_val	
1	1	1	row	row
2	2			
3	3			
4	4			
5	5			
6	6		totalRows – row + totalRows	2*totalRows – row
7	7		totalRows – row + totalRows	2*totalRows – row
8	8		totalRows – row + totalRows	2*totalRows – row
9	2 * totalRows – 1		totalRows – row + totalRows	2*totalRows - row

Row level Repetitive Action				
Variable: row		Column level Repetitive Action		
		Variable: col		
		min_val	max_val	
1	1	1	row	row
2	2			
3	3			
4	4			
5	totalRows			
6	6		2*totalRows – row	2*totalRows – row
7	7		2*totalRows – row	
8	8		2*totalRows – row	
9	2 * totalRows – 1		2*totalRows - row	

The numeric pattern of the column **max_val** of variable **col** matches exactly values(1 to 5) of **row**.
This specific part of the numeric pattern (1 to 5) can be expressed in row variable.
The condition (col >=1) gives a starting point from which it will start printing star("*") character(s) till the value of variable **row** reaches (**totalRows)**.
The variable **col** then start decreases which is captured in the expression (**2*totalRows - row**).
So in this case, there is not the one but two generalized expressions for values of the variable **col**.
This indicates that we need to use two separate **for-loop** for the variable col.

Let the following are the two versions of column level for-loop for printing star (“*”) character(s).

Column level *for-loop_1* for printing star characters / / ***inner for-loop***

Column level *for-loop_2* for printing star characters / / ***inner for-loop***

We can summarize it as:

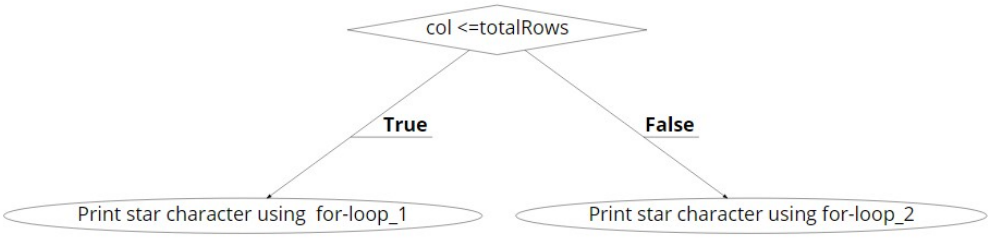
Print characters star(*) for every value of variable:row						
row=1	**row=2**	**row=3**	**row=4**	**row =totalRows**	**……**	**row= (2*totalRows - 1)**

If (condition=(col <=**totalRows**) is **True)**

Print star character using **Column level *for-loop_1***

Else

Print star character using **Column level *for-loop_2***



We can now define these two column level ***for-loop***.

Elements of* Column level *for-loop_1	
min_val	**1**
max_val	**totalRows**
variable_name	**col**
statement(s)	`System.out.print("*");`

Elements of* Column level *for-loop_2	
min_val	**totalRows + 1**
max_val	**2*totalRows - row**
variable_name	**col**
statement(s)	`System.out.print("*");`

ROW LEVEL REPETITIVE ACTION

We have assigned unique value to each row starting from 1 till 9.

We can represent value 9 as (2*5 – 1).The variable **totalRows** has initially assigned the value 5.

So, we can generalized the maximum limit of the **row** variable using the variable. Let's consider;

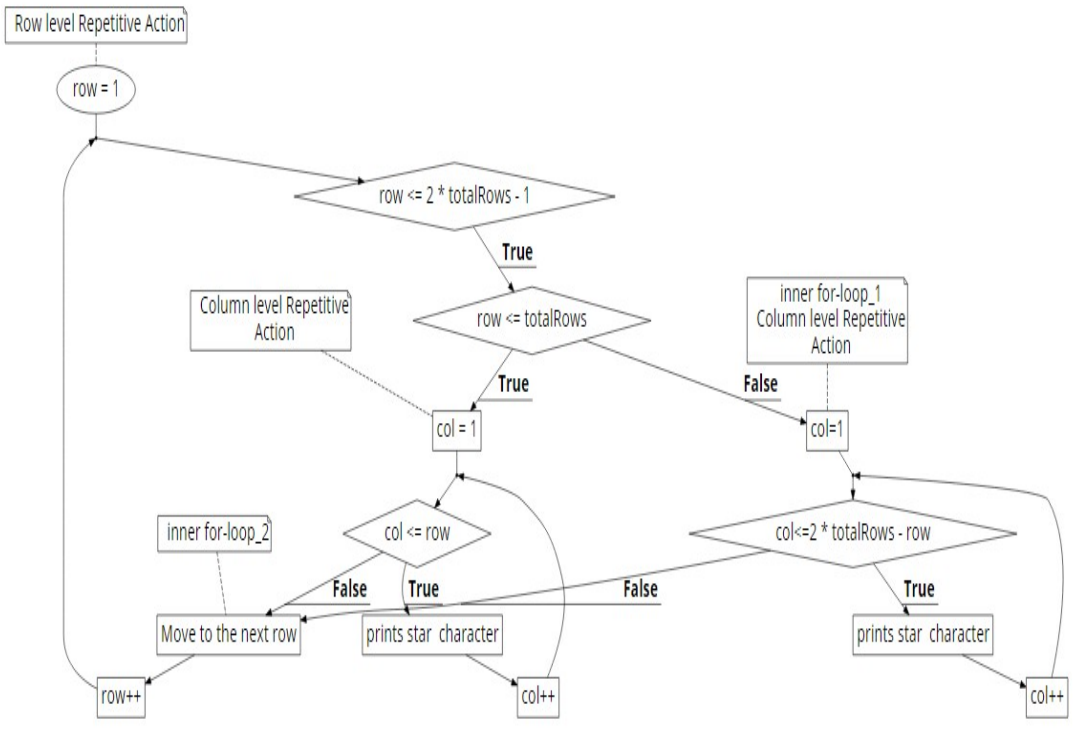
row → variable type of integer to store integer value one at a time and represents row number between 1 and (**2*totalRows** - 1).

We have logically divided row level repetition by ***for-loop*** into two parts due to two different versions of the column level **for-loop**.

Elements Of **Row Level decrementing** ***for-loop***	
min_val	**1**
totalRows	**totalRows**
variable_name	**row**
statement(s)	**Column level for-loop_1** for printing star characters `System.out.println();` // Move cursor to the next row

Elements Of **Row Level decrementing** ***for-loop***	
min_val	**totalRows + 1**
max_val	**2*totalRows - 1**
variable_name	**row**
statement(s)	**Column level for-loop_2** for printing star characters `System.out.println();`// Move cursor to the next row

Since we got all the details related to considered for-loops, we can summarize these related information with the help of flow-chart.



Let's execute the following program.

PatternG1.java

```
public class PatternG1 {

	public static void main(String[] args) {

		int totalRows = 5;// number of rows to display

		for (int row = 1; row <= (2 * totalRows - 1); row++) {

			if (row <= totalRows) {
				for (int col = 1; col <= row; col++) {
					System.out.print("* ");
				}
			} else {
				for (int col = 1; col <= (2 * totalRows - row); col++) {
					System.out.print("* ");
				}
			}
			System.out.println();
		}
	}
}
```

Output

```
*
* *
* * *
* * * *
* * * * *
* * * *
* * *
* *
*
```

The inner for-loop_1 and inner for-loop_2 differ only in the maximum limit value of the **col** variable.

It will be better if we capture maximum limit of **col** variable in a different variable.

Then we will be in a position to use only one inner for-loop for the complete column level repetition.

Let's update the previous program using single inner for-loop.

PatternG2.java

```
public class PatternG2 {

	public static void main(String[] args) {

		int totalRows = 5;// number of rows to display
		int colMax = 0;// defines the maximum limit for col variable.

		// Row level Repetitive Action :
		// Action1.Executes Column level Repetitive Action
		// Action2.Move cursor to next row.
		for (int row = 1; row <= (2 * totalRows - 1); row++) {

			// sets the maximum limit of the inner for-loop
			if (row <= totalRows) {
				colMax = row;
			} else {
				colMax = ( 2 * totalRows - row );
			}

			// Column level Repetitive Action :

			// Action1.Move cursor in the same row
			// Action2.prints star "* " character
			for (int col = 1; col <= colMax ; col++) {
				System.out.print("* ");
			}// inner for-loop
			System.out.println();// Move cursor to next row.
		}// outer for-loop
	}
}
```

Output

```
*
* *
* * *
* * * *
* * * * *
* * * *
* * *
* *
*
```

Alternative solution2

Let's start with the trace table that we got from the previous solution.

Row level Repetitive Action			Expected Output
Variable : row	Column level Repetitive Action		
	Variable : col		
	min_val	max_val	
1	1	1	*
2	1	2	* *
3	1	3	* * *
4	1	4	* * * *
5	1	5	* * * * *
6	1	4	* * * *
7	1	3	* * *
8	1	2	* *
9	1	1	*

The remaining variables are row & totalRows = 5. We can try to express column **max_val** of **col** variable in terms of numeric pattern of row values.

Row level Repetitive Action							Expected Output
Variable : row	Column level Repetitive Action						
	Variable : col						
	min_val	max_val					
1		1	1	row	row	row – 2*0	*
2		2	2	row	row	row – 2*0	* *
3		3	3	row	row	row – 2*0	* * *
4		4	4	row	row	row – 2*0	* * * *
5	1	5	5	row	row	row – 2*0	* * * * *
6		4	6 - 2	row - 2	row – 2*1	row – 2*1	* * * *
7		3	7 - 4	row - 4	row – 2*2	row – 2*2	* * *
8		2	8 - 6	row - 6	row – 2*3	row – 2*3	* *
2*5 - 1		1	9 - 8	row - 8	row – 2*4	row – 2*4	*

We can express column **max_val** of **col** variable in terms of row variable and part of the expression follows a multiple of 2 but for row values greater than 5 or **totalRows**.

It has condition when to increment the multiple of 2 or not. The condition is handled by conditional statements mostly by **if-statement** or **if-else-statement** or **if-elseif-else** .This we need to handle using such conditional statement by declaring another integer variable ,for example;

```
int count = 0;// counter to increment the multiple of 2.
```

We can give **if-statement** something like the following by incrementing the value of **variable:count**.

```
            // increment count only when row value is greater
            // than the total number of rows.
            if (row > totalRows) {
               // increment count by 1.
               count = count + 1;
             }
```

Now we can use variable count to generalize the column **max_val** of **col** variable.

Row level Repetitive Action						Expected Output
Variable : row	Column level Repetitive Action Variable : col					
	min_val	max_val				
1		1	row – 2*0	row – 2*count		*
2		2	row – 2*0	row – 2*count		* *
3		3	row – 2*0	row – 2*count		* * *
4		4	row – 2*0	row – 2*count		* * * *
5	1	5	row – 2*0	row – 2*count	row – 2 * count	* * * * *
6		4	row – 2*1	row – 2*count		* * * *
7		3	row – 2*2	row – 2*count		* * *
8		2	row – 2*3	row – 2*count		* *
(2* **totalRows** – 1)		1	row – 2*4	row – 2*count		*

Let's implement these observations as a different program.

PatternG3.java

```
public class PatternG3 {

            public static void main(String[] args) {

             int totalRows = 5;// number of rows to display
             int count = 0;// counter to increment the multiple of 2.

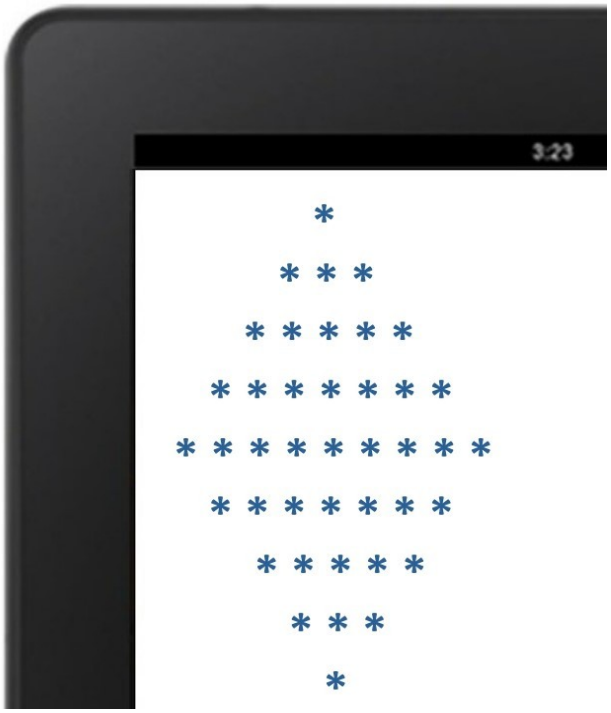
             // Row level Repetitive Action :
             // Action1.Executes Column level Repetitive Action
             // Action2.Move cursor to next row.
             for (int row = 1; row <= (2 * totalRows - 1); row++) {
                   // increment count by 1 only when row value is greater
                   // than the total number of rows.
                   if (row > totalRows) {
                         count = count + 1;
                   }

                   // Column level Repetitive Action :
                   // Action1.Move cursor in the same row
                   // Action2.prints star "* " character
                   for (int col = 1; col <= row - 2*count ; col++) {
                         System.out.print("* ");
                   }// inner for-loop
                   System.out.println();// Move cursor to next row.
             }// outer for-loop
       }
}
```

Verify results for different number of lines, let it be `totalRows` = 7,8 or 11.

Pattern1H

Our programming job is to output this pattern on a part of the computer screen as shown in the following. We can treat computer screen as a combination of (**row, column**) numeric value paired boxes which are spread all over the screen. Each box can hold a single character only.



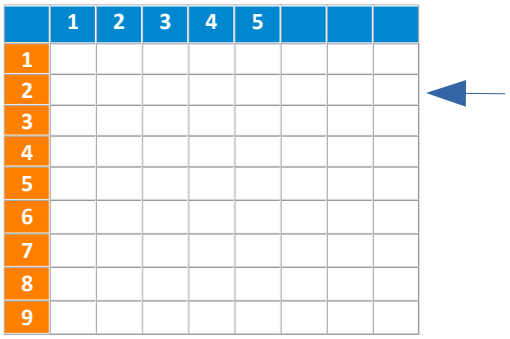
Data Representation

The complete pattern can be easily expressed as a collection of selected (**row, column**) numeric value pairs. Those selected (row, column) numeric value pairs shown in the given table structure. For example; (2, 5) means character "*" at **second row** and **fifth column** in the table.

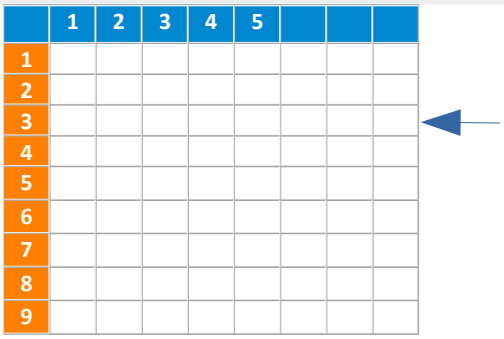
	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1					* (1,5)				
Row=2				*(2,4)	*** (2 ,5)**	*(2,6)			
Row=3			*(3,3)	*(3,4)	* (3,5)	*(3,6)	*(3,7)		
Row=4		*(4,2)	***(4,3)**	*(4,4)	*** (4,5)**	*(4,6)	***(4,7)**	*(4,8)	
Row=5	*(5,1)	*(5,2)	*(5,3)	*(5,4)	* (5,5)	*(5,6)	*(5,7)	*(5,8)	*(5,9)
Row=6		*(6,2)	***(6,3)**	*(6,4)	*** (6,5)**	*(6,6)	***(6,7)**	*(6,8)	
Row=7			*(7,3)	*(7,4)	* (7,5)	*(7,6)	*(7,7)		
Row=8				*(8,4)	*** (8,5)**	*(8,6)			
Row=9					*(9,5)				

Exercise. Complete the steps for the generation of this logic pattern for every row 1 to 9 by assuming output on the computer screen diagram.

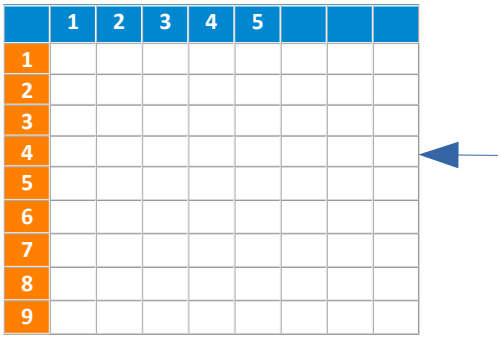
Steps for pattern generation(row = 1 to 9) with computer screen diagram.



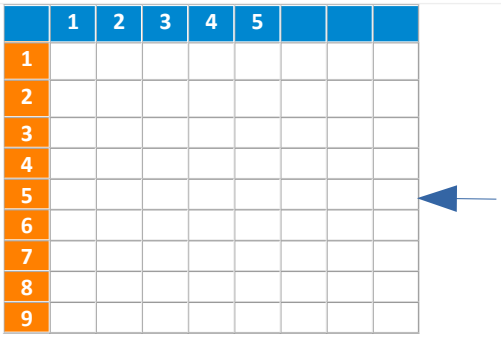
For row = 1



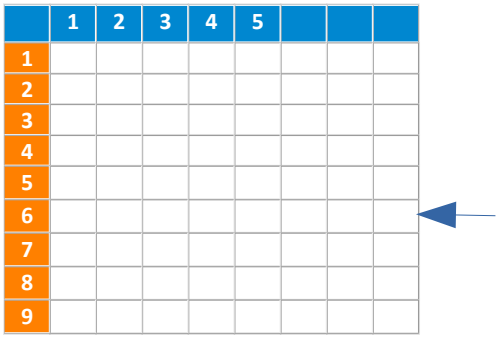
For row = 2



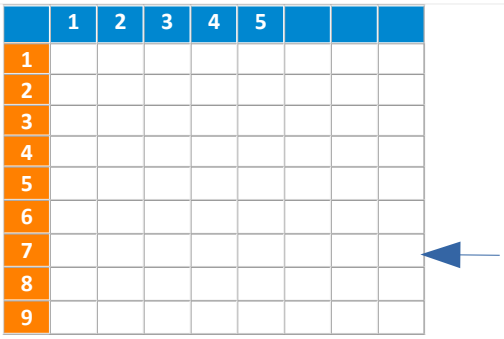
For row = 3



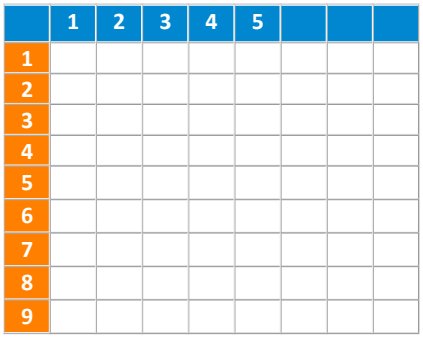
For row = 4



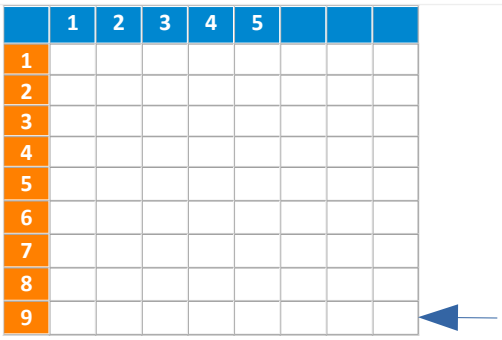
For row = 5



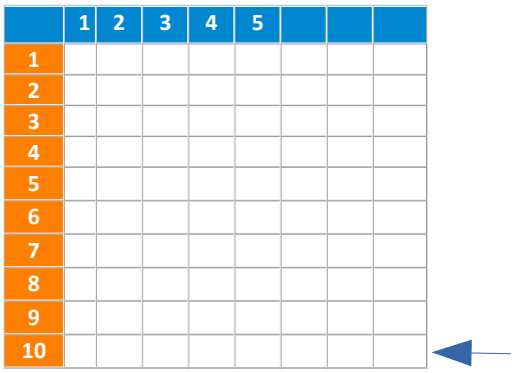
For row = 6



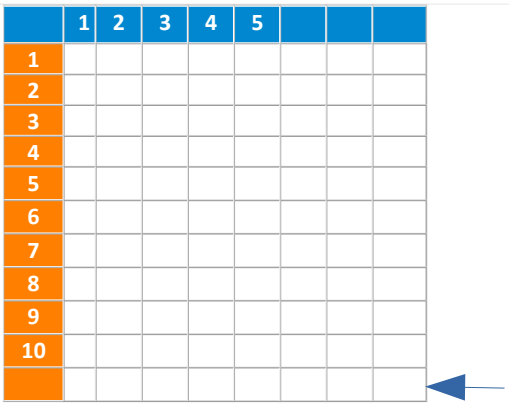
For row = 7



For row = 8



For row = 9



For row = 10

Let's convert steps into a program using inbuilt methods.

Given three types of programming statements are going to be got repeated many times in the program. We know that looping meant for repetitive action within a program. Hence, we need to logically use these printing statements within the ***for-loop***.

Purpose	Method
To move cursor to the next row.	`System.out.println()`
Move cursor to the next (row, column) position and then print space " " character in the same row.	`System.out.print(" ")`
Move cursor to the next (row, column) position and then print star "*" character in the same row.	`System.out.print("*")`

Writing a program to appropriately calculate the row and column values is the actual challenge in such types of simple pictured-based exercises. Such exercises force the brain cells to work harder without losing the motivation. We need to practice such exercise till our visualization about the logic of the program gets sufficiently improved. Without using logic trace tables if learners are able to run such program in a single chance, most of the time, then learners have achieved the mark.

COLUMN LEVEL REPETITIVE ACTION

Let's consider;

row → variable type of integer to store integer value one at a time and represents row number between 1 and 9.

col → variable type of integer to store integer value one at a time and represents column number between 1 and 9.

spaces → variable type of integer to store integer value one at a time and represents space character at column number between 1 and **totalRows**.

totalRows → variable type of integer to store integer value for total number of rows which is 5 by default.

Logic Trace Tables

(row, col) value pair depicts the position of "*" character on the display screen								
space	space	space	space	**row=1**, col=5				
space	space	space	**row=2**, col=4	**row=2**, col=5	**row=2**, col=6			
space	space	**row=3**, col=3	**row=3**, col=4	**row=3**, col=5	**row=3**, col=6	**row=3**, col=7		
space	**row=4**, col=2	**row=4**, col=3	**row=4**, col=4	**row=4**, col=5	**row=4**, col=6	**row=4**, col=7	**row=4**, col=8	
row=5, col=1	**row=5**, col=2	**row=5**, col=3	**row=5**, col=4	**row=5**, col=5	**row=5**, col=6	**row=5**, col=7	**row=5**, col=8	**row=5, col=9**
	row=6, col=2	**row=6**, col=3	**row=6**, col=4	**row=6**, col=5	**row=6**, col=6	**row=6**, col=7	**row=6**, col=8	
		row=7, col=3	**row=7**, col=4	**row=7**, col=5	**row=7**, col=6	**row=7**, col=7		
			row=8, col=4	**row=8**, col=5	**row=8**, col=6			
				row=9, col=5				

Approach to solution1

The overall observation is that at each row, either it's printing space characters or star (*) characters in a continues fashion. The columns in the pattern divided into two parts, either it's printing space " " character(s) or star("*") characters. These commas separated values given in the table are basically represents the column positions where space and star character(s) are actually positioned.

Row level Repetitive Action					Expected Output
Variable: row	Column level Repetitive Action				
	Variable : spaces	Variable: col	Character count	Total Characters	
1	1,2,3,4	5	4 Spaces + 1 star	5	*
2	1,2,3	4, 5,6	3 Spaces + 3 stars	6	* * *
3	1,2	3, 4, 5,6,7	2 Spaces + 5 stars	7	* * * * *
4	1	2, 3, 4, 5,6,7,8,	1 Space + 7 stars	8	* * * * * * *
5	0	1, 2, 3, 4, 5,6,7,8,9	0 Space + 9 stars	9	* * * * * * * * *
6	1	2, 3, 4, 5,6,7,8,	1 Space + 7 stars	8	* * * * * * *
7	1,2	3, 4, 5,6,7	2 Spaces + 5 stars	7	* * * * *
8	1,2,3	4, 5,6	3 Spaces + 3 stars	6	* * *
9	1,2,3,4	5	4 Spaces + 1 star	5	*

The printing of space(s) **comes first** before print star("*") character(s).If we take care of the condition till it print space character(s),then the remaining are the star("*") character(s). If we closely observe It's cutting the values of variable col into two halves. One part goes to printing of space character(s) and the other goes to printing of star character(s). We know that the comma separated values basically represents the column positions that uniquely identifies the position of space and star character(s).

Let's take out the minimum value (**min_val)** and maximum value(**max_val)** from this comma separated values which were given for printing space and star character(s).

Row level Repetitive Action					Expected Output
Variable: row	Column level Repetitive Action				
	Variable : spaces		Variable: col		
	min_val	max_val	min_val	max_val	
1	1	4	5	5	*
2	1	3	4	6	* * *
3	1	2	3	7	* * * * *
4	1	1	2	8	* * * * * * *
5	0	0	1	9	* * * * * * * * *
6	1	1	2	8	* * * * * * *
7	1	2	3	7	* * * * *
8	1	3	4	6	* * *
5 + 4	1	4	5	5	*

From the numeric pattern of minimum value (**min_val)** and maximum value(**max_val)** we need to derive the generalized expression. Those generalized expressions will help us to define the ***for-loop*** at the row and the column level. We know that the total number of rows given by the variable **totalRows** and has value 5 by default. So **totalRows** = 5.

The variables **row** and **col** both takes the value 1 to 9, i.e., row =1,2,3,4,5,..,9 and col=1,2,3,4,5,..,9.

Let's focus our attention for printing of star character(s) and try to find out the expression in terms of the variable **totalRows** = 5.

Row level Repetitive Action					Expected Output
Variable: row	Column level Repetitive Action				
	Variable: col (Printing star characters)				
	min_val		max_val		
1	5	5-0	5	5+0	*
2	4	5-1	6	5+1	* * *
3	3	5-2	7	5+2	* * * * *
4	2	5-3	8	5+3	* * * * * * *
5	1	5-4	9	5+4	* * * * * * * * *
6	2	5-3	8	5+3	* * * * * * *
7	3	5-2	7	5+2	* * * * *
8	4	5-1	6	5+1	* * *
5 + 5 -1	5	5-0	5	5+0	*

Part of **min_val** and **max_val** expression both follows the same numeric pattern. We can consider this as a separate integer variable as count.
Then the logical expression can be expressed in terms of **totalRows** and count.

Row level Repetitive Action					
Variable: row	Variable : count	Column level Repetitive Action			
		Variable: col (Printing star characters)			
		min_val		max_val	
1	0	5	totalRows -count	5	totalRows + count
2	1	4	totalRows -count	6	totalRows + count
3	2	3	totalRows -count	7	totalRows + count
4	3	2	totalRows -count	8	totalRows + count
5	4	1	totalRows -count	9	totalRows + count
6	3	2	totalRows -count	8	totalRows + count
7	2	3	totalRows -count	7	totalRows + count
8	1	4	totalRows -count	6	totalRows + count
2*totalRows - 1	0	5	totalRows -count	5	totalRows + count

The variable count starts increases by value 1 from the start of the row=1 till the row=**totalRows** (**total number of rows**) and then starts decreases further by value 1 till (**totalRows** – 1) times.

We can deal this using if-else conditional statement.
The condition (col >=**(totalRows – count**)) gives starting point from which ***for-loop*** will start printing star("*") character(s) till the maximum value of variable col which is **(totalRows + count**) for every value of row. The **if-condition** can be given using the comparison and logical operators for the range of values between (**totalRows – count**) and **(totalRows + count**) as:
If (col >= (**totalRows – count**) && col <= **(totalRows + count))**
We already know that;
variable col = 1,2,3,4,5 or 1,2,3,4, **…,(totalRows + count)**

Same expression

variable row = 1,2,3,….,9 or 1,2,3,….,(2 * **totalRows - 1)**

We can eliminate the logical AND(&&) condition(col <= **(totalRows + count))** in the **if-condition.** This logical AND(&&) condition already been taken care in the **loop-condition** of the column level inner ***for-loop***. So the if-condition will now reduce to the following:

If (col >= **(totalRows – count)**)

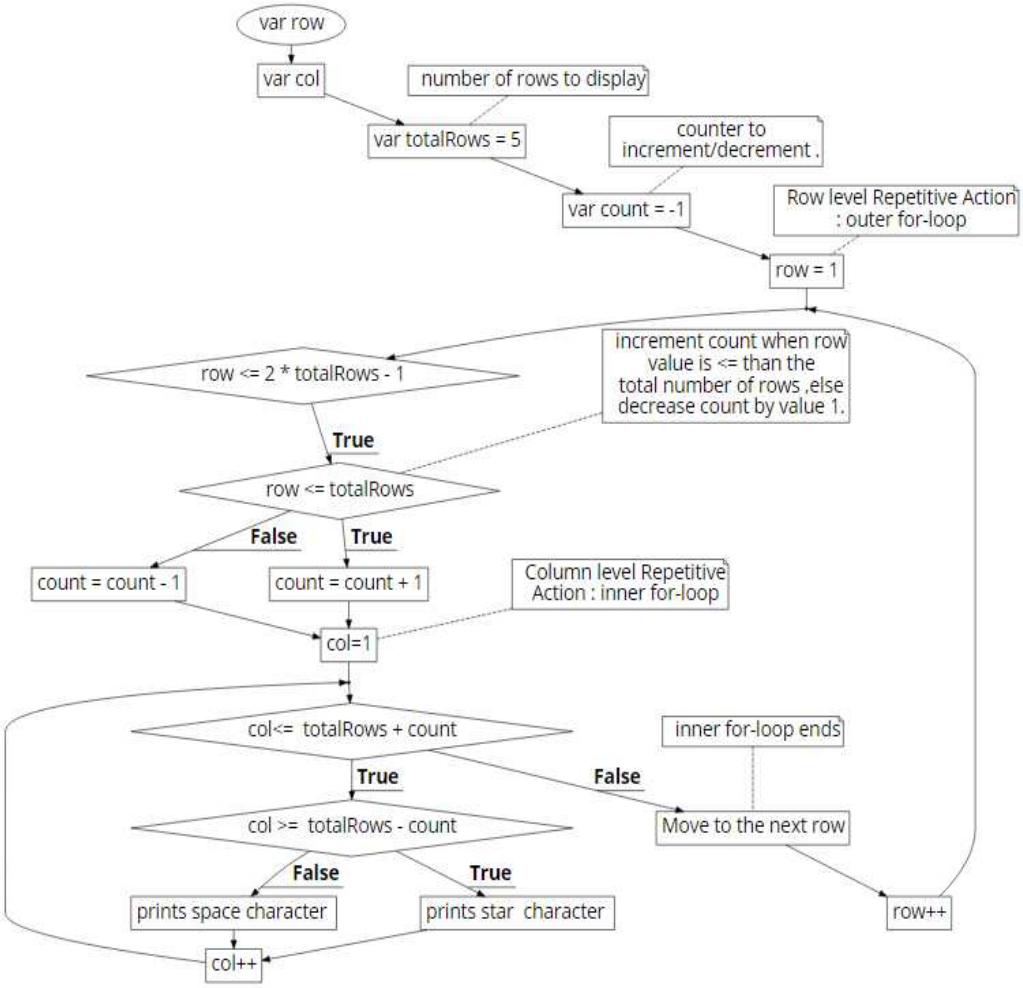
We can generalize the non-repeating numeric pattern of the column **max_val** into an expression. We can define column level ***for-loop.***

Elements of **Column level** ***for-loop***	
min_val	**1**
max_val	**(totalRows + count)**
variable_name	col
statement(s)	System.out.print("*");

We can define row level ***for-loop.***

Elements Of **Row Level decrementing** ***for-loop***	
min_val	1
max_val	**2*totalRows - 1**
variable_name	**row**
statement(s)	**Column level *for-loop*** for printing star characters `System.out.println( );//Move cursor to the next row`

We can summarize it all in a flow-chart as;



Let's execute the following program.

PatternH1.java

```
public class PatternH1 {

	public static void main(String[] args) {

		int totalRows = 7;// number of rows to display
		int count = -1;// counter to increment/decrement .

		// Row level Repetitive Action :
		// Action1.Executes Column level Repetitive Action
		// Action2.Move cursor to next row.
		for (int row = 1; row <= (2 * totalRows - 1); row++) {
			// increment count when row value is <= than the
			// total number of rows
			// else decrease count by value 1.
			if (row <= totalRows) {
				count = count + 1;
			} else {
				count = count - 1;
			}

			// Column level Repetitive Action :

			// Action1.Move cursor in the same row
			// Action2.prints star "* " character
			for (int col = 1; col <= ( totalRows + count ) ; col++) {
				if( col >= ( totalRows - count ) ) {
					System.out.print("* ");
				}else {
					System.out.print("  ");
				}

			}// inner for-loop
			System.out.println();// Move cursor to next row.
		}// outer for-loop
	}
}
```

Output

```
            *
          * * *
        * * * * *
      * * * * * * *
    * * * * * * * * *
  * * * * * * * * * * *
* * * * * * * * * * * * *
  * * * * * * * * * * *
    * * * * * * * * *
      * * * * * * *
        * * * * *
          * * *
            *
```

Problem* H1**.Continuation to previous program (PatternH1.java***), modify the program for the following situation.

In program (***PatternH1.java***), the count variable started with the value -1 given as :

```
int count = -1;// counter to increment/decrement
```

Required to match the output (**Pattern1H*)*** of the modified program (***PatternH1_2.java***) by making the following change:

```
int count = 0;// counter to increment/decrement
```

This time program has to initialise the variable count with the value `0` instead of -1 and yet still it should give the correct matching output as that of pattern program (**Pattern1H.java)**.

PatternH1_2.java

```
public class PatternH1_2 {

	public static void main(String[] args) {

		int totalRows = 7;// number of rows to display
		int count = 0;// counter to increment/decrement .

		// Row level Repetitive Action :
		// Action1.Executes Column level Repetitive Action
		// Action2.Move cursor to next row.

		// Write your logic

	}
}
```

Output

```
            *
          * * *
        * * * * *
      * * * * * * *
    * * * * * * * * *
  * * * * * * * * * * *
* * * * * * * * * * * * *
  * * * * * * * * * * *
    * * * * * * * * *
      * * * * * * *
        * * * * *
          * * *
            *
```

Alternative solution2

Let's start with the one of the logic trace table that we got from the previous solution.

Row level Repetitive Action					Expected Output
Variable : row	Column level Repetitive Action				
	Variable : col				
	Printing space character(s)		Printing star character(s)		
	min_val	max_val	min_val	max_val	
1	1	4	5	5	*
2	1	3	4	6	* * *
3	1	2	3	7	* * * * *
4	1	1	2	8	* * * * * * *
5	0	0	1	9	* * * * * * * * *
6	1	1	2	8	* * * * * * *
7	1	2	3	7	* * * * *
8	1	3	4	6	* * *
9	1	4	5	5	*

Let's focus our attention for printing of star character(s) and try to find out the expression in terms of the variables i.e. row & **totalRows**=5.

Row level Repetitive Action						
Variable : row	Column level Repetitive Action					
	Variable : col (Printing of start (*) characters)					
	min_val			max_val		
1	5	5-0	totalRows +1 - 1	5	totalRows +0	totalRows +1 - 1
2	4	5-1	totalRows +1 - 2	6	totalRows +1	totalRows +2 - 1
3	3	5-2	totalRows +1 - 3	7	totalRows +2	totalRows +3 - 1
4	2	5-3	totalRows +1 - 4	8	totalRows +3	totalRows +4 - 1
5	1	5-4	totalRows +1 - 5	9	totalRows +4	totalRows +5 - 1
6	2	2	2	8	totalRows +3	totalRows +5 - 2
7	3	3	3	7	totalRows +2	totalRows +5 - 3
8	4	4	4	6	totalRows +1	totalRows +5 - 4
9	5	5	5	5	totalRows +0	totalRows +5 - 5

We can use row=1,2,3,4,5 values to partially express the columns **min_val** and **max_val.**

Row level Repetitive Action						
Variable : row	Column level Repetitive Action					
	Variable : col (Printing of start (*) characters)					
	min_val			max_val		
1	5	5-0	totalRows +1 - row	5	totalRows + row - 1	totalRows + row - 1
2	4	5-1	totalRows +1 - row	6	totalRows + row - 1	totalRows + row - 1
3	3	5-2	totalRows +1 - row	7	totalRows + row - 1	totalRows + row - 1
4	2	5-3	totalRows +1 - row	8	totalRows + row - 1	totalRows + row - 1
5	1	5-4	totalRows +1 - row	9	totalRows + row - 1	totalRows + row - 1
6	2	2	2	8	totalRows +5 - 2	2*totalRows - 2
7	3	3	3	7	totalRows +5 - 3	2*totalRows - 3
8	4	4	4	6	totalRows +5 - 4	2*totalRows - 4
9	5	5	5	5	totalRows +5 - 5	2*totalRows - 5

Row level Repetitive Action						
Variable : row	Column level Repetitive Action					
	Variable : col (Printing of start (*) characters)					
	min_val			max_val		
1	5	5-0	totalRows +1 - row	5	totalRows + row - 1	totalRows + row - 1
2	4	5-1	totalRows +1 - row	6	totalRows + row - 1	totalRows + row - 1
3	3	5-2	totalRows +1 - row	7	totalRows + row - 1	totalRows + row - 1
4	2	5-3	totalRows +1 - row	8	totalRows + row - 1	totalRows + row - 1
5	1	5-4	totalRows +1 - row	9	totalRows + row - 1	totalRows + row - 1
6	2	2	(6 – 4)	8	2*totalRows - (6 – 4)	2*totalRows - 6 + 4)
7	3	3	(7 – 4)	7	2*totalRows - (7 – 4)	2*totalRows - 7 + 4)
8	4	4	(8 – 4)	6	2*totalRows - (8 – 4)	2*totalRows - 8 + 4)
9	5	5	(9 – 4)	5	2*totalRows - (9 – 4)	2*totalRows - 9 + 4)

We can denote numeric pattern 6,7,8,9 as variable:row. It will be more generalized and still not all.

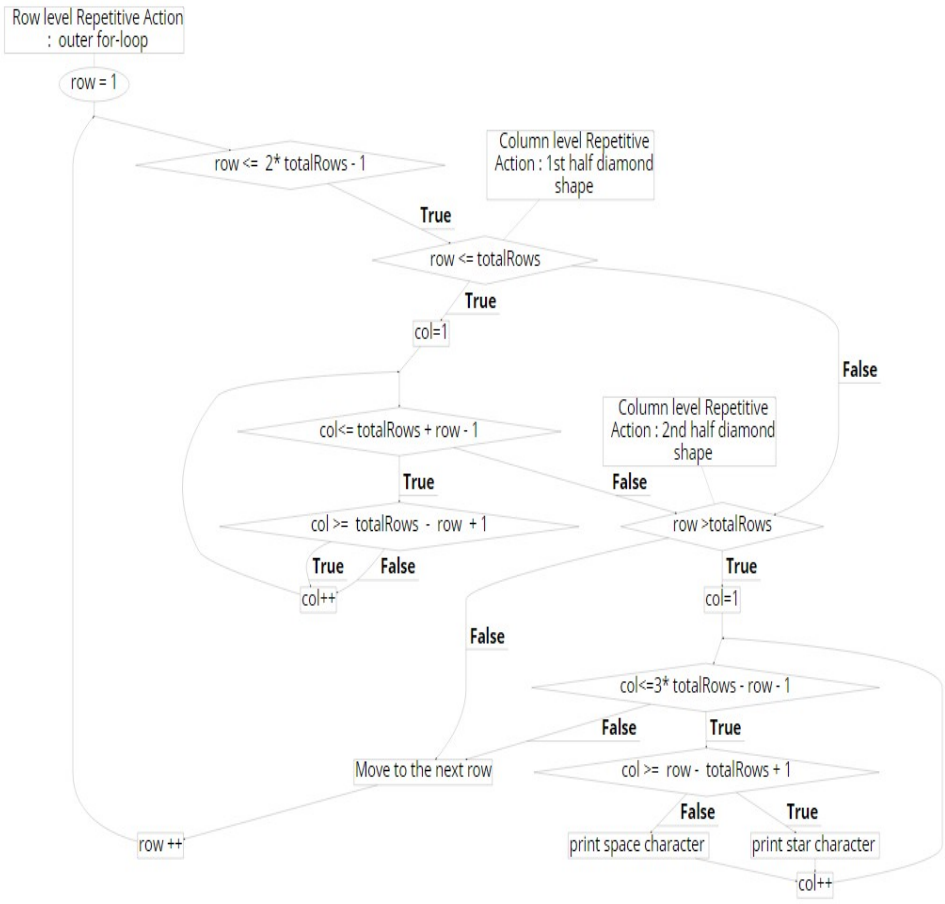
Row level Repetitive Action					
Variable : row	Column level Repetitive Action				
	Variable : col (Printing of start (*) characters)				
	min_val			max_val	
1	5	totalRows +1 - row		5	totalRows + row - 1
2	4	totalRows +1 - row		6	totalRows + row - 1
3	3	totalRows +1 - row		7	totalRows + row - 1
4	2	totalRows +1 - row		8	totalRows + row - 1
5	1	totalRows +1 - row		9	totalRows + row - 1
6	2		(row – 4)	8	2*totalRows - row + 4)
7	3		(row – 4)	7	2*totalRows - row + 4)
8	4		(row – 4)	6	2*totalRows - row + 4)
9	5		(row – 4)	5	2*totalRows - row + 4)

We can replace repeated value 4 as (totalRows - 1). It will be more generalized.

Row level Repetitive Action					
Variable : row	Column level Repetitive Action				
	Variable : col (Printing of start (*) characters)				
	min_val			max_val	
1	5	totalRows +1 - row		5	totalRows + row - 1
2	4	totalRows +1 - row		6	totalRows + row - 1
3	3	totalRows +1 - row		7	totalRows + row - 1
4	2	totalRows +1 - row		8	totalRows + row - 1
5	1	totalRows +1 - row		9	totalRows + row - 1
6	2	row – (totalRows - 1)		8	2*totalRows - row + (totalRows - 1)
7	3	row – (totalRows - 1)		7	2*totalRows - row + (totalRows - 1)
8	4	row – (totalRows - 1)		6	2*totalRows - row + (totalRows - 1)
9	5	row – (totalRows - 1)		5	2*totalRows - row + (totalRows - 1)

Row level Repetitive Action				
Variable : row	Column level Repetitive Action Variable : col (Printing of start (*) characters)			
	min_val		max_val	
1	totalRows +1 - row	**totalRows +1 - row**	totalRows + row - 1	**totalRows + row - 1**
2	totalRows +1 - row		totalRows + row - 1	
3	totalRows +1 - row		totalRows + row - 1	
4	totalRows +1 - row		totalRows + row - 1	
5	totalRows +1 - row		totalRows + row - 1	
6	row – totalRows + 1	**row – totalRows + 1**	3*totalRows - row – 1	**3*totalRows - row – 1**
7	row – totalRows + 1		3*totalRows - row – 1	
8	row – totalRows + 1		3*totalRows - row – 1	
9	row – totalRows + 1		3*totalRows - row – 1	

The following given flow-chart easily shows our logical expressions consideration.



Since two set of generalized expressions, so we need two set of column level ***for-loop.***

If - condition	min_val	max_val
Logical expression (set1)	totalRows +1 - row	**totalRows + row - 1**
Logical expression (set 2)	row – totalRows + 1	3*totalRows - row – 1

Let's execute the following program.

PatternH2_1.java

```
public class PatternH2_1 {

   public static void main(String[] args) {
      int  totalRows = 4;  //number of rows to display

      // Row level Repetitive Action :
      for( int row = 1 ; row <= ( 2* totalRows - 1 ) ; row ++ ) {

            //1st half : Column level Repetitive Action :
            if(  row <= totalRows ) {

                  for(int col = 1 ; col <=( totalRows  + row -1 );col++) {
                        if( col >= ( totalRows  -  row  + 1 )  ) {
                              System.out.print("* ");
                        }else {
                              System.out.print("  ");
                        }

                  }//inner for-loop_1
            }

            // 2nd half : Column level Repetitive Action :
            if( row >totalRows ) {
                  for(int col=1;col<=(3* totalRows - row - 1);col++){
                        if( col >= ( row -  totalRows + 1 )  ) {
                              System.out.print("* ");
                        }else {
                              System.out.print("  ");
                        }
                  }//inner for-loop_2
            }
            System.out.println();// Move cursor to next row.
      }// outer for-loop
   }
}
```

Output

```
      *
    * * *
  * * * * *
* * * * * * *
  * * * * *
    * * *
      *
```

Let's try to simplify program further by simplifying the **if-conditions** and **loop-condition** expressions.

The **if-condition** gives the minimum limit, a starting point for which program will start printing star("*") character(s) till the maximum value of variable col given by the **loop-condition.**

Let's consider;

colMin → variable type of integer to store integer value of column , a starting point from which program will start printing star("*") character.

colMax → variable type of integer to store integer value of column , a maximum value till program will stop printing star("*") character(s).

For example, let's analyze the following expression when **col** is a variable which **starts from value 0.**The variable **col=0,1,2,3 when contains the values**,

means col <= (4 – 1) or col < 4 ,both means the same.

Similarly,

When variable col=5,6,…., will have the values, which means the logical condition to cover these values can be expressed as;

col >= (4 + 1) or col > 4, both means the same.

In the same way the **if-conditions** and **loop-condition** expressions can be simplified.

Variable : row	**if-conditions**	
	Variable : colMin	
	default	**1**
row <= totalRows	col >= (totalRows - row + 1)	col > (totalRows - row)
row >totalRows	col >= (row - totalRows + 1)	col > (row - totalRows)

Variable : row	**loop-conditions**	
	Variable : colMax	
	default	**1**
row <= totalRows	col<=(totalRows + row - 1)	col<(totalRows + row)
row >totalRows	col<=(3* totalRows - row - 1)	col<(3 * totalRows – row)

Using variables **colMin & colMax** we can merge the two inner-loops by dynamically assigning these expressions to these variables.

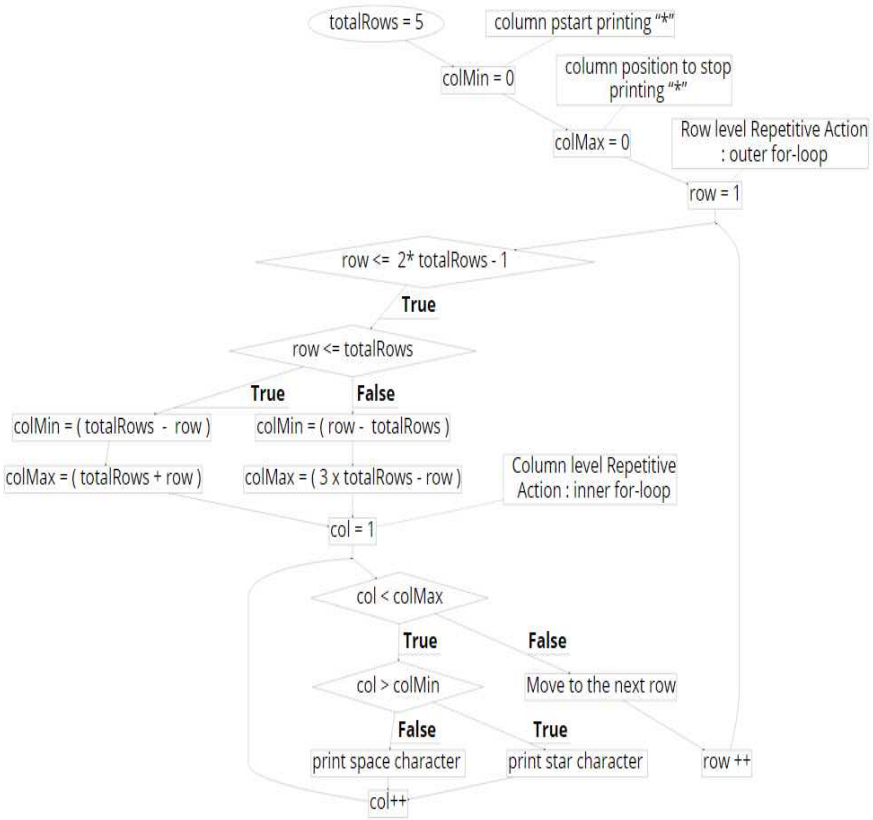
We can use **if-else** statement to update the variables **colMin & colMax** dynamically which are as follows.

```
if ( row <= totalRows ){
      colMin = ( totalRows - row );
      colMax = ( totalRows + row );
}else{
      colMin = ( row -  totalRows );
      colMax = ( 3 * totalRows - row );
}
```

Further, we can use variables **colMin & colMax** in inner ***for-loop*.**

In this way, we can try merging <u>two inner for-loops into **single inner *for-loop***.</u>

The following flow-chart gives the overall idea how program following this should get updated as per our previous discussion.



So, let's update our previous program with these changes and executes the program. Also, test for different number of lines by changing value to 7 for the variable:**totalRows.**

PatternH2_2.java

```
public class PatternH2_2 {

	public static void main( String[] args ) {

		int totalRows = 5;  //number of rows to display
		int colMin = 0;// column position to start printing “*”
		int colMax = 0;// column position to stop printing “*”

		// Row level Repetitive Action :
		// Action1.Executes Column level Repetitive Action
		// Action2.Move cursor to next row.
		for( int row = 1 ; row <= ( 2* totalRows - 1 ) ; row ++ ) {

			if ( row <= totalRows ){
				colMin = ( totalRows  -  row );
				colMax = ( totalRows + row );
			}else{
				colMin = ( row -  totalRows );
				colMax = ( 3 * totalRows - row );
			}

			//Column level Repetitive Action :
			for(int col = 1 ; col < colMax ; col++) {
				if( col > colMin  ) {
					System.out.print("* ");
				}else {
					System.out.print("  ");
				}
			}//inner for-loop

			System.out.println();// Move cursor to next row.

		}// outer for-loop
	}
}
```

Output: totalRows = 5

```
        *
      * * *
    * * * * *
  * * * * * * *
* * * * * * * * *
  * * * * * * *
    * * * * *
      * * *
        *
```

Output: totalRows = 7

```
            *
          * * *
        * * * * *
      * * * * * * *
    * * * * * * * * *
  * * * * * * * * * * *
* * * * * * * * * * * * *
  * * * * * * * * * * *
    * * * * * * * * *
      * * * * * * *
        * * * * *
          * * *
            *
```

Alternative solution3

Let's start with the logic trace table that we got from the previous solution1. The variables are row & totalRows=5.The variable spaces **representing the printing of space characters considered before printing of star(*) characters.**The variable **col representing the printing of star(*) characters .**

Row level Repetitive Action					Expected Output
Variable: row	Column level Repetitive Action				
	Variable : spaces	Variable: col	Character count	Total Characters	
1	1,2,3,4	5	4 Spaces + 1 star	5	*
2	1,2,3	4, 5,6	3 Spaces + 3 stars	6	* * *
3	1,2	3, 4, 5,6,7	2 Spaces + 5 stars	7	* * * * *
4	1	2, 3, 4, 5,6,7,8,	1 Space + 7 stars	8	* * * * * * *
5	0	1, 2, 3, 4, 5,6,7,8,9	0 Space + 9 stars	9	* * * * * * * * *
6	1	2, 3, 4, 5,6,7,8,	1 Space + 7 stars	8	* * * * * * *
7	1,2	3, 4, 5,6,7	2 Spaces + 5 stars	7	* * * * *
8	1,2,3	4, 5,6	3 Spaces + 3 stars	6	* * *
9	1,2,3,4	5	4 Spaces + 1 star	5	*

The printing of space(s) **comes first** before print star("*") character(s). If we closely observe the total character count is divided into two halves. One part goes to printing of space character(s) and the other goes to printing of star character(s) as shown in the column **Character count.** Let's take out the minimum value (**min_val)** and maximum value(**max_val)** for printing space and star character(s) from the column c**haracter count** .Hence, the following trace table can be drawn.

Row level Repetitive Action : totalRows=5							Expected Output
Variable : row	Column level Repetitive Action						
	Variable : col						
	space character(s) count			star character(s) count			
	min_val	max_val		min_val	max_val		
			1			**1**	
1	1	4	totalRows - 1	1	1	1	*
2	1	3	4 - 1	1	3	1 + 2	* * *
3	1	2	3 - 1	1	5	3 + 2	* * * * *
4	1	1	2 - 1	1	7	5 + 2	* * * * * * *
5	0	0	1 - 1	1	9	7 + 2	* * * * * * * * *
6	1	1	0 + 1	1	7	9 - 2	* * * * * * *
7	1	2	1 + 1	1	5	7 - 2	* * * * *
8	1	3	2 + 1	1	3	5 - 2	* * *
9	1	4	3 + 1	1	1	3 - 2	*

From the numeric pattern of minimum value (**min_val)** and maximum value(**max_val)** we need to derive the generalized expression. Those generalized expressions will help us to define the ***for-loop*** at the row and the column level. At the column level two inner for-loops are required one for printing space character(s) and other for printing star character(s).

The column **max_val** for space character(s) count started at value 4 (**totalRows** – 1) and then decreases by value 1 consistently till reached 0 at the row=5(total number of rows). Then it start increases by value 1 consistently till reached maximum row value as;

9=(2*5 – 1) or (2*totalRows – 1) ,since **totalRows=5**.

The column **max_val** for star character(s) count started at value 1 and increases by value 2 consistently till reached 9 at the row=5(total number of rows). Then it start decreases by value 2 consistently till **maximum row value 9 or (2*5 – 1) or (2*totalRows – 1)** ,since totalRows=5.

Let's consider;

spaceMax → variable type of integer to store integer value one at a time and starts at the value (**totalRows** – 1).

starMax → variable type of integer to store integer value one at a time and starts at the value 1.

Using variables **spaceMax & starCount** we can use two inner-loops by dynamically assigning maximum limit to print space and star character(s) respectively by these variables.

Further, we can use variables **spaceMax & starMax** in inner ***for-loops*.**

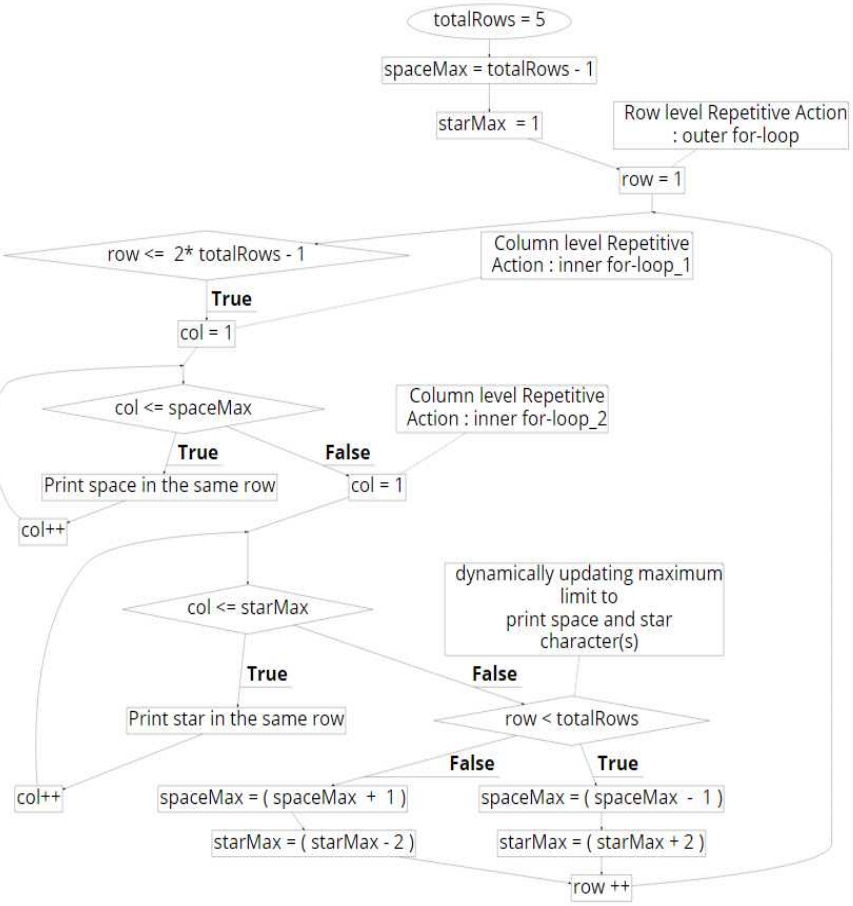
In this way, we can use two inner for-loops for **printing single row of space and star character(s)** which are as follows.

row < totalRows	**row >= totalRows**
spaceMax = (spaceMax - 1) starMax = (starMax + 2)	spaceMax = (spaceMax + 1) starMax = (starMax - 2)

We can use **if-else** statement to dynamically update these variables, **spaceMax & starMax.**

Row level Repetitive Action							
Variable : row	**Column level Repetitive Action**						
	Variable : col						
totalRows=5	**space character(s) count**			**star character(s) count**			
	min_val	**max_val**			**min_val**	**max_val**	
		Variable: spaceMax		**Value Difference**		**Variable: starMax**	**Value Difference**
1	1	totalRows	4	- 1	1	1	1
2	1	4	3	- 1	1	1	+ 2
3	1	3	2	- 1	1	3	+ 2
4	1	2	1	- 1	1	5	+ 2
5	0	1	0	- 1	1	7	+ 2
6	1	0	1	+ 1	1	9	- 2
7	1	1	2	+ 1	1	7	- 2
8	1	2	3	+ 1	1	5	- 2
(2*totalRows – 1)	1	3	4	+ 1	1	3	- 2

With the help of following flow-chart, we can get overall idea.



Let's execute the following program.

PatternH3.java

```
public class PatternH3 {

	public static void main( String[] args ) {

		int totalRows = 5;  //number of rows to display
		int spaceMax  = totalRows - 1;// maximum count to print space "  "
		int starMax   = 1;// maximum count to print star "* "

		// Row level Repetitive Action :
		// Action1.Executes Column level Repetitive Action
		// Action2.Move cursor to next row.
		for( int row = 1 ; row <= ( 2* totalRows - 1 ) ; row ++ ) {

			//Column level Repetitive Action : Print space in the same row
			for(int col = 1 ; col <= spaceMax ; col++) {
				System.out.print("  ");
			}//inner for-loop_1

			//Column level Repetitive Action : Print star in the same row
			for(int col  = 1 ; col <= starMax ; col++ ) {
				System.out.print("* ");
			}//inner for-loop_2

			System.out.println();// Move cursor to next row.

			if ( row < totalRows ){
				spaceMax = ( spaceMax  -  1 );
				starMax = ( starMax + 2 );
			}else{
				spaceMax = ( spaceMax  +  1 );
				starMax = ( starMax - 2 );
			}

		}// outer for-loop
	}
}
```

Output

```
        *
      * * *
    * * * * *
  * * * * * * *
* * * * * * * * *
  * * * * * * *
    * * * * *
      * * *
        *
```

Problem H1:

The program (**PatternH3.java**) used the conditional operator < (less than) as shown:

```
if ( row < totalRows ){
        spaceMax = (spaceMax  -  1 );
        starMax = (starMax + 2 );
}else{
        spaceMax = (spaceMax  +  1 );
        starMax = (starMax - 2 );
}
```

Write a program **PatternH4.java** which will use conditional operator <= (less than equals to) instead of conditional operator < (less than) to generate the same output as done by **PatternH3.java** program**.** Modify the program (**PatternH3.java**) to generate the pattern **Pattern1H** by using conditional operator <= (less than equals to).

```
if ( row <= totalRows ){
        spaceMax = ( spaceMax  -  1 );
        starMax = ( starMax + 2 );
}else{
        spaceMax = ( spaceMax  +  1 );
        starMax = ( starMax - 2 );
}
```

Test the output for total number of rows = 10 or **variable:totalRows = 10**.

Problems. Write programs to output the following by making use of logic trace table.

Output1

```
* * * * * * * * * *
* *             * *
*   *         *   *
*     *     *     *
*       * *       *
*       * *       *
*     *     *     *
*   *         *   *
* *             * *
* * * * * * * * * *
```

Output2

```
* * * *     * * * *
* *             * *
*   *         *   *
*     *     *     *
        * *
        * *
*     *     *     *
*   *         *   *
* *             * *
* * * *     * * * *
```

Output3

```
  * * * * * * * *
*   * * * * * * *
* *   * * * * * *
* * *   * * * * *
* * * *   * * * *
* * * * *   * * *
* * * * * *   * *
* * * * * * *   *
* * * * * * * *
```

Output4

```
*
  *
    *
      *
        *
          *
            *
              *
                *
                  *
```

PatternAA

```
                  *
                * *
              * * *
            * * * *
          * * * * *
        * * * * * *
      * * * * * * *
    * * * * * * * *
  * * * * * * * * *
* * * * * * * * *
  * * * * * * * * *
    * * * * * * * *
      * * * * * * *
        * * * * * *
          * * * * *
            * * * *
              * * *
                * *
                  *
```

PatternBB

```
                  *
                * *
              * * *
            * * * *
          * * * * *
        * * * * * *
      * * * * * * *
    * * * * * * * *
  * * * * * * * * *
* * * * * * * * *
  * * * * * * * * * *
    * * * * * * * * *
      * * * * * * * *
        * * * * * * *
          * * * * * *
            * * * * *
              * * * *
                * * *
                  * *
                    *
```

PatternCC

```
* * * * * * * * * * *
* * * * * * * * * *
* * * * * * * * *
* * * * * * *
* * * * * *
* * * * *
* * * *
* * *
* *
*
*
* *
* * *
* * * *
* * * * *
* * * * * *
* * * * * * *
* * * * * * * *
* * * * * * * * *
* * * * * * * * * *
* * * * * * * * * * *
```

PatternDD

```
          * * * * * * * * * *
            * * * * * * * * *
              * * * * * * * *
                * * * * * * *
                  * * * * * *
                    * * * * *
                      * * * *
                        * * *
                          * *
                            *
                            *
                          * *
                        * * *
                      * * * *
                    * * * * *
                  * * * * * *
                * * * * * * *
              * * * * * * * *
            * * * * * * * * *
          * * * * * * * * * *
```

PatternEE

```
* * * * * * * * *  * * * * * * * * *
* * * * * * * *      * * * * * * * *
* * * * * * *          * * * * * * *
* * * * * *              * * * * * *
* * * * *                  * * * * *
* * * *                      * * * *
* * *                          * * *
* *                              * *
*                                  *
```

PatternFF

```
*                                  *
* *                              * *
* * *                          * * *
* * * *                      * * * *
* * * * *                  * * * * *
* * * * * *              * * * * * *
* * * * * * *          * * * * * * *
* * * * * * * *      * * * * * * * *
* * * * * * * * *  * * * * * * * * *
```

PatternGG

```
                  *
                * * *
              * * * * *
            * * * * * * *
          * * * * * * * * *
        * * * * * * * * * * *
      * * * * * * * * * * * * *
    * * * * * * * * * * * * * * *
      * * * * * * * * * * * * *
        * * * * * * * * * * *
          * * * * * * * * *
            * * * * * * *
              * * * * *
                * * *
                  *
```

PatternHH

```
* * * * * * * * *  * * * * * * * * *
* * * * * * * *      * * * * * * * *
* * * * * * *          * * * * * * *
* * * * * *              * * * * * *
* * * * *                  * * * * *
* * * *                      * * * *
* * *                          * * *
* *                              * *
*                                  *
*                                  *
* *                              * *
* * *                          * * *
* * * *                      * * * *
* * * * *                  * * * * *
* * * * * *              * * * * * *
* * * * * * *          * * * * * * *
* * * * * * * *      * * * * * * * *
* * * * * * * * *  * * * * * * * * *
```

PatternII

```
*                                                  *
* *                                              * *
* * *                                          * * *
* * * *                                      * * * *
* * * * *                                  * * * * *
* * * * * *                              * * * * * *
* * * * * * *                          * * * * * * *
* * * * * * * *                      * * * * * * * *
* * * * * * * * *                  * * * * * * * * *
* * * * * * * * *                  * * * * * * * * *
* * * * * * * *                      * * * * * * * *
* * * * * * *                          * * * * * * *
* * * * * *                              * * * * * *
* * * * *                                  * * * * *
* * * *                                      * * * *
* * *                                          * * *
* *                                              * *
*                                                  *
```

ProblemGBSwitchCase: Please apply switch-case statement to generate the following output based on the value of border style values.

Border Style = 1

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
1												0											
2											0		0										
3										0				0									
4									0						0								
5										0				0									
6											0		0										
7												0											
8					0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
9				0						0				0						0			
10			0						0						0						0		
11		0						0								0						0	
12	0	0	0	0	0	0	0										0	0	0	0	0	0	0

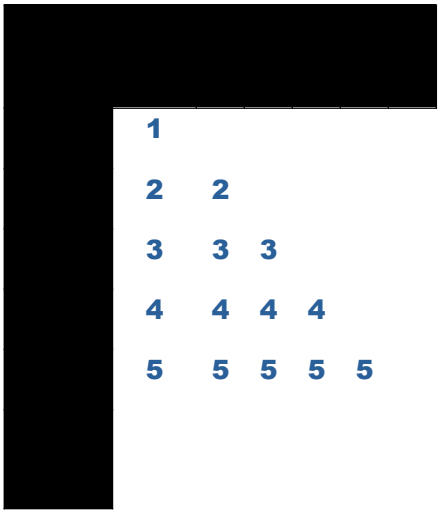
Border Style = 2

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
1												0											
2											0	0	0										
3										0	0		0	0									
4									0	0				0	0								
5										0	0		0	0									
6											0	0	0										
7												0											
8					0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
9				0	0	0	0	0	0	0				0	0	0	0	0	0	0			
10			0	0				0	0						0	0				0	0		
11		0	0	0	0	0	0	0								0	0	0	0	0	0	0	
12	0	0	0	0	0	0	0										0	0	0	0	0	0	0

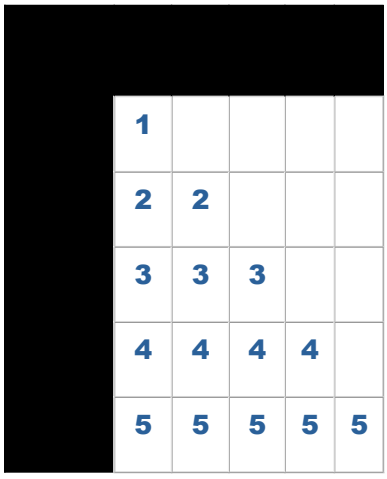
Border Style = 0

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
1												0											
2											0	0	0										
3										0	0	0	0	0									
4									0	0	0	0	0	0	0								
5										0	0	0	0	0									
6											0	0	0										
7												0											
8					0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				
9				0	0	0	0	0	0	0				0	0	0	0	0	0	0			
10			0	0	0	0	0	0	0						0	0	0	0	0	0	0		
11		0	0	0	0	0	0	0								0	0	0	0	0	0	0	
12	0	0	0	0	0	0	0										0	0	0	0	0	0	0

NumberPattern 1A



Our programming job is to print this particular logic-based pattern on a part of the computer screen. **Our approach will be assumed that the computer screen is a combination of rows and columns virtually dividing it into smaller boxes.** The smallest box is treated as a resident of just a single character only. The address in each smallest box on the display screen will logically become a unique **(row, column)** numeric combination. With this assumption computer screen will simply be a unique combination of (row, column) numeric value paired boxes which are uniformly spread all across the screen.



	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	1				
Row=2	2	2			
Row=3	3	3	3		
Row=4	4	4	4	4	
Row=5	5	5	5	5	5

Data Representation

Although it is logically, we can exactly locate the position of each printable character on the display screen by knowing it's uniquely identifiable **(row, column)** numeric paired-values. *For example;* **(4, 3)** means printing numeric character at the assumed screen location **4th row and 3rd column.**

(Row, Col) value pairs representation

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	1,1				
Row=2	2 ,1	2,2			
Row=3	3,1	3,2	3,3		
Row=4	4,1	4,2	4,3	4,4	
Row=5	5,1	5,2	5,3	5,4	5,5

The given set of row and col value pairs tries to mimic the location of each printable character of a given pattern on the device's display screen. The set of all value pairs (row = value, col = value) represent this particular pattern.

Analysis of output

It can also be interpreted in a tabular form.

row	1				
Col	1				
Output	1				

For row =1, 1<=col<=1

row	2				
Col	1	2			
Output	2	2			

For row =2, 1<=col<=2

row	3				
Col	1	2	3		
Output	3	3	3		

For row =3, 1<=col<=3

row	4				
Col	1	2	3	4	
Output	4	4	4	4	

For row =4, 1<=col<=4

row	5				
Col	1	2	3	4	5
Output	5	5	5	5	5

For row =5, 1<=col<=5

COLUMN LEVEL REPETITIVE ACTION

Row level Repetitive Action				Expected Output
variable:row	Column level Repetitive Action			
	lower_limit <= variable:col <= upper_limit			
totalRows = 5	Range	lower_limit	upper_limit	
row = 1	1 <=col<= 1	1	1	**1**
row = 2	1<=col<=2	1	2	**2 2**
row = 3	1<=col<=3	1	3	**3 3 3**
row = 4	1<=col<=4	1	4	**4 4 4 4**
row = 5	1<=col<=5	1	5	**5 5 5 5 5**

The *upper_limit* value is based on how many times the value repeated in a particular row.

The **lower_limit** of col variable has the same numeric value expression for every value of row. It's consistent for all the values rows.

Likewise, we need to come out with such a consistent expression for **upper_limit** which should suit any number of rows.

The other remaining variables are row & totalRows=5. Let's try to generalize using these variables.

For all the rows, the row value prints exactly the number of times the upper_limit of col value.

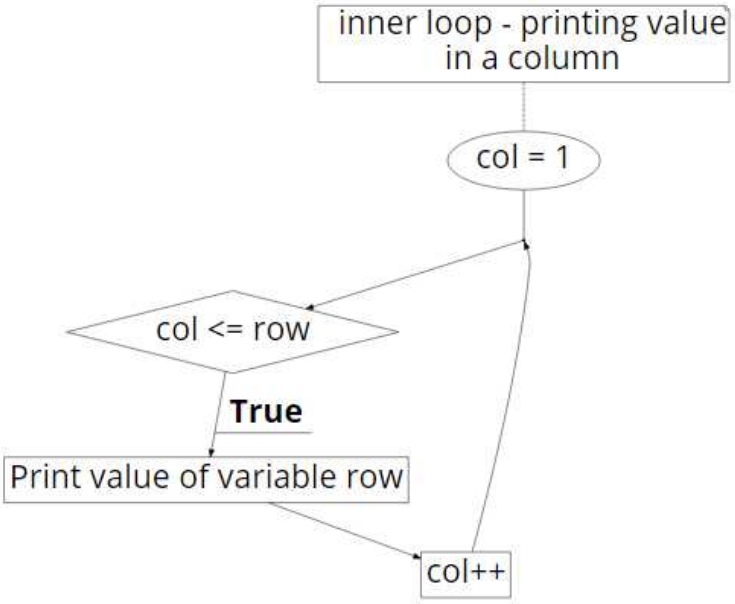
Column level Repetitive Action			
row	***lower_limit***	**upper_limit**	
		default	**1**
row = 1	1	1	row
row = 2	1	2	row
row = 3	1	3	row
row = 4	1	4	row
row = 5	1	5	row

So the *upper_limit* of col variable is generalized to row variable.

The *lower_limit & upper_limit* are consistent for all the rows. It's now generalized for any number of rows.

We can now define column level ***for-loop*** which is for every row value.

Elements of* Column level *for-loop	
min_val	**1**
max_val	**row**
variable_name	**col**
statement(s)	System.out.print(row);



ROW LEVEL REPETITIVE ACTION

The same following statements had been repeated 5 times(total number of rows).

Column level *for-loop* // *inner for-loop*

```
System.out.println();;// Move cursor to the next row
```

The minimum and maximum value can be given for the ***for-loop*** as;

min_val = 1 & *max_val* = 5 or totalRows

Let's consider;

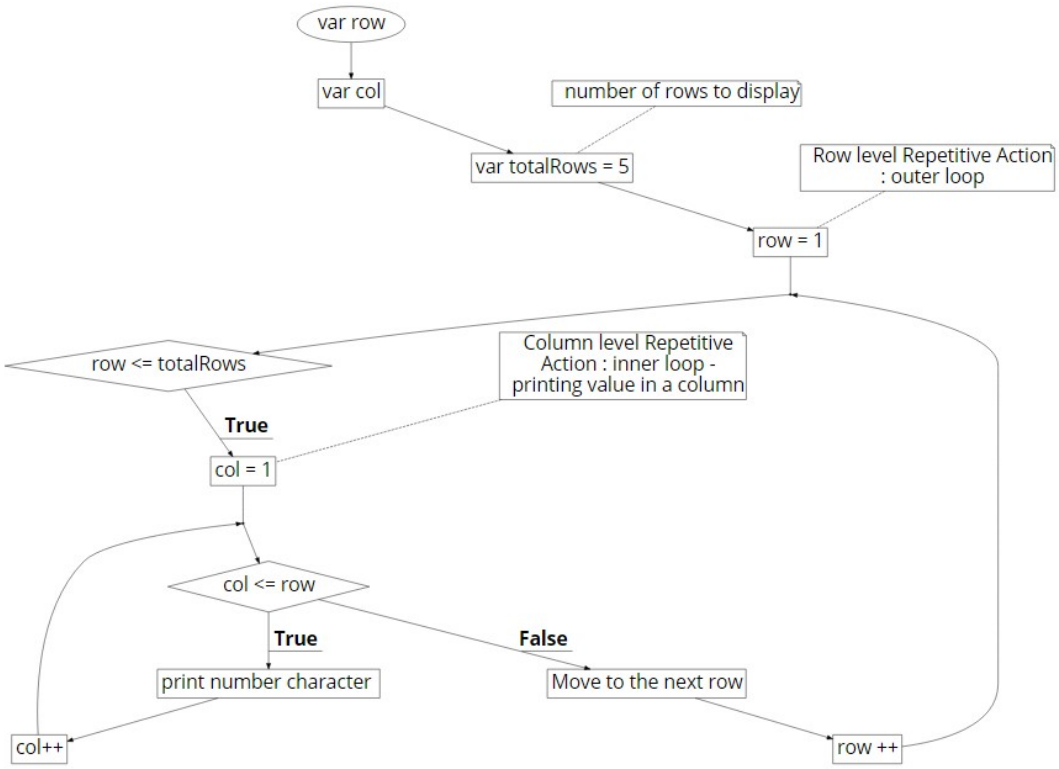
row → variable type of integer to store integer value one at a time and represents row number between 1 and 5(totalRows). The **totalRows** has initially assigned the value 5.

Outer ***for-loop*** starts at row=1 & will repeat till row <= **totalRows**.

Elements Of* Row Level *for-loop	
min_val	**1**
max_val	**totalRows**
variable_name	**row**
statement(s)	**Column level *for-loop*** System.out.println();// Move cursor to the next row

We got everything related to ***for-loop*** in order to implement row level repetition.

We can summarise all our finding in the following flow-chart.



Let's execute the following program.

NumberPattern1A.java

```
public class NumberPattern1A {

	public static void main(String[] args) {

		int  totalRows = 5;  //number of rows to display

		for( int row = 1 ; row <= totalRows ; row ++ ) {

			System.out.println();// move to the next line

			for(int col = 1 ; col <= row  ; col++) {

				// added extra space for clarity in output.
				String num = row + " ";
				System.out.print( num );
			}

		}
	}

}
```

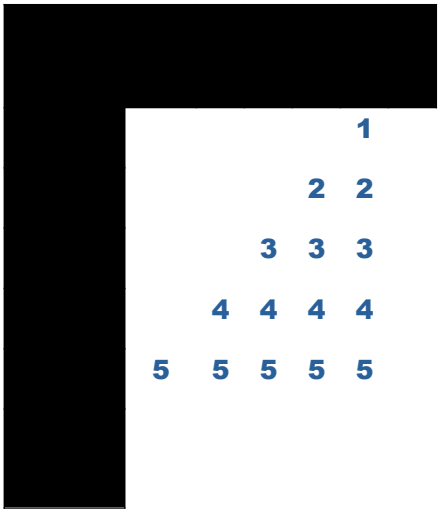
Output:totalRows = 5

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```

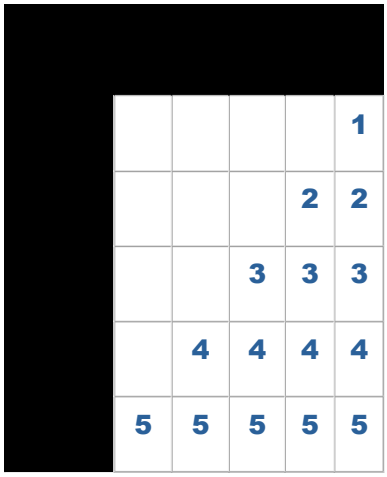
Output:totalRows = 7

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
6 6 6 6 6 6
7 7 7 7 7 7 7
```

NumberPattern 1B



Our programming job is to print this particular logic-based pattern on a part of the computer screen. **Our approach will be assumed that the computer screen is a combination of rows and columns virtually dividing it into smaller boxes.** The smallest box is treated as a resident of just a single character only. The address in each smallest box on the display screen will logically become a unique **(row, column)** numeric combination. With this assumption computer screen will simply be a unique combination of (row, column) numeric value paired boxes which are uniformly spread all across the screen.



	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1					1
Row=2				2	2
Row=3			3	3	3
Row=4		4	4	4	4
Row=5	5	5	5	5	5

Data Representation

Although it is logically, we can exactly locate the position of each printable character on the display screen by knowing it's uniquely identifiable **(row, column)** numeric paired-values. For example; **(4, 5)** means printing numeric character at the assumed screen position **4th row and 5th column.**

(Row, Col) value pairs representation

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1					1,5
Row=2				2 ,4	2,5
Row=3			3,3	3,4	3,5
Row=4		4,2	4,3	4,4	4,5
Row=5	5,1	5,2	5,3	5,4	5,5

The given set of row and col value pairs tries to mimic the location of each printable character of a given pattern on the device's display screen. The set of all value pairs (row = value, col = value) represent this particular pattern.

Analysis of output

In this case we need to take care of printing two different types of character at a particular column in a particular row. The character types are the following.

1) Space character

2) Numeric character

Tabular form representation

row	1				
col	1	2	3	4	5
Output	space	space	space	space	1

For row =1, 1 <=spaces<=4, 1>=num>=1

row	2				
col	1	2	3	4	5
Output	space	space	space	2	2

For row =2, 1 <=spaces<=3, 2>=num>=1

row	3				
col	1	2	3	4	5
Output	space	space	3	3	3

For row =3, 1 <=spaces<=2, 3>=num>=1

row	4				
col	1	2	3	4	5
Output	space	4	4	4	4

For row =4, 1 <=spaces<=1, 4>=num>=1

row	5				
col	1	2	3	4	5
Output	5	5	5	5	5

For row =5, 5>=num>=1

Observations

1) There is repetition at row level vertically, i.e., Row = 1 to 5.
2) There is repetition at col level horizontally. , i.e., Col = 1 to 5.
   This gets divided into two parts .One for repetitively printing spaces and remaining column for printing numeric characters.

For Loop	Lower limit	Upper limit
Outer loop	row=1	row<=5
Inner Loop - spaces	spaces=1	spaces <= ?(Unkown)
Inner Loop - Number	num = ?(Unkown)	num = 1

COLUMN LEVEL REPETITIVE ACTION

Row level Repetitive Action						
row	Column level Repetitive Action					
	lower_limit <=spaces<= upper_limit			upper_limit>=num>=lower_limit		
	Range	lower_limit	upper_limit	Range	upper_limit	lower_limit
1	1 <=spaces<=4	1	4	1>=num>=1	1	1
2	1 <=spaces<=3	1	3	2>=num>=1	2	1
3	1 <=spaces<=2	1	2	3>=num>=1	3	1
4	1 <=spaces<=1	1	1	4>=num>=1	4	1
5		1	0	5>=num>=1	5	1

The lower_limit for spaces & num variable has the same numeric value expression for all the rows.

It's consistent for all the rows. Likewise we need to come out with such a consistent expression for upper_limit for which it should suit any number of rows.

The other remaining variables are row & totalRows=5. Let's try get generalized logical condition using these variables.

Column level Repetitive Action							
row	lower_limit <=spaces<= upper_limit				upper_limit >=num >= lower_limit		
	lower_limit	upper_limit			upper_limit		lower_limit
		default	1	2	default	1	
1	1	4	totalRows - 1	totalRows - row	1	row	1
2	1	3	totalRows - 2	totalRows - row	2	row	1
3	1	2	totalRows - 3	totalRows - row	3	row	1
4	1	1	totalRows - 4	totalRows - row	4	row	1
5	1	0	totalRows - 5	totalRows - row	5	row	1

So the upper_limit of spaces & num variables are generalized.

The lower_limit & upper_limit are consistent for all the rows. It's now generalized for n number of rows.

So our **If-condition**(lower_limit <=spaces<= upper_limit) becomes

1 <= spaces <= (totalRows – row)

The I**f-condition**(upper_limit >=num >= lower_limit) becomes

row >=num >= 1

So it has two repetition process one for printing spaces and second for printing numeric values.

It means it requires **two inner for-loops** (nested for – loops).

For Loop	Lower limit	Upper limit
Outer loop	row=1	row<= totalRows
Inner Loop - spaces	col=1	col <= (totalRows – row)
Inner Loop - Number	num = row	num = 1

We can now define column level ***for-loop*** which is for every row value.

Elements of Column level *for-loop*	
min_val	**1**
max_val	**totalRows -row**
variable_name	**col**
statement(s)	System.out.print(spaces);

Elements of Column level *for-loop*	
min_val	**1**
max_val	**row**
variable_name	**num**
statement(s)	System.out.print(num);

ROW LEVEL REPETITIVE ACTION

The same following statements had been repeated 5 times(total number of rows).

Column level *for-loop* for printing space characters// ***inner for-loop***

Column level *for-loop* for printing star characters// ***inner for-loop***

```
System.out.println();// Moves cursor to the next row
```

We can observe from the pattern that the total number of rows are 5.

We have assigned unique value to each row starting from 1 till 5.

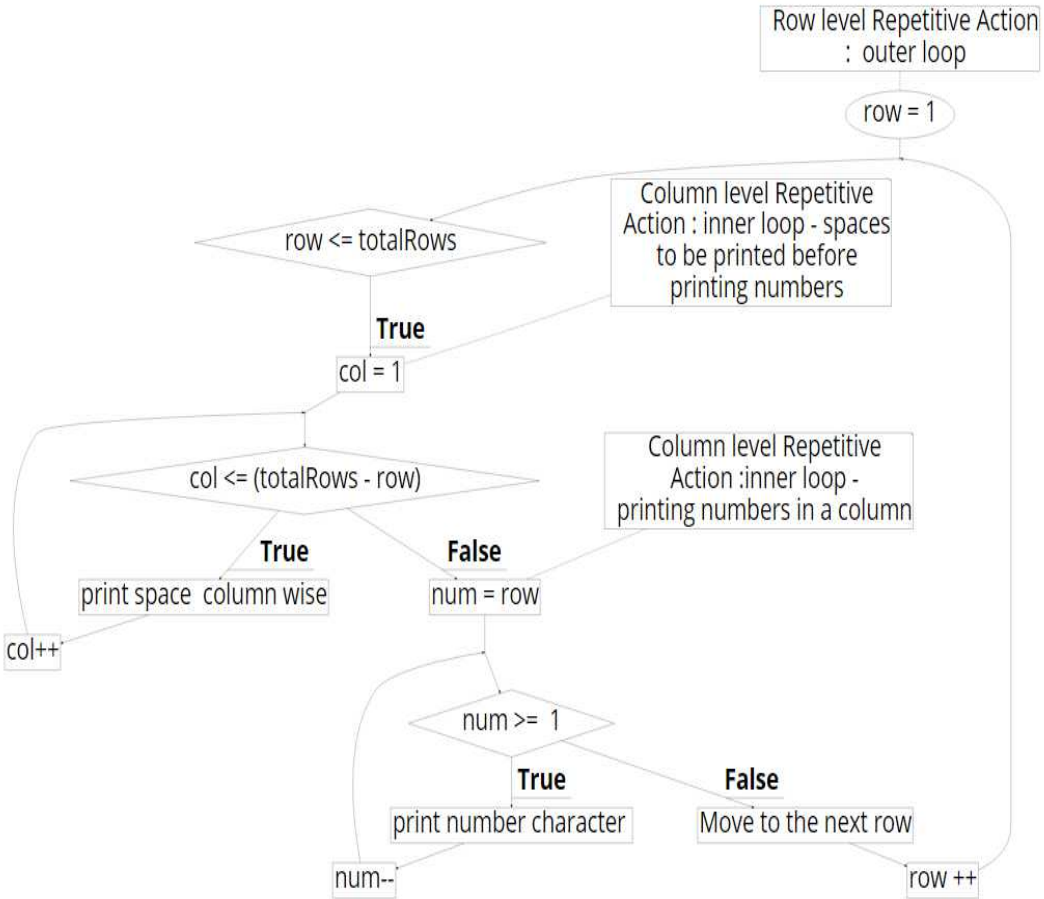
So, the minimum and maximum value can be given for the ***for-loop*** at row level as;

min_val = 1 & *max_val* = 5 or totalRows

Outer ***for-loop*** starts at row=1 & will repeat till row <= **totalRows**.

Elements Of **Row Level** ***for-loop***	
min_val	**1**
max_val	**totalRows**
variable_name	**row**
statement(s)	**Column level *for-loop*** for printing space characters. **Column level *for-loop*** for printing star characters System.out.println();;// Move cursor to the next row

We got everything related to ***for-loop*** required for row level repetition.



Let's execute the following program.

NumberPattern1B.java

```
public class NumberPattern1B {

	public static void main(String[] args) {

		int  totalRows = 5;  //number of rows to display

		for( int row = 1 ; row <= totalRows ; row ++ ) {

			System.out.println();// move control to the next line

			// spaces to be printed before printing numbers
			for(int col = 1 ; col <= (totalRows - row)   ; col++) {
				System.out.print( "  " );
			}

			for(int num = row ; num >=  1 ; num--) {

				// added extra space for clarity in output.
				String numStr = row  + " ";
				System.out.print( numStr );
			}
		}
	}
}
```

Output: totalRows = 5

```
        1
      2 2
    3 3 3
  4 4 4 4
5 5 5 5 5
```

Output: totalRows = 7

```
          1
        2 2
      3 3 3
    4 4 4 4
  5 5 5 5 5
6 6 6 6 6 6
7 7 7 7 7 7 7
```

Please, test for different number of lines, let it be 7. It should work correctly, so that we can be confirmed that our solution is generalized and fit for any number of lines.

Alternative solution

We observed previously that for row=5, the values of col and numeric output are just reversed.

Let's device a solution by decrementing the for-loop for col variable (5 to 1) so the numeric output will match with the value of row variable.

row	5				
col	1	2	3	4	5
output	5	4	3	2	1

For row =5, 5>=num>=1 , 1 <=col<=5

row	1				
col	5	4	3	2	1
output	space	space	space	space	1

row =1 , 1>=col>=1 (print row value) or row >=col>=1 (print row value)

row	2				
col	5	4	3	2	1
output	space	space	space	2	2

row = 2 , 2>=col>=1 (print row value) or row >=col>=1 (print row value)

row	3				
col	5	4	3	2	1
output	space	space	3	3	3

row = 3 , 3>=col>=1 (print row value) or row >=col>=1 (print row value)

row	4				
col	5	4	3	2	1
output	space	4	4	4	4

row = 4 , 4>=col>=1 (print row value) or row >=col>=1 (print row value)

row	5				
col	5	4	3	2	1
output	5	5	5	5	5

row = 5 , 5>=col>=1 (print row value) or row >=col>=1 (print row value)

So, **if –condition** = (col >=1 and col<=row)

col >=1 is redundant condition ,since it's a lower limit in the inner for-loop for col = 5 to 1.

So, **if –condition** reduced to (col<=row)

The overall observation is at each row, either it's printing space characters or row values in a continues fashion. If we take care of only the logical condition of printing spaces, then it's quite obvious that the condition can be expressed in if-else.

The printing of space or numeric characters are continuous.

Let's execute the following program.

NumberPattern1B_2.java

```
public class NumberPattern1B_2 {

	public static void main(String[] args) {

		int  totalRows = 5;  //number of rows to display

		for( int row = 1 ; row <= totalRows ; row ++ ) {

			System.out.println();// move control to the next line

			//prints variable col value for (row >= col >=1)
			for(int col = totalRows ; col >= 1 ; col--) {
				if( col<=row )   {
					System.out.print( row + " " );
				}else {
					System.out.print( "  " );
				}
			}
		}
	}
}
```

Output: totalRows = 5

```
          1
        2 2
      3 3 3
    4 4 4 4
  5 5 5 5 5
```

Output: totalRows = 7

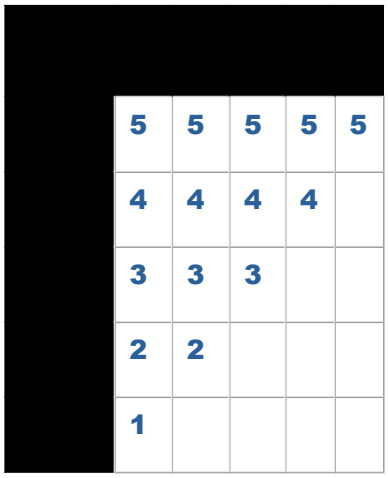
```
            1
          2 2
        3 3 3
      4 4 4 4
    5 5 5 5 5
  6 6 6 6 6 6
7 7 7 7 7 7 7
```

Please, test for different number of lines, let it be 7. It should work correctly.

NumberPattern 1C



Our programming job is to print this particular logic-based pattern on a part of the computer screen. **Our approach will be assumed that the computer screen is a combination of rows and columns virtually dividing it into smaller boxes.** The smallest box is treated as a resident of just a single character only. The address in each smallest box on the display screen will logically become a unique **(row, column)** numeric combination. With this assumption computer screen will simply be a unique combination of (row, column) numeric value paired boxes which are uniformly spread all across the screen.



	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	5	5	5	5	5
Row=2	4	4	4	4	
Row=3	3	3	3		
Row=4	2	2			
Row=5	1				

Data Representation

Although it is logically, we can exactly locate the position of each printable character on the display screen by knowing it's uniquely identifiable **(row, column)** numeric paired-values. *For example;* **(2, 3)** means printing numeric character at the assumed screen location **2nd row and 3rd column.**

(Row, Col) value pairs representation

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	1,1	1,2	1,3	1,4	1,5
Row=2	2,1	2,2	2,3	2,4	
Row=3	3,1	3,2	3,3		
Row=4	4,1	4,2			
Row=5	5,1				

The given set of row and col value pairs tries to mimic the location of each printable character of a given pattern on the device's display screen. The set of all value pairs (row = value, col = value) represent this particular pattern.

Analysis of output

Let's device a solution by incrementing the for-loop for col variable (1 to 5) and determining the if-condition(lower_limit <=col<= upper_limit) of col variable to give the desired numeric output.

row	1				
col	1	2	3	4	5
output	5	5	5	5	5

row =1 , 1 <=Col <= 5 (print upper_limit=5)

row	2				
col	1	2	3	4	5
output	4	4	4	4	

row = 2 , (1 <=Col <= 4) (print upper_limit=4)

row	3				
col	1	2	3	4	5
output	3	3	3		

row = 3 , (1 <=Col <= 3) (print upper_limit=3)

row	4				
col	1	2	3	4	5
output	2	2			

row = 4 , (1 <=Col <= 2) (print upper_limit=2)

row	5				
col	1	2	3	4	5
output	1				

row = 5 , (1 <=Col <= 1) (print upper_limit=1)

COLUMN LEVEL REPETITIVE ACTION

The numeric output will be the value of row variable printed that many times equals the upper_limit of col variable. The upper_limit of col variable varies with the value of row variable.

Row level Repetitive Action			Expected Output
row	Column level Repetitive Action		
	lower_limit <=col<= upper_limit		
	lower_limit	upper_limit	
1	1	5	**5 5 5 5 5**
2	1	4	**4 4 4 4**
3	1	3	**3 3 3**
4	1	2	**2 2**
5	1	1	**1**

The lower_limit has same numeric value expression for the rows .It's consistent for all the rows.

Likewise we need to come out with such a consistent expression for upper_limit which should suit any number of rows. The other remaining variables are **row, totalRows**=5.

Let's try to generalize logical expression using these variables.

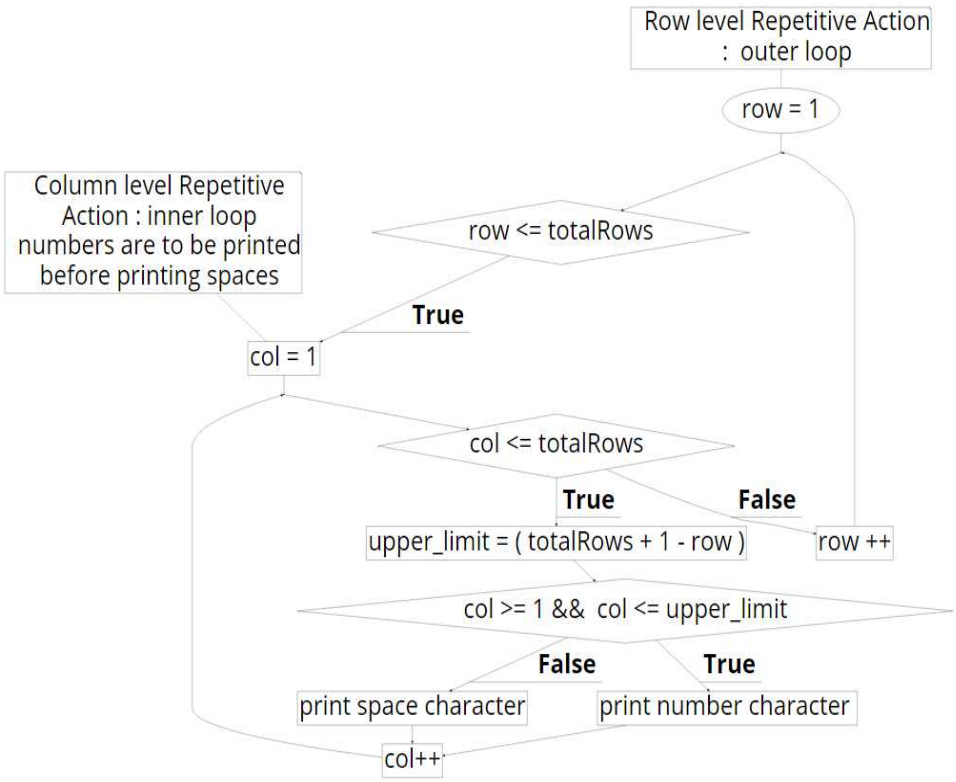
Column Level Repetitive Action					
row	**lower_limit <=col<= upper_limit**				
	lower_limit	**upper_limit**			
		default	**1**	**2**	**3**
1	1	5	totalRows	(totalRows +1) - 1	(totalRows +1) - row
2	1	4	totalRows -1	(totalRows +1) - 2	(totalRows +1) - row
3	1	3	totalRows - 2	(totalRows +1) - 3	(totalRows +1) - row
4	1	2	totalRows - 3	(totalRows +1) - 4	(totalRows +1) - row
5	1	1	totalRows - 4	(totalRows +1) - 5	(totalRows +1) - row

So the upper_limit of col variable is generalized to (totalRows + 1 – row).

The lower_limit & upper_limit are consistent for all the rows. It's now generalized for n number of rows. So our If-condition becomes

If-condition	lower_limit <=col<= upper_limit
Mathematical expression	1 <= col <= ((totalRows +1) – row)
Java syntax	col >= 1 && col <= (totalRows + 1 - row)

The flow-chart can be shown which is as follows.



Since inner for-loop's *lower_limit* matches with the if-condition's *lower_limit*.

```
for(int col = 1 ; col <= totalRows ; col++) {
        if( col >= 1 &&  col <= ( totalRows + 1 – row ) ) {
                ---- statements ------
        }else{
         // print space(s)
      }
}
```

So, we can safely neglect *lower_limit* in the if-condition as it is already been taken care by it's own for-loop's starting value. Hence, our final if-condition will be,

If-condition	lower_limit <=col<= upper_limit
Mathematical expression	col <= ((totalRows +1) – row)
Java syntax	col <= (totalRows + 1 - row)

If we need to print something after some space(s), then in that case putting program logic to

print space(s) make sense. The other way round will not be a sensible exercise.

NumberPattern1C_1.java

```
public class NumberPattern1C_1 {

      public static void main(String[] args) {

            int  totalRows = 5;  //number of rows to display

            for( int row = 1 ; row <= totalRows ; row ++ ) {

                  System.out.println();// move control to the next line

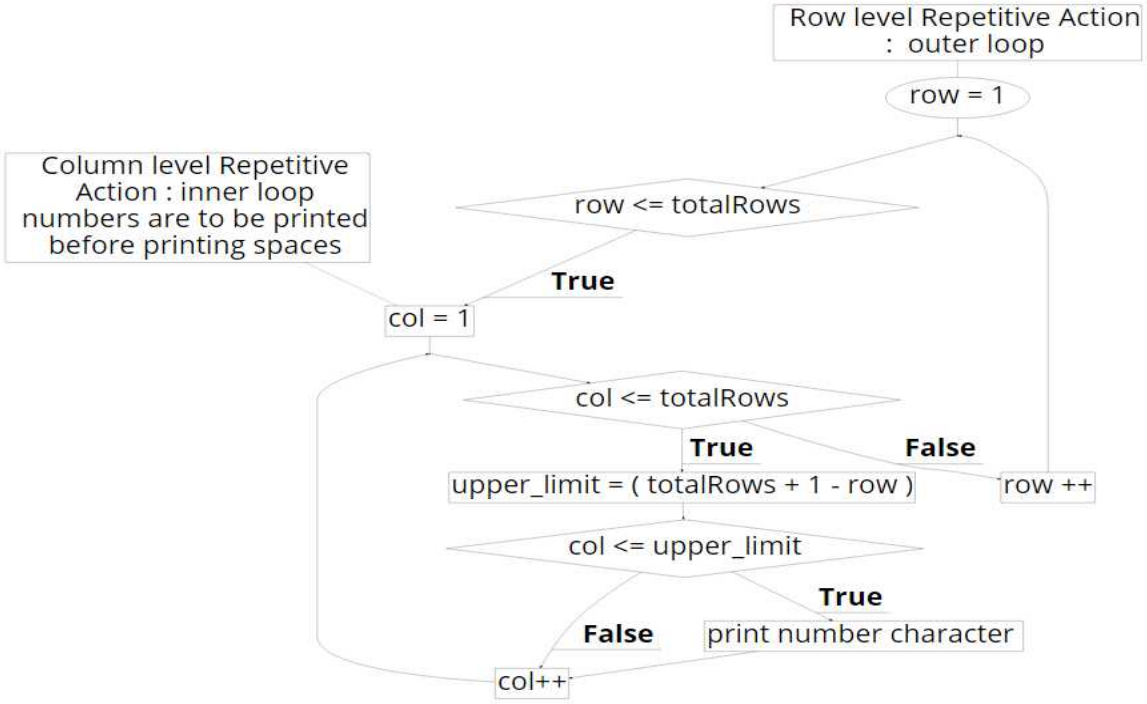
                  // numbers to be printed before printing spaces
                  for(int col = 1 ; col <= totalRows    ; col++) {

                        int upper_limit = ( totalRows + 1 - row );

                        if( col >= 1 &&  col <= upper_limit  ) {
                              System.out.print( upper_limit + " " );
                        }else {
                              System.out.print( "  " );
                        }
                  }//inner loop

            }// outer loop
      }
}
```

So in this particular case we can avoid participation of else-condition which further simplifies our condition logic. We can further refine our flow-chart given as follows.



NumberPattern1C_2.java

```
public class NumberPattern1C_2 {

	public static void main(String[] args) {

		int  totalRows = 5;  //number of rows to display

		for( int row = 1 ; row <= totalRows ; row ++ ) {

			System.out.println();// move control to the next line

			// spaces to be printed before printing numbers
			for(int col = 1 ; col <= totalRows    ; col++) {
				int upper_limit = ( totalRows + 1 - row );
				if( col <= upper_limit  ) {
					System.out.print( upper_limit + " " );
				}
			}
		}
	}
}
```

We know that the total number of column level repetition are the total number of rows.

Furthermore, inner loop's if-condition is responsible for decreasing the count of printing of numeric characters in columns. So the total number of repetition should also go on decreases as the number

of rows goes on increases. Rows where not all column places are filled with numeric character(s) are unnecessarily looping more. We can stop this extra looping by dynamically matching the *upper_limit* of inner for-loop with the if-condition.

Hence, we can completely remove the if-condition and replace the logical expression of *upper_limit* of inner for-loop with the logical expression used in **If-condition**.

NumberPattern1C_3.java

```
public class NumberPattern1C_3 {

	public static void main(String[] args) {

		int totalRows = 5;  //number of rows to display

		for( int row = 1 ; row <= totalRows ; row ++ ) {

			System.out.println();// move control to the next line

			int upper_limit = ( totalRows + 1 - row );

			for(int col = 1 ; col <= upper_limit ; col++ ) {
				System.out.print( upper_limit + " " );
			}
		}
	}
}
```

Output: totalRows = 5

```
5 5 5 5 5
4 4 4 4
3 3 3
2 2
1
```

Output: totalRows = 7

```
7 7 7 7 7 7 7
6 6 6 6 6 6
5 5 5 5 5
4 4 4 4
3 3 3
2 2
1
```

Please, test the output for different number of lines, let it be 7. It should work correctly.

Alternative solution

This time let's use decrementing outer for-loop using row variable (5 to 1) and try to determine the logical condition in order to position the numeric character(s) in our row by column picture format.

row	5				
col	1	2	3	4	5
output	5	5	5	5	5

row =5 , 1 <=Col <= 5 (print upper_limit=5)

row	4				
col	1	2	3	4	5
output	4	4	4	4	

row = 4 , (1 <=Col <= 4) (print upper_limit=4)

row	3				
col	1	2	3	4	5
output	3	3	3		

row = 3 , (1 <=Col <= 3) (print upper_limit=3)

row	2				
col	1	2	3	4	5
output	2	2			

row = 2 , (1 <=Col <= 2) (print upper_limit=2)

row	1				
col	1	2	3	4	5
output	1				

row = 1 , (1 <=Col <= 1) (print upper_limit=1)

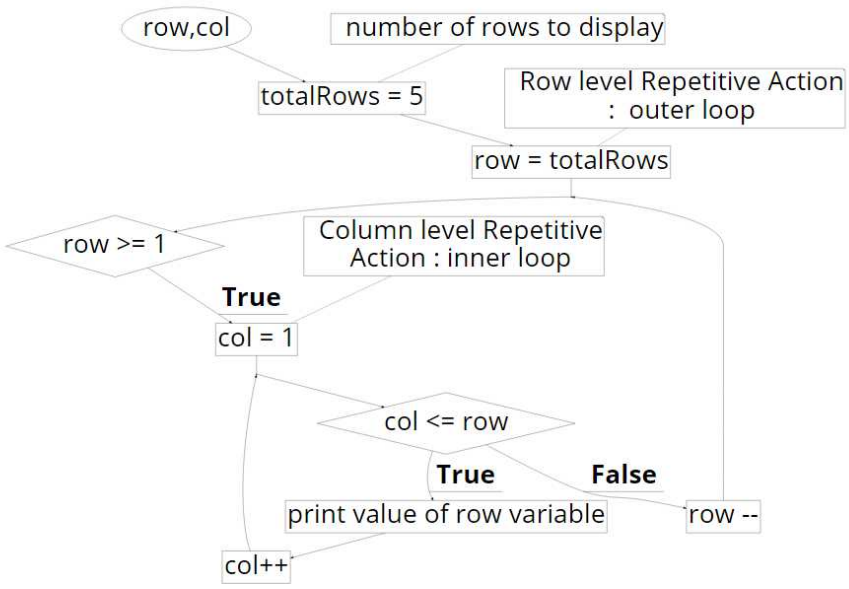
Column level Repetitive Action			Expected Output
row	lower_limit <=col<= upper_limit		
	lower_limit	upper_limit	
5	1	5	**5 5 5 5 5**
4	1	4	**4 4 4 4**
3	1	3	**3 3 3**
2	1	2	**2 2**
1	1	1	**1**

The numeric output will be the value of row variable printed that many times equals the upper_limit of row variable. The upper_limit of row variable varies and matches with the value of row variable.

Column level Repetitive Action			Expected Output
row	If-condition(lower_limit <=col<= upper_limit)		
	lower_limit	upper_limit	
5	1	row	**5 5 5 5 5**
4	1	row	**4 4 4 4**
3	1	row	**3 3 3**
2	1	row	**2 2**
1	1	row	**1**

So our loop-condition(lower_limit <=col<= upper_limit) becomes

loop-condition	lower_limit <=col<= upper_limit
Mathematical expression	1 <= col <= row
Java syntax	for(int col = 1; col <=row ; col++)



Hence, our final program will be,

NumberPattern2C.java

```
public class NumberPattern2C {

	public static void main(String[] args) {

		int totalRows = 5;  //number of rows to display

		for( int row = totalRows ; row >= 1 ; row -- ) {

			System.out.println();// move control to the next line

			// row value to be printed
			for(int col = 1 ; col <= row    ; col++) {
				System.out.print( row + " " );
			}
		}
	}
}
```

Output: totalRows = 5

```
5 5 5 5 5
4 4 4 4
3 3 3
2 2
1
```

Output: totalRows = 7

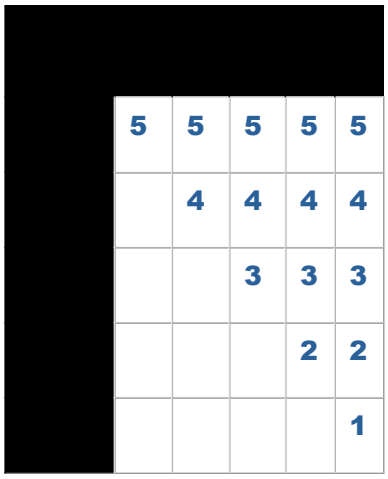
```
7 7 7 7 7 7 7
6 6 6 6 6 6
5 5 5 5 5
4 4 4 4
3 3 3
2 2
1
```

Please, test for different number of lines, let it be 7. It should work correctly.

NumberPattern 1D



Our programming job is to print this particular logic-based pattern on a part of the computer screen. **Our approach will be assumed that the computer screen is a combination of rows and columns virtually dividing it into smaller boxes.** The smallest box is treated as a resident of just a single character only. The address in each smallest box on the display screen will logically become a unique **(row, column)** numeric combination. With this assumption computer screen will simply be a unique combination of (row, column) numeric value paired boxes which are uniformly spread all across the screen.



	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	5	5	5	5	5
Row=2		4	4	4	4
Row=3			3	3	3
Row=4				2	2
Row=5					1

Data Representation

Although it is logically, we can exactly locate the position of each printable character on the display screen by knowing it's uniquely identifiable **(row, column)** numeric paired-values. *For example;* **(3, 4)** means printing numeric character at the assumed screen location **3rd row and 4th column.**

(Row, Col) value pairs representation

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	1,1	1,2	1,3	1,4	1,5
Row=2		2,2	2,3	2,4	2,5
Row=3			3,3	3,4	3,5
Row=4				4,4	4,5
Row=5					5,5

The given set of row and col value pairs tries to mimic the location of each printable character of a given pattern on the device's display screen. The set of all value pairs (row = value, col = value) represent this particular pattern.

Analysis of output

In this case we need to take care of printing two different types of characters at a particular column in a particular row. The character types are the following.

1) Space character

2) Numeric character

row	5				
Col	1	2	3	4	5
Output	5	5	5	5	5

For row =1, 0 <=spaces<=0, 1<=num<=5

row	4				
Col	1	2	3	4	5
Output	space	4	4	4	4

For row =2, 1 <=spaces<=1, 1<=num<=4

row	3				
Col	1	2	3	4	5
Output	space	space	3	3	3

For row =3, 1 <=spaces<=2, 1<=num<=3

row	2				
Col	1	2	3	4	5
Output	space	space	space	2	2

For row =4, 1 <=spaces<=3, 1<=num<=2

row	1				
Col	1	2	3	4	5
Output	space	space	space	space	1

For row =5, 1 <=spaces<=4, 1<=num<=1

Observations

1) There is repetition at row level vertically, i.e., row = 1 to 5.
2) There is repetition at col level horizontally. , i.e., col = 1 to 5.
   This gets divided into two parts. One for repetitively printing spaces and remaining column for printing numeric characters.

For Loop	Lower limit	Upper limit
Outer loop(decrementing)	row=1	row>=5
Inner Loop - spaces	spaces=1	spaces <= ?(Unkown)
Inner Loop - Number	num = 1	num =?(Unkown)

COLUMN LEVEL REPETITIVE ACTION

So it has two repetition process one for printing spaces and second for printing numeric values.

It means require 2 inner for-loops and 1 outer loop for iterating rows. The relation between row value and upper limit for inner for-loop for spaces and numbers.

Row level Repetitive Action					Expected Output
row	Column level Repetitive Action				
	lower_limit<=col_sp<= upper_limit		lower_limit <=col_num <= upper_limit		
	lower_limit	upper_limit	lower_limit	upper_limit	
5	1	0	1	5	5 5 5 5 5
4	1	1	1	4	4 4 4 4
3	1	2	1	3	3 3 3
2	1	3	1	2	2 2
1	1	4	1	1	1

The lower_limit for spaces & num variable has the same numeric value expression for all the rows.

It's consistent for all the rows. Likewise we need to come out with such a consistent expression

for upper_limit for which it should suit any number of rows.

The other remaining variables are row & totalRows=5. Let's try to generalize using these variables.

Column level Repetitive Action						
row	lower_limit <=col_sp<= upper_limit			lower_limit <=col_num <= upper_limit		
	lower_limit	upper_limit		upper_limit	lower_limit	
		default	1		default	
5	1	0	totalRows - row	1	5	5 5 5 5 5
4	1	1	totalRows - row	1	4	4 4 4 4
3	1	2	totalRows - row	1	3	3 3 3
2	1	3	totalRows - row	1	2	2 2
1	1	4	totalRows - row	1	1	1

Now the uppler_limit of col_sp variable related inner for-loop is generalized, same expression on all the rows. The lower_limit of col_num variable related inner for-loop is not generalized as not having same expression on all the rows.

Since lower_limit of col_num variable values matches with row variable values.

So, we can use row variable to make it consistent given as follows.

Column level Repetitive Action						
row	lower_limit <=col_sp<= upper_limit			lower_limit <=col_num <= upper_limit		
	lower_limit	upper_limit		upper_limit	lower_limit	
		default	1		default	1
5	1	0	totalRows - row	1	5	row
4	1	1	totalRows - row	1	4	row
3	1	2	totalRows - row	1	3	row
2	1	3	totalRows - row	1	2	row
1	1	4	totalRows - row	1	1	row

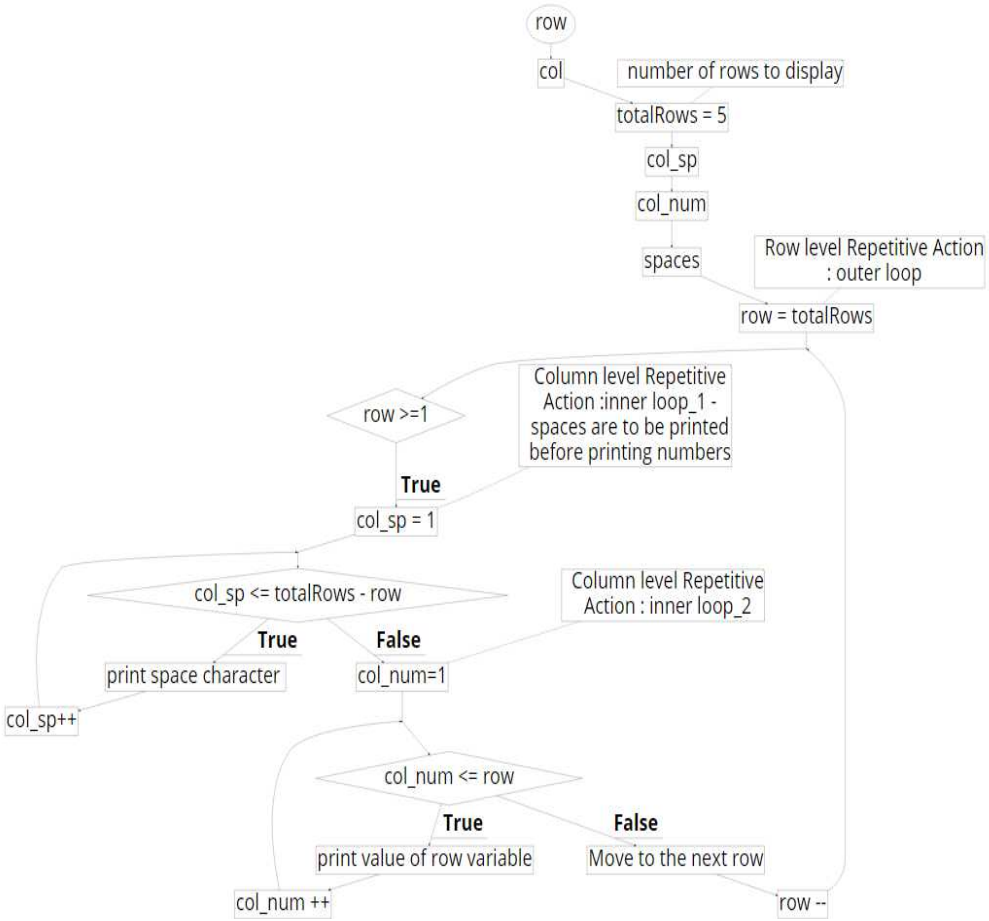
The lower_limit of col_sp variable is same for all the rows .The upper_limit of col_num variable is same for all the rows. It means the upper_limit of col_sp & col_num variables are generalized for any number of rows. So our inner for-loop condition for printing space characters becomes;

loop-condition	lower_limit <=col_sp<= upper_limit
Mathematical expression	1 <= col_sp <= (totalRows – row)
Java syntax	for(col_sp = 1;col_sp<=(totalRows – row); col_sp++)

So our inner for-loop condition for printing numeric characters becomes;

loop-condition	lower_limit <=col_num<= upper_limit
Mathematical expression	1 <= col_num <= row
Java syntax	for(col_num=1; col_num <= row ; col_num ++)

The flow-chart for our observations can be given as follows.



Let's write the program as per the flow-chart.

Please do test for different number of lines, let **change** it to be 7 and execute it. It should work correctly.

NumberPattern1D.java

```
public class NumberPattern1D {

	public static void main(String[] args) {

		int  totalRows = 5;  //number of rows to display

		for( int row = totalRows ; row >=1  ; row -- ) {

			System.out.println();// move control to the next line

			// spaces to be printed
			for( int col = 1; col <= ( totalRows - row ) ; col++ ) {
				System.out.print( "  " );
			}

			// row value to be printed
			for( int num=1; num<= row ; num++ ){
				System.out.print( row + " " );
			}

		}
	}
}
```

Output: totalRows = 5

```
5 5 5 5 5
  4 4 4 4
    3 3 3
      2 2
        1
```

Output: totalRows = 7

```
7 7 7 7 7 7 7
  6 6 6 6 6 6
    5 5 5 5 5
      4 4 4 4
        3 3 3
          2 2
            1
```

Alternative solution

Since the column values need to split clearly between printing spaces and numbers. We can try to do this using if-else condition construct.

NumberPattern2D.java

```
public class NumberPattern2D {

	public static void main(String[] args) {

		int  totalRows = 5;  //number of rows to display

		for( int row = totalRows ; row >=1  ; row -- ) {

			System.out.println();// move control to the next line

			for(int col = 1; col <= ( totalRows  ) ; col++) {

				if( col <= ( totalRows - row ) ) {

					// spaces to be printed
					System.out.print( "  " );

				}else {

					// row value to be printed
					System.out.print( row + " " );

				}

			}
		}
	}
}
```

Output

```
5 5 5 5 5
  4 4 4 4
    3 3 3
      2 2
        1
```

Alternative solution

Let's device a solution by decrementing the **for-loop** for col variable (5 to 1) so the numeric output will match with the value of row variable.

row	5				
col	5	4	3	2	1
output	5	5	5	5	5

row =5 , 5>=col>=1 (print row value) or row >=col>=1 (print row value)

row	4				
col	5	4	3	2	1
output	space	4	4	4	4

row = 4 , 4>=col>=1 (print row value) or row >=col>=1 (print row value)

row	3				
col	5	4	3	2	1
output	space	space	3	3	3

row = 3 , 3>=col>=1 (print row value) or row >=col>=1 (print row value)

row	2				
col	5	4	3	2	1
output	space	space	space	2	2

row = 2 , 2>=col>=1 (print row value) or row >=col>=1 (print row value)

row	1				
col	5	4	3	2	1
output	space	space	space	space	1

row = 1 , 1>=col>=1 (print row value) or row >=col>=1 (print row value)

Row level Repetitive Action				**Expected Output**
row	**Column level Repetitive Action**			
	upper_limit >=col>= upper_limit			
	upper_limit		**lower_limit**	
	default	**1**		
5	5	row	1	5 5 5 5 5
4	4	row	1	4 4 4 4
3	3	row	1	3 3 3
2	2	row	1	2 2
1	1	row	1	1

So, if –condition can be given as

if-condition	upper_limit >=col>= lower_limit
Mathematical expression	row >= col >= 1
Java syntax	If(col>=1 && col <= row)

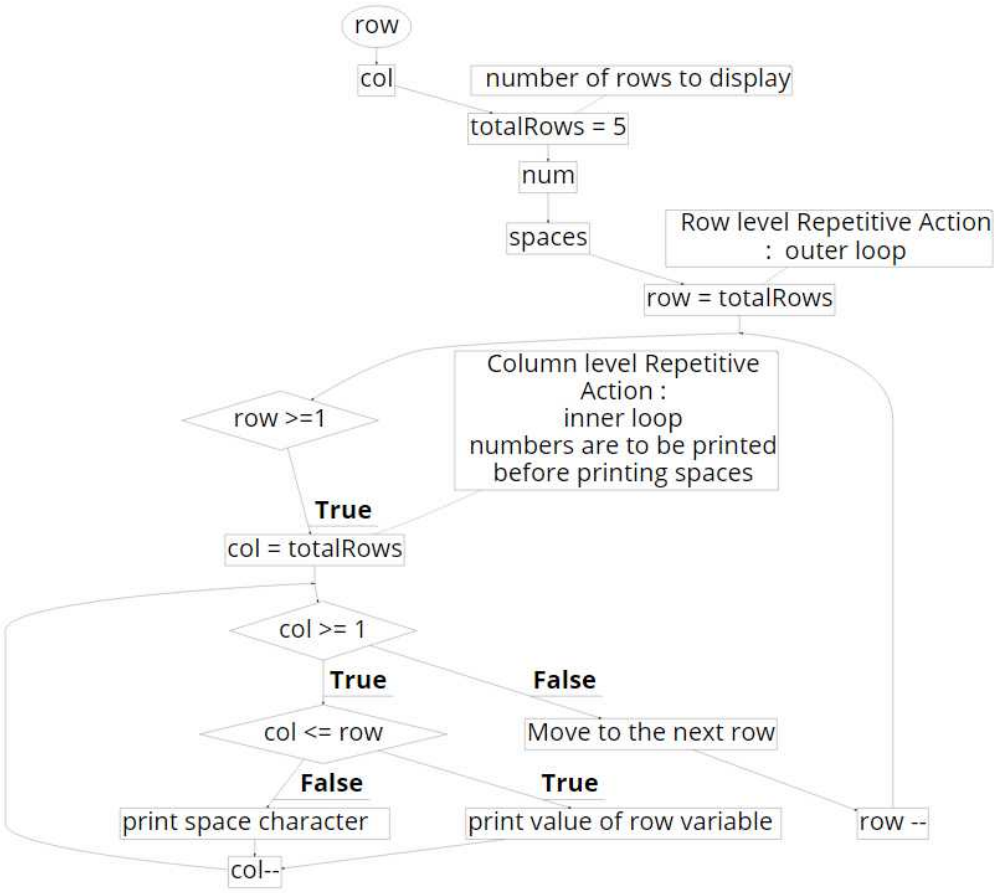
The part of if-condition col >=1 is already taken care in the lower_limit of inner for-loop.

So, if –condition is now becomes;

if-condition	upper_limit >=col>= lower_limit
Mathematical expression	col<=row
Java syntax	if(col <= row)

The overall observation at each row is either it's printing space characters or row values in a continues fashion in every column but in a different way.

If we take care of condition of printing numbers only, then the else part will for printing spaces and can be easily expressed in **if-else statement** for printing desired output.



Let's try to execute the following program.

NumberPattern3D.java

```
public class NumberPattern3D {

	public static void main(String[] args) {

		int  totalRows = 5;  //number of rows to display

		for( int row = totalRows ; row >=1  ; row -- ) {

			System.out.println();// move control to the next line

			for( int col = totalRows; col >= 1 ; col-- ) {

				if( col <=  row ) {
					// row value to be printed
					System.out.print( row + " " );
				}else {
					// spaces to be printed
					System.out.print( "  " );
				}

			}
		}
	}
}
```

Output:totalRows = 5

```
5 5 5 5 5
  4 4 4 4
    3 3 3
      2 2
        1
```

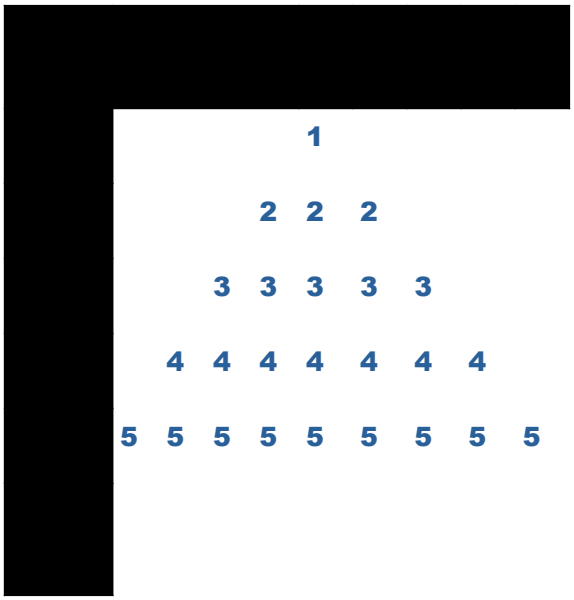
Output:totalRows = 7

```
7 7 7 7 7 7 7
  6 6 6 6 6 6
    5 5 5 5 5
      4 4 4 4
        3 3 3
          2 2
            1
```

Please, test for different number of lines, let it be 7. It should work correctly.

NumberPattern 1E

Our programming job is to print this particular logic-based pattern on a part of the computer screen. **Our approach will be assumed that the computer screen is a combination of rows and columns virtually dividing it into smaller boxes.** The smallest box is treated as a resident of just a single character only. The address in each smallest box on the display screen will logically become a unique **(row, column)** numeric combination. With this assumption computer screen will simply be a unique combination of (row, column) numeric value paired boxes which are uniformly spread all across the screen.



	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1					1				
Row=2				2	2	2			
Row=3			3	3	3	3	3		
Row=4		4	4	4	4	4	4	4	
Row=5	5	5	5	5	5	5	5	5	5

Data Representation

Although it is logically, we can exactly locate the position of each printable character on the display screen by knowing it's uniquely identifiable **(row, column)** numeric paired-values. *For example;* **(4, 3)** means printing numeric character at the assumed screen location **4th row and 3rd column.**

(Row, Col) value pairs representation

	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1					1,5				
Row=2				2,4	2,5	2,6			
Row=3			3,3	3,4	3,5	3,6	3,7		
Row=4		4,2	4,3	4,4	4,5	4,6	4,7	4,8	
Row=5	5,1	5,2	5,3	5,4	5,5	5,6	5,7	5,8	5,9

Analysis of output

row	1								
col	1	2	3	4	5	6	7	8	9
output					1				

For row = 1, 5 <=Col <= 5, (printing value of row)

row	2								
col	1	2	3	4	5	6	7	8	9
output				2	2	2			

For row = 2, 4 <=Col <= 6, (printing value of row)

row	3								
col	1	2	3	4	5	6	7	8	9
output			3	3	3	3	3		

For row = 3, 3 <=Col <= 7, (printing value of row)

row	4								
col	1	2	3	4	5	6	7	8	9
output		4	4	4	4	4	4	4	

For row = 4, 2 <=Col <= 8, (printing value of row)

row	5								
col	1	2	3	4	5	6	7	8	9
output	5	5	5	5	5	5	5	5	5

For row = 5, 1 <=Col <= 9, (printing value of row)

COLUMN LEVEL REPETITIVE ACTION

Total maximum of 9 columns are needed to print the bottom of pattern.

Row level Repetitive Action				Expected Output
row	Column level Repetitive Action (lower_limit <= col <= upper_limit)			
	range	lower_limit	upper_limit	
1	5 <=Col <= 5	5	5	1
2	4 <=Col <= 6	4	6	2 2 2
3	3 <=Col <= 7	3	7	3 3 3 3 3
4	2 <=Col <= 8	2	8	4 4 4 4 4 4 4
5	1 <=Col <= 9	1	9	5 5 5 5 5 5 5 5 5

The lower_limit & upper_limit ,both are not consistent for all the rows. We need to come out with consistent expressions for both upper_limit and lower_limit which should suit for n number of rows. The other remaining variables are row, totalRows=5.

Let's try to generalize using these variables.

Column level Repetitive Action						
row	lower_limit <= col <= upper_limit					
	lower_limit			upper_limit		
		1	2		1	2
1	5	totalRows	totalRows +1 - 1	5	totalRows	totalRows +1 - 1
2	4	totalRows -1	totalRows +1 - 2	6	totalRows + 1	totalRows + 2 - 1
3	3	totalRows -2	totalRows +1 - 3	7	totalRows + 2	totalRows + 3 - 1
4	2	totalRows -3	totalRows +1 - 4	8	totalRows + 3	totalRows + 4 - 1
5	1	totalRows -4	totalRows +1 - 5	9	totalRows + 4	totalRows + 5 - 1

Column level Repetitive Action				
row	If-condition(lower_limit <= col <= upper_limit)			
	lower_limit		upper_limit	
	default	3	default	3
1	5	totalRows +1 - row	5	totalRows + row - 1
2	4	totalRows +1 - row	6	totalRows + row - 1
3	3	totalRows +1 - row	7	totalRows + row - 1
4	2	totalRows +1 - row	8	totalRows + row - 1
5	1	totalRows +1 - row	9	totalRows + row - 1

The lower_limit & upper_limit are consistent for all the rows. It's now generalized for any number of rows to print numeric value. So our If-condition becomes

if-condition	`lower_limit <=col<= upper_limit`
Mathematical expression	`(totalRows +1 - row) <= col <= (totalRows + row - 1)`
Java syntax	`if( col >= ( ( totalRows - row ) + 1 ) &&` `col <= ( ( totalRows + row ) - 1 ) )`

When the if-condition satisfied, it will print the row value at that position at a particular row or else will print space character. Since we know that the total maximum of 9 columns are needed to print the bottom of pattern. The for-loop condition is given as;

loop-condition	`lower_limit <=col<= upper_limit`
Mathematical expression	`1<= col <= 9`
Java syntax	`for( col = 1 ; col <= 9 ; col++ )`

The *upper_limit* of col variable of inner for-loop which is a **value 9** is required to be generalized.

The hard-coded values in logical condition makes program to be fail for different input values. Somehow, it has to be generalized using variables considered for program.

loop-condition	lower_limit <=col<= upper_limit
Mathematical expression	1<= col <= totalRows + 4
	1<= col <= totalRows + (totalRows – 1)
	1<= col <= (2*totalRows – 1)
Java syntax	for(col = 1 ; col <= (2* totalRows - 1) ; col++)

We are now able to generalized using variable **totalRows**.

Let's try to verify the upper limit for total columns which we will display for each row

for totalRows = 5 , the col <= 9 (2*5 – 1)

for totalRows = 4 , the col <= 7 (2*4 – 1)

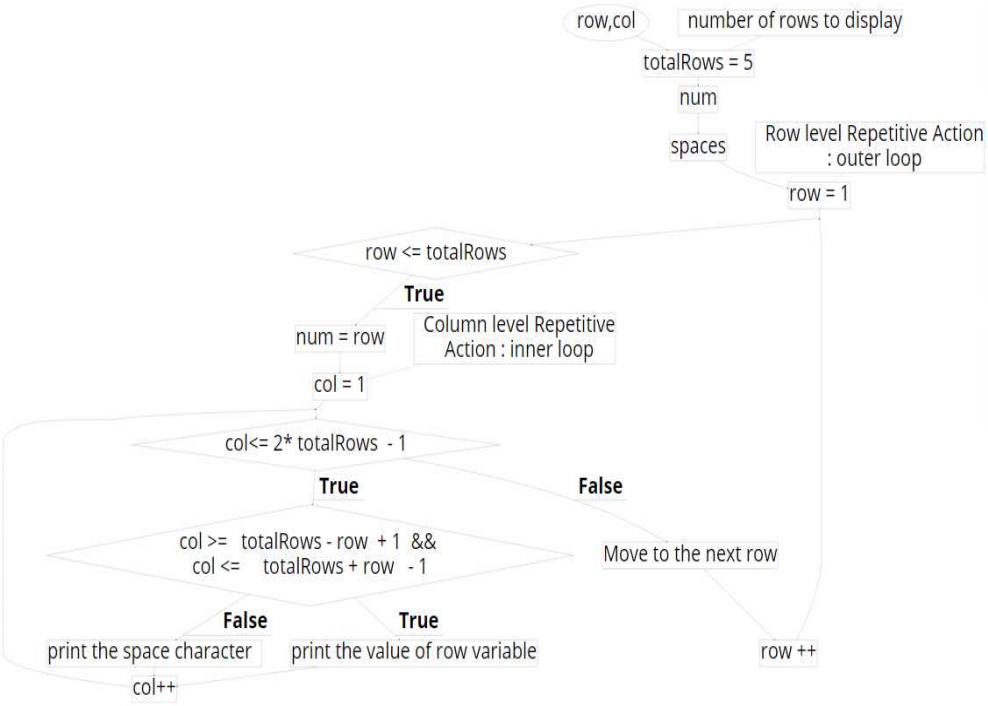
for totalRows = 3 , the col <= 5 (2*3 – 1)

for totalRows = 2 , the col <= 3 (2*2 – 1)

for totalRows = 1 , the col <= 1 (2*1 – 1)

So this generalization for total columns as per given rows holds valid.

All these observations can be easily captured in a flow-chart given as follows.



Let's write the program based on the observations captured in a flow-chart.

NumberPattern1E.java

```
public class NumberPattern1E {

	public static void main(String[] args) {

		int  totalRows = 5;  //number of rows to display

		for( int row = 1 ; row <= totalRows ; row ++ ) {

			System.out.println();// move control to the next line

			//repetition happens 2* totalRows  - 1 times horizontally

			int num = row;
			for(int col = 1 ; col <= ( 2* totalRows  - 1 ) ; col++) {

				if( col >= ( ( totalRows - row ) + 1 ) &&
					col <= (  (  totalRows + row )  - 1 ) ) {

					// row value to be printed
					System.out.print(row + " ");

				}else {

					// spaces to be printed
					System.out.print("  ");

				}

			}//end  inner for - loop
		}
	}

}
```

Output:totalRows=5

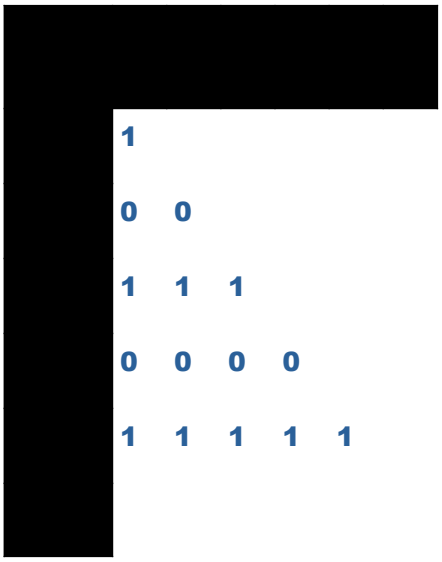
```
        1
      2 2 2
    3 3 3 3 3
  4 4 4 4 4 4 4
5 5 5 5 5 5 5 5 5
```

Output:totalRows=8

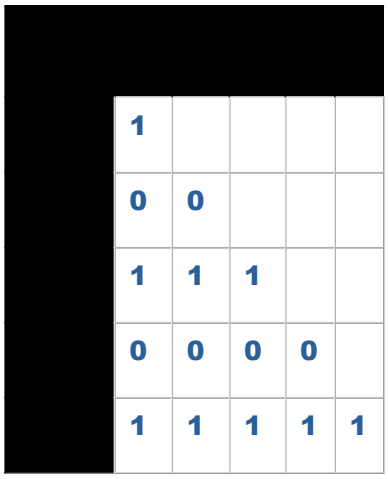
```
              1
            2 2 2
          3 3 3 3 3
        4 4 4 4 4 4 4
      5 5 5 5 5 5 5 5 5
    6 6 6 6 6 6 6 6 6 6 6
  7 7 7 7 7 7 7 7 7 7 7 7 7
8 8 8 8 8 8 8 8 8 8 8 8 8 8 8
```

Please, test for different number of lines, let it be 8. It should work correctly.

NumberPattern 1F



Our programming job is to print this particular logic-based pattern on a part of the computer screen. **Our approach will be assumed that the computer screen is a combination of rows and columns virtually dividing it into smaller boxes.** The smallest box is treated as a resident of just a single character only. The address in each smallest box on the display screen will logically become a unique **(row, column)** numeric combination. With this assumption computer screen will simply be a unique combination of (row, column) numeric value paired boxes which are uniformly spread all across the screen.



	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	1				
Row=2	0	0			
Row=3	1	1	1		
Row=4	0	0	0	0	
Row=5	1	1	1	1	1

Data Representation

Although it is logically, we can exactly locate the position of each printable character on the display screen by knowing it's uniquely identifiable **(row, column)** numeric paired-values. *For example;* **(4, 3)** means printing numeric character at the assumed screen location **4th row and 3rd column.**

(Row, Col) value pairs representation

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	1,1				
Row=2	2 ,1	2,2			
Row=3	3,1	3,2	3,3		
Row=4	4,1	4,2	4,3	4,3	
Row=5	5,1	5,2	5,3	5,4	5,5

The given set of row and col value pairs tries to mimic the location of each printable character of a given pattern on the device's display screen. The set of all value pairs (row = value, col = value) represent this particular pattern.

Analysis of output

It can also be interpreted in a tabular form.

row	1				
Col	1				
Output	1				

For row =1, 1<=col<=1

row	2				
Col	1	2			
Output	0	0			

For row =2, 1<=col<=2

row	3				
Col	1	2	3		
Output	1	1	1		

For row =3, 1<=col<=3

row	4				
Col	1	2	3	4	
Output	0	0	0	0	

For row =4, 1<=col<=4

row	5				
Col	1	2	3	4	5
Output	1	1	1	1	1

For row =5, 1<=col<=5

Column Level Repetitive Action

Row level Repetitive Action				**Expected Output**
row	**Column level Repetitive Action**			
	Values repeated	**lower_limit <=col<= upper_limit**		
		lower_limit	**upper_limit**	
1	1	1	1	**1**
2	0	1	2	**0 0**
3	1	1	3	**1 1 1**
4	0	1	4	**0 0 0 0**
5	1	1	5	**1 1 1 1 1**

The *upper_limit* value is based on how many times the value repeated in a particular row.

The *lower_limit* of col variable has the same numeric value expression for every value of row. It's consistent for all the values rows. Likewise we need to come out with such a consistent expression for upper_limit which should suit any number of rows using available variables. The variables are row & totalRows=5.

Moreover, the repeated values in column appears alternatively in row. It seems we can establish a relationship between appearing of 0 or 1 value with the row value.

If we observe it is following even-odd rule. For even value of row, means when row value is a multiple of 2, we need to print zeros or ones otherwise. Thus, we can try to output 0 or 1 depends upon the value of row variable. Let's try to generalize columns **upper_limit** and **values_repeated** by expressing in terms of available variables.

Column level Repetitive Action						Expected Output
row	values_repeated		lower_limit <=col<= upper_limit			
			lower_limit	upper_limit		
	default	1		default	1	
1	1	row%2==1	1	1	row	**1**
2	0	row%2==0	1	2	row	**0 0**
3	1	row%2==1	1	3	row	**1 1 1**
4	0	row%2==0	1	4	row	**0 0 0 0**
5	1	row%2==1	1	5	row	**1 1 1 1 1**

For all the rows, the value prints exactly the number of times the upper_limit of col value.

So the *upper_limit* of col variable is generalized to **row** variable.

The *lower_limit & upper_limit* are consistent for all the rows.

It's now generalized for any number of rows.

We can now define column level ***for-loop*** which is for every row value.

Elements of Column level *for-loop*	
min_val	**1**
max_val	**row**
variable_name	**col**
statement(s)	System.out.print(row);

Also, printing 0 or 1 can be given by checking the if-condition (row%2==1) or (row%2==0).

General Rule

Such situation may come where we need to check whether logical condition is following pattern of (**row+col**) values or (**row-col**) values or (**row*col**) values or (**row%col**) values or it could be anything like that.

The general rule is, somehow we need to express hard-coded values in terms of available variable(s) if following same numeric pattern/variations or simple maths formula made up of available variable(s).

ROW LEVEL REPETITIVE ACTION

The same following statements had been repeated 5 times(total number of rows).

Column level *for-loop* // *inner for-loop*

```
System.out.println( );// Move cursor to the next row
```

The minimum and maximum value can be given for the ***for-loop*** as;
min_val = 1 & *max_val* = 5 or totalRows
Let's consider;

row → variable type of integer to store integer value one at a time and represents row number between 1 and 5(totalRows). The **totalRows** has initially assigned the value 5.

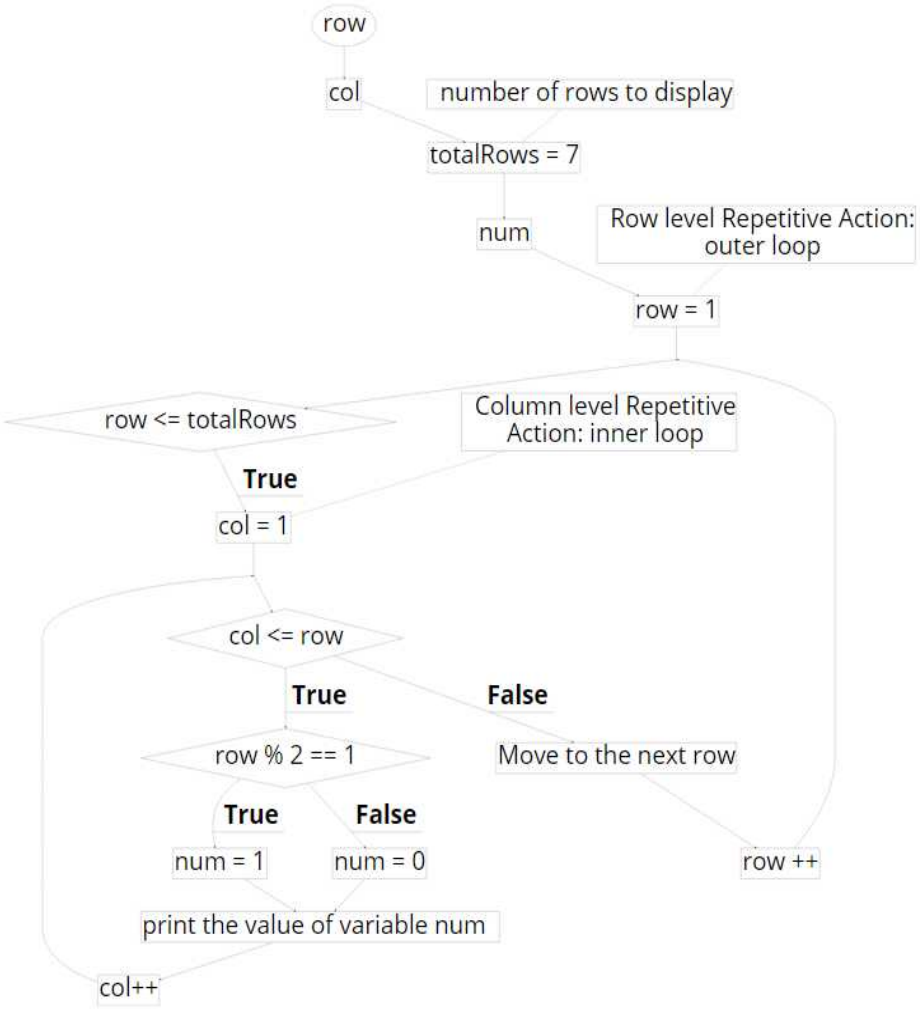
Outer ***for-loop*** starts at row=1 & will repeat till row <= **totalRows**.

We got everything related to ***for-loop*** for row level repetition.

Elements Of Row Level *for-loop*	
min_val	**1**
max_val	**totalRows**
variable_name	**row**

statement(s)	**Column level *for-loop***
	// Moves cursor to the next row
	System.out.println( );

We can summarise our observations in a flow-chart as shown in the following.



Let's execute the following program.

NumberPattern1F.java

```
public class NumberPattern1F {

	public static void main(String[] args) {

		int totalRows = 7; // number of rows to display
		int num;

		// Row level Repetitive Action:
		for (int row = 1; row <= totalRows; row++) {

			// Column level Repetitive Action:

			// print the value of variable num
			for (int col = 1; col <= row; col++) {
				if (row % 2 == 1) {
					num = 1;
				} else {
					num = 0;
				}

				System.out.print(num + " ");
			} // inner loop - printing value in a column

			// Move cursor to the next row
			System.out.println();
		} // outer loop

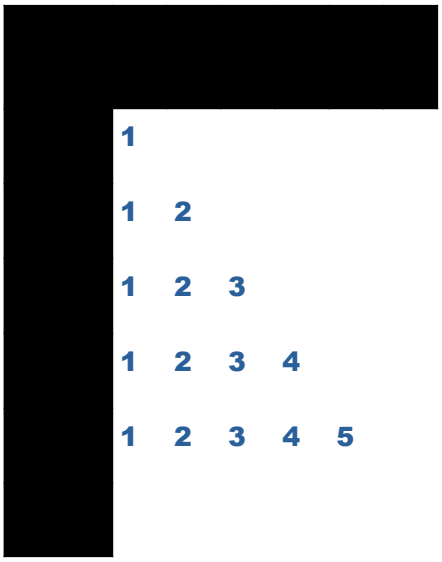
	}

}
```

Output

```
1
0 0
1 1 1
0 0 0 0
1 1 1 1 1
0 0 0 0 0 0
1 1 1 1 1 1 1
```

NumberPattern AA



Our programming job is to print this particular logic-based pattern on a part of the computer screen. **Our approach will be assumed that the computer screen is a combination of rows and columns virtually dividing it into smaller boxes.** The smallest box is treated as a resident of just a single character only. The address in each smallest box on the display screen will logically become a unique **(row, column)** numeric combination. With this assumption computer screen will simply be a unique combination of (row, column) numeric value paired boxes which are uniformly spread all across the screen.

1				
1	2			
1	2	3		
1	2	3	4	
1	2	3	4	5

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	1				
Row=2	1	2			
Row=3	1	2	3		
Row=4	1	2	3	4	
Row=5	1	2	3	4	5

Data Representation

Although it is logically, we can exactly locate the position of each printable character on the display screen by knowing it's uniquely identifiable **(row, column)** numeric paired-values. *For example;* **(4, 3)** means printing numeric character at the assumed screen location **4th row and 3rd column.**

(Row, Col) value pairs representation

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	1,1				
Row=2	2 ,1	2,2			
Row=3	3,1	3,2	3,3		
Row=4	4,1	4,2	4,3	4,3	
Row=5	5,1	5,2	5,3	5,4	5,5

The given set of row and col value pairs tries to mimic the location of each printable character of a given pattern on the device's display screen. The set of all value pairs (row = value, col = value) represent this particular pattern.

Analysis of output

It can also be interpreted in a tabular form.

row	1				
Col	1				
Output	1				

For row =1, 1<=col<=1

row	2				
Col	1	2			
Output	1	2			

For row =2, 1<=col<=2

row	3				
Col	1	2	3		
Output	1	2	3		

For row =3, 1<=col<=3

row	4				
Col	1	2	3	4	
Output	1	2	3	4	

For row =4, 1<=col<=4

row	5				
Col	1	2	3	4	5
Output	1	2	3	4	5

For row =5, 1<=col<=5

Observations

1) There is repetition required at row level vertically, i.e., row = 1 to 5.
2) There is repetition required at col level horizontally. , i.e., col = 1 to 5.
   So it's a 2-way repetition process means require 2 for-loops (nested for – loop).

For Loop	Lower limit	Upper limit
Outer loop	**row=1**	**row<=5**
Inner Loop	**col=1**	**(Unknown)** **Need to find a relation how exactly it's varying.**

COLUMN LEVEL REPETITIVE ACTION

Row level Repetitive Action			Expected Output
row	Column Level Repetitive Action		
	loop-condition(lower_limit <=col<= upper_limit)		
	lower_limit	upper_limit	
row = 1	1	1	1
row = 2	1	2	1 2
row = 3	1	3	1 2 3
row = 4	1	4	1 2 3 4
row = 5	1	5	1 2 3 4 5

The lower_limit has same numeric value expression for the rows .It's consistent for all the rows. Likewise, we need to come out with such a consistent expression for upper_limit which should suit any number of rows. The other remaining variables are row & **totalRows=5**. Let's try to generalize numeric pattern using these variables.

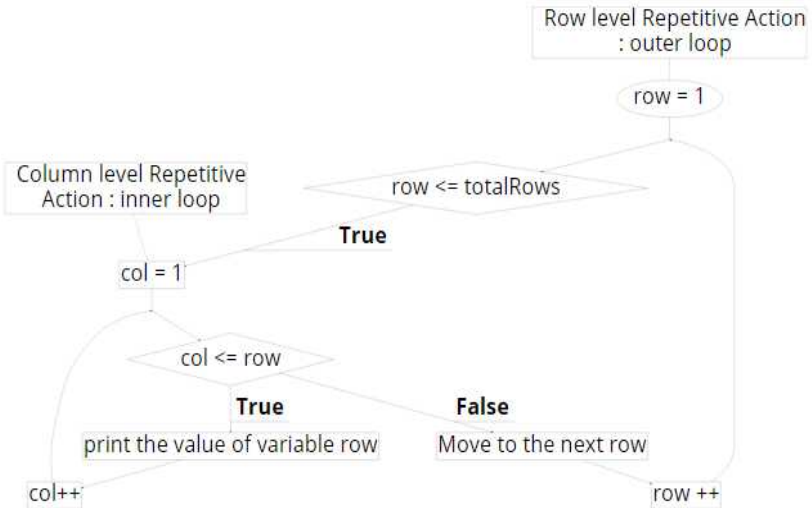
Row level Repetitive Action					Expected Output
row	Column Level Repetitive Action				
	loop-condition(lower_limit <=col<= upper_limit)				
	lower_limit	upper_limit			
row = 1	1	1	row	row	1
row = 2		2	row		1 2
row = 3		3	row		1 2 3
row = 4		4	row		1 2 3 4
row = 5		5	row		1 2 3 4 5

So the upper_limit of col variable is generalized to (row). For all the rows, the col values matches exactly the output that we need.The lower_limit & upper_limit are consistent for all the rows.

It's now generalized for any number of rows. So our **loop-condition** becomes;

loop-condition	lower_limit <=col<= upper_limit
Mathematical expression	1 <= col <= row
Java syntax	`for(int col = 1 ; col <= row ; col++)`

The related flow-chart can be given as follows.



Let's implement the program.

NumberPatternAA.java

```
public class NumberPatternAA {

	public static void main(String[] args) {

		int totalRows = 5;  //number of rows to display

		for( int row = 1 ; row <= totalRows ; row ++ ) {

			System.out.println();// move control to the next line

			for(int col = 1 ; col <= row  ; col++) {

				// added extra space for clarity in output.
				String num = col + " ";
				System.out.print( num );
			}

		}
	}
}
```

Output: totalRows = 5

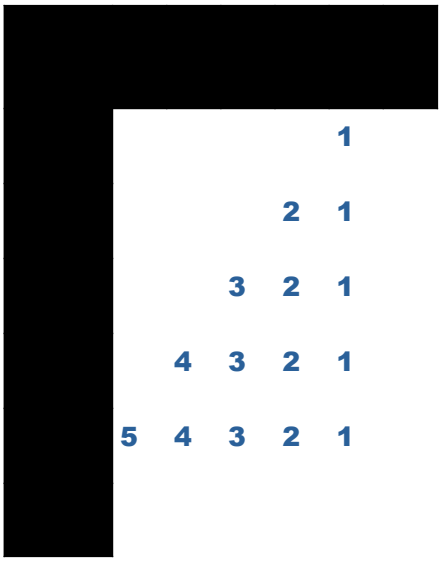
```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Output: totalRows = 10

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
1 2 3 4 5 6
1 2 3 4 5 6 7
1 2 3 4 5 6 7 8
1 2 3 4 5 6 7 8 9
1 2 3 4 5 6 7 8 9 10
```

Please, test for different number of lines, let it be 10. It should work correctly.

NumberPattern BB



Our programming job is to print this particular logic-based pattern on a part of the computer screen. **Our approach will be assumed that the computer screen is a combination of rows and columns virtually dividing it into smaller boxes.** The smallest box is treated as a resident of just a single character only. The address in each smallest box on the display screen will logically become a unique **(row, column)** numeric combination. With this assumption computer screen will simply be a unique combination of (row, column) numeric value paired boxes which are uniformly spread all across the screen.

				1
			2	1
		3	2	1
	4	3	2	1
5	4	3	2	1

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1					1
Row=2				2	1
Row=3			3	2	1
Row=4		4	3	2	1
Row=5	5	4	3	2	1

Data Representation

Although it is logically, we can exactly locate the position of each printable character on the display screen by knowing it's uniquely identifiable **(row, column)** numeric paired-values. *For example;* **(4, 3)** means printing numeric character at the assumed screen location **4th row and 3rd column.**

(Row, Col) value pairs representation

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1					1,5
Row=2				2,4	2,5
Row=3			3,3	3,4	3,5
Row=4		4,2	4,3	4,4	4,5
Row=5	5,1	5,2	5,3	5,4	5,5

The given set of row and col value pairs tries to mimic the location of each printable character of a given pattern on the device's display screen. The set of all value pairs (row = value, col = value) represent this particular pattern.

Analysis of output

In this case we need to take care of printing two different types of characters at a particular column in a particular row:

1) Space character

2) Numeric character

row	1				
col	1	2	3	4	5
output	space	space	space	space	1

For row =1, 1 <=spaces<=4, 1>=num>=1

row	2				
Col	1	2	3	4	5
Output	space	space	space	2	1

For row =2, 1 <=spaces<=3, 2>=num>=1

row	3				
Col	1	2	3	4	5
Output	space	space	3	2	1

For row =3, 1 <=spaces<=2, 3>=num>=1

row	4				
Col	1	2	3	4	5
Output	space	4	3	2	1

For row =4, 1 <=spaces<=1, 4>=num>=1

row	5				
Col	1	2	3	4	5
Output	5	4	3	2	1

For row =5, 5>=num>=1

Observations

1) There is repetition at row level vertically, i.e., Row = 1 to 5.
2) There is repetition at col level horizontally. , i.e., Col = 1 to 5.
   This gets divided into two parts .One for repetitively printing spaces and remaining column for printing numeric characters.

For Loop	Lower limit	Upper limit
Outer loop	row=1	row<=5
Inner Loop - spaces	spaces=1	spaces <= ?(Unkown)
Inner Loop - Number	num = ?(Unkown)	num = 1

COLUMN LEVEL REPETITIVE ACTION

Row level Repetitive Action							Expected Output
row	Column level Repetitive Action						
	lower_limit <=spaces<= upper_limit			upper_limit>=num >= lower_limit			
	lower_limit		upper_limit	upper_limit	lower_limit		
row = 1	1	1	4	1	1	1	1
row = 2	1		3	2	1		2 1
row = 3	1		2	3	1		3 2 1
row = 4	1		1	4	1		4 3 2 1
row = 5	1		0	5	1		5 4 3 2 1

The lower_limit for spaces & num variable has the same numeric value expression for all the rows. It's consistent for all the rows. Likewise we need to come out with such a consistent expression for upper_limit for which it should suit any number of rows.
The other remaining variables are row & totalRows=5. Let's try to generalize using these variables.

Row level Repetitive Action							
row	Column level Repetitive Action						
	lower_limit <=spaces<= upper_limit				upper_limit>=num >= lower_limit		
	lower_limit	upper_limit			upper_limit		lower_limit
row = 1	1	4	totalRows - 1	totalRows - row	1	row	1
row = 2		3	totalRows - 2	totalRows - row	2	row	
row = 3		2	totalRows - 3	totalRows - row	3	row	
row = 4		1	totalRows - 4	totalRows - row	4	row	
row = 5		0	totalRows - 5	totalRows - row	5	row	

Row level Repetitive Action						
row	Column level Repetitive Action					
	lower_limit <=spaces<= upper_limit			upper_limit>=num >= lower_limit		
	lower_limit	upper_limit		upper_limit		lower_limit
row = 1	1	totalRows - row	totalRows - row	row	row	1
row = 2		totalRows - row		row		
row = 3		totalRows - row		row		
row = 4		totalRows - row		row		
row = 5		totalRows - row		row		

So the upper_limit of spaces & num variables are generalized.
The lower_limit & upper_limit are consistent for all the rows.
It's now generalized for any number of rows.
Since it has two repetition process;
a) one repetition process for printing spaces.
b) second repetition process for printing numeric values.

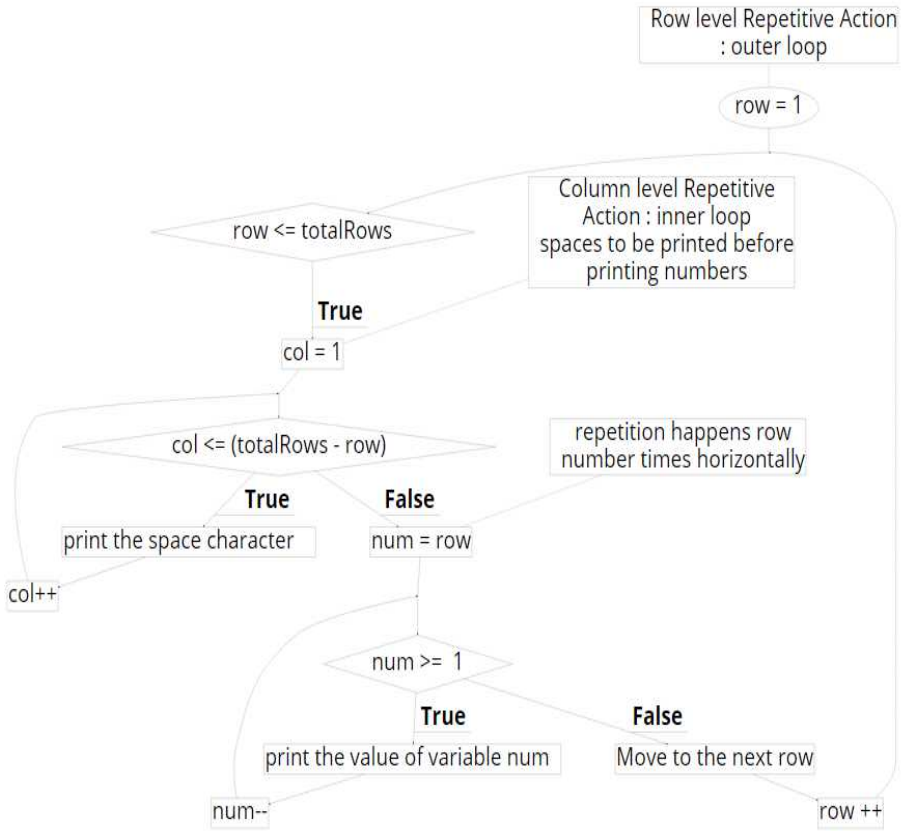
So our inner for-loop condition for printing space characters becomes;

loop-condition	`lower_limit <=col<= upper_limit`
Mathematical expression	`1 <= col <= (totalRows - row)`
Java syntax	`for(int col = 1; col <= (totalRows - row); col++)`

So our inner for-loop condition for printing numeric characters becomes;

loop-condition	`lower_limit <=num<= upper_limit`
Mathematical expression	`1 <= num <= row`
Java syntax	`for ( int num = row; num >= 1; num-- )`

The flow-chart can be used to summarize all the gathered information.
It is given as follows.



Let's execute the following program. Please, test for different number of lines, let it be 7.

NumberPatternBB.java

```
public class NumberPatternBB {
	public static void main(String[] args) {
		int totalRows = 5; // number of rows to display
		for ( int row = 1; row <= totalRows; row++ ) {
			System.out.println();// move control to the next line
			// spaces to be printed before printing numbers
			for(int col = 1; col <= (totalRows - row); col++ ) {
				System.out.print("  ");
			}
			for ( int num = row; num >= 1; num-- ) {
				// added extra space for clarity in output.
				String numStr = num + " ";
				System.out.print(numStr);
			}
		}
	}
}
```

Output:totalRows = 5

```
        1
      2 1
    3 2 1
  4 3 2 1
5 4 3 2 1
```

Output:totalRows = 7

```
          1
        2 1
      3 2 1
    4 3 2 1
  5 4 3 2 1
6 5 4 3 2 1
7 6 5 4 3 2 1
```

Alternative solution

We observed previously that for row=5, the values of col and numeric output are just reversed. Let's device a solution by decrementing the for-loop for col variable (5 to 1) so the numeric output will match partially or wholly with the values of col variable.

row	1				
col	5	4	3	2	1
output	space	space	space	space	1

row =1 , 1>=col>=1 (print col value) or row >=col>=1 (print col value)

row	2				
col	5	4	3	2	1
output	space	space	space	2	1

row = 2 , 2>=col>=1 (print col value) or row >=col>=1 (print col value)

row	3				
col	5	4	3	2	1
output	space	space	3	2	1

row = 3 , 3>=col>=1 (print col value) or row >=col>=1 (print col value)

row	4				
col	5	4	3	2	1
output	space	4	3	2	1

row = 4 , 4>=col>=1 (print col value) or row >=col>=1 (print col value)

row	5				
col	5	4	3	2	1
output	5	4	3	2	1

row = 5 , 5>=col>=1 (print col value) or row >=col>=1 (print col value)

So, if –condition now becomes **(col >=1 and col<=row) .**
col >=1 is redundant condition, since it's a lower limit in the inner for-loop for col = 5 to 1.
So, if –condition is reduced to **(col<=row)**

if-condition	lower_limit <=col<= upper_limit
Mathematical expression	1 <= col <= row
Java syntax	if(col<=row)

The overall observation is at each row, either it's printing space characters or col values in a continues fashion. It's quite obvious that we can make use of **if-else statement** as printing of characters are continuous. Let's update the program.

NumberPatternBB2.java

```
public class NumberPatternBB2 {

	public static void main(String[] args) {

		int  totalRows = 5;  //number of rows to display

		// Row level Repetitive Action :

		for( int row = 1 ; row <= totalRows ; row ++ ) {

			// Column level Repetitive Action :
			// Action1.Move cursor in the same row
			// Action2.print character
			for( int col = totalRows ; col >= 1 ; col-- ) {

				if( col<=row )   {

					System.out.print( col + " " );

				}else {

					System.out.print( "  " );

				}

			}// inner for-loop

			System.out.println();// move control to the next line

		}

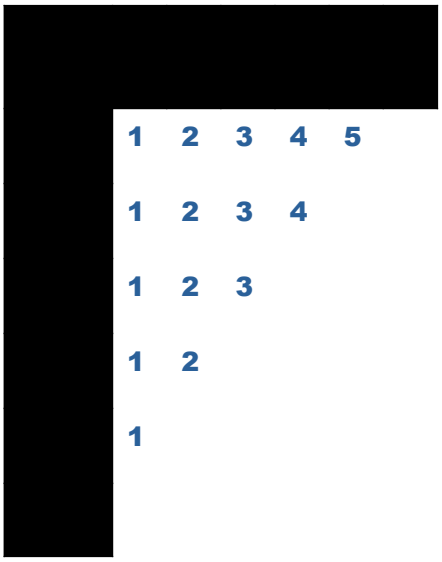
	}

}
```

Output

```
        1
      2 1
    3 2 1
  4 3 2 1
5 4 3 2 1
```

NumberPattern CC



Our programming job is to print this particular logic-based pattern on a part of the computer screen. **Our approach will be assumed that the computer screen is a combination of rows and columns virtually dividing it into smaller boxes.** The smallest box is treated as a resident of just a single character only. The address in each smallest box on the display screen will logically become a unique **(row, column)** numeric combination. With this assumption computer screen will simply be a unique combination of (row, column) numeric value paired boxes which are uniformly spread all across the screen.

1	2	3	4	5
1	2	3	4	
1	2	3		
1	2			
1				

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1					1
Row=2				2	1
Row=3			3	2	1
Row=4		4	3	2	1
Row=5	5	4	3	2	1

Data Representation

Although it is logically, we can exactly locate the position of each printable character on the display screen by knowing it's uniquely identifiable **(row, column)** numeric paired-values. *For example;* **(2, 3)** means printing numeric character at the assumed screen location **2nd row and 3rd column.**

(Row, Col) value pairs representation

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	1,1	1,2	1,3	1,4	1,5
Row=2	2,1	2,2	2,3	2,4	
Row=3	3,1	3,2	3,3		
Row=4	4,1	4,2			
Row=5	5,1				

The given set of row and col value pairs tries to mimic the location of each printable character of a given pattern on the device's display screen. The set of all value pairs (row = value, col = value) represent this particular pattern.

Analysis of output

Let's device a solution by applying incrementing for-loop using range of values of variable:col(1 to 5). The numeric output may match partially or wholly with the values of col variable. So **if-condition** can be expressed using col variable.

row	1				
col	1	2	3	4	5
output	1	2	3	4	5

row =1 , 1 <=Col <= 5 (print col value)

row	2				
col	1	2	3	4	5
output	1	2	3	4	

row = 2 , (1 <=Col <= 4) (print col value)

row	3				
col	1	2	3	4	5
output	1	2	3		

row = 3 , (1 <=Col <= 3) (print col value)

row	4				
col	1	2	3	4	5
output	1	2			

row = 4 , (1 <=Col <= 2) (print col value)

row	5				
col	1	2	3	4	5
output	1				

row = 5 , (1 <=Col <= 1) (print col value)

COLUMN LEVEL REPETITIVE ACTION

Row level Repetitive Action			Expected Output
row	Column level Repetitive Action		
	If-condition(lower_limit <=col<= upper_limit)		
	lower_limit	upper_limit	
1	1	5	1 2 3 4 5
2	1	4	1 2 3 4
3	1	3	1 2 3
4	1	2	1 2
5	1	1	1

The lower_limit has same numeric value expression for the rows .

It's consistent for all the rows. Likewise we need to come out with such a consistent expression for *upper_limit* which should suit any number of rows.

The considered variables are row, totalRows=5.Let try to generalize using these variables.

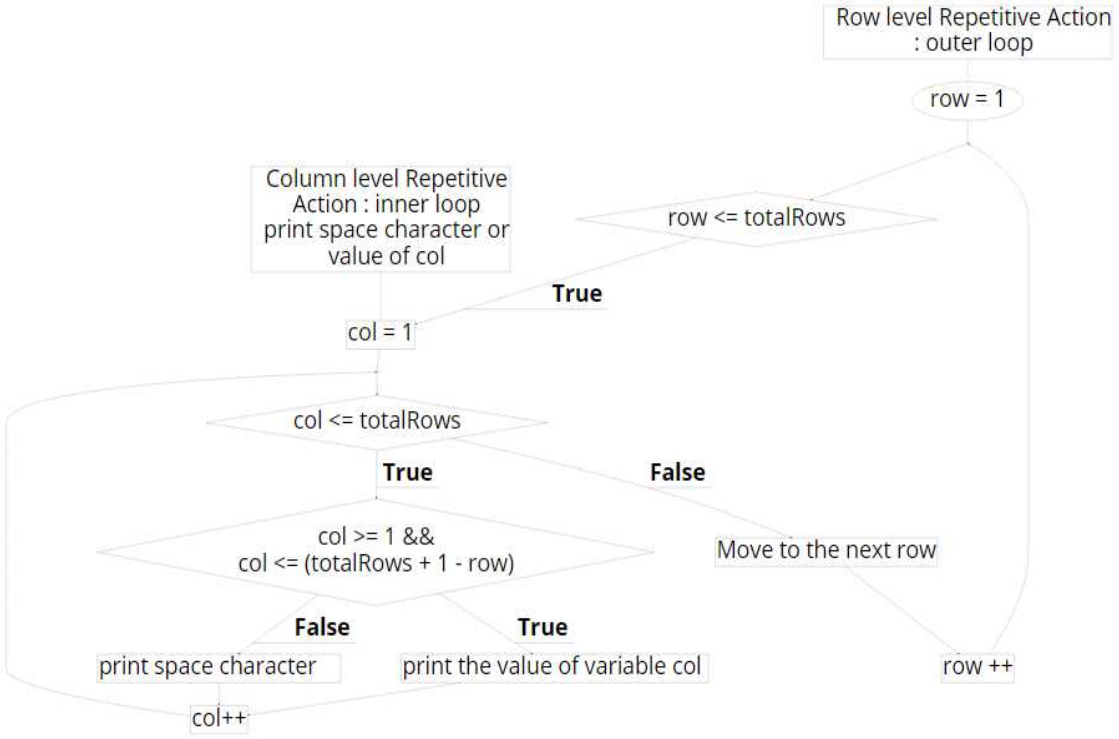
The lower_limit of col variable generalized to value 1, since being consistent for all the row values.

Row level Repetitive Action						
row	Column level Repetitive Action					
	If-condition(lower_limit <=col<= upper_limit)					
	lower_limit	upper_limit				
1	1	5	totalRows	(totalRows +1) - 1	(totalRows +1) - row	(totalRows+1) - row
2		4	totalRows -1	(totalRows +1) - 2	(totalRows +1) - row	
3		3	totalRows - 2	(totalRows +1) - 3	(totalRows +1) - row	
4		2	totalRows - 3	(totalRows +1) - 4	(totalRows +1) - row	
5		1	totalRows - 4	(totalRows +1) - 5	(totalRows +1) - row	

So the upper_limit of col variable is generalized to (**totalRows + 1 – row**). Thus, lower_limit & upper_limit are consistent for all the rows. It's now generalized for any number of rows. So our If-condition becomes;

if-condition	`lower_limit <=col<= upper_limit`
Mathematical expression	`1 <= col <= ( (totalRows +1) - row )`
Java syntax	`if( col >= 1 && col <= (totalRows + 1 - row) )`

We can summarize our findings in a flow-chart given as follows.



Let's implement the program.

NumberPatternCC1.java

```
public class NumberPatternCC1 {

	public static void main(String[] args) {

		int  totalRows = 5;  //number of rows to display

		// Row level Repetitive Action :
		for( int row = 1 ; row <= totalRows ; row ++ ) {

			// Column level Repetitive Action :
			// Action1.Move cursor in the same row
			// Action2.print character

			// spaces to be printed before printing numbers
			for( int col = 1 ; col <= totalRows ; col++ ) {

				if( col >= 1 &&
				    col <= (totalRows + 1 - row) ) {
					System.out.print( col + " " );
				}else {
					System.out.print( "  " );
				}

			}// inner for-loop

			// move control to the next line
			System.out.println();
		}
	}
}
```

Output

```
1 2 3 4 5
1 2 3 4
1 2 3
1 2
1
```

The inner for-loop's *lower_limit* matches with the if-condition's lower_limit.

```
for(int col = 1 ; col <= totalRows    ; col++) {
        if( col >= 1 &&  col <= (totalRows + 1 - row) ) {
                ---- statements ------
        }
}
```

We can safely neglect the *lower_limit* in if-condition as it will become redundant condition, since already been taken care in inner for-loop. Hence, our final if-condition can be given as;

if-condition	lower_limit <=col<= upper_limit
Mathematical expression	col <= ((totalRows +1) - row)
Java syntax	if(col <= (totalRows + 1 - row))

NumberPatternCC2.java

```
public class NumberPatternCC2 {

	public static void main(String[] args) {

		int  totalRows = 5;  //number of rows to display

		// Row level Repetitive Action :
		for( int row = 1 ; row <= totalRows ; row ++ ) {

			// Column level Repetitive Action :
			// Action1.Move cursor in the same row
			// Action2.print character

			// spaces to be printed before printing numbers
			for(int col = 1 ; col <= totalRows ; col++) {

				if( col <= (totalRows + 1 - row) ) {
					System.out.print( col + " " );
				}else {
					System.out.print( "  " );
				}
			}// inner for-loop

			// move control to the next line
			System.out.println();
		}
	}
}
```

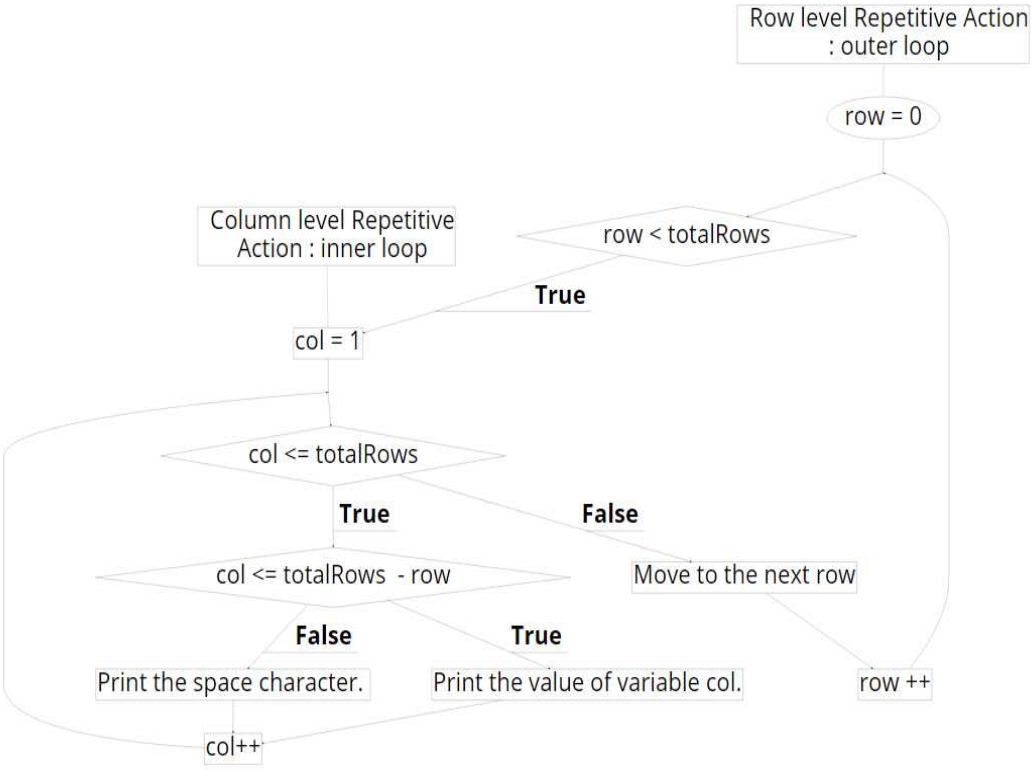
Output

```
1 2 3 4 5
1 2 3 4
1 2 3
1 2
1
```

Further, simplifying on subtracting 1 from row values and upper_limit of col values by utilizing the previous obtained trace table.

Column level Repetitive Action				
row		**col**		
	1	**lower_limit**	**upper_limit**	
			1	**2**
1 -1	0	1	(totalRows +1) – row - 1	(totalRows – row)
2 - 1	1	1	(totalRows +1) – row - 1	(totalRows – row)
3 -1	2	1	(totalRows +1) – row - 1	(totalRows – row)
4 - 1	3	1	(totalRows +1) – row - 1	(totalRows – row)
5 - 1	4	1	(totalRows +1) – row - 1	(totalRows – row)

if-condition	lower_limit <=col<= upper_limit
Mathematical expression	col <= (totalRows – row)
Java syntax	`if( col <= (totalRows - row) )`



Let's update the program with our latest logical condition modification.

NumberPatternCC3.java

```
public class NumberPatternCC3 {

	public static void main(String[] args) {

		int  totalRows = 5;  //number of rows to display

		// Row level Repetitive Action :
		for( int row = 0 ; row < totalRows ; row ++ ) {

			// Column level Repetitive Action :
			// Action1.Move cursor in the same row
			// Action2.print character

			// spaces to be printed before printing numbers
			for(int col = 1 ; col <= totalRows ; col++) {

				if( col <= (totalRows  - row) ) {
					System.out.print( col + " " );
				}else {
					System.out.print( "  " );
				}

			}// inner for-loop

			// move control to the next line
			System.out.println();
		}
	}
}
```

Output

```
1 2 3 4 5
1 2 3 4
1 2 3
1 2
1
```

NumberPattern DD



Our programming job is to print this particular logic-based pattern on a part of the computer screen. **Our approach will be assumed that the computer screen is a combination of rows and columns virtually dividing it into smaller boxes.** The smallest box is treated as a resident of just a single character only. The address in each smallest box on the display screen will logically become a unique **(row, column)** numeric combination. With this assumption computer screen will simply be a unique combination of (row, column) numeric value paired boxes which are uniformly spread all across the screen.

5	4	3	2	1
	4	3	2	1
		3	2	1
			2	1
				1

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	5	4	3	2	1
Row=2		4	3	2	1
Row=3			3	2	1
Row=4				2	1
Row=5					1

Data Representation

Although it is logically, we can exactly locate the position of each printable character on the display screen by knowing it's uniquely identifiable **(row, column)** numeric paired-values. *For example;* **(4, 5)** means printing numeric character at the assumed screen location **4th row and 5th column.**

(Row, Col) value pairs representation

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	1,1	1,2	1,3	1,4	1,5
Row=2		2,2	2,3	2,4	2,5
Row=3			3,3	3,4	3,5
Row=4				4,4	4,5
Row=5					5,5

The given set of row and col value pairs tries to mimic the location of each printable character of a given pattern on the device's display screen. The set of all value pairs (row = value, col = value) represent this particular pattern.

Analysis of output

Let's apply decrementing for-loop for col variable (5 to 1) or (5 >=col>=1) so that the numeric output will match partially or wholly with the values of col variable.

row	1				
col	5	4	3	2	1
output	5	4	3	2	1

row =1 , 5 >=col>=1 (print col value)

row	2				
col	5	4	3	2	1
output		4	3	2	1

row =2 , 4 >=col>=1 (print col value)

row	3				
col	5	4	3	2	1
output			3	2	1

row =3 , 3 >=col>=1 (print col value)

row	4				
col	5	4	3	2	1
output				2	1

row =4 , 2>=col>=1 (print col value)

row	5				
col	5	4	3	2	1
output					1

row =5 , 1>=Col >= 1 (print col value)

COLUMN LEVEL REPETITIVE ACTION

Row level Repetitive Action			Expected Output
row	Column level Repetitive Action		
	upper_limit >=col>= lower_limit		
	lower_limit	upper_limit	
1	1	5	**5 4 3 2 1**
2	1	4	**4 3 2 1**
3	1	3	**3 2 1**
4	1	2	**2 1**
5	1	1	**1**

The lower_limit has same consistent expression as numeric value for the all the rows.

Likewise, we need to come out with such a consistent expression for upper_limit which should suit any number of rows. The available variables are row& totalRows=5. Let's try to generalize using these variables.

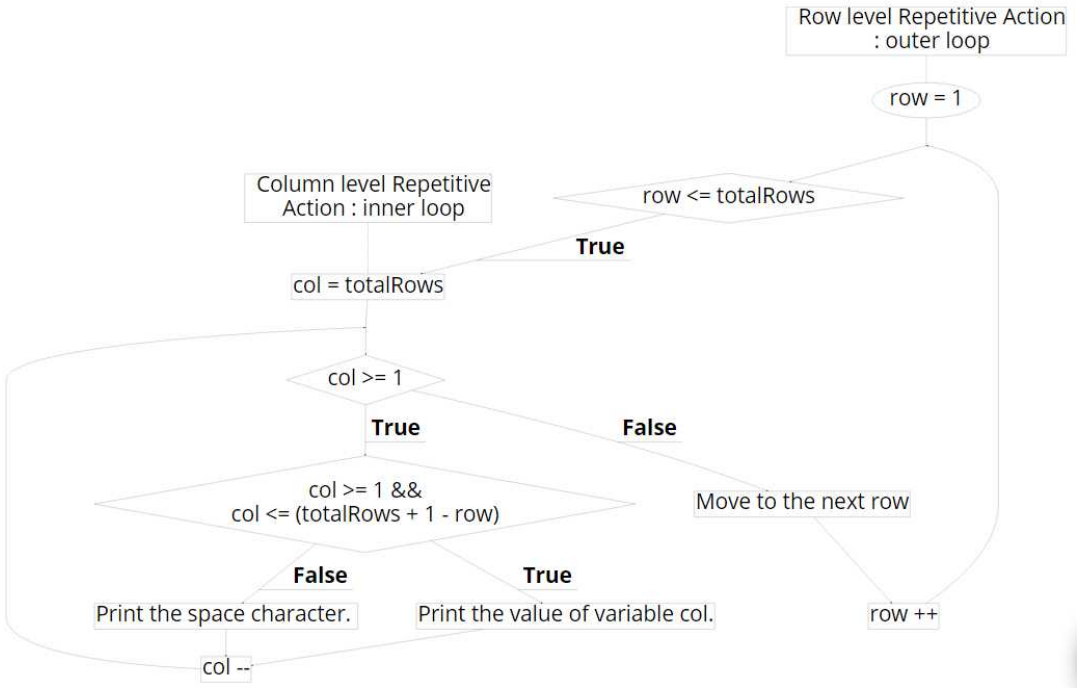
Column level Repetitive Action					
row	If-condition(upper_limit >=col>= lower_limit)				
	lower_limit	upper_limit			
			1	2	3
1	1	5	totalRows	totalRows +1 - 1	totalRows +1 - row
2	1	4	totalRows - 1	totalRows +1 - 2	totalRows +1 - row
3	1	3	totalRows - 2	totalRows +1 - 3	totalRows +1 - row
4	1	2	totalRows - 3	totalRows +1- 4	totalRows +1 - row
5	1	1	totalRows - 4	totalRows +1- 5	totalRows +1 - row

The lower_limit & upper_limit are become consistent for all the rows. The upper_limit is generalized to (**totalRows + 1 – row**).This logical expression is such a formula, commonly applied on all rows and columns which in turn helps to give matching output.

So our If-condition becomes;

if-condition	`upper_limit >=col>= lower_limit`
Mathematical expression	`1 <= col <= ( (totalRows +1) - row )`
Java syntax	`if( col >= 1 && col <= (totalRows + 1 - row) )`

The flow-chart can be given as follows.



NumberPatternDD1.java

```
public class NumberPatternDD1 {

	public static void main(String[] args) {

		int  totalRows = 5;  //number of rows to display

		// Row level Repetitive Action:
		for( int row = 1 ; row <= totalRows ; row ++ ) {

			// Column level Repetitive Action:
			// Action1.Move cursor in the same row
			// Action2.print character

			// spaces to be printed before printing numbers
			for( int col = totalRows ; col >= 1 ; col -- ) {

				if( col >= 1 &&  col <= (totalRows + 1 - row) ) {
					System.out.print( col + " " );
				}else {
					System.out.print( "  " );
				}

			}// inner for-loop

			// move control to the next line
			System.out.println();
		}
	}
}
```

Output

```
5 4 3 2 1
  4 3 2 1
    3 2 1
      2 1
        1
```

The decrementing inner for-loop's lower_limit matches with the if-condition's lower_limit.

```
for(int col = totalRows    ; col >= 1    ; col--) {

        if( col >= 1 &&  col <= (totalRows + 1 - row) ) {

                ---- statements ------

        }

}
```

We can safely neglect the **lower_limit** in if-condition as it is an extra condition.

Hence, our final if-condition will be;

if-condition	upper_limit >=col>= lower_limit
Mathematical expression	col <= ((totalRows +1) - row)
Java syntax	if(col <= (totalRows + 1 - row))

NumberPatternDD2.java

```
public class NumberPatternDD2 {

	public static void main(String[] args) {

		int  totalRows = 5;  //number of rows to display

		// Row level Repetitive Action :
		for( int row = 1 ; row <= totalRows ; row ++ ) {

			// Column level Repetitive Action :
			// Action1.Move cursor in the same row
			// Action2.print character

			// spaces to be printed before printing numbers
			for( int col = totalRows ; col >= 1 ; col -- ) {

				if( col <= (totalRows + 1 - row) ) {
					System.out.print( col + " " );
				}else {
					System.out.print( "  " );
				}

			}

			// move control to the next line
			System.out.println();
		}
	}
}
```

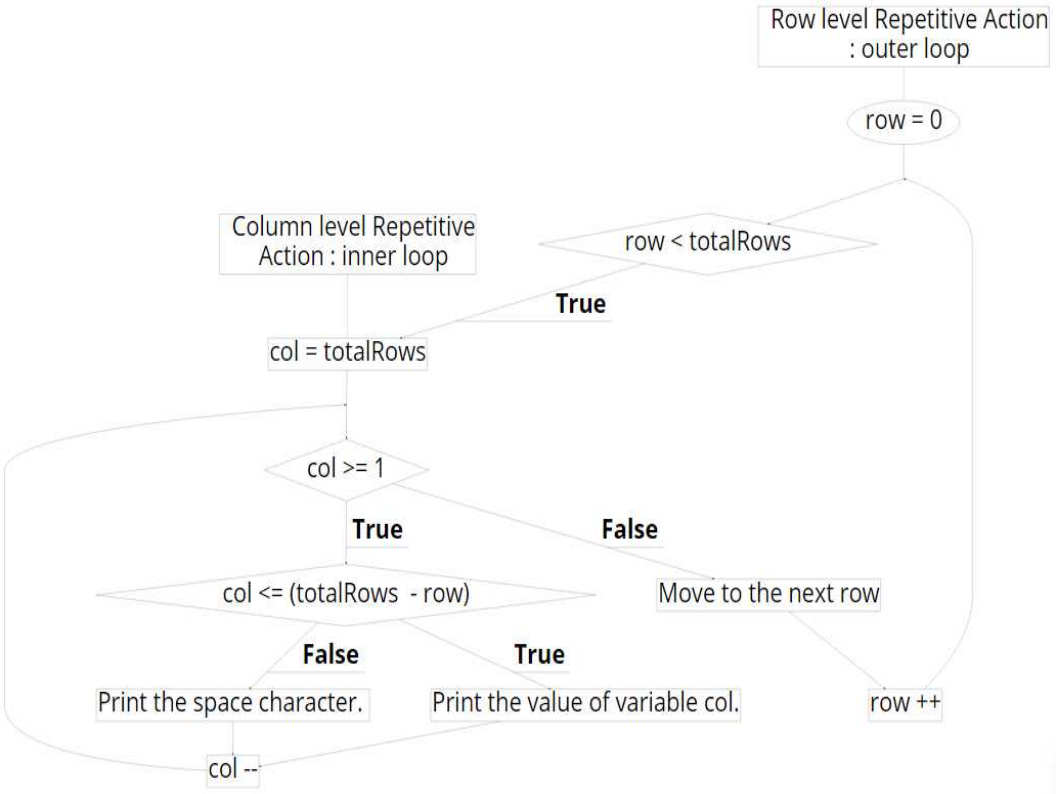
Further, simplifying upper_limit by subtracting 1 as shown in the following;

Column level Repetitive Action				
row		upper_limit >=col>= lower_limit		
		lower_limit	upper_limit	
1 - 1	0	1	(totalRows +1) – row - 1	(totalRows – row)
2 - 1	1	1	(totalRows +1) – row - 1	(totalRows – row)
3 - 1	2	1	(totalRows +1) – row - 1	(totalRows – row)
4 - 1	3	1	(totalRows +1) – row - 1	(totalRows – row)
5 - 1	4	1	(totalRows +1) – row - 1	(totalRows – row)

Hence, our improved if-condition will be,

if-condition	`upper_limit >=col>= lower_limit`
Mathematical expression	`col <= ( (totalRows - row )`
Java syntax	`if( col <= (totalRows - row) )`

The modification can be easily expressed in the flow-chart.



The updated final solution can be expressed in the following program.

NumberPatternDD3.java

```
public class NumberPatternDD3 {

	public static void main( String[] args ) {

		int  totalRows = 5;  //number of rows to display

		// Row level Repetitive Action :
		for( int row = 0 ; row < totalRows ; row ++ ) {

			// Column level Repetitive Action :
			// Action1.Move cursor in the same row
			// Action2.print character

			// spaces to be printed before printing numbers
			for( int col = totalRows ; col >= 1    ; col -- ) {

				if(  col <= (totalRows  - row) ) {
					System.out.print( col + " " );
				}else {
					System.out.print( "  " );
				}

			}

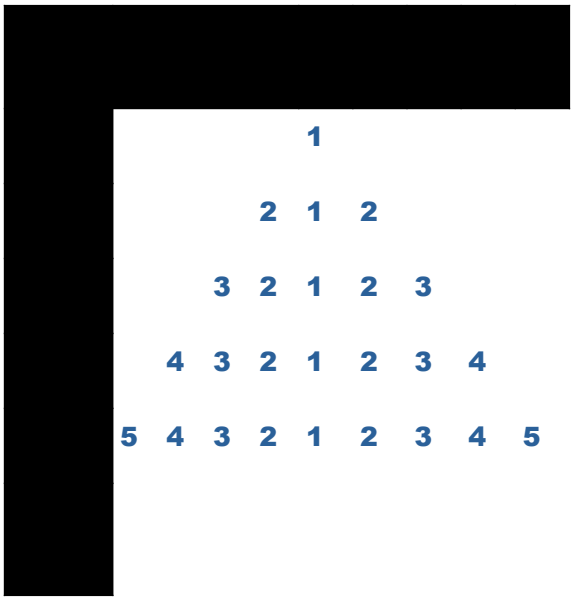
			// move control to the next line
			System.out.println();
		}
	}
}
```

Output

```
5 4 3 2 1
  4 3 2 1
    3 2 1
      2 1
        1
```

NumberPattern EE

Our programming job is to print this particular logic-based pattern on a part of the computer screen. **Our approach will be assumed that the computer screen is a combination of rows and columns virtually dividing it into smaller boxes.** The smallest box is treated as a resident of just a single character only. The address in each smallest box on the display screen will logically become a unique **(row, column)** numeric combination. With this assumption computer screen will simply be a unique combination of (row, column) numeric value paired boxes which are uniformly spread all across the screen.



	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1					1				
Row=2				2	1	2			
Row=3			3	2	1	2	3		
Row=4		4	3	2	1	2	3	4	
Row=5	5	4	3	2	1	2	3	4	5

Data Representation

Although it is logically, we can exactly locate the position of each printable character on the display screen by knowing it's uniquely identifiable **(row, column)** numeric paired-values. *For example;* **(4, 3)** means printing numeric character at the assumed screen location **4th row and 3rd column.**

(Row, Col) value pairs representation

	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1					1,5				
Row=2				2,4	2,5	2,6			
Row=3			3,3	3,4	3,5	3,6	3,7		
Row=4		4,2	4,3	4,4	4,5	4,6	4,7	4,8	
Row=5	5,1	5,2	5,3	5,4	5,5	5,6	5,7	5,8	5,9

Analysis of output

row	1								
col	1	2	3	4	5	6	7	8	9
output					1				

For row = 1, 5 <=Col <= 5

row	2								
col	1	2	3	4	5	6	7	8	9
output				2	1	2			

For row = 2, 4 <=Col <= 6

row	3								
col	1	2	3	4	5	6	7	8	9
output			3	2	1	2	3		

For row = 3, 3 <=Col <= 7

row	1								
col	1	2	3	4	5	6	7	8	9
output		4	3	2	1	2	3	4	

For row = 4, 2 <=Col <= 8

row	1							
col	1	3	4	5	6	7	8	9
output	5	3	2	1	2	3	4	5

For row = 5, 1 <=Col <= 9

COLUMN LEVEL REPETITIVE ACTION

Row level Repetitive Action				Expected Output
row	Column level Repetitive Action			
	lower_limit <= col <= upper_limit			
	range	lower_limit	upper_limit	
row = 1	5 <=Col <= 5	5	5	**1**
row = 2	4 <=Col <= 6	4	6	**2 1 2**
row = 3	3 <=Col <= 7	3	7	**3 2 1 2 3**
row = 4	2 <=Col <= 8	2	8	**4 3 2 1 2 3 4**
row = 5	1 <=Col <= 9	1	9	**5 4 3 2 1 2 3 4 5**

The lower_limit & upper_limit ,both are not consistent for all the rows. We need to come out with consistent expressions for both upper_limit and lower_limit which should suit for any number of rows. The other remaining variables are **row, totalRows = 5**. Let's try to generalize using these variables.

Column level Repetitive Action						
row	If-condition(lower_limit <= col <= upper_limit)					
	lower_limit			upper_limit		
		1	2		1	2
1	5	totalRows	totalRows +1 - 1	5	totalRows	totalRows +1 - 1
2	4	totalRows -1	totalRows +1 - 2	6	totalRows + 1	totalRows + 2 - 1
3	3	totalRows -2	totalRows +1 - 3	7	totalRows + 2	totalRows + 3 - 1
4	2	totalRows -3	totalRows +1 - 4	8	totalRows + 3	totalRows + 4 - 1
5	1	totalRows -4	totalRows +1 - 5	9	totalRows + 4	totalRows + 5 - 1

Column level Repetitive Action					
row	lower_limit <= col <= upper_limit				
	lower_limit		upper_limit		
	default	3	default	3	
1	5	totalRows +1 - row	5	totalRows + row - 1	
2	4	totalRows +1 - row	6	totalRows + row - 1	
3	3	totalRows +1 - row	7	totalRows + row - 1	
4	2	totalRows +1 - row	8	totalRows + row - 1	
5	1	totalRows +1 - row	9	totalRows + row - 1	

The lower_limit & upper_limit are become consistent for all the rows.

The upper_limit is generalized to (**totalRows + row – 1**).

This logical expression is such a formula, commonly applied on all rows and columns which in turn helps to give matching output. So our If-condition becomes;

if-condition	`lower_limit <=col_sp<= upper_limit`
Mathematical expression	`(totalRows +1 - row) <= col <= (totalRows + row - 1)`
Java syntax	`if( col >= ( ( totalRows - row) + 1 ) && col <= ( ( totalRows + row )- 1 ) )`

When the if-condition satisfied, it will print the numeric character at that position at a particular row or else will print the space character. We can observe two things from the given output which can be given as;

a) The values at each row starts with the row value and goes on decreases till the value 1 and further increases till it reaches the value of row value. The value basically oscillating like a pendulum of a wall clock. Let's choose a variable **num** which represents a row value, every time a new row starts. The value of variable **num** then oscillates to give output in a single row. It can be shown by following table.

Table for num variable starting at row value.

row	num	Col<totalRows	Col=totalRows	Col>totalRows
1	1		1	
2	2	2	1	**2**
3	3	3 2	1	**2 3**
4	4	4 3 2	1	**2 3 4**
5	5	5 4 3 2	1	2 3 4 5

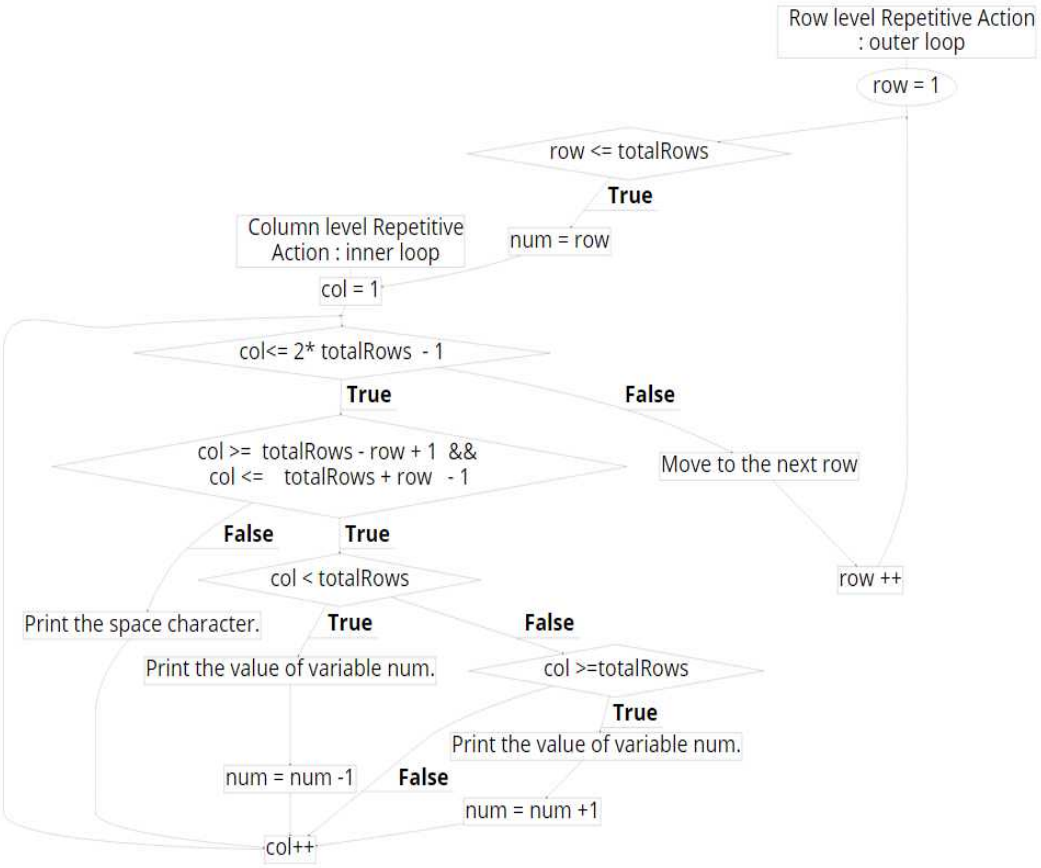
```
If(  col < totalRows ){
      num value gets printed.
      num value get decreased by 1.
 }
If( col >= totalRows ){
      num value gets printed.
      num value get increased by 1.
 }
```

b) The column counts are more than the **totalRows** count other than the first row and the input value of variable **totalRows** is 5.The column count is different on each row. Considering all these factors the upper_limit of inner for–loop need to be generalized.

loop-condition	`lower_limit <=col<= upper_limit`
Mathematical expression	`1 <= col <= 9`
	`1 <=col <= totalRows + 4`
	`1 <=col <= totalRows + (totalRows - 1)`
	`1<=col<=2* totalRows - 1`
Java syntax	`for(int col = 1 ; col <= ( 2* totalRows  - 1 ) ; col++)`

So this generalization to calculate total columns for a given row value is satisfactory.

The flow-chart can be given as;



NumberPatternEE.java

```
public class NumberPatternEE {

	public static void main(String[] args) {

		int  totalRows = 8;  //number of rows to display

		// Row level Repetitive Action :
		for( int row = 1 ; row <= totalRows ; row ++ ) {

			//repetition count (2* totalRows  - 1) happens horizontally
			int num = row;

			// Column level Repetitive Action :
			// Action1.Move cursor in the same row
			// Action2.print character
			for(int col = 1 ; col <= ( 2* totalRows  - 1 ) ; col++) {

				if( col >= ( ( totalRows - row) + 1 ) &&
				    col <= (  ( totalRows + row )  - 1 ) ) {

					if(col < totalRows ) {
						// 1st half triangle
						System.out.print( num + " ");
						num = num -1;
					}else if(col >=totalRows) {
						// 2nd half traingle
						System.out.print( num + " ");
						num = num +1;
					}

				}else {
					System.out.print("  ");
				}
			}//end  inner for - loop

			// move control to the next line
			System.out.println();
		}
	}

}
```

Output

```
              1
            2 1 2
          3 2 1 2 3
        4 3 2 1 2 3 4
      5 4 3 2 1 2 3 4 5
    6 5 4 3 2 1 2 3 4 5 6
  7 6 5 4 3 2 1 2 3 4 5 6 7
8 7 6 5 4 3 2 1 2 3 4 5 6 7 8
```

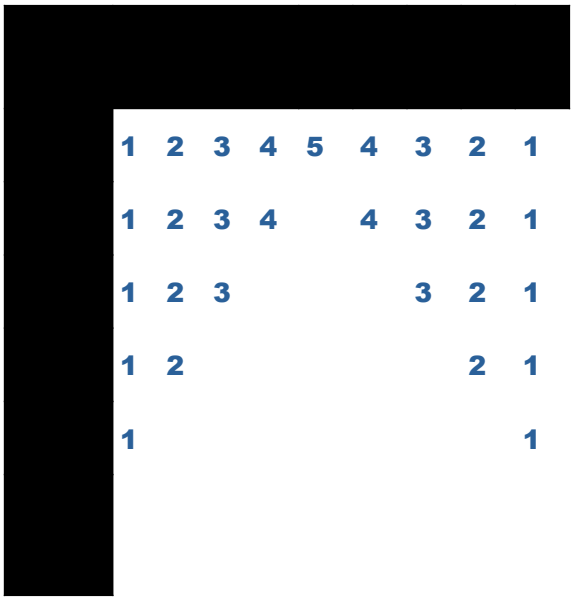
ProblemGBEE: **Write logic trace table and a program for the following output.**

Output

```
        5
      4 5 6
    3 4 5 6 7
  2 3 4 5 6 7 8
1 2 3 4 5 6 7 8 9
```

NumberPattern FF

Our programming job is to print this particular logic-based pattern on a part of the computer screen. **Our approach will be assumed that the computer screen is a combination of rows and columns virtually dividing it into smaller boxes.** The smallest box is treated as a resident of just a single character only. The address in each smallest box on the display screen will logically become a unique **(row, column)** numeric combination. With this assumption computer screen will simply be a unique combination of (row, column) numeric value paired boxes which are uniformly spread all across the screen.



	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1	1	2	3	4	5	4	3	2	1
Row=2	1	2	3	4		4	3	2	1
Row=3	1	2	3				3	2	1
Row=4	1	2						2	1
Row=5	1								1

Data Representation

Although it is logically, we can exactly locate the position of each printable character on the display screen by knowing it's uniquely identifiable **(row, column)** numeric paired-values. *For example;* **(2, 3)** means printing character at the assumed screen location **2nd row and 3rd column.**

(Row, Col) value pairs representation

	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1	(1,1) 1	(1,2) 2	(1,3) 3	(1,4) 4	(1,5) 5	(1,6) 4	(1,7) 3	(1,8) 2	(1,9) 1
Row=2	(2,1) 1	(2,2) 2	(2,3) 3	(2,4) 4	2,5 space	(2,6) 4	(2,7) 3	(2,8) 2	(2,9) 1
Row=3	(3,1) 1	(3,2) 2	(3,3) 3	3,4 space	3,5 space	3,6 space	(3,7) 3	(3,8) 2	(3,9) 1
Row=4	(4,1) 1	(4,2) 2	4,3 space	4,4 space	4,5 space	4,6 space	4,7 space	(4,8) 2	(4,9) 1
Row=5	(5,1) 1	5,2 space	5,3 space	5,4 space	5,5 space	5,6 space	5,7 space	5,8 space	(5,9) 1

Analysis of output

In general, three repetitive action takes place.

a) Printing of numbers starting from 1 in incrementing order.

b) Printing of spaces

c) Printing of numbers in decrementing order and it's upper limit will start from last updated incremented value.

COLUMN LEVEL REPETITIVE ACTION

Row level Repetitive Action			
row	Column level Repetitive Action		
	Variable :numInc	Variable :space (space counts)	Variable :numDec
1	1, 2 ,3 ,4, 5	0(no space)	4, 3, 2, 1
2	1, 2, 3, 4	1	4, 3, 2, 1
3	1, 2, 3	1,2,3	3, 2, 1
4	1, 2	1,2,3,4,5	2, 1
5	1	1,2,3,4,5,6,7	1

Row level Repetitive Action						
row	Column level Repetitive Action					
	Variable :numInc		Variable :space		Variable :numDec	
	lower_limit	upper_limit	lower_limit	upper_limit	upper_limit	lower_limit
1	1	5	0	0	4	1
2	1	4	1	1	4	1
3	1	3	1	3	3	1
4	1	2	1	5	2	1
5	1	1	1	7	1	1

We know that total number of rows is given by the variable **totalRows**=5.We need to generalize to get unique expression. The **upper_limit** of variable **numInc** and **numDec** can be expressed in terms of variable **totalRows**. The **lower_limit** of variable **numInc** and **numDec** are already generalized.

Row level Repetitive Action						
row	Column level Repetitive Action					
	Variable :numInc		Variable :space		Variable :numDec	
	lower_limit	upper_limit	lower_limit	upper_limit	upper_limit	lower_limit
		1			1	
1	1	totalRows - 0	0	0	totalRows - 1	1
2		totalRows - 1	1	1	totalRows - 1	
3		totalRows - 2	1	3	totalRows - 2	
4		totalRows - 3	1	5	totalRows - 3	
5		totalRows - 4	1	7	totalRows - 4	

If we take values of row variable starting from 0 instead of 1, then it will help us to generalize the **upper_limit** of variable **numInc** and **numDec.** Let try to generalize both of these using variable **row**.

Row level Repetitive Action						
row	Column level Repetitive Action					
	Variable :numInc		Variable :space		Variable :numDec	
	lower_limit	upper_limit	lower_limit	upper_limit	upper_limit	lower_limit
		1			1	
row = 0	1	totalRows - 0	0	0	totalRows - 1	1
row = 1		totalRows - 1	1	1	totalRows - 1	
row = 2		totalRows - 2	1	3	totalRows - 2	
row = 3		totalRows - 3	1	5	totalRows - 3	
row = 4		totalRows - 4	1	7	totalRows - 4	

The part of **upper_limit** of variable **numInc** and **numDec** matches with numeric pattern of row variable. Hence, we can use row variable to generalize the expressions.

Row level Repetitive Action						
row	Column level Repetitive Action					
	Variable :numInc		Variable :space		Variable :numDec	
	lower_limit	upper_limit	lower_limit	upper_limit	upper_limit	lower_limit
		1			1	
row = 0	1	totalRows - row	0	0	totalRows - 1	1
row = 1		totalRows - row	1	1	totalRows - row	
row = 2		totalRows - row	1	3	totalRows - row	
row = 3		totalRows - row	1	5	totalRows - row	
row = 4		totalRows - row	1	7	totalRows - row	

For the uppler_limit of variable **numDec** we got two expressions. Somehow by putting a condition we need to switch the expression depends upon the value of row variable. Let's take one temporary variable **ul _ numDec** to represent upper_limit of the variable **numDec,** then it can be given as **;**

Variable : ul _ numDec

```
if(row == 0 ){
   ul_ numDec is ( totalRows – 1)
 }else{
   ul_ numDec is (totalRows - row).
 }
```

Row level Repetitive Action						
row	Column level Repetitive Action					
	Variable :numInc		Variable :space		Variable :numDec	
	lower_limit	upper_limit	lower_limit	upper_limit	upper_limit	lower_limit
		1			1	
row = 0	1	totalRows - row	0	0	ul _ numDec	1
row = 1			1	1	ul _ numDec	
row = 2			1	3	ul _ numDec	
row = 3			1	5	ul _ numDec	
row = 4			1	7	ul _ numDec	

Till now, we are able to generalize variables **numInc** and **numDec,** since their lower_limit and upper_limit are showing same expression for all the row values.

What remained now is to generalize the **upper_limit** for the variable **spaces.**

Column level Repetitive Action							
row	Variable :space						
	lower_limit	upper_limit					
			1	2	3	4	5
0	0	0	0	0	0	0	0
1	1	1	1	1	1 + 1 - 1	2*1 - 1	2*row -1
2	1	3	2 + 1	2 + 2 - 1	2 + 2 - 1	2*2 - 1	2*row -1
3	1	5	3 + 2	3 + 3 - 1	3 + 3 - 1	2*3 - 1	2*row -1
4	1	7	4 + 3	4 + 4 - 1	4 + 4 - 1	2*4 - 1	2*row -1

For the uppler_limit of variable **space at row = 0,** the value will be (2*0 – 1) or -1.The lower_limit value of variable **space at row = 0 is also 0,** so the loop-condition (0 < -1) will fail and no space will get printed. This holds true even for **loop-condidtion** (1 < -1). Hence, we are in a position to get generalized expression for lower_limit and upper_limit.

loop-condition	`lower_limit <=space<= upper_limit`
Mathematical expression	`1<=space<=(2* row - 1)`
Java syntax	`for( space = 1 ; space <= ( 2* row - 1 ) ; space++)`

This time somehow we are able to generalize the expressions otherwise we need to follow if-else condition as we did for the variable **numDec** using temporary variable **ul _ numDec.**

Alternate solution

Row level Repetitive Action							
row	Column level Repetitive Action						
	Variable :col		Variable :col		Variable :col		
	Print numbers		Print spaces			Print numbers	
1	1, 2 ,3 ,4		Print 5(no space)		6, 7, 8, 9		4, 3, 2, 1
2	1, 2, 3, 4		5		6, 7, 8, 9		4, 3, 2, 1
3	1, 2, 3		4,5,6		7, 8, 9		3, 2, 1
4	1, 2		3,4,5,6,7		8, 9		2, 1
5	1		2,3,4,5,6,7,8		9		1

Row level Repetitive Action							
row	Column level Repetitive Action						
	Variable :col		Variable :col		Variable :col		
	Print numbers		Print spaces		Print numbers		
	lower_limit	upper_limit	lower_limit	upper_limit	lower_limit	upper_limit	Output
1	1	4	Print 5(no space)		6	9	4, 3, 2, 1
2	1	4	5	5	6	9	4, 3, 2, 1
3	1	3	4	6	7	9	3, 2, 1
4	1	2	3	7	8	9	2, 1
5	1	1	2	8	9	9	1

For printing characters at the first row, means at row=1, this pattern doesn't required to print any space. So for the sake of initialization we can give it's upper_limit value as -1 and lower_limit value as 0.If we observe for printing spaces, the numeric pattern is such that the lower_limit value is less than upper_limit value for every row except the first row. This will help to allow, condition to be fail and we will be not able to print space in the first row. This what we need.

Moreover it's one of the common practice to initialize variable with invalid data in order to verify validity of the programming logic whenever that programming logic gets executed. The values of variable acts like a check point whether the programming logic executes as per expected or not and this our purpose of doing exercises on programming patterns to be able to visualize the flow of data.

Row level Repetitive Action							
row	Column level Repetitive Action						
	Variable :col		Variable :col		Variable :col		
	Print numbers		Print spaces		Print numbers		
	lower_limit	upper_limit	lower_limit	upper_limit	lower_limit	upper_limit	Output
1	1	totalRows - 1	0	-1	6	9	4, 3, 2, 1
2	1	totalRows - 1	totalRows - 0	totalRows + 0	6	9	4, 3, 2, 1
3	1	totalRows - 2	totalRows - 1	totalRows + 1	7	9	3, 2, 1
4	1	totalRows - 3	totalRows - 2	totalRows + 2	8	9	2, 1
5	1	totalRows - 4	totalRows - 3	totalRows + 3	9	9	1

If we take values of row variable starting from 0 instead of 1, then it will help us to generalize the **upper_limit .** Let try to generalize both of these using variable. If we observe, we can try to generalize lower_limit & upper_limit of printing spaces column using **row** variable. With using simple math we can alter lower_limit & upper_limit so that part of expression which is following a numeric pattern may match with the numeric pattern of **row** variable.

Row level Repetitive Action							
row	Column level Repetitive Action						
	Variable :col		Variable :col		Variable :col		
	Print numbers		Print spaces		Print numbers		
	Lower_limit	Upper_limit	Lower_limit	Upper_limit	lower_limit	Upper_limit	Output
0	1	totalRows - 1	0	-1	6	9	4, 3, 2, 1
1	1	totalRows - 1	totalRows - 0	totalRows + 0	6	9	4, 3, 2, 1
2	1	totalRows - 2	totalRows - 1	totalRows + 1	7	9	3, 2, 1
3	1	totalRows - 3	totalRows - 2	totalRows + 2	8	9	2, 1
4	1	totalRows - 4	totalRows - 3	totalRows + 3	9	9	1

Row level Repetitive Action							
row	Column level Repetitive Action						
	Variable :col		Variable :col		Variable :col		
	Print numbers		Print spaces		Print numbers		
	lower_limit	upper_limit	lower_limit	upper_limit	lower_limit	upper_limit	Output
0	1	totalRows - 1	0	-1	6	9	4, 3, 2, 1
1	1	totalRows - row	totalRows – 1 + 1	totalRows + 1 - 1	6	9	4, 3, 2, 1
2	1	totalRows - row	totalRows – 2 + 1	totalRows + 2 - 1	7	9	3, 2, 1
3	1	totalRows - row	totalRows – 3 + 1	totalRows + 3 - 1	8	9	2, 1
4	1	totalRows - row	totalRows – 4 + 1	totalRows + 4 - 1	9	9	1

Row level Repetitive Action							
row	Column level Repetitive Action						
	Variable :col		Variable :col		Variable :col		
	Print numbers		Print spaces		Print numbers		
	lower _limit	upper _limit	lower _limit	upper _limit	lower _limit	upper _limit	Output
0	1	totalRows - 1	0	-1	6	9	4, 3, 2, 1
1	1	totalRows - row	totalRows – row + 1	totalRows + row - 1	6	9	4, 3, 2, 1
2	1	totalRows - row	totalRows – row + 1	totalRows + row - 1	7	9	3, 2, 1
3	1	totalRows - row	totalRows – row + 1	totalRows + row - 1	8	9	2, 1
4	1	totalRows - row	totalRows – row + 1	totalRows + row - 1	9	9	1

If we build our programming logic giving conditions around set1 of printing numbers and printing spaces, then in the last else-part will take of set2 of printing numbers. By putting a condition we need to switch the expression of **lower_limit** or **upper_limit** depends upon the value of row variable.

Print numbers(set 1)

lower_limit variable : lw_lt_num

```
if(row == 0){
   lw_lt_num = ( totalRows - 1 )
}else{
  lw_lt_num = ( totalRows - row )
}
```

Print spaces

lower_limit variable : lw_lt_spaces

```
if(row == 0){
   lw_lt_spaces = 0;
}else{
  lw_lt_spaces =( totalRows - row + 1 );
}
```

Print spaces

upper_limit variable : up_lt_spaces

```
if(row == 0){
   up_lt_spaces = -1;
}else{
   up_lt_spaces =( totalRows + row - 1 )
}
```

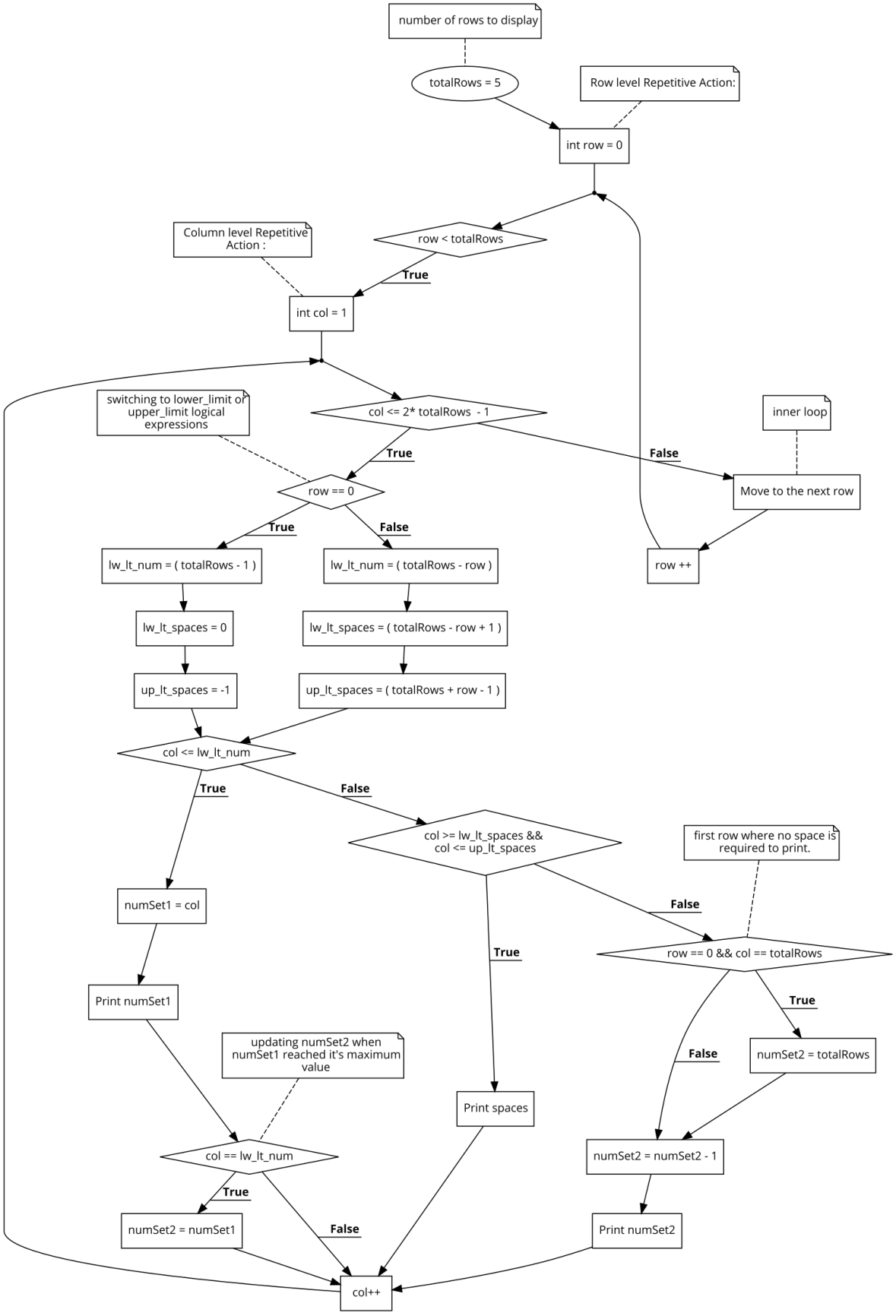
We know that total number of rows is given by the **variable:totalRows** = 5.

The **variable:col** takes the range of values from 1 to 9.

Col = 1, 2, 3… 9 **or**
Col = 1, 2, 3… (10 -1) **or**
Col = 1, 2, 3…, (2*5 -1) **or**
Col = 1, 2, 3…, (2* **totalRows** -1)

Hence, we expressed **upper_limit** of **variable:col** as (2* **totalRows** -1) in a generalized form.

We can summarize all our observations in a flow-chart given as follows.



NumberPatternFF.java

```
public class NumberPatternFF {

	public static void main(String[] args) {

		int  totalRows = 5;  //number of rows to display
		int lw_lt_num, lw_lt_spaces, up_lt_spaces;
		int numSet1;
		String numSet1Str;
		int numSet2 = 0;
		String numSet2Str;
		String spaces = "  ";// 2 spaces
		// Row level Repetitive Action :
		for( int row = 0 ; row < totalRows ; row ++ ) {
			// Column level Repetitive Action :
			for( int col = 1 ; col <= ( 2* totalRows  - 1 ) ; col++) {
				// switching to lower_limit or upper_limit
				// logical expressions
				if(row == 0){
					lw_lt_num = ( totalRows - 1 );
					lw_lt_spaces = 0;
					up_lt_spaces = -1;
				}else{
					lw_lt_num = ( totalRows - row );
					lw_lt_spaces = ( totalRows - row + 1 );
					up_lt_spaces = ( totalRows + row - 1 );
				}

				if( col <= lw_lt_num ){
					numSet1 = col;
					numSet1Str = numSet1 + " ";
					System.out.print( numSet1Str );
					// updating numSet2 when numSet1 reached
					// it's maximum value
					if(col == lw_lt_num ){
						numSet2 = numSet1;
					}
				}else if( col >= lw_lt_spaces &&
					col <= up_lt_spaces ) {
					System.out.print( spaces );
				}else {
					// first row where no space is required to print.
					if(row == 0 && col == totalRows ){
						numSet2 = totalRows;
					}
					numSet2Str = numSet2 + " ";
					numSet2 = numSet2 - 1;
					System.out.print(numSet2Str);

				}
			}// inner loop
			// Move cursor to the next row
			System.out.println();

		}// outer loop
	}

}
```

```
Output
1 2 3 4 5 4 3 2 1
1 2 3 4   4 3 2 1
1 2 3       3 2 1
1 2           2 1
1               1
```

If-else Shortcut

Short-cut in any matter is a special case to be applied in a specific situation. Not only in subject of programming people unknowingly confuse themselves simply by the fact that they choose a special case and tried to apply it commonly. Even in choice of words, two words may give almost similar meaning and when we actually apply those in different situations then only we will be in a position to explore it's true potential meaning.

The shortcut to if-else option is there in almost every programming language which is also known as **ternary operator** and is given by the symbol (:).

When to apply if-else shortcut

The ternary operator is a sort of syntactic sugar for if-else statements usually applied in a situation where both **if-block** and **else-block** assign different value to the same result variable. For example, let's suppose that if-else statement is something like the following;

If-else statement	If-else shortcut or ternary operator (:)
If(condition){ **result = *one_value*;** **}else{** **result = *other_value*;** **}**	**result = (condition)? *one_value* : *other_value* ;**
Simple if-else can be easily replaced by the use of if-else shortcut represented by the one liner **ternary operator.** In such situations only it makes true sense to use ternary **operator** to increase code readability.	• Two outcome result values are possible which are separated by a symbol colon (:). • If **condition** is true, then *one_value* will get assigned to the variable result. • If **condition** is false, then *one_value* will get assigned to the variable result.

In the program **NumberPatternFF.java** we can use ternary operator to improve that part of source code readability.

Separate if-else

```
if(row == 0){
    lw_lt_num = ( totalRows - 1 );
}else{
    lw_lt_num = ( totalRows - row );
}
```

```
if(row == 0){
     lw_lt_spaces = 0;
}else{
     lw_lt_spaces = ( totalRows - row + 1 );
}
```

```
if(row == 0){
     up_lt_spaces = -1;
}else{
     up_lt_spaces = ( totalRows + row - 1 );
}
```

Many If-else combined in single if-else

```
if(row == 0){
       lw_lt_num = ( totalRows - 1 );
       lw_lt_spaces = 0;
       up_lt_spaces = -1;
}else{
       lw_lt_num = ( totalRows - row );
       lw_lt_spaces = ( totalRows - row + 1 );
       up_lt_spaces = ( totalRows + row - 1 );
}
```

Equivalent If-else shortcut or ternary operator (:) expressions

```
lw_lt_num = (row == 0)? ( totalRows - 1 ) : ( totalRows - row );
```

```
lw_lt_spaces = (row == 0)? 0 : ( totalRows - row + 1 );
```

```
up_lt_spaces = (row == 0)? -1 : (totalRows + row - 1 );
```

I am not really a great fan of exceptions but just to share information, even **nested ternary operator is possible** like nested if-else. In most of the cases it makes hard to read the conditions and associated evaluation expressions. In some cases, a proper formatting of nested ternary operators may help but still code implemented will not be very easy read for junior level programmers at-least. As per my opinion, it is better to avoid nested ternary operators.

The updated program demonstrating the use of ternary operator (:).This is done in order to replace many simpler **if-else statements** to gain readability regarding programming code.

Let's check from the implementation.

NumberPatternFF2.java

```
public class NumberPatternFF2 {

	public static void main(String[] args) {

		int  totalRows = 5;  //number of rows to display
		int lw_lt_num, lw_lt_spaces, up_lt_spaces;
		int numSet1;
		String numSet1Str;
		int numSet2 = 0;
		String numSet2Str;
		String spaces = "  ";// 2 spaces
		// Row level Repetitive Action :
		for( int row = 0 ; row < totalRows ; row ++ ) {
			// Column level Repetitive Action :
			for( int col = 1 ; col <= ( 2* totalRows  - 1 ) ; col++) {
				// use of ternary operator, an simpler if-else shortcut
				lw_lt_num = (row == 0)?
						( totalRows - 1 ) : ( totalRows - row );
				lw_lt_spaces = (row == 0)?
						0 : ( totalRows - row + 1 );
				up_lt_spaces = (row == 0)?
						-1 : (totalRows + row - 1 );

				if( col <= lw_lt_num ){
					numSet1 = col;
					numSet1Str = numSet1 + " ";
					System.out.print( numSet1Str );

					// updating numSet2 when numSet1 reach
					// it's maximum value
					if(col == lw_lt_num ){
						numSet2 = numSet1;
					}
				}else if( col >= lw_lt_spaces &&
					col <= up_lt_spaces ) {
					System.out.print( spaces );
				}else {
					// first row, where no space will be print.
					if(row == 0 && col == totalRows ){
						numSet2 = totalRows;
					}
					numSet2Str = numSet2 + " ";
					numSet2 = numSet2 - 1;
					System.out.print(numSet2Str);

				}
			}// inner loop

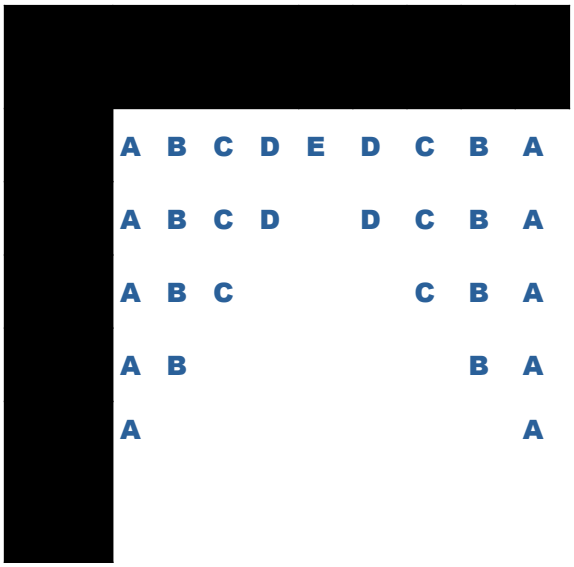
			// Move cursor to the next row
			System.out.println();

		}// outer loop
	}

}
```

NumberPattern GG

Our approach will be assumed that the computer screen is a combination of rows and columns virtually dividing it into smaller boxes. The smallest box is treated as a resident of just a single character only. The address in each smallest box on the display screen will logically become a unique **(row, column)** numeric combination. With this assumption computer screen will simply be a unique combination of (row, column) numeric value paired boxes which are uniformly spread all across the screen.



	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1	A	B	C	D	E	D	C	B	A
Row=2	A	B	C	D		D	C	B	A
Row=3	A	B	C				C	B	A
Row=4	A	B						B	A
Row=5	A								A

Data Representation

Although it is logically, we can exactly locate the position of each printable character on the display screen by knowing it's uniquely identifiable **(row, column)** numeric paired-values. *For example;* **(4, 3)** means printing character at the assumed screen location **4th row and 3rd column.**

(Row, Col) value pairs representation

	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1	(1,1) A	(1,2) B	(1,3) C	(1,4) D	(1,5) E	(1,6) D	(1,7) C	(1,8) B	(1,9) A
Row=2	(2,1) A	(2,2) B	(2,3) C	(2,4) D	2,5 space	(2,6) D	(2,7) C	(2,8) B	(2,9) A
Row=3	(3,1) A	(3,2) B	(3,3) C	3,4 space	3,5 space	3,6 space	(3,7) C	(3,8) B	(3,9) A
Row=4	(4,1) A	(4,2) B	4,3 space	4,4 space	4,5 space	4,6 space	4,7 space	(4,8) B	(4,9) A
Row=5	(5,1) A	5,2 space	5,3 space	5,4 space	5,5 space	5,6 space	5,7 space	5,8 space	(5,9) A

This pattern is a special case of **NumberPatternFF** and is about printing of alphabets instead of numbers. Every character what we see in keyboard whether it is alphabets, numbers or other special character has a unique code which is basically a number. So every character even the numbers also internally represented by a numeric value called **Unicode**. We can use in-built **typecasting** feature of Java to get the alphabets from Unicode values. It is used to convert variable of one data-type to another. For example;

```
// typecasting - converting integer data-type to char data-type
int unicode = 65;
char alpahabet = (char)unicode;
```

Type casting operator

Analysis of output

In general, three repetitive action takes place.

a) Printing of alphabets starting from A in incrementing order.

b) Printing of spaces

c) Printing of alphabets in decrementing order and it's upper limit will start from last updated incremented value.

So in an increment value order of Unicode we can get alphabets. We can verify this from following program which prints all 26 alphabets, capital or small along with Unicode.

Capital Alphabets	Unicode	Small Alphabets	Unicode
A	**65**	**a**	**97**
B	**66**	**b**	**98**
.....	.....	.....	.....
Z	90	z	122

UniCodeToAlphabets.java

```
public class UniCodeToAlphabets {
	public static void main( String[] args ) {
		String spaces = "  ";
		System.out.println("Alphabets and it's equivalent unicode." );
		System.out.println("--------------------------------------" );
		for ( int cnt = 0; cnt < 26; cnt++ ) {
			int unicodeCap = (65 + cnt);
			int unicodeSmall = (97 + cnt);
			String capLetter = (char) unicodeCap +
					"=" + (unicodeCap);
			String smallLetter = (char) unicodeSmall +
					"=" + (unicodeSmall);
			String print = capLetter + spaces +
					"|" + spaces + smallLetter ;
			System.out.println(print);
		}
	}
}
```

Output

```
Alphabets and it's equivalent unicode.
--------------------------------------
A=65  |  a=97
B=66  |  b=98
C=67  |  c=99
.....    .....
.....    .....
Y=89  |  y=121
Z=90  |  z=122
```

NumberPatternGG.java

```
public class NumberPatternGG {

	public static void main( String[] args ) {

		int  totalRows = 5;  //number of rows to display
		int lw_lt_num, lw_lt_spaces, up_lt_spaces;
		int numSet1;
		String numSet1Str;
		int numSet2 = 0;
		String numSet2Str;
		String spaces = "  ";// 2 spaces

		// Row level Repetitive Action :
		for( int row = 0 ; row < totalRows ; row ++ ) {

		// Column level Repetitive Action :
		for( int col = 1 ; col <= ( 2* totalRows  - 1 ) ; col++) {

			// switching the lower_limit or
			// upper_limit logical expressions
			lw_lt_num = (row == 0)?
					( totalRows - 1 ) : ( totalRows - row );
			lw_lt_spaces = (row == 0)?
					  0 : ( totalRows - row + 1 );
			up_lt_spaces = (row == 0)?
					 -1 : (totalRows + row - 1 );

			if( col <= lw_lt_num ){
				numSet1 = col + ( 65 - 1);
				numSet1Str = (char)numSet1 + " ";
				System.out.print( numSet1Str );
			// updating numSet2 when numSet1 reach it's maximum value
				if(col == lw_lt_num ){
					numSet2 = numSet1;
				}

			}else if( col >= lw_lt_spaces &&
				col <= up_lt_spaces ) {
				System.out.print( spaces );
			}else {
				// first row where no space is required to print.
				if(row == 0 && col == totalRows ){
					numSet2 = totalRows + ( 65 - 1);
				}
				numSet2Str = (char) numSet2 + " ";
				numSet2 = numSet2 - 1;
				System.out.print(numSet2Str);

			}
		}// inner loop

		   // Move cursor to the next row
		   System.out.println();

		}// outer loop

	}

}
```

```
Output
```

```
A B C D E D C B A
A B C D   D C B A
A B C       C B A
A B           B A
A               A
```

With little bit modification to the program **NumberPatternGG.java,** if we try we can generate the similar pattern in small alphabets. Instead of using the following for capital letters

```
numSet1 = col + ( 65 - 1); // Unicode for capital alphabet 'A' is 65
```

We can use the following

```
numSet1 = col + ( 97 - 1);  // Unicode for small alphabet 'a' is 97
```

Although with minor change we can get output in small alphabets but real point is to understand is that the most of the code will get repeated. We can avoid such repetition of programming logic by the concept of **functions** or methods. We can create our own function and use it as on demand in the same way as we are using in-built method/function like `System.out.println().`

Writing our own functions

Till now, we have used many times the concept of generalized expression based on variable(s) to represent hard-coded values. We can still go ahead to generalize programming logic too. One of major advantage of function with input parameter(s) is that it helps to achieve different types of output by reusing the same programming logic.

Writing functions are a very sensible idea that suggests how we can encapsulate different programming logics differently so that repetition of same piece of programming logic should not happen in other parts of the programs. Before we apply this concept we need to understand the commonly used syntax of a function in a class with **main method** which is as follows:

static return_type methodName(*parameter1, parameter2,…*){

    //programming logic

}

Where *parameter1, parameter2…* are the input(s) that a function may require and passing parameters in function is optional. The **return_type** is void if it is not returning any value or may return (int,char,..etc) using the return keyword. Since our main method is **static**, so just to keep technical stuff easy our method **printPattern** too using **static** keyword and moreover it is a part of object-oriented programming ,which is out of scope of this book**.** For the following three different types of output we can use the same **printPattern method**, encapsulating a common programming logic. The **printPattern** will print different pattern due to the differences in input parameters.

printPattern(param11, param12)	printPattern(param21, param22)	printPattern(param31, param32)
A A	a a	1 1
A B B A	a b b a	1 2 2 1
A B C C B A	a b c c b a	1 2 3 3 2 1
A B C D D C B A	a b c d d c b a	1 2 3 4 4 3 2 1
A B C D E D C B A	a b c d e d c b a	1 2 3 4 5 4 3 2 1

PatternByFunctionCall.java

```
public class PatternByFunctionCall {

	public static void main( String[] args ) {

		// main program
		int unicode_A = 65;// unicode for letter A
		String spaces4 = "    ";// 4 spaces - formatting output
		int unicode_a = 97;// unicode for letter a
		String spaces3 = "   ";// 3 spaces - formatting output
		int unicode_1 = 49;// unicode for digit 1.

		System.out.println("Print Capital Letter Pattern" );
		System.out.println("-----------------------------------" );
		printPattern(unicode_A, spaces4 );// function call

		System.out.println("Print Small Letter Pattern" );
		System.out.println("-----------------------------------" );
		printPattern(unicode_a, spaces3 );// function call

		System.out.println("Print Number Pattern FF" );
		System.out.println("-----------------------------------" );
		printPattern(unicode_1, spaces3 );// function call
	}

	// method/function definition
	static void printPattern( int unicode, String spaces ) {

		int totalRows = 5; // number of rows to display
		int lw_lt_num;
		int lw_lt_spaces;
		int up_lt_spaces;
		int numSet1;
		String numSet1Str;
		int numSet2 = 0;
		String numSet2Str;
		spaces = "  ";// 2 spaces

		// Row level Repetitive Action :
		for ( int row = 0; row < totalRows; row++ ) {

			// Column level Repetitive Action :
			// 1)Move cursor in the same row. 2)print character
			for ( int col = 1; col <= (2 * totalRows - 1); col++ ) {

			// switching the lower_limit or upper_limit
			// logical expressions
			lw_lt_num = (row == 0) ?
			            (totalRows - 1) : (totalRows - row);
			lw_lt_spaces = (row == 0) ?
			               0 : (totalRows - row + 1);
			up_lt_spaces = (row == 0) ?
			               -1 : (totalRows + row - 1);
```

```
                        if ( col <= lw_lt_num ) {
                              numSet1 = col + (unicode - 1);
                              numSet1Str = (char) numSet1 + " ";
                              System.out.print(numSet1Str );

                          // updating numSet2 when numSet1 reach
                          // it's maximum value
                          if ( col == lw_lt_num ) {
                              numSet2 = numSet1;
                          }

                        } else if ( col >= lw_lt_spaces &&
                                    col <= up_lt_spaces ) {
                              System.out.print(spaces );
                        } else {
                        // first row where no space is required to print.
                              if ( row == 0 && col == totalRows ) {
                                    numSet2 = totalRows + ( unicode - 1);
                              }
                              numSet2Str = (char) numSet2 + " ";
                              numSet2 = numSet2 - 1;
                              System.out.print(numSet2Str );

                        }
                    } // inner loop
                    System.out.println();// Move cursor to the next row

              } // outer loop

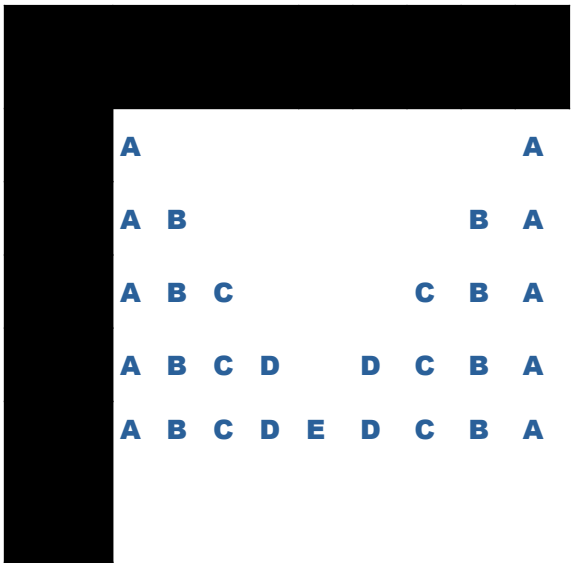
       }// method ends here
}
```

Output

```
Print Capital Letter Pattern
---------------------------------
A B C D E D C B A
A B C D   D C B A
A B C       C B A
A B           B A
A               A
Print Small Letter Pattern
---------------------------------
a b c d e d c b a
a b c d   d c b a
a b c       c b a
a b           b a
a               a
Print Number Pattern FF
---------------------------------
1 2 3 4 5 4 3 2 1
1 2 3 4   4 3 2 1
1 2 3       3 2 1
1 2           2 1
1               1
```

NumberPattern EE

Our approach will be assumed that the computer screen is a combination of rows and columns virtually dividing it into smaller boxes. The smallest box is treated as a resident of just a single character only. The address in each smallest box on the display screen will logically become a unique **(row, column)** numeric combination. With this assumption computer screen will simply be a unique combination of (row, column) numeric value paired boxes which are uniformly spread all across the screen.



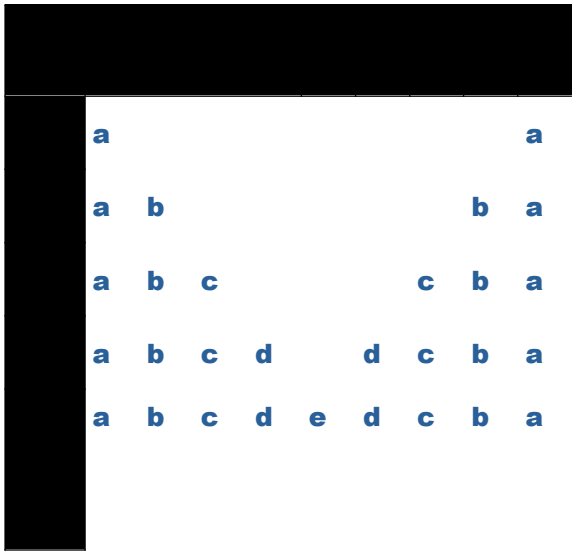
	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1	A								A
Row=2	A	B						B	A
Row=3	A	B	C				C	B	A
Row=4	A	B	C	D		D	C	B	A
Row=5	A	B	C	D	E	D	C	B	A

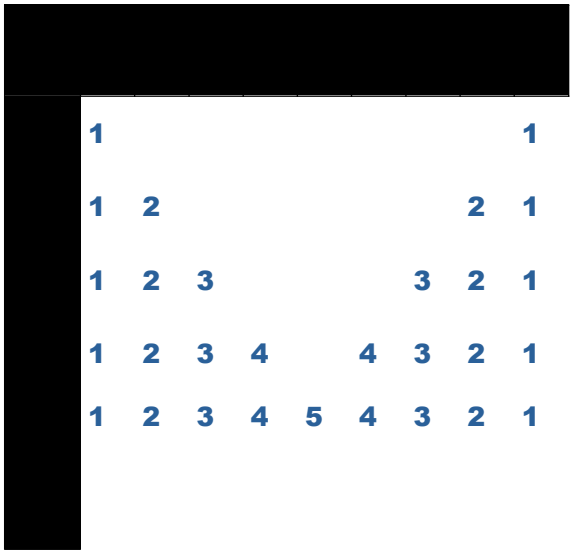
Data Representation

Although it is logically, we can exactly locate the position of each printable character on the display screen by knowing it's uniquely identifiable **(row, column)** numeric paired-values. *For example;* **(4, 3)** means printing character at the assumed screen location **4th row and 3rd column.**

(Row, Col) value pairs representation

	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1	(1,1) A	1,2 space	1,3 space	1,4 space	1,5 space	1,6 space	1,7 space	1,8 space	(1,9) A
Row=2	(2,1) A	(2,2) B	2,3 space	2,4 space	2,5 space	2,6 space	2,7 space	(2,8) B	(2,9) A
Row=3	(3,1) A	(3,2) B	(3,3) C	3,4 space	3,5 space	3,6 space	(3,7) C	(3,8) B	(3,9) A
Row=4	(4,1) A	(4,2) B	(4,3) C	(4,4) D	4,5 space	(4,6) D	4,7 C	(4,8) B	(4,9) A
Row=5	(5,1) A	(5,2) B	(5,3) C	(5,4) D	(5,5) E	(5,6) D	5,7 C	5,8 B	(5,9) A





	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1	a								a
Row=2	a	b						b	a
Row=3	a	b	c				c	b	a
Row=4	a	b	c	d		d	c	b	a
Row=5	a	b	c	d	e	d	c	b	a

	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1	1								1
Row=2	1	2						2	1
Row=3	1	2	3				3	2	1
Row=4	1	2	3	4		4	3	2	1
Row=5	1	2	3	4	5	4	3	2	1

Analysis of output

This pattern is just reverse of pattern **NumberPatternGG.** What we have printed in first row in the pattern **NumberPatternGG**, same we need to print in the last row. So, by following the same trick we can generate this pattern. We can conclude two things about this pattern;

a) The outer for-loop which indicates the movement of cursor row-wise, need to reverse it.It means we need to follow decrementing for-loop using row variable.

b) The column wise the it will print in the same manner as it was done in **NumberPatternGG.**

ReversePatternByFunctionCall.java

```
public class ReversePatternByFunctionCall {

	public static void main( String[] args ) {

		int unicode_A = 65;// unicode for letter A
		String spaces4 = "    ";// 4 spaces - formatting output
		int unicode_a = 97;// unicode for letter a
		String spaces3 = "   ";// 3 spaces - formatting output
		int unicode_1 = 49;// unicode for digit 1.
		System.out.println("Print Capital Letter Pattern" );
		System.out.println("-----------------------------------" );
		printPattern(unicode_A, spaces4 );

		System.out.println();
		System.out.println("Print Small Letter Pattern" );
		System.out.println("-----------------------------------" );
		printPattern(unicode_a, spaces3 );

		System.out.println();
		System.out.println("Print Number Pattern FF" );
		System.out.println("-----------------------------------" );
		printPattern(unicode_1, spaces3 );
	}

	// method/function definition
	static void printPattern( int unicode, String spaces ) {

		int totalRows = 5; // number of rows to display
		int lw_lt_num;
		int lw_lt_spaces;
		int up_lt_spaces;
		int numSet1;
		String numSet1Str;
		int numSet2 = 0;
		String numSet2Str;

		spaces = "  ";// 2 spaces

		// Row level Repetitive Action :
		for ( int row = (totalRows-1) ; row >= 0 ; row -- ) {

			// Column level Repetitive Action :
			// 1)Move cursor in the same row. 2)print character
			for ( int col = 1; col <= (2 * totalRows - 1); col++ ) {

			// switching the lower_limit or
			// upper_limit logical expressions
			lw_lt_num = (row == 0) ?
			            (totalRows - 1) : (totalRows - row);
			lw_lt_spaces = (row == 0) ?
			               0 : (totalRows - row + 1);
			up_lt_spaces = (row == 0) ?
			              -1 : (totalRows + row - 1);

				if ( col <= lw_lt_num ) {
					numSet1 = col + (unicode - 1);
					numSet1Str = (char) numSet1 + " ";
					System.out.print(numSet1Str );
```

```
                                    // updating numSet2 when numSet1 reach
                                    // it's maximum value
                                    if ( col == lw_lt_num ) {
                                           numSet2 = numSet1;
                                    }

                                } else if ( col >= lw_lt_spaces &&
                                          col <= up_lt_spaces ) {
                                             System.out.print(spaces );

                                } else {
                                    // first row where no space is required to print.
                                    if ( row == 0 && col == totalRows ) {
                                           numSet2 = totalRows + (65 - 1);
                                    }
                                    numSet2Str = (char) numSet2 + " ";
                                    numSet2 = numSet2 - 1;
                                    System.out.print(numSet2Str );

                                }

                        } // inner loop

                        // Move cursor to the next row
                        System.out.println();

                } // outer loop

        }// method ends here

}
```

Output

```
Print Capital Letter Pattern
-----------------------------------
A               A
A B           B A
A B C       C B A
A B C D   D C B A
A B C D E D C B A

Print Small Letter Pattern
-----------------------------------
a               a
a b           b a
a b c       c b a
a b c d   d c b a
a b c d E D C B A

Print Number Pattern FF
-----------------------------------
1               1
1 2           2 1
1 2 3       3 2 1
1 2 3 4   4 3 2 1
1 2 3 4 E D C B A
```

There is an error in the output related to small letter pattern and number pattern. Please correct the program to give the appropriate output.

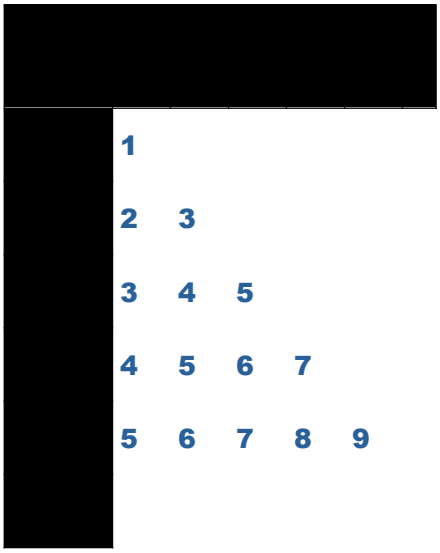
Problem: Outputting following patterns using the concept of function.

	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1	A	B	C	D	E	D	C	B	A
Row=2	A	B	C	D		D	C	B	A
Row=3	A	B	C				C	B	A
Row=4	A	B						B	A
Row=5	A								A
Row=6	A	B						B	A
Row=7	A	B	C				C	B	A
Row=8	A	B	C	D		D	C	B	A
Row=9	A	B	C	D	E	D	C	B	A

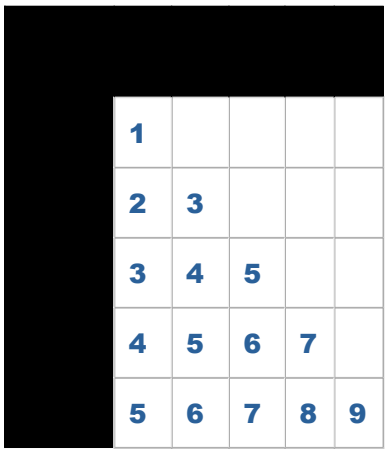
	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1	a	b	c	d	e	d	c	b	a
Row=2	a	b	c	d		d	c	b	a
Row=3	a	b	c				c	b	a
Row=4	a	b						b	a
Row=5	a								a
Row=6	a	b						b	a
Row=7	a	b	c				c	b	a
Row=8	a	b	c	d		d	c	b	a
Row=9	a	b	c	d	e	d	c	b	a

	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1	1	2	3	4	5	4	3	2	1
Row=2	1	2	3	4		4	3	2	1
Row=3	1	2	3				3	2	1
Row=4	1	2						2	1
Row=5	1								1
Row=6	1	2						2	1
Row=7	1	2	3				3	2	1
Row=8	1	2	3	4		4	3	2	1
Row=9	1	2	3	4	5	4	3	2	1

NumberPattern AAA



Our programming job is to print this particular logic-based pattern on a part of the computer screen. **Our approach will be assumed that the computer screen is a combination of rows and columns virtually dividing it into smaller boxes.** The smallest box is treated as a resident of just a single character only. The address in each smallest box on the display screen will logically become a unique **(row, column)** numeric combination. With this assumption computer screen will simply be a unique combination of (row, column) numeric value paired boxes which are uniformly spread all across the screen.



	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	1				
Row=2	2	3			
Row=3	3	4	5		
Row=4	4	5	6	7	
Row=5	5	6	7	8	9

Data Representation

Although it is logically, we can exactly locate the position of each printable character on the display screen by knowing it's uniquely identifiable **(row, column)** numeric paired-values. *For example;* **(4, 3)** means printing numeric character at the assumed screen location **4th row and 3rd column.**

(Row, Col) value pairs representation

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	1,1				
Row=2	2 ,1	2,2			
Row=3	3,1	3,2	3,3		
Row=4	4,1	4,2	4,3	4,4	
Row=5	5,1	5,2	5,3	5,4	5,5

The given set of row and col value pairs tries to mimic the location of each printable character of a given pattern on the device's display screen. The set of all value pairs (row = value, col = value) represent this particular pattern.

Analysis of output.

It can also be interpreted in a tabular form.

row	1				
Col	1				
Output	1				

For row =1, 1<=col<=1

row	2				
Col	1	2			
Output	2	3			

For row =2, 1<=col<=2

row	3				
Col	1	2	3		
Output	3	4	5		

For row =3, 1<=col<=3

row	4				
Col	1	2	3	4	
Output	4	5	6	7	

For row =4, 1<=col<=4

row	5				
Col	1	2	3	4	5
Output	5	6	7	8	9

For row =5, 1<=col<=5

For all the rows, we need to generalize the lower_limit & upper_limit so it should be suitable for any number of rows.

Observations

3) There is repetition required at row level vertically, i.e., Row = 1 to 5.
4) There is repetition required at col level horizontally. , i.e., Col = 1 to 5.
   So it's a 2-way repetition process means require 2 for-loops(nested for – loops).

For Loop	Lower limit	Upper limit
Outer loop	**row=1**	**row<=5**
Inner Loop	**col=(Unknown)** **Need to find a relation how exactly it's varying**	**(Unknown)** **Need to find a relation how exactly it's varying.**

COLUMN LEVEL REPETITIVE ACTION

Row level Repetitive Action							Expected Output
row	Column level Repetitive Action						
	loop-condition(lower_limit <=col<= upper_limit)						
	lower_limit		upper_limit				
	default	1	default	1	2	3	
1	1	row	1	1	1+1 - 1	row + row - 1	**1**
2	2	row	3	2 + 1	2+2 - 1	row + row - 1	**2 3**
3	3	row	5	3 + 2	3+3 - 1	row + row - 1	**3 4 5**
4	4	row	7	4 + 3	4+4 - 1	row + row - 1	**4 5 6 7**
5	5	row	9	5 + 4	5+5 - 1	row + row - 1	**5 6 7 8 9**

The lower_limit matches the row value, so that it can be generalized to variable row.

Row level Repetitive Action							Expected Output
row	Column level Repetitive Action						
	loop-condition(lower_limit <=col<= upper_limit)						
	lower_limit		upper_limit				
	default	1	default	1	2	3	
1	1	row	1	1	1+1 - 1	2*row - 1	**1**
2	2	row	3	2 + 1	2+2 - 1	2*row - 1	**2 3**
3	3	row	5	3 + 2	3+3 - 1	2*row - 1	**3 4 5**
4	4	row	7	4 + 3	4+4 - 1	2*row - 1	**4 5 6 7**
5	5	row	9	5 + 4	5+5 - 1	2*row - 1	**5 6 7 8 9**

Row level Repetitive Action							Expected Output
row	Column level Repetitive Action						
	loop-condition(lower_limit <=col<= upper_limit)						
	lower_limit		upper_limit				
	1	2	default	2	3	4	
1	row	row	1	1+1 - 1	2*row - 1	2*row - 1	**1**
2	row		3	2+2 - 1	2*row - 1		**2 3**
3	row		5	3+3 - 1	2*row - 1		**3 4 5**
4	row		7	4+4 - 1	2*row - 1		**4 5 6 7**
5	row		9	5+5 - 1	2*row - 1		**5 6 7 8 9**

The lower_limit & upper_limit are become consistent for all the rows. The lower _limit is generalized to **row** and upper_limit is generalized to (**2*row – 1**).These logical expressions are such that on commonly applying for all rows and columns helps to give matching output.

So our final logical expression becomes;

loop-condition	`lower_limit <=col<= upper_limit`
Mathematical expression	`row <= col <= (2*row – 1)`
Java syntax	`for( col = row ; col <= 2*row - 1; col++ )`

NumberPatternAAA.java

```
public class NumberPatternAAA {

	public static void main(String[] args) {

		int  totalRows = 5;  //number of rows to display

		// Row level Repetitive Action :
		for( int row = 1 ; row <= totalRows ; row ++ ) {

			// Column level Repetitive Action:
			// 1)Move cursor in the same row. 2)print character
			for( int col = row ; col <= ( 2*row - 1 ) ; col++ ) {

				// added extra space for clarity in output.
				 String num = col + " ";
				 System.out.print( num );
			}

			// move control to the next line
			System.out.println();
		}
	}
}
```

Output

```
1
2 3
3 4 5
4 5 6 7
5 6 7 8 9
```

ProblemGBAAA: **Modify program NumberPatternAAA.java so that it should show following output using switch-case for values 1 and 2 respectively.**

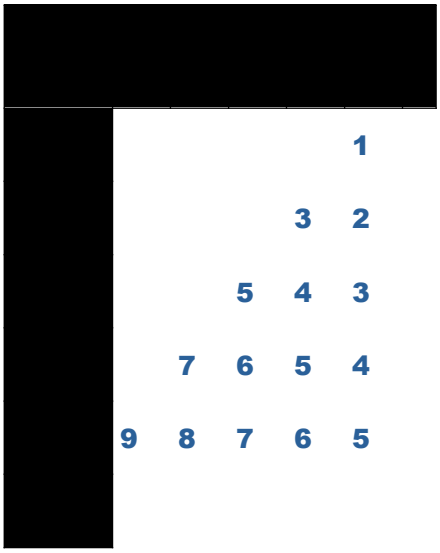
Output

```
1
  3
3   5
  5   7
5   7   9
```

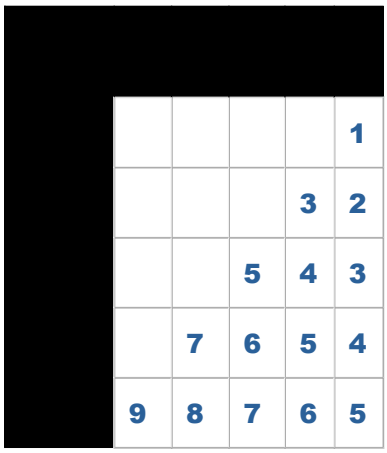
Output

```
2
  4
4   6
  6   8
```

NumberPattern BBB



Our programming job is to print this particular logic-based pattern on a part of the computer screen. **Our approach will be assumed that the computer screen is a combination of rows and columns virtually dividing it into smaller boxes.** The smallest box is treated as a resident of just a single character only. The address in each smallest box on the display screen will logically become a unique **(row, column)** numeric combination. With this assumption computer screen will simply be a unique combination of (row, column) numeric value paired boxes which are uniformly spread all across the screen.



	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1					1
Row=2				3	2
Row=3			5	4	3
Row=4		7	6	5	4
Row=5	9	8	7	6	5

Data Representation

Although it is logically, we can exactly locate the position of each printable character on the display screen by knowing it's uniquely identifiable **(row, column)** numeric paired-values. *For example;* **(4, 3)** means printing numeric character at the assumed screen location **4th row and 3rd column.**

(Row, Col) value pairs representation

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1					1,5
Row=2				2,4	2,5
Row=3			3,3	3,4	3,5
Row=4		4,2	4,3	4,4	4,5
Row=5	5,1	5,2	5,3	5,4	5,5

The given set of row and col value pairs tries to mimic the location of each printable character of a given pattern on the device's display screen. The set of all value pairs (row = value, col = value) represent this particular pattern.

Analysis of output

In this case we need to take care of printing two different types of characters at a particular column in a particular row:

1) Space character

2) Numeric character

row	1				
Col	1	2	3	4	5
Output	space	space	space	space	1

For row =1, 1 <=spaces<=4, 1>=num>=1

row	2				
Col	1	2	3	4	5
Output	space	space	space	3	2

For row =2, 1 <=spaces<=3, 3>=num>=2

row	3				
Col	1	2	3	4	5
Output	space	space	5	4	3

For row =3, 1 <=spaces<=2, 5>=num>=3

row	4				
Col	1	2	3	4	5
Output	space	7	6	5	4

For row =4, 1 <=spaces<=1, 7>=num>=4

row	5				
Col	1	2	3	4	5
Output	9	8	7	6	5

For row =5, 9>=num>=5

Observations

1) There is repetition at row level vertically, i.e., Row = 1 to 5.
2) There is repetition at col level horizontally. , i.e., Col = 1 to 5.
   This gets divided into two parts .One for repetitively printing spaces and remaining column for printing numeric characters.

For Loop	Lower limit	Upper limit
Outer loop	row=1	row<=5
Inner Loop - spaces	col=1	col <= ?(Unkown)
Inner Loop - Number	num = ?(Unkown)	num = 1

COLUMN LEVEL REPETITIVE ACTION

Row level Repetitive Action					Expected Output
row	Column Level Repetitive Action				
	lower_limit <= col <= upper_limit		upper_limit >=num >= lower_limit		
	lower_limit	upper_limit	upper_limit	lower_limit	
1	1	4	1	1	**1**
2	1	3	3	2	**3 2**
3	1	2	5	3	**5 4 3**
4	1	1	7	4	**7 6 5 4**
5	1	0	9	5	**9 8 7 6 5**

We need to come out with a consistent expression for upper_limit for which it should suit any number of rows. We can refer previous pattern problem **PatternAAA's** result for the num variable. The other variables are row & totalRows=5. Let's try to generalize using these variables.

Row level Repetitive Action							
row	Column Level Repetitive Action						
	lower_limit <= col <= upper_limit				upper_limit >=num >= lower_limit		
	lower_limit	upper_limit			upper_limit		lower_limit
1	1	4	totalRows - 1	totalRows - row	1	row	row
2	1	3	totalRows - 2	totalRows - row	3	row + 1	row
3	1	2	totalRows - 3	totalRows - row	5	row + 2	row
4	1	1	totalRows - 4	totalRows - row	7	row + 3	row
5	1	0	totalRows - 5	totalRows - row	9	row + 4	row

Again trying to express the hard coded numeric values of upper_limit related to variable num in terms of variable row.

Column Level Repetitive Action							
row	lower_limit <= col <= upper_limit		upper_limit >=num >= lower_limit				
	lower_limit	upper_limit	upper_limit				lower_limit
1	1	totalRows - row	1	row	row + 1 - 1	row + row - 1	row
2	1	totalRows - row	3	row + 1	row + 2 - 1	row + row - 1	row
3	1	totalRows - row	5	row + 2	row + 3 - 1	row + row - 1	row
4	1	totalRows - row	7	row + 3	row + 4 - 1	row + row - 1	row
5	1	totalRows - row	9	row + 4	row + 5 - 1	row + row - 1	row

Column Level Repetitive Action							
row	lower_limit <= col <= upper_limit		upper_limit >=num >= lower_limit				
	lower_limit	upper_limit	upper_limit				lower_limit
1	1	totalRows - row	1	row + 1 - 1	row + row - 1	2*row - 1	row
2	1	totalRows - row	3	row + 2 - 1	row + row - 1	2*row - 1	row
3	1	totalRows - row	5	row + 3 - 1	row + row - 1	2*row - 1	row
4	1	totalRows - row	7	row + 4 - 1	row + row - 1	2*row - 1	row
5	1	totalRows - row	9	row + 5 - 1	row + row - 1	2*row - 1	row

Column Level Repetitive Action								
row	lower_limit <= col <= upper_limit				upper_limit >=num >= lower_limit			
	lower_limit		upper_limit		upper_limit		lower_limit	
1	1		totalRows - row		2*row - 1		row	
2	1		totalRows - row		2*row - 1		row	
3	1	**1**	totalRows - row	**totalRows - row**	2*row - 1	**2*row - 1**	row	**row**
4	1		totalRows - row		2*row - 1		row	
5	1		totalRows - row		2*row - 1		row	

So the upper_limit of col & num variables are generalized. The lower_limit & upper_limit are consistent for all the rows. It's now generalized for any number of rows.

So our loop-condition for space count on each row using variable col becomes;

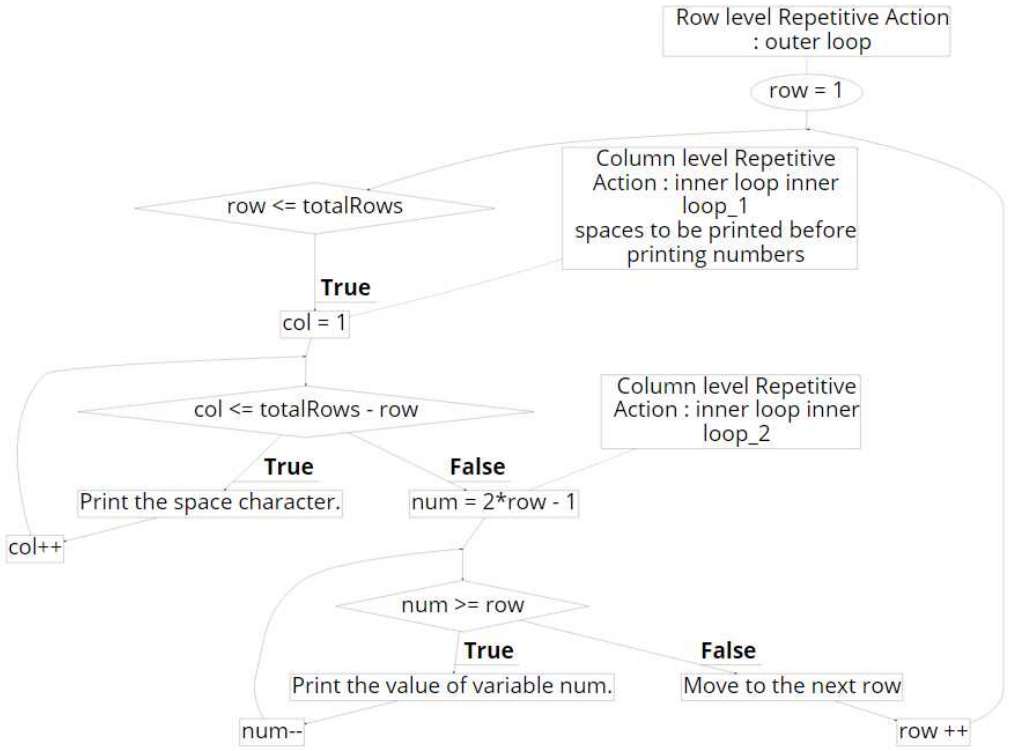
loop-condition	`lower_limit <=col<= upper_limit`
Mathematical expression	`1 <= col <= (totalRows – row)`
Java syntax	`for( col = 1 ; col <= (totalRows - row) ; col++)`

So our loop-condition for printing numbers on each row using variable num becomes;

loop-condition	`upper_limit >=num >= lower_limit`
Mathematical expression	`1 <= col <= (totalRows – row)`
Java syntax	`for( num = 2*row - 1  ; num >= row  ; num--)`

So it has two repetition process one for printing spaces and second for printing numeric values. It means require 2 inner for-loops (nested for – loops).

The flow-chart can be given as follows to capture the details.



NumberPatternBBB1.java

```
public class NumberPatternBBB1 {

	public static void main( String[] args ) {

		int  totalRows = 5;  //number of rows to display

		// Row level Repetitive Action :
		for( int row = 1 ; row <= totalRows ; row ++ ) {

			// Column level Repetitive Action :
			// 1)Move cursor in the same row. 2)print character

			// spaces to be printed before printing numbers
			for( int col = 1 ; col <= (totalRows - row) ; col++ ) {
				System.out.print( "  " );
			}// inner loop_1

			for( int num = ( 2*row - 1 ) ; num >= row  ; num-- ) {
				// added extra space for clarity in output.
				String numStr = num  + " ";
				System.out.print( numStr );
			}// inner loop_2

			// move control to the next line
			System.out.println();
		}
	}
}
```

Output

```
        1
      3 2
    5 4 3
  7 6 5 4
9 8 7 6 5
```

Alternative solution

Let's device a solution by merging two inner for-loops into single by giving the if-else construct.

We can directly get the idea from for-loop condition of the both the inner for-loops. It's either printing space or number. Number starts with the formula (**2*row - 1)** and decreases till it reaches the row value. The lower_limit is an initialization part and need to do it only once.

If we take care of initialisation and step-decrementing part then the loop-condition can be converted into the **if-condition.**

NumberPatternBBB2.java

```
public class NumberPatternBBB2 {

	public static void main(String[] args) {

		int  totalRows = 5;  //number of rows to display
		// Row level Repetitive Action :
		for( int row = 1 ; row <= totalRows ; row ++ ) {

			// Column level Repetitive Action :
			// 1)Move cursor in the same row. 2)print character

			// spaces to be printed before printing numbers
			int num = 2*row - 1;//  horizontal movements

			// spaces to be printed before printing numbers
			for(int col = 1 ; col <= (totalRows)   ; col++) {

				if( col <= (totalRows - row) ) {
					System.out.print( "  " );
				}else if( num >= row ) {
					// added extra space for clarity in output.
					String numStr = num  + " ";
					System.out.print( numStr );
					num = num - 1;
				}

			}

			// move control to the next line
			System.out.println();
		}
	}
}
```

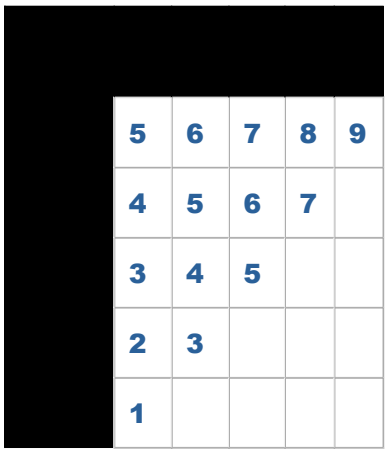
Output

```
        1
      3 2
    5 4 3
  7 6 5 4
9 8 7 6 5
```

NumberPattern CCC



Our programming job is to print this particular logic-based pattern on a part of the computer screen. **Our approach will be assumed that the computer screen is a combination of rows and columns virtually dividing it into smaller boxes.** The smallest box is treated as a resident of just a single character only. The address in each smallest box on the display screen will logically become a unique **(row, column)** numeric combination. With this assumption computer screen will simply be a unique combination of (row, column) numeric value paired boxes which are uniformly spread all across the screen.



	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	5	6	7	8	9
Row=2	4	5	6	7	
Row=3	3	4	5		
Row=4	2	3			
Row=5	1				

Data Representation

Although it is logically, we can exactly locate the position of each printable character on the display screen by knowing it's uniquely identifiable **(row, column)** numeric paired-values. *For example;* **(4, 3)** means printing numeric character at the assumed screen location **4th row and 3rd column.**

(Row, Col) value pairs representation

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	5,1	5,2	5,3	5,4	5,5
Row=2	4,1	4,2	4,3	4,4	
Row=3	3,1	3,2	3,3		
Row=4	2,1	2,2			
Row=5	1,1				

The given set of row and col value pairs tries to mimic the location of each printable character of a given pattern on the device's display screen. The set of all value pairs (row = value, col = value) represent this particular pattern.

Analysis of output

When Row = 1, Col = 1,2,3,4,5 (1 to 5) : repetition will be 5 times horizontally (1 <=Col <= 5)

When Row = 2, Col = 1,2,3,4 (1 to 4) : repetition will be 4 times horizontally (1 <=Col <= 4)

When Row = 3, Col = 1,2,3 (1 to 3) : repetition will be 3 times horizontally (1 <=Col <= 3)

When Row = 4, Col = 1,2 (1 to 2) : repetition will be 2 times horizontally (1 <=Col <=2)

When Row = 5, Col = 1 : will be 1 time (1 <=Col <=1)

We need to take care of logic of numeric output only and not to take care of logic required

to print spaces. The logic for spaces is to be considered only if spaces comes before numeric output.

So, it's irrelevant in this case.

row	5				
col	5	6	7	8	9
output	5	6	7	8	9

row =5 , (5 <=Col <= 9) (print till upper_limit=9)

row	4				
col	4	5	6	7	
output	4	5	6	7	

row = 4 , (4 <=Col <= 7) (print till upper_limit=7)

row	3				
col	3	4	5		
output	3	4	5		

row = 3 , (3 <=Col <= 5) (print till upper_limit=5)

row	2				
col	2	3			
output	2	3			

row = 2 , (2 <=Col <= 2) (print till upper_limit=3)

row	1				
col	1				
output	1				

row = 1 , (1 <=Col <= 1) (print till upper_limit=1)

The row value equals to the lower_limit of col variable. The upper_limit value of col variable varies in each row.

Let's device a solution by **decrementing the outer for-loop using variable row** (5 to 1) and determining the loop-condition(lower_limit <=col<= upper_limit) of col variable to give the desired numeric output. Let's try to get generalized expression using the variable **row**.

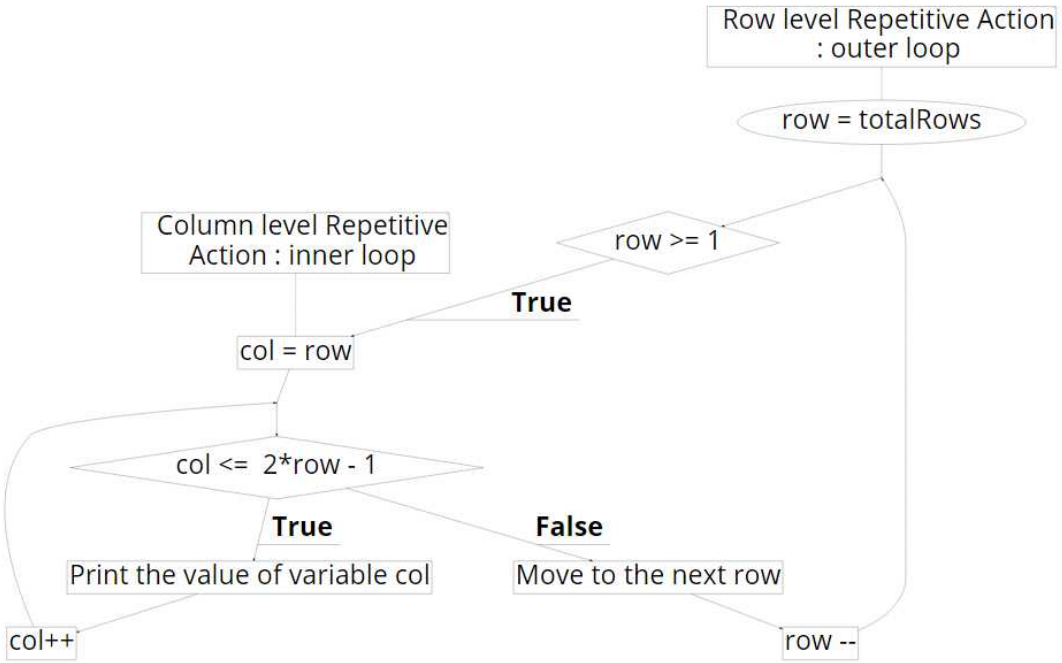
COLUMN LEVEL REPETITIVE ACTION

Row level Repetitive Action(decrementing loop)									Expected Output
row	Column Level Repetitive Action								
	loop-condition (lower_limit <=col<=upper_limit)								
	lower_limit		upper_limit						
	1		1	2	3	4	5		
5	5	row	9	5+4	row + 4	row + 5-1	row + row - 1	2* row - 1	5 6 7 8 9
4	4	row	7	4+3	row + 3	row + 4-1	row + row - 1	2* row - 1	4 5 6 7
3	3	row	5	3+2	row + 2	row + 3-1	row + row - 1	2* row - 1	3 4 5
2	2	row	3	2+1	row + 1	row + 2-1	row + row - 1	2* row - 1	2 3
1	1	row	1	1+0	row + 0	row + 1-1	row + row - 1	2* row - 1	1

So our inner loop-condition to give numeric output on each row using variable col becomes;

loop-condition	`lower_limit <=col<= upper_limit`
Mathematical expression	`row <= col <= (2* row - 1)`
Java syntax	`for( col = row ; col <= ( 2*row - 1 ) ; col++)`

The flow-chart can be given as follows.



Let execute the following program to verify the output.

NumberPatternCCC1.java

```
public class NumberPatternCCC1 {

	public static void main(String[] args) {

		int totalRows = 5;  //number of rows to display
		String colStr;
		String spaces = "  "; //2 spaces

		// Row level Repetitive Action :
		for( int row = totalRows ; row >= 1 ; row -- ) {

			// Column level Repetitive Action :
			// 1)Move cursor in the same row. 2)print character
			for( int col = row ; col <= ( 2*row - 1 ) ; col++ ) {
				colStr = col  + " ";
				System.out.print( colStr );

			}// inner loop

			// Move cursor to the next row
			System.out.println();

		}// outer loop
	}

}
```

Output

```
5 6 7 8 9
4 5 6 7
3 4 5
2 3
1
```

ProblemGBCCC1: **Modify program `NumberPatternCCC1.Java` so that it should give following output.**

Output

```
        9
      7
    5
  3
1
```

Alternative solution

Let's try to device a solution using incrementing for-loop for row variable (1 to 5) and determining the loop-condition for col variable.

The row value in this case doesn't matches the lower_limit of col variable on each row. So row variable we can't use it directly to express lower_limit of col variable.

We shall try other variable totalRows=5 (default number of rows) to generalize it.

Row level Repetitive Action(incrementing loop)						Expected Output
row	Column Level Repetitive Action					
	loop-condition(lower_limit <=col<= upper_limit)					
	lower_limit			upper_limit		
	default	1	2	default	1	
1	5	totalRows	totalRows +1 - 1	9	2* totalRows - 1	5 6 7 8 9
2	4	totalRows -1	totalRows +1 - 2	7	2* totalRows - 3	4 5 6 7
3	3	totalRows -2	totalRows +1 - 3	5	2* totalRows - 5	3 4 5
4	2	totalRows -3	totalRows +1 - 4	3	2* totalRows - 7	2 3
5	1	totalRows -4	totalRows +1 - 5	1	2* totalRows - 9	1

Part of lower_limit's logical expression matches with the row value. Since all numeric values are matching with value of row variable in each row. We should use row variable to remove numeric values occurring in the lower_limit's logical expression.

Column Level Repetitive Action					Expected Output
row	*loop-condition(lower_limit <=col<= upper_limit)*				
	lower_limit		upper_limit		
	default	3	default	1	
1	5	totalRows +1 - row	9	2* totalRows - 1	**5 6 7 8 9**
2	4	totalRows +1 - row	7	2* totalRows - 3	**4 5 6 7**
3	3	totalRows +1 - row	5	2* totalRows - 5	**3 4 5**
4	2	totalRows +1 - row	3	2* totalRows - 7	**2 3**
5	1	totalRows +1 - row	1	2* totalRows - 9	**1**

Now it's turn to generalize the upper_limit of col variable.

Column Level Repetitive Action					Expected Output
row	*loop-condition(lower_limit <=col<= upper_limit)*				
	lower_limit	upper_limit			
	3	default	1	2	
1	totalRows +1 - row	9	2* totalRows - 1	2* totalRows - 1	**5 6 7 8 9**
2	totalRows +1 - row	7	2* totalRows - 3	2* totalRows - 2 -1	**4 5 6 7**
3	totalRows +1 - row	5	2* totalRows - 5	2* totalRows - 3 -2	**3 4 5**
4	totalRows +1 - row	3	2* totalRows - 7	2* totalRows - 4 -3	**2 3**
5	totalRows +1 - row	1	2* totalRows - 9	2* totalRows - 5 -4	**1**

Let's try to generalize part of the upper_limit using row variable and can be given as;

Column Level Repetitive Action				Expected Output
row	*loop-condition(lower_limit <=col<= upper_limit)*			
	lower_limit	upper_limit		
	3	4	5	
1	totalRows +1 - row	2* totalRows - row	2* totalRows - row -1 +1	5 6 7 8 9
2	totalRows +1 - row	2* totalRows - row - 1	2* totalRows - row -2+1	4 5 6 7
3	totalRows +1 - row	2* totalRows - row - 2	2* totalRows - row - 3+1	3 4 5
4	totalRows +1 - row	2* totalRows - row - 3	2* totalRows - row – 4+1	2 3
5	totalRows +1 - row	2* totalRows - row - 4	2* totalRows - row – 5+1	1

Again, we can use row variable in the upper_limit to get rid of hard-coded numeric values.

Column Level Repetitive Action				Expected Output
row	*loop-condition(lower_limit <=col<= upper_limit)*			
	lower_limit	upper_limit		
	3	6	7	
1	totalRows +1 - row	2* totalRows - row -row +1	2* (totalRows – row) + 1	5 6 7 8 9
2	totalRows +1 - row	2* totalRows - row -row +1	2* (totalRows – row) + 1	4 5 6 7
3	totalRows +1 - row	2* totalRows - row -row +1	2* (totalRows – row) + 1	3 4 5
4	totalRows +1 - row	2* totalRows - row -row +1	2* (totalRows – row) + 1	2 3
5	totalRows +1 - row	2* totalRows - row -row +1	2* (totalRows – row) + 1	1

The lower_limit and upper_limit expressions both have +1.
Moreover, If we run outer loop through [1 <=row<= totalRows] or [0 <=row< totalRows] ,in either case it is the same.
By deducting 1 from row , lower_limit and upper_limit expressions will make it simpler.

Column Level Repetitive Action			Expected Output
row	loop -condition(lower_limit <=col< upper_limit)		
	lower_limit expression	upper_limit expression	
1 -1	totalRows +1 – row - 1	2* (totalRows – row) + 1 - 1	5 6 7 8 9
2 -1	totalRows +1 – row - 1	2* (totalRows – row) + 1 - 1	4 5 6 7
3 -1	totalRows +1 – row - 1	2* (totalRows – row) + 1- 1	3 4 5
4 -1	totalRows +1 – row - 1	2* (totalRows – row) + 1- 1	2 3
5-1	totalRows +1 – row - 1	2* (totalRows – row) + 1- 1	1

Column Level Repetitive Action					Expected Output
row	*loop -condition(lower_limit <=col< upper_limit)*				
	lower_limit		upper_limit		
	expression		expression		
0	totalRows – row	totalRows – row	2* (totalRows – row)	2* (totalRows – row)	5 6 7 8 9
1	totalRows – row		2* (totalRows – row)		4 5 6 7
2	totalRows – row		2* (totalRows – row)		3 4 5
3	totalRows – row		2* (totalRows – row)		2 3
4	totalRows – row		2* (totalRows – row)		1

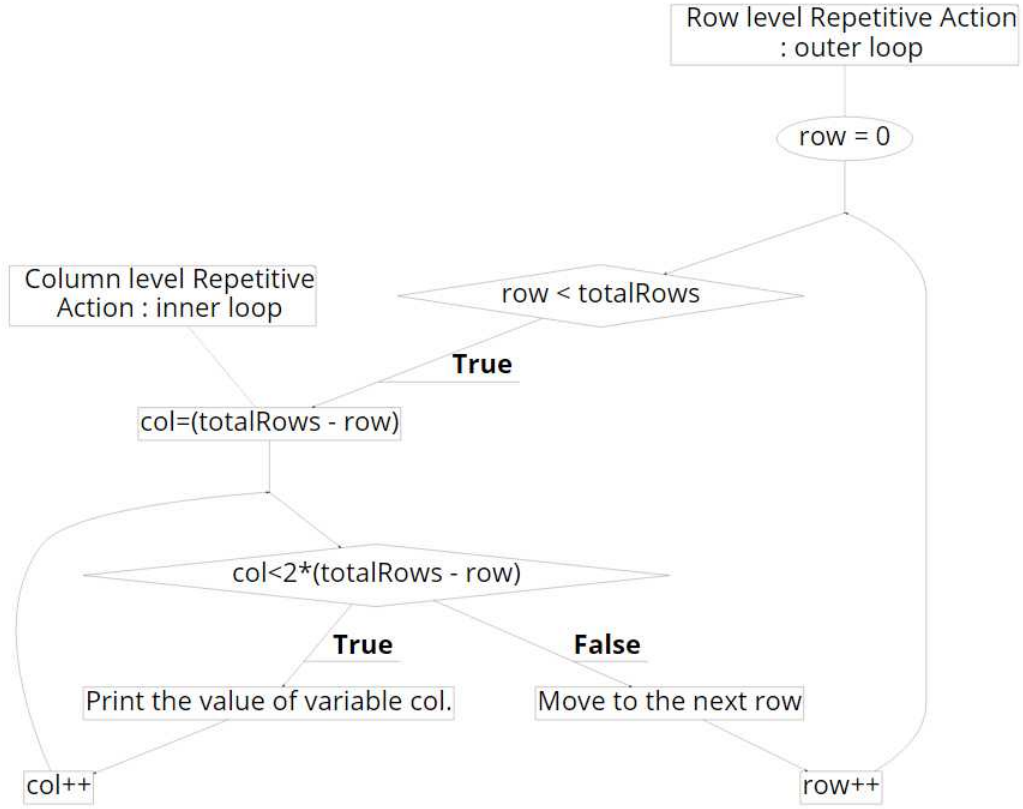
So our outer for-loop condition becomes ;

loop-condition	`lower_limit <=row< upper_limit`
Mathematical expression	`0 <= row < totalRows`
Java syntax	`for( row = 0; row < totalRows; row++)`

So our inner for-loop condition becomes ;

loop-condition	`lower_limit <=col< upper_limit`
Mathematical expression	`(totalRows - row) <=col < 2* (totalRows - row)`
Java syntax	`for(col=(totalRows-row);col< 2*(totalRows - row);col++ )`

The flow-chart can be given as follows.



NumberPatternCCC2.java

```java
public class NumberPatternCCC2 {

	public static void main(String[] args) {

	   int totalRows = 5;  //number of rows to display
	   String numStr;

	   // Row level Repetitive Action :
	   for( int row = 0; row < totalRows; row++) {

	      // Column level Repetitive Action :
	      // 1)Move cursor in the same row. 2)print character
	      for( int col=(totalRows - row);col<2*(totalRows - row); col++ ) {
	            numStr = col  + " ";
	            System.out.print( numStr );
	      }// inner loop

	      // Move cursor to the next row
	      System.out.println();

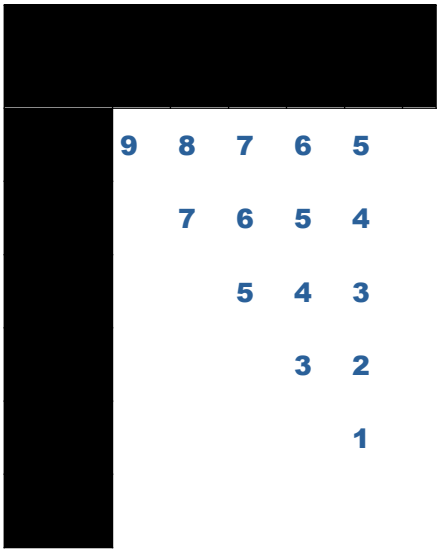
	   }// outer loop
	}

}
```

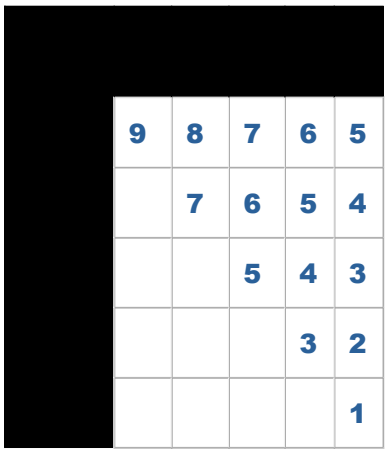
Output

```
5 6 7 8 9
4 5 6 7
3 4 5
2 3
1
```

NumberPattern DDD



Our programming job is to print this particular logic-based pattern on a part of the computer screen. **Our approach will be assumed that the computer screen is a combination of rows and columns virtually dividing it into smaller boxes.** The smallest box is treated as a resident of just a single character only. The address in each smallest box on the display screen will logically become a unique **(row, column)** numeric combination. With this assumption computer screen will simply be a unique combination of (row, column) numeric value paired boxes which are uniformly spread all across the screen.



	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	9	8	7	6	5
Row=2		7	6	5	4
Row=3			5	4	3
Row=4				3	2
Row=5					1

Data Representation

Although it is logically, we can exactly locate the position of each printable character on the display screen by knowing it's uniquely identifiable **(row, column)** numeric paired-values. *For example;* **(2, 3)** means printing numeric character at the assumed screen location **2nd row and 3rd column.**

(Row, Col) value pairs representation

	Col=1	Col=2	Col=3	Col=4	Col=5
Row=1	1,1	1,2	1,3	1,4	1,5
Row=2		2,2	2,3	2,4	2,5
Row=3			3,3	3,4	3,5
Row=4				4,4	4,5
Row=5					5,5

The given set of row and col value pairs tries to mimic the location of each printable character of a given pattern on the device's display screen. The set of all value pairs (row = value, col = value) represent this particular pattern.

Analysis of output

In this case we need to take care of printing two different types of characters at a particular column in a particular row:

1) Space character

2) Numeric character

row	5				
Col	1	2	3	4	5
Output	9	8	7	6	5

For row =1, 0 <=spaces<=0, 9>=num>=5

row	4				
Col	1	2	3	4	5
Output	space	7	6	5	4

For row =2, 1 <=spaces<=1, 9>=num>=5

row	3				
Col	1	2	3	4	5
Output	space	space	5	4	3

For row =3, 1 <=spaces<=2, 5>=num>=3

row	2				
Col	1	2	3	4	5
Output	space	space	space	3	2

For row =4, 1 <=spaces<=3, 3>=num>=2

row	1				
Col	1	2	3	4	5
Output	space	space	space	space	1

For row =5, 1 <=spaces<=4, 1>=num>=1

Observations

1) There is repetition at row level vertically, i.e., Row = 1 to 5.
2) There is repetition at col level horizontally. , i.e., Col = 1 to 5.
   This gets divided into two parts. One for repetitively printing spaces and remaining column for printing numeric characters.

For Loop	Lower limit	Upper limit
Outer loop(decrementing)	**row=1**	**Row>=5**
Inner Loop - spaces	**col=1**	**col<= ?(Unkown)**
Inner Loop – Number (decrementing)	**num = row**	**num =?(Unkown)**

COLUMN LEVEL REPETITIVE ACTION

Row level Repetitive Action						Expected Output
row	Column level Repetitive Action					
	lower_limit <=col<= upper_limit		upper_limit >=num >= lower_limit			
	lower_limit	upper_limit	upper_limit	lower_limit		
row = 5	1	0	9	5	row	9 8 7 6 5
row = 4	1	1	7	4	row	7 6 5 4
row = 3	1	2	5	3	row	5 4 3
row = 2	1	3	3	2	row	3 2
row = 1	1	4	1	1	row	1

The lower_limit for spaces & num variable has the same numeric value expression for all the rows. It's consistent for all the rows. Likewise, we need to come out with such a consistent expression for upper_limit for which it should suit any number of rows.
The other remaining variables are row & totalRows=5. Let's try to generalize using these variables.

Column level Repetitive Action						
row	lower_limit <=col<= upper_limit				upper_limit >=num >= lower_limit	
	lower_limit	upper_limit			upper_limit	lower_limit
row = 5	1	0	5-5	totalRows - row	9	row
row = 4		1	5-4	totalRows - row	7	
row = 3		2	5-3	totalRows - row	5	
row = 2		3	5-2	totalRows - row	3	
row = 1		4	5-1	totalRows - row	1	

The uppler_limit of row variable related inner for-loop is generalized, consistent expression on all the rows.The uppler_limit of num variable related inner for-loop is not generalized as not having consistent numeric or logical expression on all the rows.

Column level Repetitive Action									
row	lower_limit <=col<= upper_limit				upper_limit >=num >= lower_limit				
	lower_limit	upper_limit			upper_limit			lower_limit	
row = 5	1	0	totalRows - row	totalRows - row	9	10-1	2*5-1	row	
row = 4		1	totalRows - row		7	8-1	2*4-1		
row = 3		2	totalRows - row		5	6-1	2*3-1		
row = 2		3	totalRows - row		3	4-1	2*2-1		
row = 1		4	totalRows - row		1	2-1	2*1-1		

Column level Repetitive Action									
row	lower_limit <=col<= upper_limit		upper_limit >=num >= lower_limit						
	lower_limit	upper_limit	upper_limit						lower_limit
row = 5	1	totalRows - row	9	10-1	2*	5	-1	2*row -1	row
row = 4			7	8-1	2*	4	-1	2*row -1	
row = 3			5	6-1	2*	3	-1	2*row -1	
row = 2			3	4-1	2*	2	-1	2*row -1	
row = 1			1	2-1	2*	1	-1	2*row -1	

row	Column level Repetitive Action							
	lower_limit <=col<= upper_limit		upper_limit >=num >= lower_limit					
	lower_limit	upper_limit	upper_limit					lower_limit
row = 5	1	totalRows - row	2*	5	-1	2*row -1	2*row - 1	row
row = 4			2*	4	-1	2*row -1		
row = 3			2*	3	-1	2*row -1		
row = 2			2*	2	-1	2*row -1		
row = 1			2*	1	-1	2*row -1		

So the upper_limit of col & num variables are generalized. The lower_limit & upper_limit are consistent for all the rows. It's now generalized for any number of rows. So it has two repetition process one for printing spaces and second for printing numeric values. It means require two inner for-loops in order to print characters at the column level and one outer loop for iterating rows.

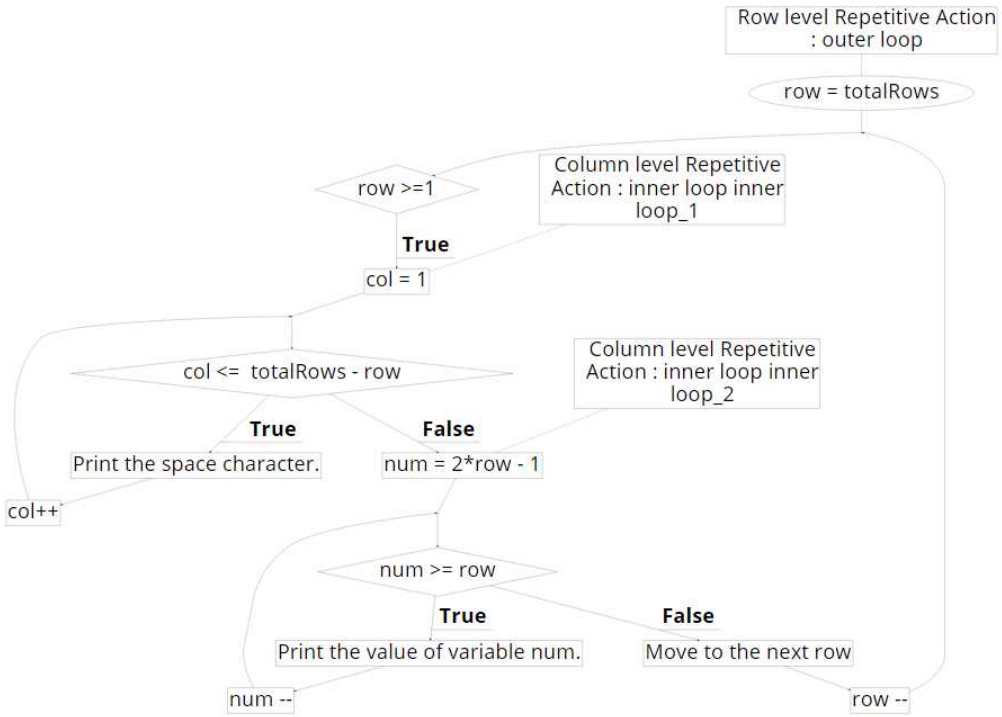
So our inner for-loop condition for printing spaces becomes;

loop-condition	`lower_limit <=col<= upper_limit`
Mathematical expression	`1 <= col <= (totalRows – row)`
Java syntax	`for( col = 1; col <= ( totalRows - row ); col++)`

So our inner for-loop condition for printing numbers becomes;

loop-condition	`upper_limit >=num>= lower_limit`
Mathematical expression	`(2*row – 1) >= num>= row`
Java syntax	`for( num = 2*row - 1 ; num >= row ; num -- )`

We can summarize all the related information in a flow-chart as given following.



Let's execute the following program.

NumberPatternDDD1.java

```
public class NumberPatternDDD1 {

	public static void main(String[] args) {

		int totalRows = 5;  //number of rows to display
		int num;
		String numStr;
		String spaces = "  ";   // 2 spaces

		// Row level Repetitive Action :
		for( int row = totalRows ; row >=1  ; row -- ) {

			// Column level Repetitive Action :
			// 1)Move cursor in the same row.
			// 2)print space character to before
			for( int col = 1; col <= ( totalRows - row ); col++ ) {
				System.out.print( spaces );
			}// inner loop_1

			// num value to be printed
			for( num = 2*row - 1 ; num >= row ; num -- ){
				numStr = num  + " ";
				System.out.print( numStr );
			}// inner loop_2

			// Move cursor to the next row
			System.out.println();
		}// outer loop
	}

}
```

Output

```
9 8 7 6 5
  7 6 5 4
    5 4 3
      3 2
        1
```

Alternative solution

Since the column values get clear split between printing spaces and numbers. We can try to reproduce the same result using if-else condition construct. Using the **loop-condition** for spaces as an if-condition and printing number in **else-condition.**

NumberPatternDDD2.java

```
public class NumberPatternDDD2 {

	public static void main(String[] args) {

		int totalRows = 5; // number of rows to display
		int num;
		String numStr;
		String spaces = "  "; // 2 spaces

		// Row level Repetitive Action :
		for ( int row = totalRows; row >= 1; row-- ) {

			num = 2 * row - 1;

			// Column level Repetitive Action :
			for (int col = 1; col <= (totalRows); col++) {

				// when spaces are to be printed
				if (col <= (totalRows - row)) {
					System.out.print(spaces);
				} else {
					numStr = num + " ";
					System.out.print(numStr);
					num--;
				}
			} // inner loop

			// Move cursor to the next row
			System.out.println();

		} // outer loop
	}

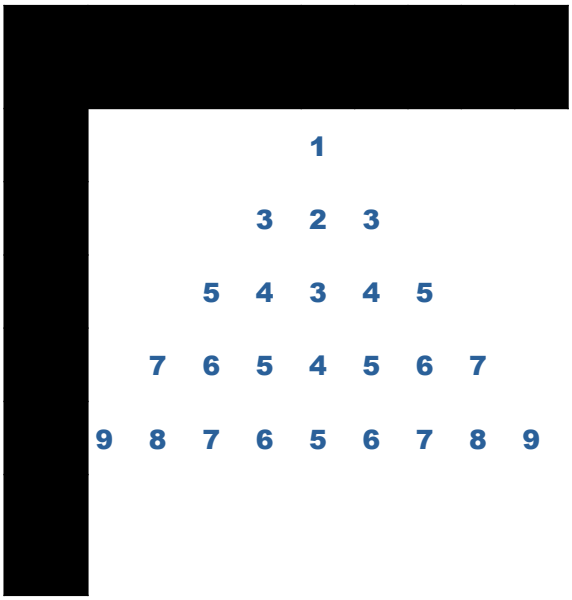
}
```

Output

```
9 8 7 6 5
  7 6 5 4
    5 4 3
      3 2
        1
```

NumberPatternEEE

Our programming job is to print this particular logic-based pattern on a part of the computer screen. **Our approach will be assumed that the computer screen is a combination of rows and columns virtually dividing it into smaller boxes.** The smallest box is treated as a resident of just a single character only. The address in each smallest box on the display screen will logically become a unique **(row, column)** numeric combination. With this assumption computer screen will simply be a unique combination of (row, column) numeric value paired boxes which are uniformly spread all across the screen.



	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1					1				
Row=2				3	2	3			
Row=3			5	4	3	4	5		
Row=4		7	6	5	4	5	6	7	
Row=5	9	8	7	6	5	6	7	8	9

Data Representation

Although it is logically, we can exactly locate the position of each printable character on the display screen by knowing it's uniquely identifiable **(row, column)** numeric paired-values. *For example;* **(4, 3)** means printing numeric character at the assumed screen location **4th row and 3rd column.**

(Row, Col) value pairs representation

	Col=1	Col=2	Col=3	Col=4	Col=5	Col=6	Col=7	Col=8	Col=9
Row=1					1,5				
Row=2				2,4	2,5	2,6			
Row=3			3,3	3,4	3,5	3,6	3,7		
Row=4		4,2	4,3	4,4	4,5	4,6	4,7	4,8	
Row=5	5,1	5,2	5,3	5,4	5,5	5,6	5,7	5,8	5,9

Analysis of output

row	1								
col	1	2	3	4	5	6	7	8	9
output					1				

For row = 1, 5 <=Col <= 5

row	2								
col	1	2	3	4	5	6	7	8	9
output				3	2	3			

For row = 2, 4 <=Col <= 6

row	3								
col	1	2	3	4	5	6	7	8	9
output			5	4	3	4	5		

For row = 3 , 3 <=Col <= 7,

row	4								
col	1	2	3	4	5	6	7	8	9
output		7	6	5	4	5	6	7	

For row = 4, 2 <=Col <= 8,

row	5								
col	1	2	3	4	5	6	7	8	9
output	9	8	7	6	5	6	7	8	9

For row = 5, 1 <=Col <= 9

COLUMN LEVEL REPETITIVE ACTION

Row level Repetitive Action			Expected Output
row	Column level Repetitive Action		
	lower_limit <= col <= upper_limit		
	lower_limit	upper_limit	
1	5	5	1
2	4	6	3 2 3
3	3	7	5 4 3 4 5
4	2	8	7 6 5 4 5 6 7
5	1	9	9 8 7 6 5 6 7 8 9

The lower_limit & upper_limit ,both are not consistent for all the rows. We need to come out with consistent expressions for both upper_limit and lower_limit which should suit for n number of rows. The other remaining variables are row, totalRows=5. Let's try to generalize using these variables.

Column level Repetitive Action						
row	lower_limit <= col <= upper_limit					
	lower_limit			upper_limit		
	default	1	2	default	1	
1	5	totalRows	totalRows +1 - 1	5	totalRows	totalRows +1 - 1
2	4	totalRows -1	totalRows +1 - 2	6	totalRows + 1	totalRows + 2 - 1
3	3	totalRows -2	totalRows +1 - 3	7	totalRows + 2	totalRows + 3 - 1
4	2	totalRows -3	totalRows +1 - 4	8	totalRows + 3	totalRows + 4 - 1
5	1	totalRows -4	totalRows +1 - 5	9	totalRows + 4	totalRows + 5 - 1

Column level Repetitive Action				
row	lower_limit <= col <= upper_limit			
	lower_limit		upper_limit	
	default	3	default	3
1	5	totalRows +1 - row	5	totalRows + row - 1
2	4	totalRows +1 - row	6	totalRows + row - 1
3	3	totalRows +1 - row	7	totalRows + row - 1
4	2	totalRows +1 - row	8	totalRows + row - 1
5	1	totalRows +1 - row	9	totalRows + row - 1

The lower_limit & upper_limit are consistent for all the rows. It's now generalized for any number of rows, which gives the exact position where to print numeric value. `So our If-condition becomes;

if-condition	`lower_limit <=col<= upper_limit`
Mathematical expression	`(totalRows +1 - row) <= col <= (totalRows + row - 1)`
Java syntax	`if( col >= ( (totalRows - row) + 1 ) &&` `col <= ( ( totalRows + row ) - 1 ) )`

When the if-condition satisfied, it will print the numeric value at that position at a particular row or else will print the space character. We can observe two things from the given output which can be given as;

a) The values at each row starts with the (2*row -1) value and goes on decreases till row value and further increases till it reaches the (2*row -1) value. The value basically oscillating like a pendulum of a wall clock. Let's choose a variable **num** which represents a (2*row -1) value, every time a new row starts. The value of variable num then oscillates to give output in a single row. It can be shown by following table.

Table for **num** variable starting at (2*row -1) value.

row	num	col<totalRows	col=totalRows	col>totalRows
1	1		1	
2	2	3	2	**3**
3	3	5 4	3	**4 5**
4	4	7 6 5	4	**5 6 7**
5	5	9 8 7 6	5	6 7 8 9

col<totalRows	col>=totalRows

if(col<totalRows){
num =(2*row -1) value gets printed.
num value get decreased by 1.
}
	if(col>=totalRows){
	num value gets printed.
	num value get increased by 1 till (2*row -1)
	}

b) The column counts are more than the **totalRows** count other than the first row. The **totalRows** value is 5 .The column count is different on each row. The upper_limit of inner ***for–loop*** need to be generalized.

loop-condition	`lower_limit <=col<= upper_limit`
Mathematical expression	`1 <= col <= 9`
	`1 <=col <= totalRows + 4`
	`1 <=col <= totalRows + (totalRows – 1)`
	`1<=col<=2* totalRows - 1`
Java syntax	`for( col = 1 ; col <= ( 2* totalRows - 1 ) ; col++)`

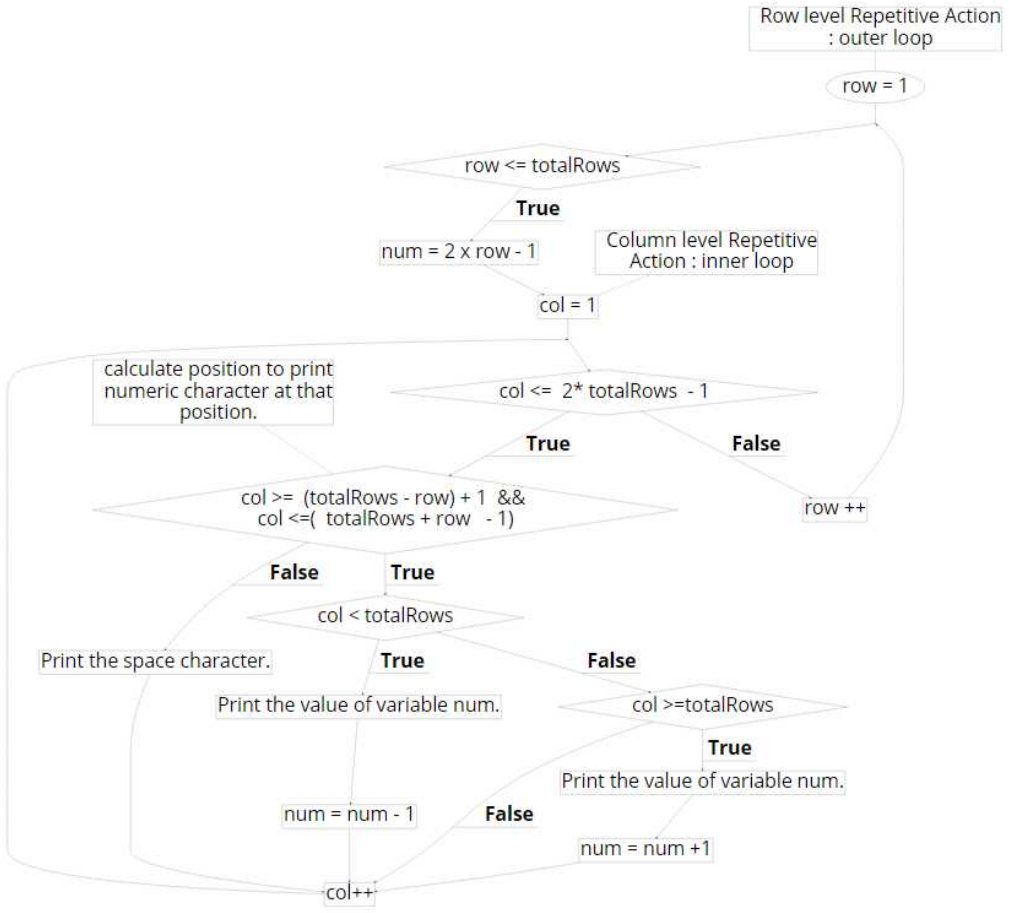
Let's verify the upper limit for col variables or total columns which will used to display each row

for totalRows = 5 ,the col <= 9 equals (2*5 – 1)

for totalRows = 4 ,the col <= 7 equals (2*4 – 1)

So this generalization to calculate total columns for a given row value is satisfactory.

The flow-chart can be given as follows.



Let write the program based on our findings and flow-chart.

NumberPatternEEE.java

```
public class NumberPatternEEE {

	public static void main( String[] args ) {

		int totalRows = 5;  //number of rows to display
		int num;
		String numStr;
		String spaces = "  ";   // 2 spaces

		// Row level Repetitive Action :
		for( int row = 1 ; row <= totalRows ; row ++ ) {
		num = 2*row - 1;

			// Column level Repetitive Action :
			for( int col = 1 ; col <= ( 2* totalRows  - 1 ) ; col++ ) {

				// calculates position to print numeric character
				if( col >= ( (totalRows - row) + 1 ) &&
				    col <= (  (  totalRows + row )  - 1 ) ) {
				  // print a number
				  if(col < totalRows ) {
					numStr = num  + " ";
					System.out.print( numStr );
					num = num - 1;
				  }else if(col >=totalRows) {
					numStr = num  + " ";
					System.out.print( numStr );
					num = num +1;
				  }

				}else {
					System.out.print(spaces);// print spaces
				}
			}// inner loop

			// Move to next row
			System.out.println();

		}// outer loop

	}

}
```

Output

```
        1
      3 2 3
    5 4 3 4 5
  7 6 5 4 5 6 7
9 8 7 6 5 6 7 8 9
```

ProblemGBBBB2: **Modify program NumberPatternBBB2.java so that it should able to show the following output.**

Output

```
        1
      2
    3
  4
5
```

ProblemGBDD1: **Modify program NumberPatternDDD1.java so that it should able to show the following output.**

Output

```
9
  7
    5
      3
        1
```

ProblemGBEEE: **Modify program NumberPatternEEE.java so that it should able to show the following output.**

Output

```
        1
      3   3
    5       5
  7           7
9               9
```

ProblemGBEEE1: **Modify program PatternEEE.java so that it should show the following output.**

Output

```
        1
      3 2 3
    5   3   5
  7     4     7
9       5       9
```

ProblemGB. *W*rite programs to generate the following outputs.

Output

```
1>=1

2>=1 2>=2

3>=1 3>=2 3>=3

4>=1 4>=2 4>=3 4>=4

5>=1 5>=2 5>=3 5>=4 5>=5
```

Output

```
  1<2 1<3 1<4 1<5 1<6 1<7 1<8 1<9 1<10

    2<3 2<4 2<5 2<6 2<7 2<8 2<9 2<10

      3<4 3<5 3<6 3<7 3<8 3<9 3<10

        4<5 4<6 4<7 4<8 4<9 4<10

          5<6 5<7 5<8 5<9 5<10

            6<7 6<8 6<9 6<10

              7<8 7<9 7<10

                8<9 8<10

                  9<10
```

Output

```
<<~>><<~>><<~>><<~>><<~>><<~>><<~>><<~>><<~>><<~>>

  <<~>><<~>><<~>><<~>><<~>><<~>><<~>><<~>><<~>>

    <<~>><<~>><<~>><<~>><<~>><<~>><<~>><<~>>

      <<~>><<~>><<~>><<~>><<~>><<~>><<~>>

        <<~>><<~>><<~>><<~>><<~>><<~>>
```

Output

```
2

3 4

4 5 6

5 6 7 8

6 7 8 9 10

7 8 9 10 11 12
```

Output

```
1

2 1

3 2 1

4 3 2 1

5 4 3 2 1

6 5 4 3 2 1

7 6 5 4 3 2 1
```

Output

```
0

1 0

2 1 0

3 2 1 0

4 3 2 1 0

5 4 3 2 1 0
```

Strings

It is common to modify or manipulate text at runtime like displaying text or part of text in some way, dynamically creating web-page url addresses or file paths, extracting specific information from a text like many develops web-scrapping tools specially to extract text from web-pages. Every programming language provides in-built string libraries for string manipulation. We will experiment with some of commonly used in-built string methods or functions.

String concatenation

We can output strings by joining one another using in-built **concat( )** method or using concatenation operator with the + (plus).We had already use concatenation operator (+) many times in our early exercises.

String name = "Steve" + " " + "Jobs";

String name1 = "Steve";

String name2 = "Jobs";

name1.concat(" ");

name1.concat(name2);

It's a wrong assumption that both variables **name** and **name1** will give the same output.

It's one of the most common mistake which many of beginner programmers does .

StringConcat.java

```
public class StringConcat {

	public static void main( String[] args ) {

		String name  = "Steve" + " " + "Jobs";
		String name1 = "Steve";
		String name2 = "Jobs";

		System.out.println( name  );
		System.out.println( "X -> Common mistake while using concat()" );
		// no assignment operator is being used to store the final result
		name1.concat(" ");
		name1.concat( name2 );
		System.out.println( name1  );//common mistake
		System.out.println(  "Correct use of concat() " );
		// don't forget to use assignment operator just like
		// we had used in case of numeric calculations
		name1 = name1.concat(" ");
		name1 = name1.concat( name2 );
		System.out.println( name1 );

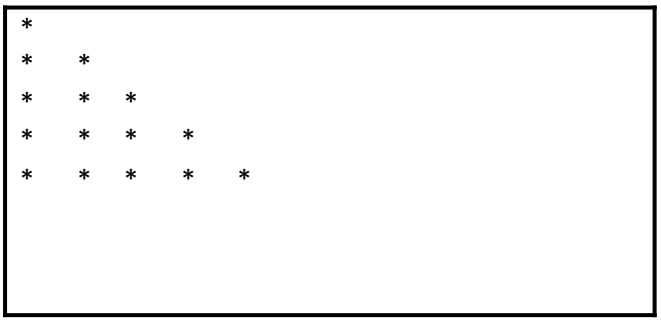
	}
}
```

Output

```
Steve Jobs
X -> Common mistake while using concat()
Steve
Correct use of concat()
Steve Jobs
```

Patterns by Single Looping

We can output some of the patterns what we have earlier achieved using nested loop (one for-loop within another for-loop) can be done by single ***for-loop*** using **concat()** method.



Pattern A1

StringPatternA1.java

```
public class StringPatternA1 {

	public static void main( String[] args ) {

		int totalRows = 5;  // number of rows to display
		String line = ""; // initialize with blank string.

		for( int row = 1 ; row <= totalRows ; row ++ ) {

			// Column level Repetitive Action :
			// a space is added for clarity in the output
			line = line.concat("*" + " " );

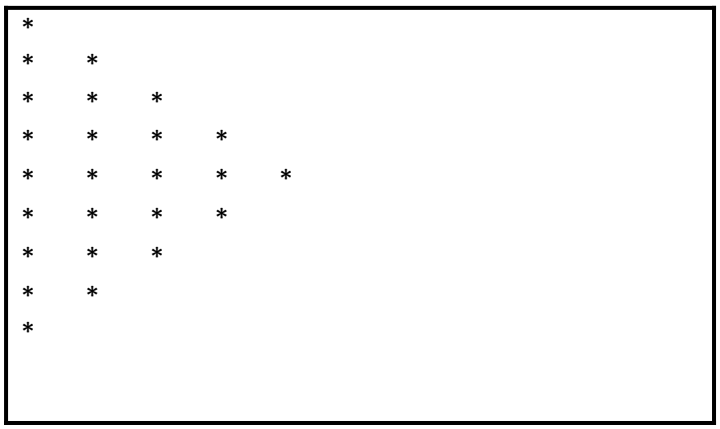
			System.out.print(line);

			// Move cursor to the next row
			System.out.println();
		}
	}

}
```

Output

```
*
* *
* * *
* * * *
* * * * *
```



Pattern G1

StringPatternG1.java

```
public class StringPatternG1 {

	public static void main( String[] args ) {

		int totalRows = 5;  // number of rows to display
		int count = 0;     // counter to increment by 1.
		String line = ""; // initialized with blank string.

		for( int row = 1; row <= ( 2 * totalRows - 1 ); row++ ) {
			// Column level Repetitive Action :
			if ( row <= totalRows ) {
				line = line.concat("*");
			} else {
			// increment count only when row > totalRows
			 count = count + 1;
			 line = line.substring( 0 , totalRows - count );
			}
			System.out.print(line);

			// Move to next row
			System.out.println();
		}
	}

}
```

Output

```
*
**
***
****
*****
****
***
**
*
```

ProblemSPG1: In the program `StringPatternG1.java`, the star "*" characters are printed quite closed to each other. In order to get better output suppose an extra space is added as shown in the following program by the **arrow mark,** the output doesn't came as per our expectation.

What modifications in the following program should be applied so that we should able to get the expected output.

StringPatternG1.java

```
public class StringPatternG1 {

	public static void main( String[] args ) {

		int totalRows = 5;  // number of rows to display
		int count = 0;    // counter to increment by 1.
		String line = ""; // initialize with blank string.

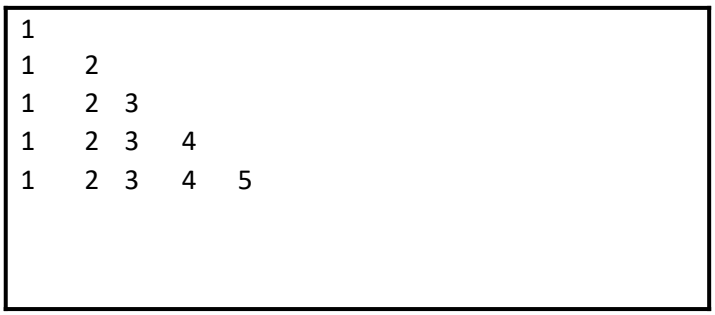
		for( int row = 1; row <= ( 2 * totalRows - 1 ); row++ ) {

			// Column level Repetitive Action :
			if ( row <= totalRows ) {
				// an extra space added here
			   line = line.concat("*" + " ");
			} else {
			// increment count only when row > totalRows
			 count = count + 1;
			 line = line.substring( 0 , totalRows - count );
			}
			System.out.print(line);

			// Move to next row
			System.out.println();
		}
	}

}
```

Output by this program	Expected Output
*	*
* *	* *
* * *	* * *
* * * *	* * * *
* * * * *	* * * * *
* *	* * * *
* *	* * *
*	* *
*	*



NumberPattern AA1

StringPatternAA1.java

```
public class StringPatternAA1 {

	public static void main( String[] args ) {

		int totalRows = 5; // number of rows to display
		String line = ""; // Initialize with blank string.

		for( int row = 1 ; row <= totalRows ; row ++ ) {

			// Column level Repetitive Action:
			line = line.concat( row + " " );
			System.out.print(line);

			// Move cursor to the next row
			System.out.println();
		}
	}
}
```

Output

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

```
1
1   2
1   2   3
1   2   3   4
1   2   3   4   5
1   2   3   4
1   2   3
1   2
1
```

NumberPatternGG1

StringPatternGG1.java

```
public class StringPatternGG1 {

	public static void main( String[] args ) {

		int totalRows = 5;  // number of rows to display
		int count = 0;      // counter to increment by 1.
		String line = ""; // initialize with blank string.

		for( int row = 1; row <= ( 2 * totalRows - 1 ); row++ ) {

			// Column level Repetitive Action :
			if ( row <= totalRows ) {
				// updating the string variable line using
				// concatenating operator +
				line = line + row;
			} else {
			// increment count only when row > totalRows
			 count = count + 1;
			 line = line.substring( 0 , totalRows - count );
			}
			// Prints the content of variable line and moves to the
			// next row. This way in the next turn,the printing of
			// variable line will happen at the next line.
			System.out.println(line);
		}

	}

}
```

Output

```
1
12
123
1234
12345
1234
123
12
1
```

Accessing each string's character

In order know the composition of string, we can use the in-built **charAt (index)** method. The "**index**" means the position of a character in a given string.

Let suppose "Program" is a string, then we can give it in a program as;

String str = "Program";

What shall we expect from **charAt (index)?** It can be given as follows:

charAt(index)	Character at index position
str.charAt(0)	P
str.charAt(1)	r
str.charAt(2)	o
str.charAt(3)	g
str.charAt(4)	r
str.charAt(5)	a
str.charAt(6)	m

Let's confirm our understanding using a given program.

StringCharAt.java

```
public class StringCharAt {

	public static void main(String[] args) {

		String str = "Program";
		int index ;

		for( index = 0 ; index < str.length() ; index++) {
		  String output = "str.charAt( "+ index + ") = "
		                  + str.charAt(index);
		  System.out.println(output);
		}
	}

}
```

Output

```
str.charAt( 0) = P
str.charAt( 1) = r
str.charAt( 2) = o
str.charAt( 3) = g
str.charAt( 4) = r
str.charAt( 5) = a
str.charAt( 6) = m
```

ZERO is a valid value in the **charAt (0)** method and points to the first position character in a given string.

Exercise: Write a program for the following staircase look alike picture.

```
P_|_P
r__|__r
o___|___o
g____|____g
r_____|_____r
a______|______a
m_______|_______m
m_______|_______m
i_________|_________i
n__________|__________n
g___________|___________g
```

Staircase.java

```
public class Staircase {

	public static void main( String[] args ) {
		String str  = "Programming";
		String spaces = "";
		for( int index = 0 ; index < str.length() ; index++) {
			spaces = spaces.concat("_");
			String print  = str.charAt(index) + spaces ;
			print = print + "|"+ spaces + str.charAt (index);
			System.out.println( print );
		}
	}
}
```

Problem A: Rewrite the program **Staircase.java** without using **concat**() method.
The first step will be to write the logic trace table with appropriate row & columns.
From the analysis of logic trace table, a programmer can be able to find out generalized way to get the solution to print series of "_"characters. A different character could also be taken like "*" ,"=" or "#" to get the count.
Problem B.If there is a variable national_id in a program and you assign a 5 digit length value, for example : 78901. It should output in digits made up of "#" characters.

```
 # # # #       # # # # #       # # # #       # # # #       #

       #       #       #       #     #       #     #       #

       #       # # # # #       # # # #       #     #       #

       #       #       #               #       #     #       #

       #       # # # # #               #       # # # #    # # # # #
```

Write a program to print any such 5-digit number.

Substrings

Choosing **start** and **end** as some positions in a given string in an increasing order.

To get a copy of the part of a given string based on the **start** and **end** positions in a given string is commonly referred as substring. One substring could be a part of another substring.

The method can be given as **substring (start, end)** or **substring (start).**If **end** position is not known, method will return copy of a substring till last from a given **start** position.

SubStringResults.java

```
public class SubStringResults {

	public static void main(String[] args) {

		String str = "Programming";
		int index ;
		String subStr = "";
		int LENGTH = str.length();
		System.out.println( "Length of the String  = " + LENGTH);
		System.out.println( "====================");

		for( index = 0 ; index <= LENGTH ; index++) {
			subStr = str.substring(0, index);
			System.out.println("str.substring( 0, " +  index + ") = "
					+ subStr);
		}

		index = LENGTH - 1;
		subStr = str.substring(5, index);
		System.out.println("str.substring( 5, " +  index + ") = "
				+ subStr);
		System.out.println( "====================");
		System.out.println("Please notice first and"
				 + " last substring() method results.");
	}

}
```

Output

```
Length of the String  = 11
====================
str.substring( 0, 0) =
str.substring( 0, 1) = P
str.substring( 0, 2) = Pr
str.substring( 0, 3) = Pro
str.substring( 0, 4) = Prog
str.substring( 0, 5) = Progr
str.substring( 0, 6) = Progra
str.substring( 0, 7) = Program
str.substring( 0, 8) = Programm
str.substring( 0, 9) = Programmi
str.substring( 0, 10) = Programmin
str.substring( 0, 11) = Programming
str.substring( 5, 10) = ammin
====================
Please notice first and last substring() method results.
```

Exercise: *Generate the following output using **substring()** method for a given string "Programming".*

```
P
Pr
Pro
Prog
Progr
Progra
Program
Programm
Programmi
Programmin
Programming
Programmin
Programmi
Programm
Program
Progra
Progr
Prog
Pro
Pr
P
```

PatternBySubString.java

```
public class PatternBySubString {

	public static void main(String[] args) {

		String str = "Programming";
		int index ;
		String subStr = "";
		int LENGTH = str.length();
		System.out.println( "Length of String  = " + LENGTH);
		System.out.println( "====================");
		for( index = 1 ; index <= LENGTH ; index++) {
			subStr = str.substring(0, index);
			System.out.println( subStr);

		}
		// using decrementing loop in a string
		for( index = LENGTH - 1; index >= 1 ; index -- ) {

			subStr = str.substring(0, index);

			System.out.println( subStr);

		}

	}

}
```

Problem1. Print the following output using string in-built functions.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1											P										
2										r	P	r									
3									o	r	P	r	o								
4								g	o	r	P	r	o	g							
5							r	g	o	r	P	r	o	g	r						
6						a	r	g	o	r	P	r	o	g	r	a					
7					m	a	r	g	o	r	P	r	o	g	r	a	m				
8				m	m	a	r	g	o	r	P	r	o	g	r	a	m	m			
9			i	m	m	a	r	g	o	r	P	r	o	g	r	a	m	m	i		
10		n	i	m	m	a	r	g	o	r	P	r	o	g	r	a	m	m	i	n	
11	g	n	i	m	m	a	r	g	o	r	P	r	o	g	r	a	m	m	i	n	g

Problem2. Print the following output using string in-built functions.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1											P										
2										r		r									
3									o				o								
4								g						g							
5							r								r						
6						a										a					
7					m												m				
8				m														m			
9			i																i		
10		n																		n	
11	g																				g
12		n																		n	
13			i																i		
14				m														m			
15					n												n				
16						a										a					
17							r								r						
18								g						g							
19									o				o								
20										r		r									
21											P										

Problem3. Please verify wavelength your logical thinking capabilities. Write a generalized program, so that for different size=1,2,3 inputs, the size of the wave should get doubled or triple. You can write separate program for each different type of wave.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
1	*	*	*	*			*	*	*	*			*	*	*	*						
2	*			*			*			*			*			*						
3	*			*			*			*			*			*						
4	*			*			*			*			*			*			*			
5				*			*			*			*			*			*			
6				*			*			*			*			*			*			
7				*	*	*	*			*	*	*	*			*	*	*	*			

SIZE = 1

1	*						*	*	*	*	*	*	*	*								
2	*						*							*								
3	*						*							*								
4	*						*							*								
5	*						*							*								
6	*						*							*								
7	*						*							*								
8	*						*							*								
9	*						*							*								
10	*						*							*								
11	*						*							*								
12	*						*							*								
13	*	*	*	*	*	*	*							*	*	*	*	*	*	*		

SIZE = 2

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
1				*												*						
2			*		*										*		*					
3		*				*								*				*				
4	*						*						*						*			
5								*				*								*		
6									*		*										*	
7										*												*

SIZE = 1

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
1							*						*						*			
2						*	*					*	*					*	*			
3					*		*				*		*				*		*			
4				*			*			*			*			*			*			*
5			*				*		*				*		*				*		*	
6		*					*	*					*	*					*	*		
7	*						*						*						*			

SIZE = 1

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
1	*						*						*									
2	*	*					*	*					*	*								
3	*		*				*		*				*		*							
4	*			*			*			*			*			*						
5	*				*		*				*		*				*					
6	*					*	*					*	*					*				
7	*						*						*						*			

SIZE = 1

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
1	1	1	1	1			2	2	2	2			3	3	3	3						
2	1			1			2			2			3			3						
3	1			1			2			2			3			3						
4	1			1			2			2			3			3			3			
5				1			1			2			2			3			3			
6				1			1			2			2			3			3			
7				1	1	1	1			2	2	2	2			3	3	3	3			

SIZE = 1

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
1	1	1	1	1			0	0	0	0			1	1	1	1						
2	1			1			0			0			1			1						
3	1			1			0			0			1			1						
4	1			1			0			0			1			1			1			
5				1			1			0			0			1			1			
6				1			1			0			0			1			1			
7				1	1	1	1			0	0	0	0			1	1	1	1			

SIZE = 1

Problem4. Write a program for each of the following different wave design.

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
1		*	*					*	*					*	*							
2	*			*			*			*			*			*						
3	*			*			*			*			*			*						
4	*			*			*			*			*			*			*			
5				*			*			*			*			*			*			
6				*			*			*			*			*			*			
7					*	*					*	*					*	*				

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
1		<	>					<	>					<	>							
2	<			>			<			>			<			>						
3	<			>			<			>			<			>						
4	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-			
5				<			>			<			>			<			>			
6				<			>			<			>			<			>			
7					<	>					<	>					<	>				

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
1		<	>					<	>					<	>							
2	<			>			<			>			<			>						
3	<			>			<			>			<			>						
4	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-			
5				[			]			[			]			[			]			
6				[			]			[			]			[			]			
7					[	]					[	]					[	]				

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
1		<	>					[	]					<	>							
2	<			>			[			]			<			>						
3	<			>			[			]			<			>						
4	<			>			[			]			<			>						
5				<			>			[			]			<			>			
6				<			>			[			]			<			>			
7					<	>					[	]					<	>				

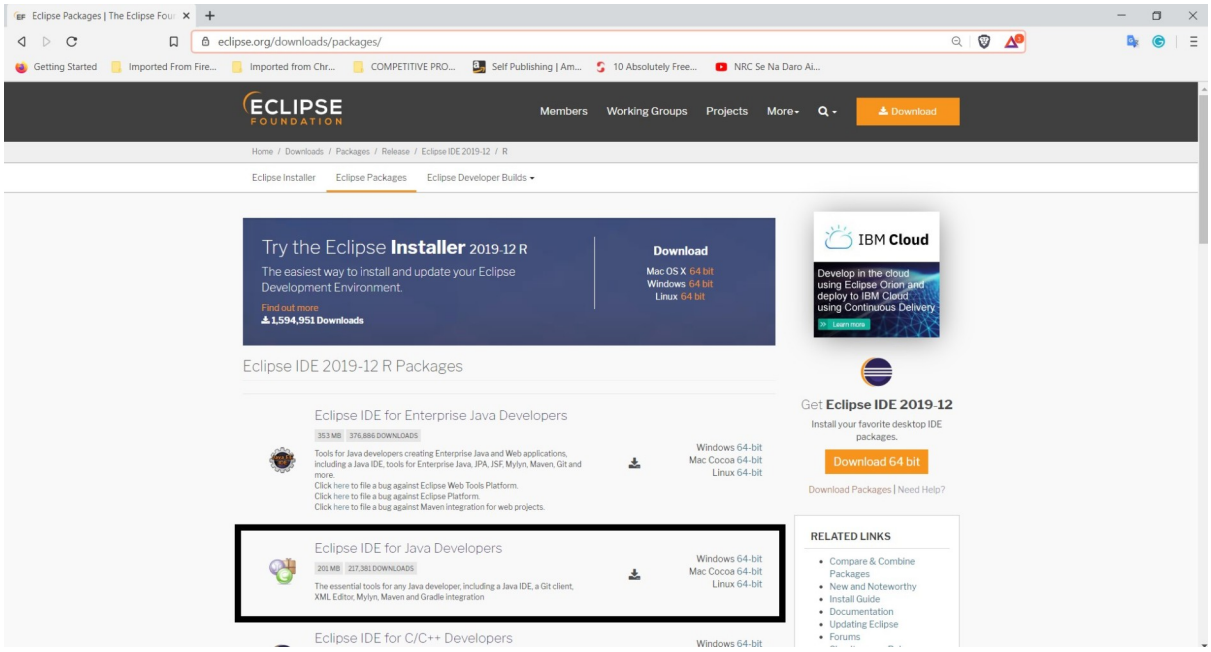
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
1		3	4					15	16					27	28							
2	2			5			14			17			26			29						
3	1			6			13			18			25			30						
4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
5				7			12			19			24			31			36			
6				8			11			20			23			32			35			
7					9	10					21	22					33	34				

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22
1		1	2					5	6					9	10							
2	1			2			5			6			9			10						
3	1			2			5			6			9			10						
4	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0			
5				3			4			7			8			11			12			
6				3			4			7			8			11			12			
7					3	4					7	8					11	12				

Appendix-A: Installation of eclipse IDE and project set-up

We need to follow certain steps to set-up an eclipse IDE to create java projects and to run our programs. The steps are as follows:

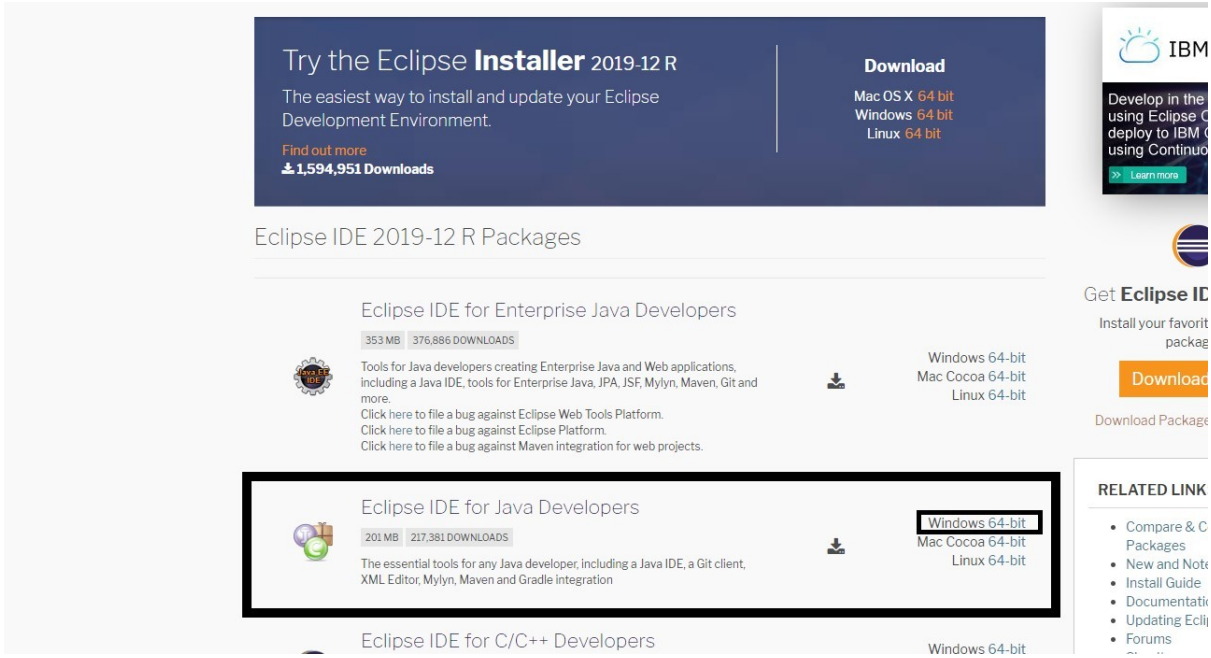
1.Visit the link: http://www.eclipse.org/downloads/eclipse-packages/



2. Please choose **Eclipse IDE for Java Developers** from the list of IDEs.

It is shown by highlighting rectangular box.

3. Choose the eclipse Installer 64 bit depends upon your OS and download it.

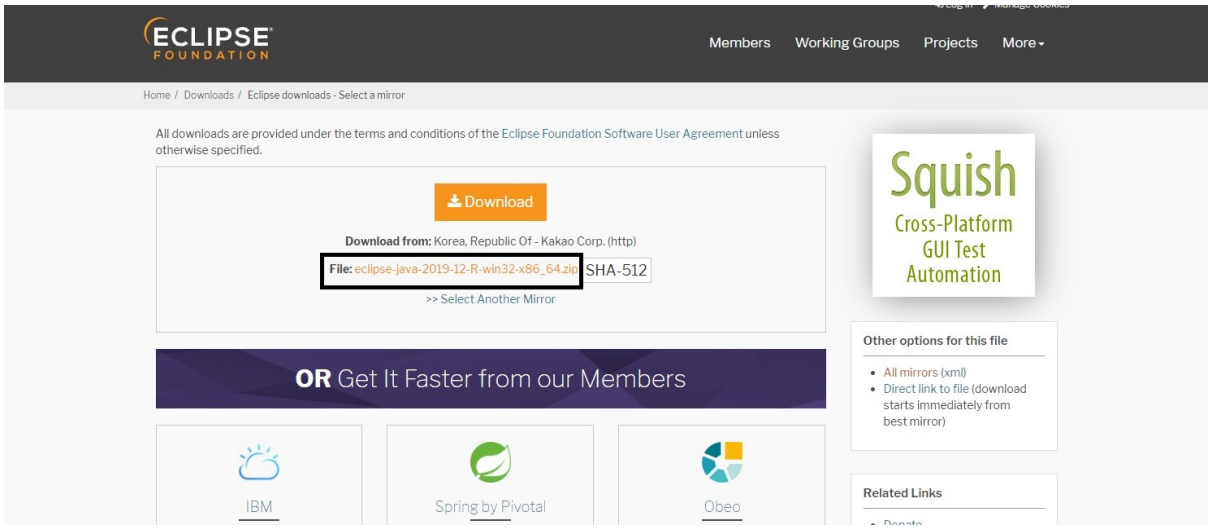


For installation on Windows, please click "**Windows 64-bit**" link.

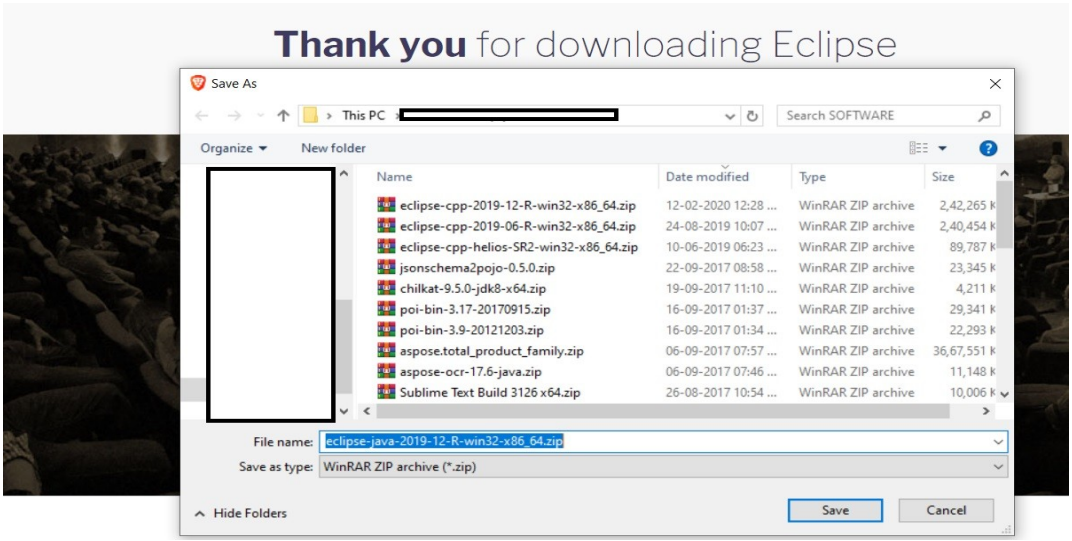
On clicking the "**Windows 64-bit**" link, it will next bring you to the download page.

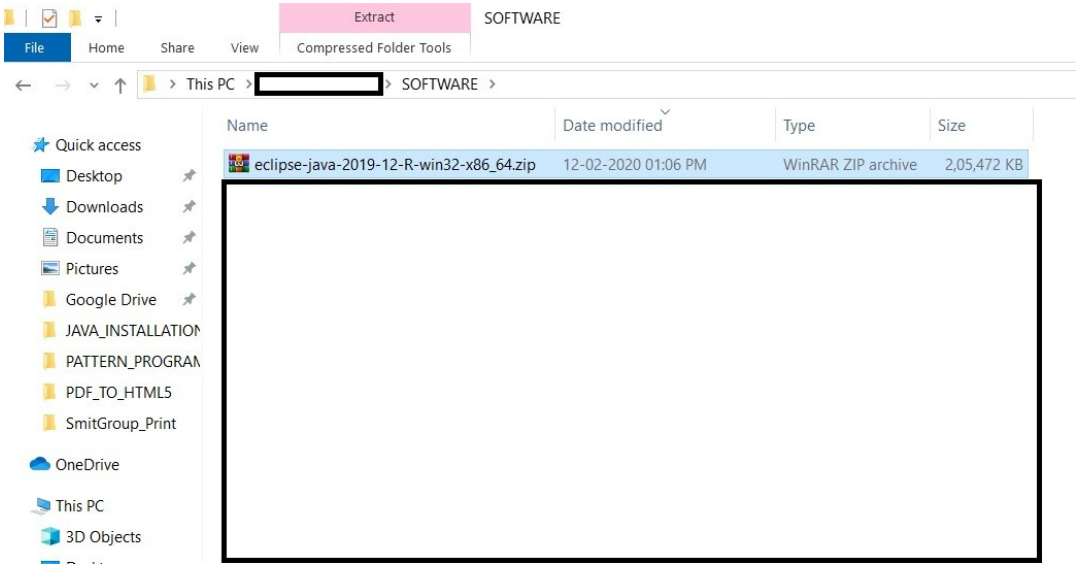
4. Please check the file name starts with the label **File: eclipse-Java-2019**.

Please click the "**Download**" button.

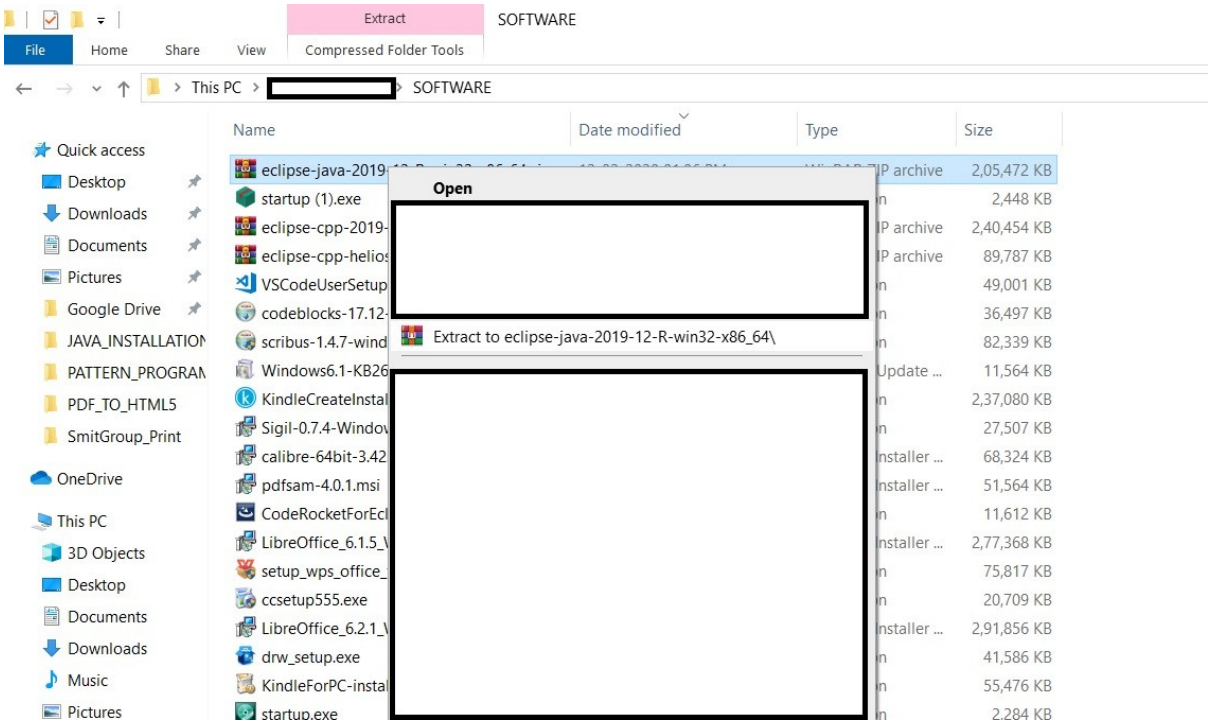


5. Please save it in your local.



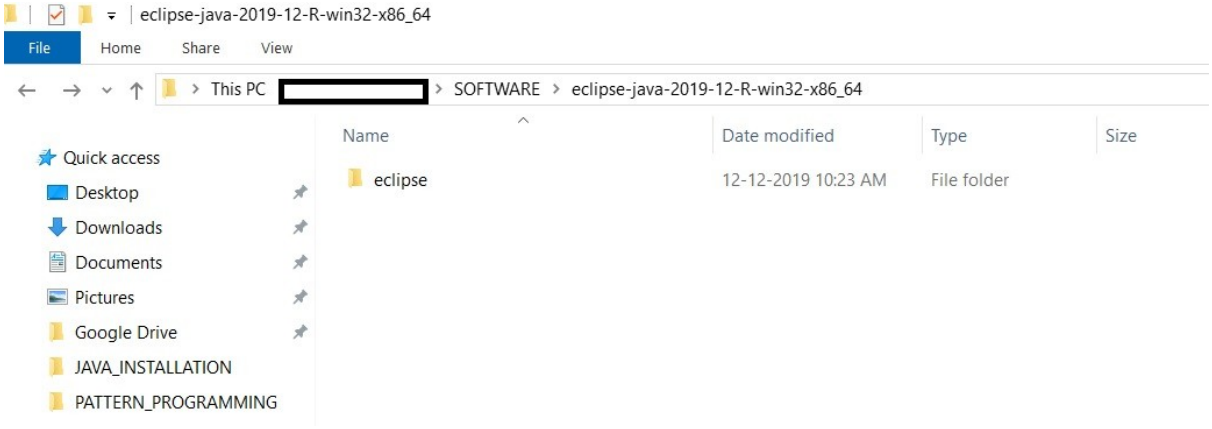


6. Please extract the saved file.

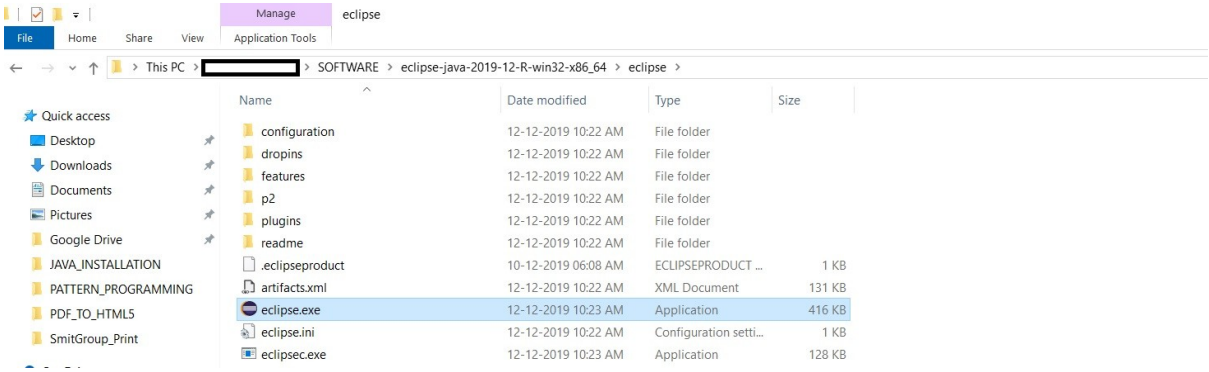


After extracting the file we can observe the folder **eclipse-java-2019-12-R-win32-x86_64.**

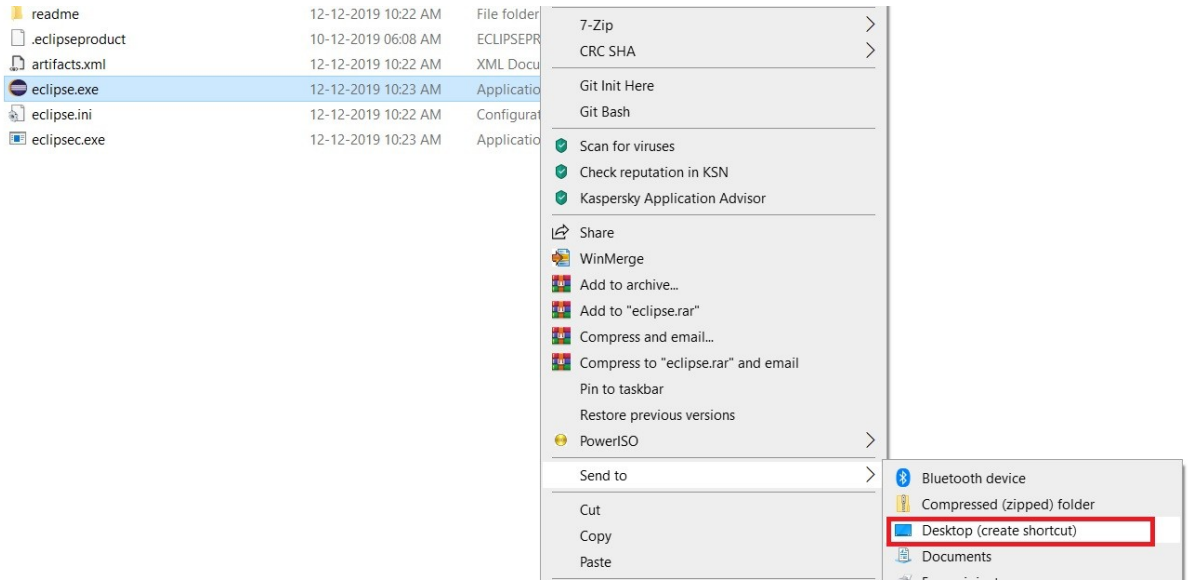
After exploring the folder **eclipse-java-2019-12-R-win32-x86_64**, we can observe folder **eclipse**.



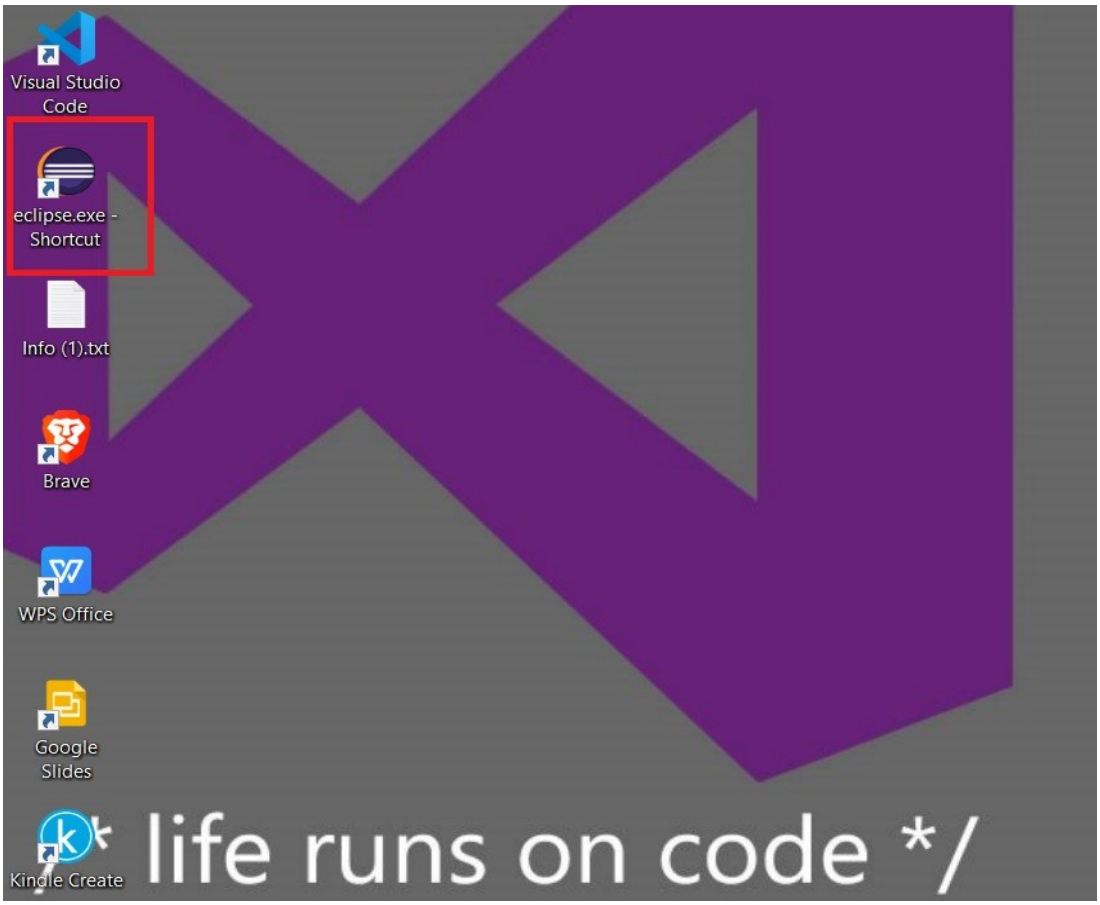
The eclipse folder consist of **eclipse.exe** file.



7. Please select the **eclipse.exe** file and apply mouse right click, a menu will be displayed to create the shortcut option. This will create a short for the **eclipse.exe** on user's desktop.



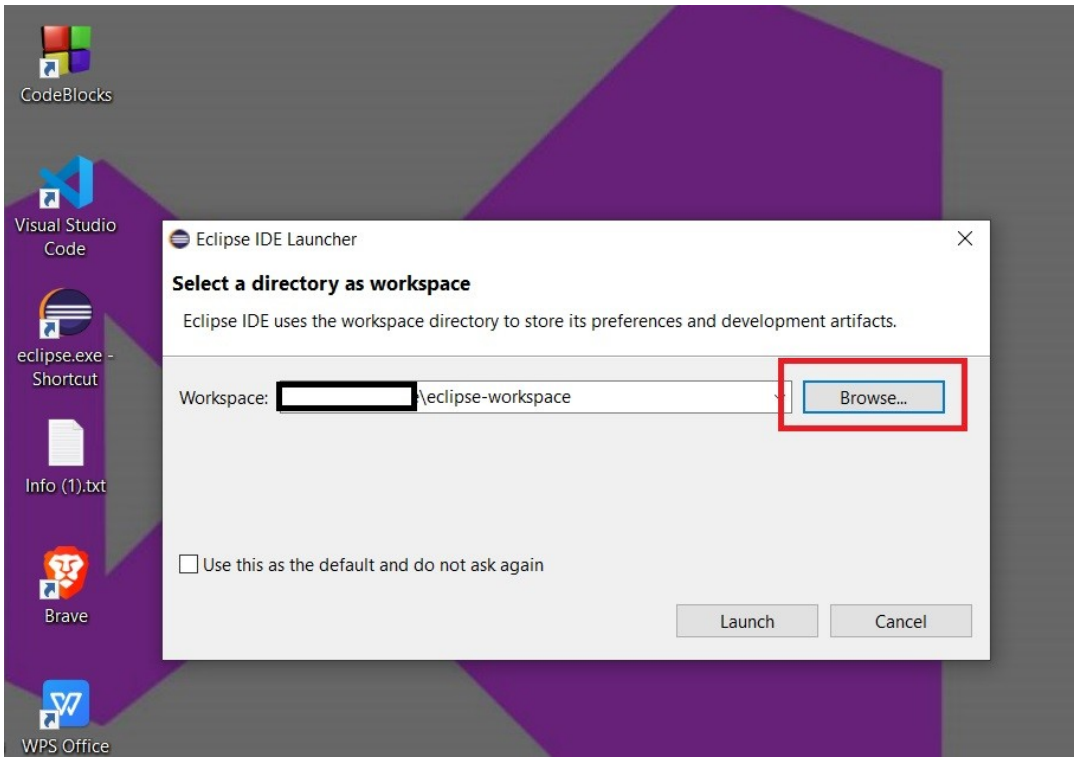
8. Eclipse IDE's shortcut on user's desktop will look the following.



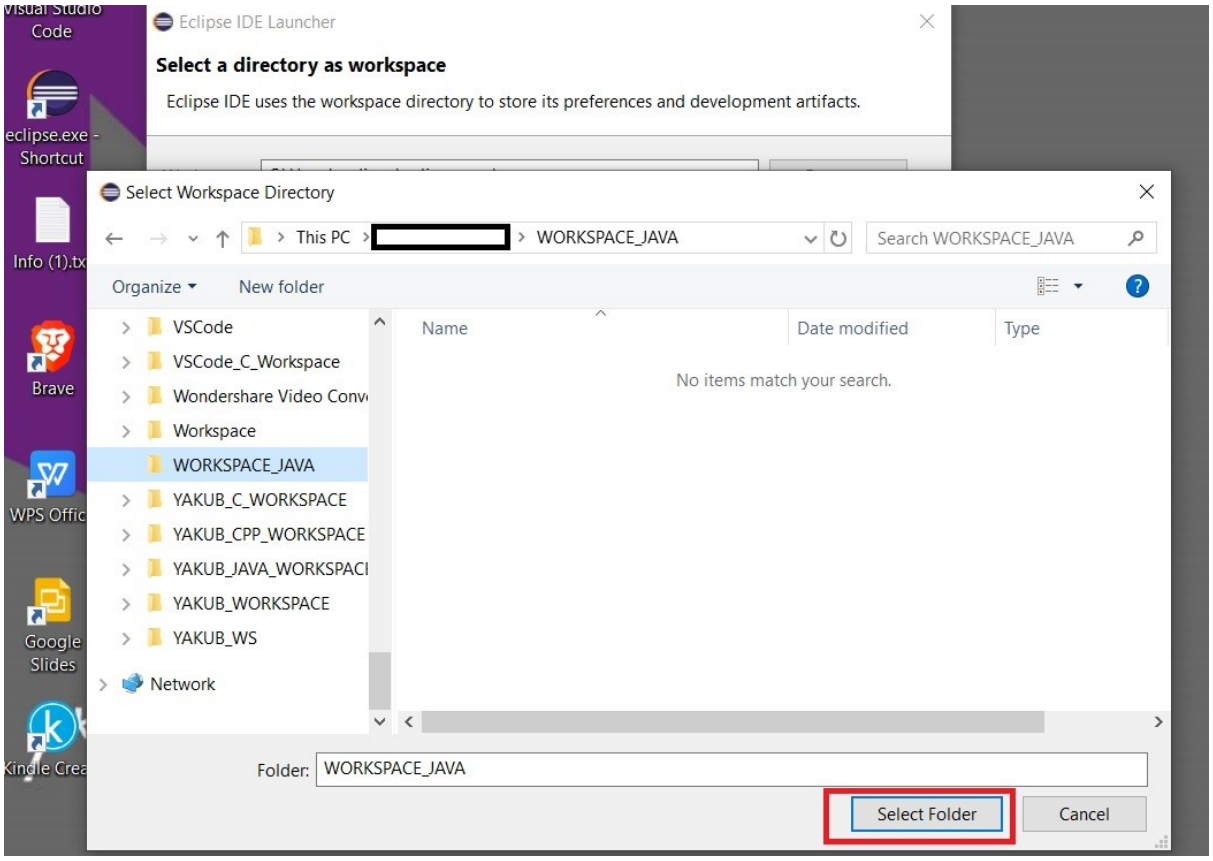
9. Please click the Eclipse IDE's shortcut to launch the IDE.



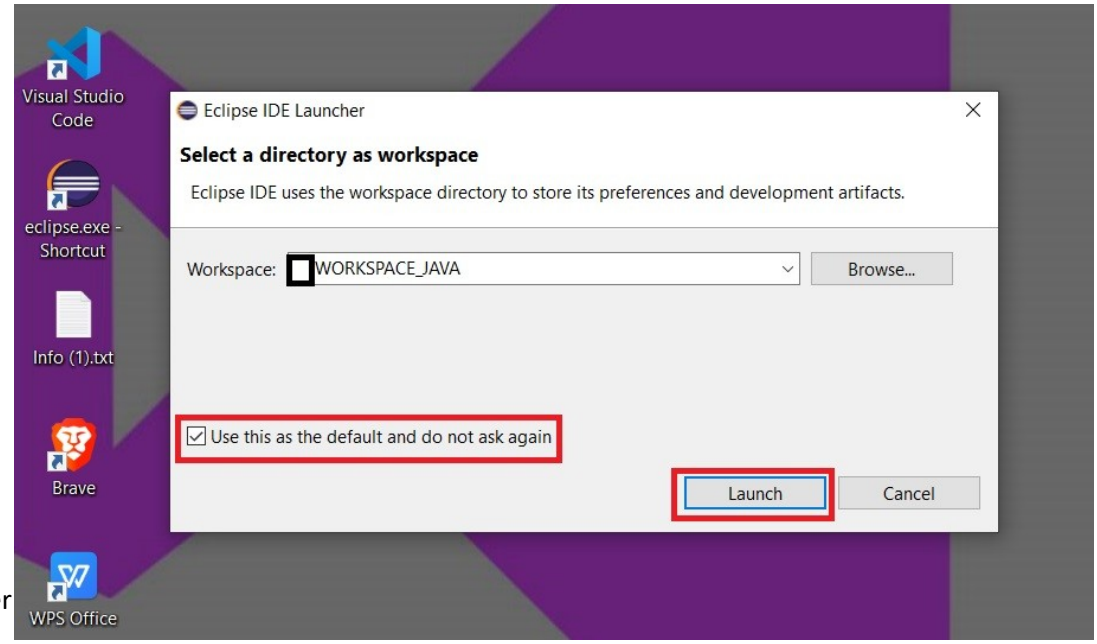
10.The Eclipse IDE launcher will ask for the workspace directory where we will actually create our Java project. We need to click the "**Browse**" button in order to map to the our workspace directory.



11. You can create a workplace directory with the name WORKSPACE_JAVA in your local or can create it with a different name also, just like we create any folder with some name of our choice. Please click the button "**Select Folder**".



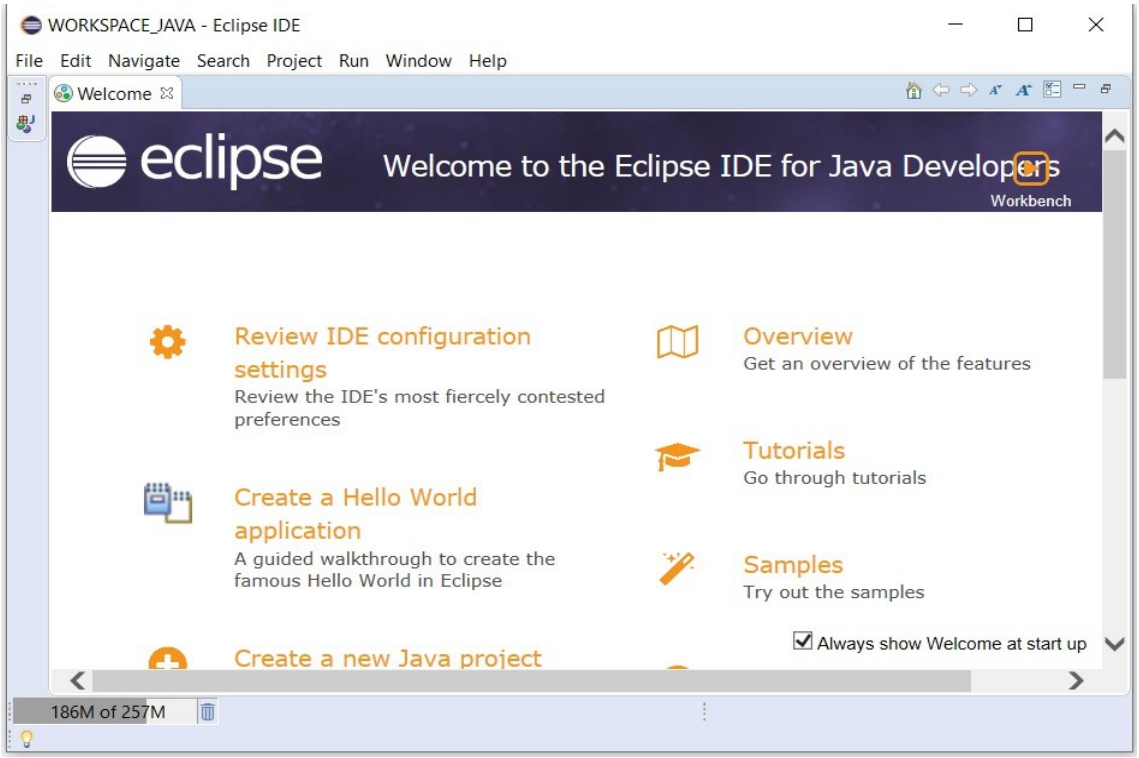
12. Our choose workspace directory's name will appear against the label Workspace: and we can make it as the default workspace by checking the checkbox. So whenever in future when we click the Eclipse IDE's shortcut which is available on the desktop, Eclipse IDE launcher will appear with this default workplace selected. However, we change it to point to a different workspace, in case, if we have any. Please click the "**Launch**" button.



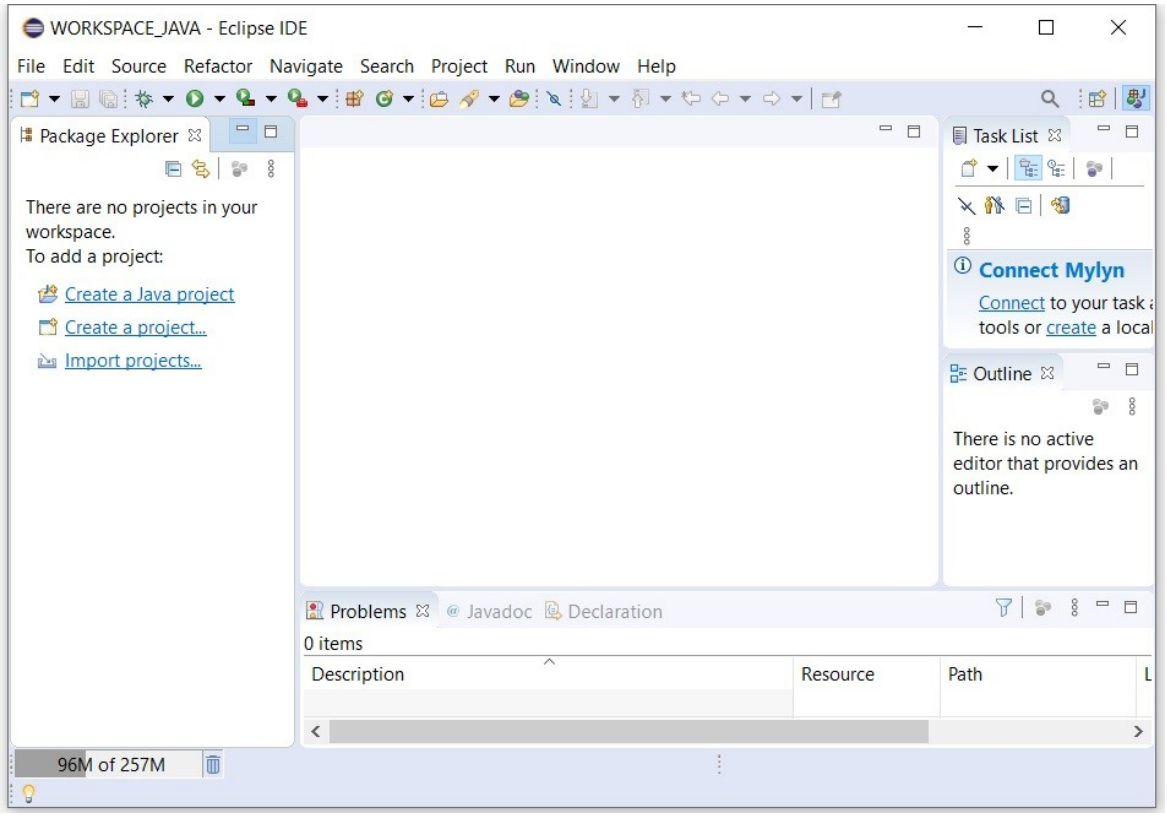
13. After clicking the "Launch" button, it will show the start-up of eclipse IDE.



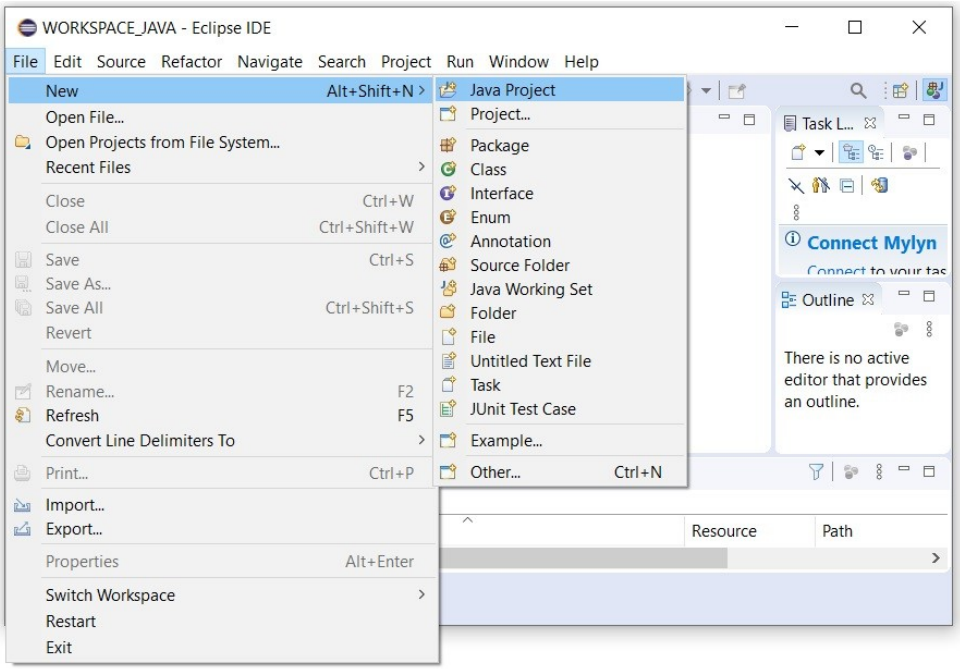
14. When Eclipse is fully ready you will see the Welcome Screen.



You can close the "**Welcome**" screen. On closing it we can see the project environment.

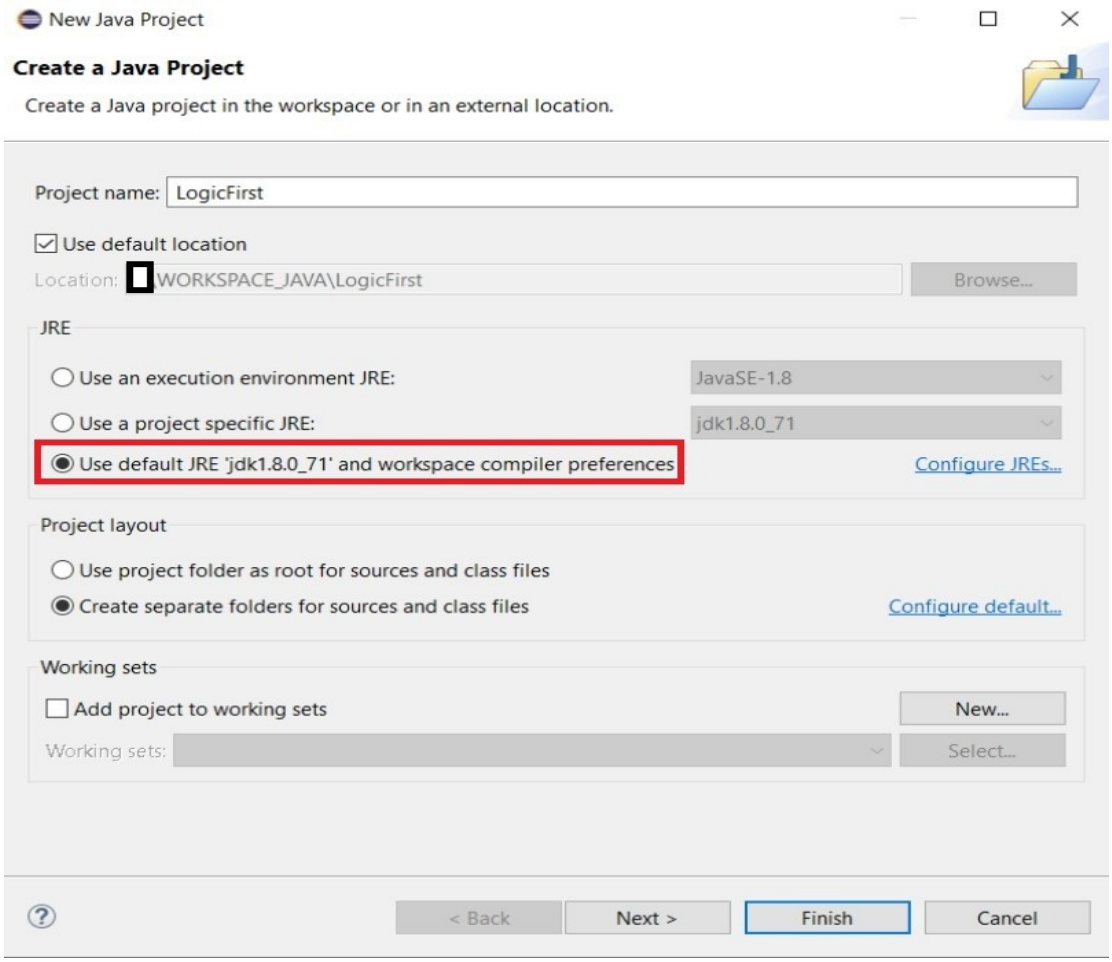


15. Navigate to **File -> New - > Java Project**.



16. The 'Create a Java Project' box will pop-up. You can give project name of your choice.

I kept as "**LogicFirst**" and select the option "**Use default JRE**".Click the "**Finish**" button.



17. The java project "**LogicFirst**" gets created in the eclipse IDE.
The "**src**" is the source code folder where we going to put our **java** programs, that is, **.java** files.
The simple java program's structure can be given as follows :

```
File name will be (Name of the program).java

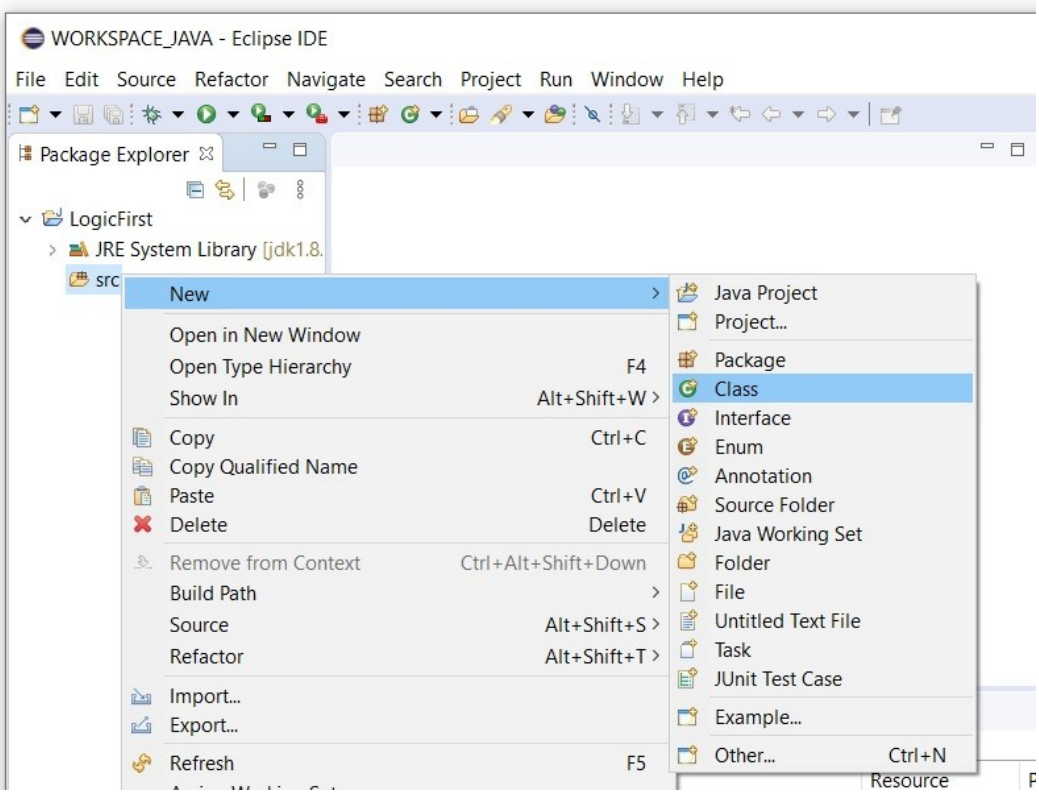
(Name of the program)     ( Default main() method is a must to run program)


public class Welcome{

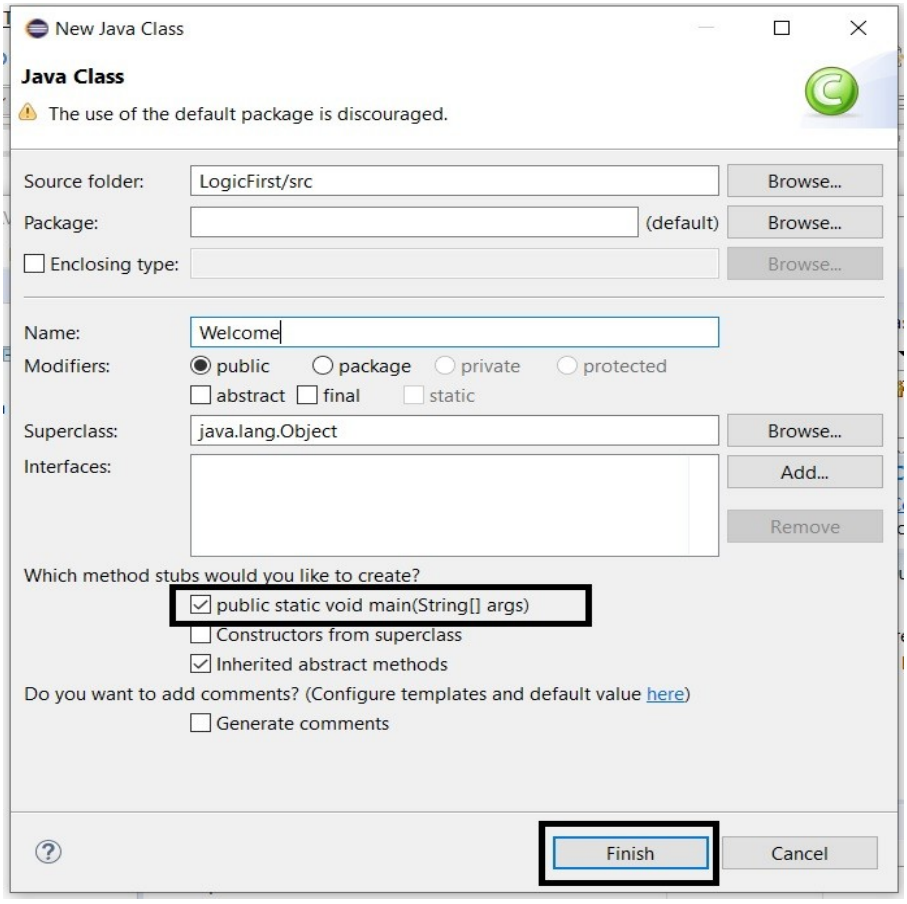
      public static void main ( String[] args ){
            // program logic goes here
               statement(s)
      }
}
```

Java program is a text file and every file has a name and extension to do the classification among the files. The name of Java file is the name that appears after the Java inbuilt keywords "public class".In this case it's **Welcome** and extension of Java program file is "**.java**".So name of the program in this case is **Welcome.java.**

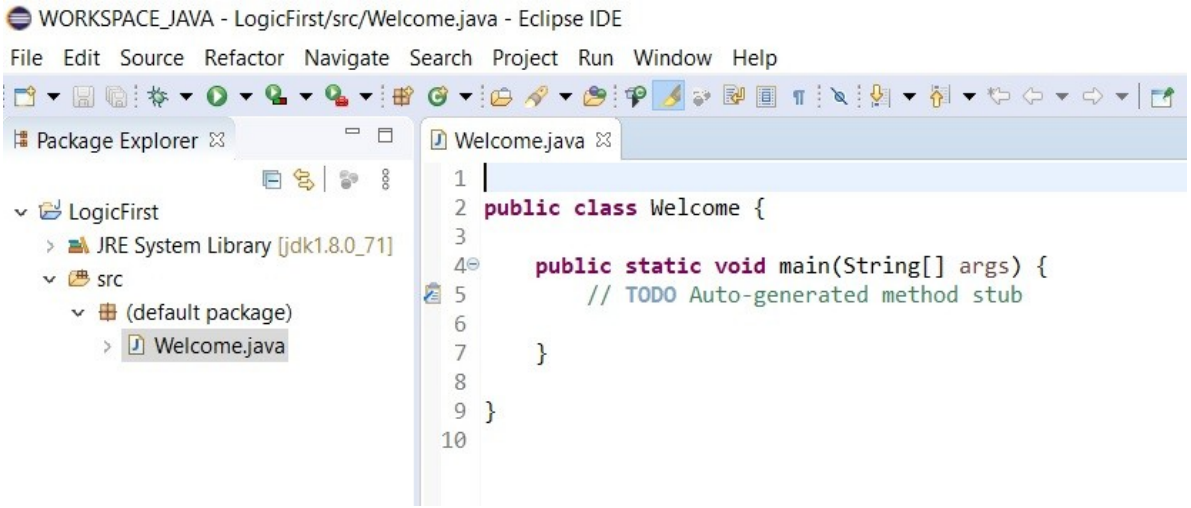
18. Let 's create Java file. Select the "**src**" folder ->Right Click->New->Class



19. The 'New Java Class' box will pop-up. You can give **Java Class** name of your choice.
I named it "Welcome" and do check the checkbox for "**public static void main(String[] args)**".
Click the "**Finish**" button.



20. The basic template for **Welcome.java** program gets created. We need to write the Java programming statement(s) to see it in action when we run this program.



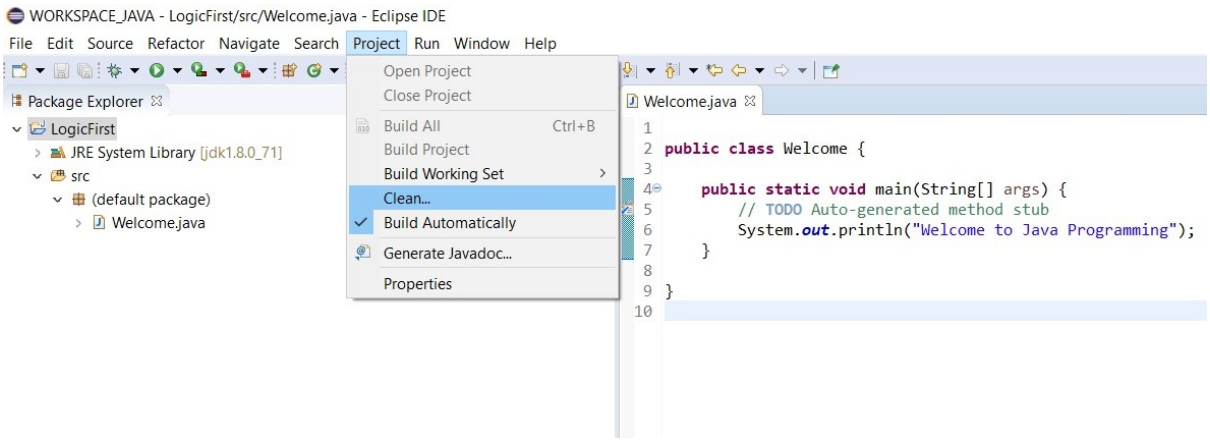
Let's consider our first working example of java program is to output the following text;

Welcome to Java Programming

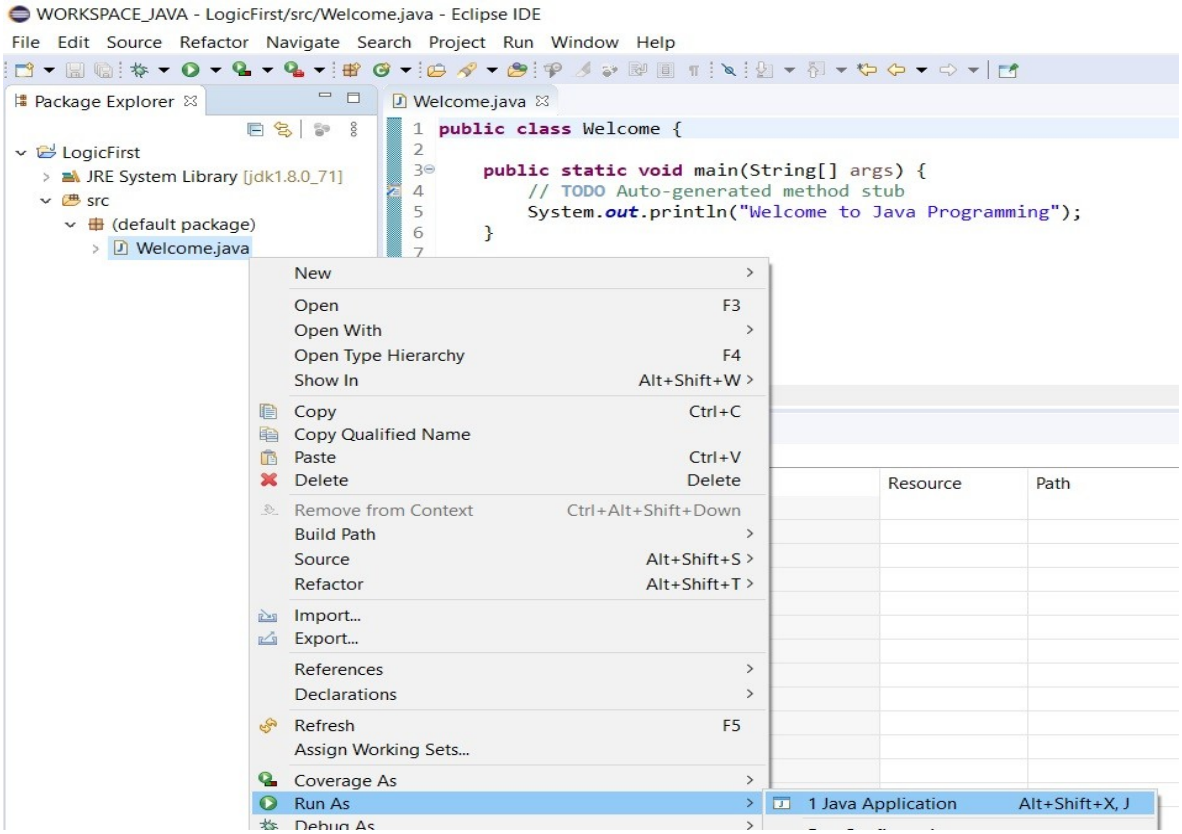
The following statement outputs any mentioned text to the console.

```
System.out.println("Welcome to Java Programming");
```

21. Let's complete our program of outputting the welcome message and build the project by selecting the "**LogicFirst**" Java project. Please use the option from the menu **Project → clean** as shown in the following screenshot. Also, notice that "**Build Automatically**" is checked by default. Although, Eclipse IDE take care of building the project automatically, it is better to use option "**clean**" which forcefully rebuilds the project. In a rare occasion, sometimes it doesn't compile the latest Java file changes and build the project properly. Hence, option "clean" is better option in case of final testing of Java project or in doubt.

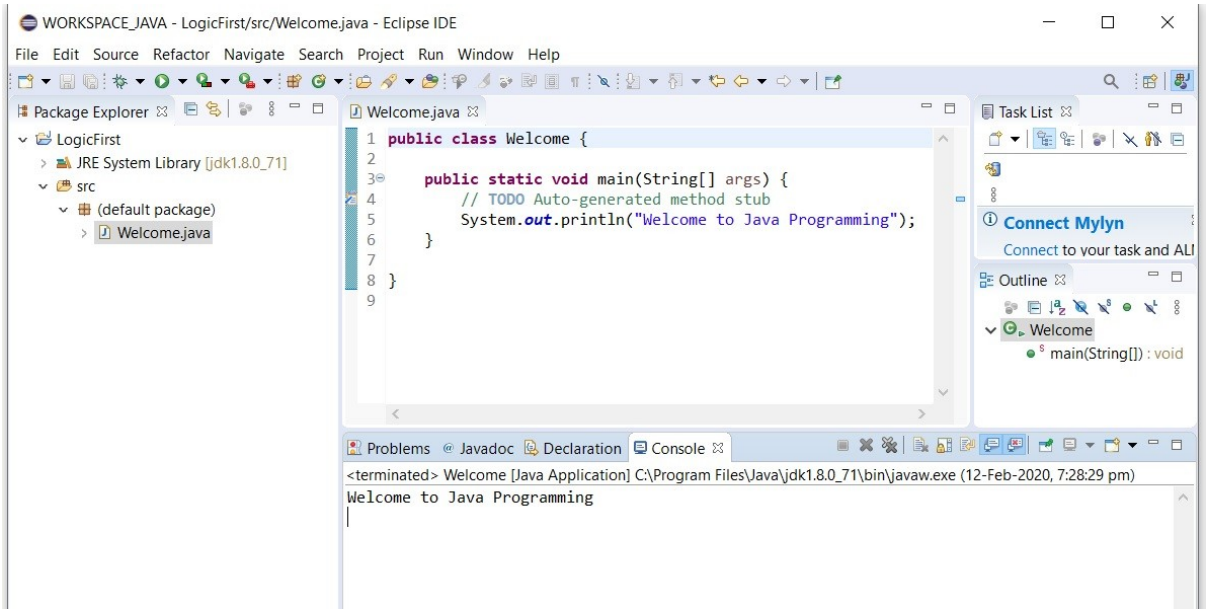


22. Let's try to run **Welcome.java** program and in the same way we can run any other Java program that contains the **main()** method.



Select the **Welcome.java** program file ->**Right Click->Run As->Java Application.**

23. The final output of the program will be automatically displayed in the console window.



References:

1. https://www.analyticsvidhya.com/blog/2013/06/art-structured-thinking-analyzing/
2. https://www.w3schools.com/js/default.asp
3.Object Oriented Programming with C++ and Java [2004] D. Samanta - PHI

Acknowledgments

I personally wish to thank everyone that helped me to make this, my first book possible.

My family who supported me throughout its completion.

My youngest brother, Sadique S. Naikwadi helped me in contributing technical reviews.

He is a freelancer developer and always been famous for his programming skills since college times. He worked on many different languages and never hesitate in trying programming stuffs.

He is always been reached by college mates and beginner programmers for improving their programming skills. His suggestions helped me a lot to put technical stuffs in an easy way and he discussed in detail, the issues of a struggling programmers. There was a time when programming is alien to him and the set of exercises which are covered in this book was once a test-bed for him and he was able to pick programming very nicely. With all these important factors, he is the best person around me to give me an honest technical reviews. Moreover, whenever he gets chance, he runs test-bed based on this book's exercises to help programmatically week learners.

I like to thank Mrs. Shabana Mursal and her family. She teaches in a school, and she made me understand how different teaching style is important to teach students of different mindset.

Her inputs compelled me to put a completely new section which I would have never thought of including it.

Thank you all so much for taking interest to help me.

I am not teaching you anything.

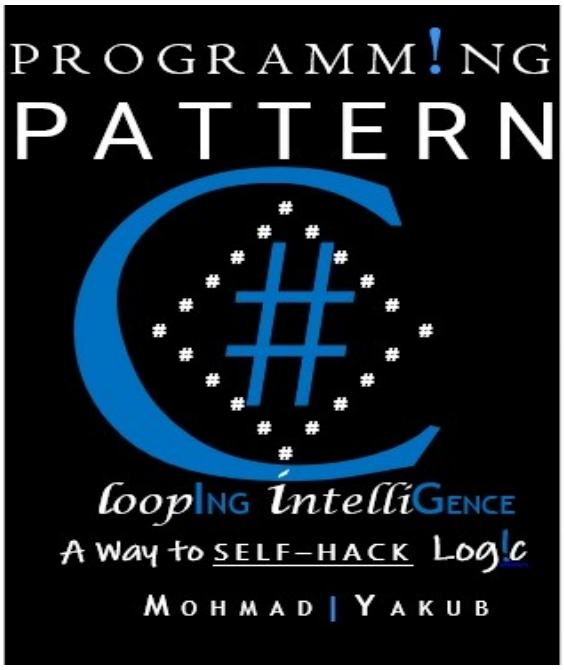
I just help you to explore yourself.

- Bruce Lee

Expect more books by mid-end of this year 2021.

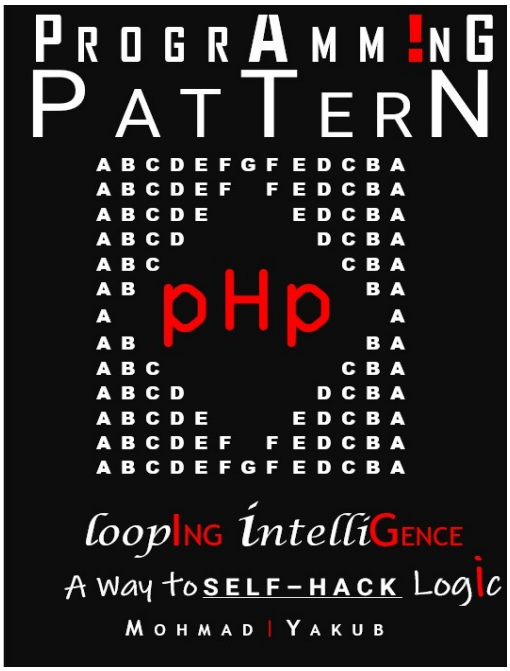
PROGRAMMING PATTERN C#

Looping Intelligence



PROGRAMMING PATTERN pHp

Looping Intelligence



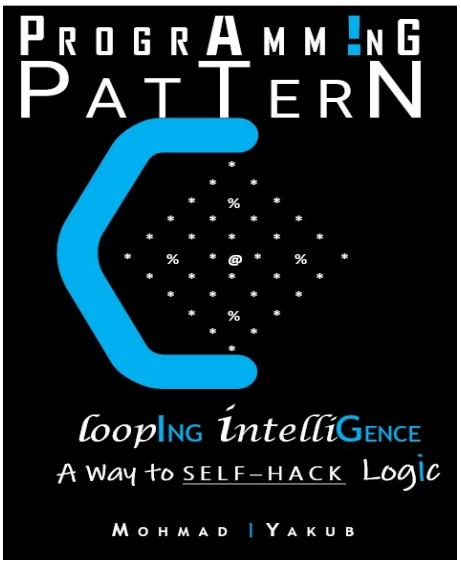
PROGRAMMING PATTERN C + +

Looping Intelligence



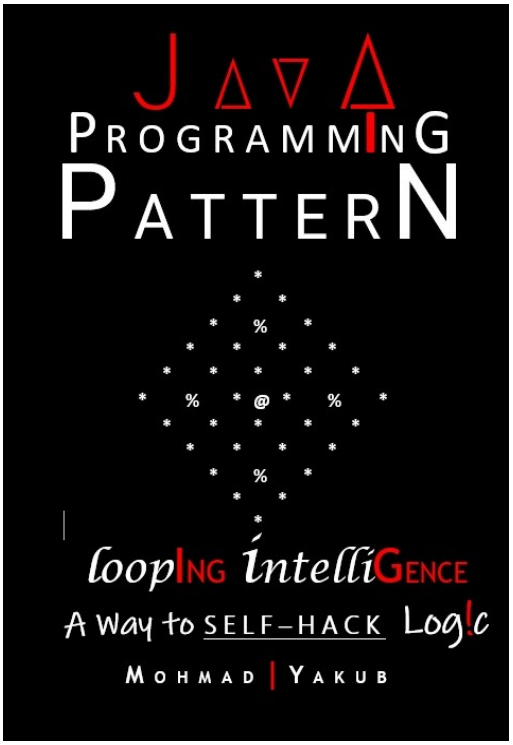
PROGRAMMING PATTERN C

Looping Intelligence



PROGRAMMING PATTERN JAVA

Looping Intelligence



Absolute Beginner's

Kung Fu In Programming

***Logic By Common Sense*.**

